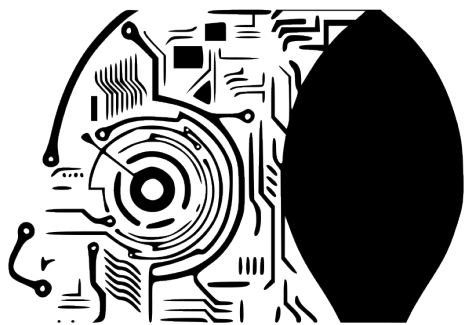




MODELOS DE I.A.

<https://martinezpenya.es/ModelosIA/>

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Modelos de IA (CE Inteligencia Artificial y Big Data)

IES Eduardo Primo Marques (Carlet)

David Martinez Peña

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Índice de contenidos

1. 🚀 Información importante	10
1.1 🚫 Denominación del curso	10
1.2 📋 Contenidos y temporalización	10
1.3 🎯 Resultados de aprendizaje, pesos y calificación	11
1.4 📝 Evaluación	12
1.5 📖 Legislación vigente	12
2. UD00	13
2.1 UD00 — Presentación del módulo y curso rápido de Docker	13
1. Introducción	13
2. Resultados de aprendizaje y objetivos	13
3. Presentación del curso y del módulo (RA7-i)	14
4. Cómo se evalúa el módulo	15
5. Entorno de trabajo del curso	15
6. Curso rápido de Docker: conceptos (RA7-i)	19
7. Curso rápido de Docker: uso básico (RA7-i)	23
8. El Dockerfile: construir imágenes reproducibles (RA7-i)	24
9. Docker Compose: varios servicios a la vez (RA7-i)	26
10. El contenedor de prácticas de IA (RA7-i)	27
11. Copias de seguridad de imágenes y contenedores (RA7-i)	30
12. Puntos clave de la unidad	30
13. Glosario	31
14. FAQ	31
15. Sesiones	32
A. Anexo · chuleta de comandos	32
B. Anexo · catálogo de contenedores para experimentar	33
16. Recursos	34
17. Evaluación	35
18. Recuperación	35
2.2 Diapositivas de la UD00	36
2.3 UD00 — Ejercicios	37
A. Sobre el módulo y la evaluación	37
B. Conceptos de Docker	37
C. Dockerfile	37
D. Docker Compose	37
E. Prácticos	37

F. Instalación, versiones y mantenimiento	38
Soluciones	38
2.4 Talleres	39
UD00 · Taller 1 — Verificación del entorno y primer contenedor	39
UD00 · Taller 2 — Contenedor de prácticas de IA	42
3. UD01	44
3.1 Caracterización de sistemas y utilización de modelos de Inteligencia Artificial	44
Fundamentos de los Sistemas Inteligentes	44
Tipos de Inteligencia Artificial. Escuelas y clasificaciones	57
Utilización de modelos de Inteligencia Artificial	62
Técnicas de la Inteligencia Artificial	64
Campos de Aplicaciones de la Inteligencia Artificial	70
Nuevas Formas de Interacción	82
Mapa conceptual	85
3.2 Diapositivas de la UD01	86
4. UD02	87
4.1 UD02 — Modelos de IA y resolución de problemas	87
1. Introducción	87
2. Resultado de aprendizaje y criterios de evaluación	87
3. Objetivos de la unidad	88
4. Sistema de resolución de problemas (RA2-a)	88
5. Clasificación de modelos de IA (RA2-b)	90
6. Modelos de automatización de tareas (RA2-c)	91
7. Razonamiento impreciso: lógica difusa (RA2-d)	92
8. Sistemas basados en reglas (RA2-e)	94
9. Adecuación del modelo (RA2-f)	96
10. Caso de estudio: Robocode como sistema de resolución de problemas completo	96
11. Puntos clave de la unidad	97
12. Glosario	97
13. FAQ	98
14. Planificación sesión a sesión	99
15. Tabla final RA/CE	99
16. Recursos	99
17. Evaluación	100
18. Recuperación	100
4.2 Diapositivas de la UD02	101
4.3 UD02 — Ejercicios	102
A. Sistema de resolución de problemas (RA2-a)	102

B. Clasificación de modelos de IA (RA2-b)	102
C. Automatización de tareas (RA2-c)	102
D. Lógica difusa (RA2-d)	103
E. Sistemas basados en reglas (RA2-e)	103
F. Adecuación del modelo (RA2-f)	103
4.4 Talleres	104
UD02 · Taller 1 — Control difuso con scikit-fuzzy	104
UD02 · Taller 2 — Sistema basado en reglas con experta	106
UD02 · Taller 3 — Preparar el entorno para Robocode (Java o Python)	109
UD02 · Taller 4 — Control de versiones con GitHub	114
UD02 · Taller 5 — Documentar con Markdown	121
4.5 Actividades Entregables	124
UD02 — Actividad entregable: Robocode Tank Royale	124
UD02 · Comparativa Java vs Python para Robocode	127
UD02 · Robocode Tank Royale en Java	131
UD02 · Robocode Tank Royale en Python	142
5. UD05	159
5.1 UD05 — Sistemas expertos y controladores inteligentes	159
1. Introducción	159
2. Resultado de aprendizaje y criterios de evaluación	159
3. Objetivos de la unidad	160
4. Arquitectura y dinámica de los sistemas expertos (RA5-a)	160
5. Estructuras elementales de representación (RA5-a/b)	163
6. Representar y simular comportamientos con experta (RA5-b)	164
7. Sistemas híbridos reglas/datos (RA5-b)	166
8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d)	167
9. Variación de características y dinámica (RA5-c)	173
10. Estrategias de control de un sistema experto (RA5-d)	174
11. Controladores inteligentes (RA5-e)	175
12. Aplicaciones y tendencias (contenidos del RA5)	177
13. Puntos clave de la unidad	178
14. Glosario	178
15. FAQ	179
16. Sesiones	180
17. Recursos	180
18. Evaluación	181
19. Recuperación	181
5.2 Diapositivas de la UD05	182

5.3 UD05 · Ejercicios	183
A. Del conocimiento a la arquitectura (RA5-a)	183
B. Representación del conocimiento (RA5-a/b)	183
C. Simular comportamientos con experta (RA5-b)	183
D. Sistemas híbridos reglas/datos (RA5-b)	184
E. Lógica difusa (RA5-b/d)	184
F. Variación de características y dinámica (RA5-c)	184
G. Estrategias de control (RA5-d)	184
H. Controladores inteligentes (RA5-e)	184
I. Aplicaciones y tendencias (contenidos RA5)	184
J. Caso práctico	185
5.4 Talleres	186
UD05 · Taller 1 — Simular un sistema experto con experta	186
UD05 · Taller 2 — Lógica difusa: el problema de las propinas	189
UD05 · Taller 3 — Sistema experto como controlador de un proceso	192
5.5 UD05 · Actividades guiadas	195
Referencia: un sistema experto grande, resuelto	195
5.6 UD05 · Actividades entregables	196
Rúbricas	196
6. UD03	199
6.1 UD03 — Procesamiento del Lenguaje Natural	199
1. Introducción	199
2. Resultado de aprendizaje y criterios de evaluación	199
3. Objetivos de la unidad	200
4. Qué es el PLN (RA3-a)	200
5. El potencial (RA3-c)	202
6. La ambigüedad, el problema de fondo (RA3-c)	202
7. Desambiguación y etiquetado morfológico (RA3-a, RA3-c)	203
8. Las demás limitaciones (RA3-c)	204
9. Cuándo es factible aplicar PLN (RA3-d)	205
10. Quién hace un proyecto de PLN (RA3-b, RA3-e, RA3-f)	206
11. Las herramientas en Python (RA3-c, RA3-g)	207
12. Construir un sistema orientado a una tarea (RA3-g)	209
13. Puntos clave de la unidad	211
14. Glosario	212
15. FAQ	213
16. Sesiones	214
17. Recursos	214

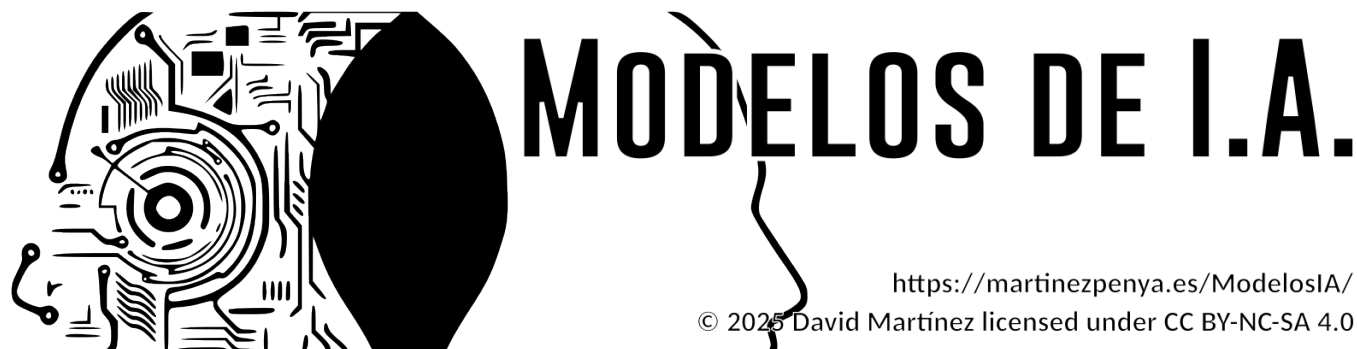
18. Evaluación	215
19. Recuperación	215
6.2 Diapositivas de la UD03	216
6.3 UD03 · Ejercicios	217
A. Qué es el PLN (RA3-a)	217
B. El potencial (RA3-c)	217
C. La ambigüedad (RA3-c)	217
D. Desambiguación y POS (RA3-a, RA3-c)	217
E. Las otras limitaciones (RA3-c)	218
F. Cuándo es factible (RA3-d)	218
G. El papel del lingüista (RA3-b)	218
H. El trabajo cooperativo (RA3-e)	218
I. La formación del investigador (RA3-f)	219
J. Las herramientas (RA3-c, RA3-g)	219
K. Construir un sistema (RA3-g)	219
L. Integración	219
6.4 Talleres	221
UD03 · Taller 1 — Del texto al vector: análisis de sentimiento	221
UD03 · Taller 2 — Un sistema de PLN de punta a punta	224
6.5 UD03 · Actividades guiadas	226
N01 · Introducción al PLN	226
N02 · Clasificación de texto con PyTorch	226
N03 · Modelos de lenguaje: afinar DistilBERT	226
N04 · spaCy, primeros pasos	226
N05 · Ampliación: clasificador de géneros musicales	227
6.6 UD03 · Actividades entregables	228
EX1 · Representación de texto, por dos caminos	228
EX2 · Clasificador de preguntas	228
EX3 · NLTK y Python: etiquetado con <code>cess_esp</code>	228
EX4 · Análisis de sentimiento en reseñas de cine	229
EX5 · Asistente virtual por voz	229
Rúbricas	229
7. UD04	231
7.1 UD04 — Análisis de sistemas robotizados	231
1. Introducción	231
2. Resultado de aprendizaje y criterios de evaluación	231
3. Objetivos de la unidad	232
4. Métodos y aplicaciones de la robótica (RA4-a)	232

5. Qué clase de problema resuelve la robótica	235
6. Modelado y control cinemático (RA4-a)	236
7. Los problemas y sus soluciones (RA4-b)	238
8. Percepción robótica (RA4-b)	240
9. Incertidumbre y aprendizaje (RA4-b)	241
10. Técnicas de programación de robots (RA4-c)	242
11. Humanos y robots (RA4-c, RA4-d)	0
12. Diseño e implementación de sistemas robotizados (RA4-d)	0
13. Practicar sin robot: los dos simuladores de la unidad	0
14. Puntos clave de la unidad	0
15. Glosario	0
16. FAQ	0
17. Sesiones	0
18. Recursos	0
19. Evaluación	0
20. Recuperación	0
7.2 Diapositivas de la UD04	0
7.3 UD04 · Ejercicios	0
A. Métodos y aplicaciones de la robótica (RA4-a)	0
B. Sensores y actuadores (RA4-a)	0
C. Qué problema resuelve la robótica (RA4-a)	0
D. Modelado y control cinemático (RA4-a)	0
E. Problemas y soluciones (RA4-b)	0
F. Espacio de configuración y planificación (RA4-b)	0
G. Percepción (RA4-b)	0
H. Aprendizaje en robótica (RA4-b)	0
I. Técnicas de programación de robots (RA4-c)	0
J. Humanos y robots (RA4-c, RA4-d)	0
K. Diseño e implementación (RA4-d)	0
L. Simulación con Python (RA4-a, RA4-b)	0
7.4 Talleres	0
UD04 · Taller 1 — Cinemática de un manipulador	0
UD04 · Taller 2 — Diseño de un sistema robotizado	0
7.5 UD04 · Actividades guiadas	0
1 · Introducción a OpenCV	0
2 · Vehículos de Braitenberg	0
3 · Ejemplos de robots en AITK	0
7.6 UD04 · Actividades entregables	0

EX1 · Ejercicios de OpenCV	0
EX2 · Navegar con cámara, por reglas	0
EX3 · Navegar con cámara, con lógica difusa	0
EX4 · Generar los datos de entrenamiento	0
EX5 · Controlar el robot con una red neuronal	0
EX6 · Aprendizaje por refuerzo con NEAT	0
Rúbricas	0
8. UD06	0
8.1 UD06 — Aplicación de principios legales y éticos de la IA	0
1. Introducción	0
2. Resultado de aprendizaje y criterios de evaluación	0
3. Objetivos de la unidad	0
4. Riesgos y deontología de la IA (RA6-a)	0
5. Privacidad y protección de datos (RA6-b)	0
6. Cumplimiento estricto de la legalidad (RA6-c)	0
7. Security by design (RA6-d)	0
8. Privacy by design (RA6-e)	0
9. Sesgos de género y algorítmicos (RA6-f)	0
10. Puntos clave de la unidad	0
11. Glosario	0
12. FAQ	0
13. Sesiones	0
14. Recursos	0
15. Evaluación	0
16. Recuperación	0
8.2 Diapositivas de la UD06	0
8.3 UD06 · Ejercicios	0
A. Riesgos y deontología (RA6-a)	0
B. Privacidad y protección de datos (RA6-b)	0
C. Cumplimiento legal (RA6-c)	0
D. Security by design (RA6-d)	0
E. Privacy by design (RA6-e)	0
F. Sesgos de género y algorítmicos (RA6-f)	0
G. Casos prácticos	0
8.4 Talleres	0
UD06 · Taller 1 — Análisis de un caso ético	0
UD06 · Taller 2 — Auditoría de sesgos con Fairlearn	0
8.5 Debates	0

UD06 · Debate 1 — Límites éticos de la Inteligencia Artificial	0
UD06 · Debate 2 — El algoritmo contra el crimen	0
8.6 UD06 · Actividades guiadas	0
Antes de cada debate	0
Para ampliar	0
Actualidad: veinticuatro noticias para el debate	0
8.7 UD06 · Actividades entregables	0
Resumen de entregas	0
Rúbrica de los debates	0
Tertulia de ciencia-ficción	0
9. 🙋 Sobre mí...	0
9.1 🧑 David Martínez Peña	0
9.2 📧 Contacto:	0
10. Fuentes de información	0

1. 🚀 Información importante



1.1 🏠 Denominación del curso

📖 **Curso de especialización de Inteligencia Artificial y Big Data**

🏠 Modelos de Inteligencia Artificial (MIA) · Código **5071** · **90 h** · 4 ECTS

i **Curso 2026-2027**

La docencia empieza el **1 de octubre de 2026** y el módulo termina el **28 de mayo de 2027**; junio se reserva a la **convocatoria ordinaria**. Las clases son de **lunes a jueves**: los viernes no hay clase y se aprovechan para las tareas y los talleres en casa.

Vacaciones: Navidad del 22 de diciembre al 6 de enero · Pascua del 25 de marzo al 5 de abril.

Festivos que caen en día de clase: 12 de octubre, 7 y 8 de diciembre, y **Fallas, 17 y 18 de marzo**. El 9 de octubre (Día de la Comunitat Valenciana) y el 19 de marzo caen en viernes, así que no afectan.

Entre Fallas y Pascua, **marzo es el mes más interrumpido del curso**: tenlo en cuenta para planificar entregas.

1.2 📋 Contenidos y temporalización

Las unidades se imparten en este orden. El identificador **UDxx** va ligado a su resultado de aprendizaje (UD05 es siempre RA5), no al orden en que se imparte.

UD	Título	RA	Horas	Semanas	Fechas
UD00	Presentación y curso rápido de Docker	—	6	1-2	1 oct - 8 oct
UD01	Caracterización de sistemas de IA	RA1	12	3-6	12 oct - 6 nov
UD02	Modelos de IA y resolución de problemas	RA2	12	7-10	9 nov - 3 dic
UD05	Sistemas expertos	RA5	12	11-15	7 dic - 14 ene
UD03	Procesamiento del Lenguaje Natural	RA3	12	16-19	18 ene - 11 feb
UD04		RA4	12	20-23	15 feb - 11 mar

UD	Título	RA	Horas	Semanas	Fechas
	Análisis de sistemas robotizados				
UD06	Principios legales y éticos de la IA	RA6	6	24-28	15 mar - 23 abr
UD07	Proyecto integrador (común al curso)	RA7	15	29-33	26 abr - 28 may
Cierre	Presentaciones RA7 y prueba ordinaria (solo RA no superados)	—	3	34-35	1 jun - 18 jun
Total			90	30 nominales 33 con clase	

**Por qué este orden**

La UD05 (sistemas expertos) se imparte justo después de la UD02 porque se construye sobre las reglas y la lógica difusa que se ven allí. Y la UD06 (ética) ocupa el tramo de Fallas y Pascua, el más fragmentado del curso, porque es la unidad con menos carga de laboratorio.

El **contenido del módulo acaba a finales de abril**. Después viene el **proyecto intermodular (RA7)**, común a todo el curso de especialización: durante unas cinco semanas tu equipo trabaja el proyecto en las horas de todos los módulos.

1.3 Resultados de aprendizaje, pesos y calificación

RA	CE	Descripción	Nota mínima	Peso
RA1	4	Caracteriza sistemas de Inteligencia Artificial relacionándolos con la mejora de la eficiencia operativa de las organizaciones y empresas.	5	13,33 %
RA2	6	Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.	5	13,33 %
RA3	7	Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.	5	13,33 %
RA4	4	Analiza sistemas robotizados, evaluando opciones de diseño e implementación.	5	13,33 %
RA5	5	Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.	5	13,33 %
RA6	6	Aplica principios legales y éticos al desarrollo de la Inteligencia Artificial integrándolos como parte del proceso.	5	13,33 %
RA7	9	Desarrolla un proyecto integrador que combine técnicas de IA y análisis	5	20 %

RA	CE	Descripción	Nota mínima	Peso
		de datos masivos para resolver un problema real o simulado, gestionando todo el ciclo de vida del proyecto.		
				100 %

Cada RA se califica **1 a 10, sin decimales** (Orden 8/2025, art. 5.1): **40 %** tareas, talleres y ejercicios + **60 %** prueba escrita. Para superar el módulo hace falta **5 o más en cada RA**.

1.4 Evaluación

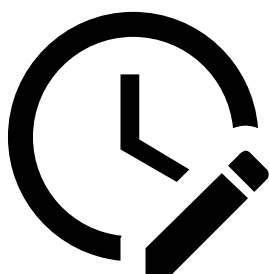
-  La evaluación del módulo se realiza sobre los **Resultados de Aprendizaje (RA)** del currículo. Cada RA tiene asociados sus **criterios de evaluación (CE)**, que son los que determinan el grado de adquisición de las competencias del módulo.
-  La nota final del módulo sale de la ponderación de los RA. Cada RA se evalúa de forma independiente, con **calificación de 1 a 10, sin decimales** (Orden 8/2025, art. 5.1).
-  Cada RA tiene **una prueba escrita en Moodle** al cerrar su unidad, con preguntas de test y de desarrollo sobre el contenido de la unidad.
-  Hay que obtener al menos un **5 en cada RA** para aprobar el módulo.
-  Si un RA queda por debajo de 5, se recupera con las actividades y pruebas diseñadas para **ese** resultado de aprendizaje. La convocatoria ordinaria de junio se dirige **solo a los RA no superados**: no hay prueba global del módulo.
-  La **UD00** (presentación y Docker) **no se califica**: el entorno de trabajo es un requisito. Sus tres entregables —los dos talleres y el notebook— se marcan como **hecho / no hecho**, sin nota pero **obligatorios**: son la prueba de que tienes el entorno operativo, y sin ellos no se pueden hacer las prácticas de la UD02.

Importante

-  Aprobar las evaluaciones parciales **no garantiza** aprobar el módulo.
-  Puedes aprobar las dos evaluaciones con buena nota, tener **un solo RA suspendido** y suspender el módulo.

1.5 Legislación vigente

-  [RD 279/2021, de 20 de abril](#) — establece el curso de especialización y fija los **RA y CE** del módulo.
-  [RD 497/2024, de 21 de mayo](#) — modifica determinados reales decretos de cursos de especialización.
-  [Decreto 95/2026, de 9 de junio \(DOGV\)](#) — currículo en la Comunitat Valenciana: fija el módulo en **90 h**.
-  [Horario y organización del curso \(GVA\)](#)



23 de agosto de 2026

2. UD00

2.1 UD00 — Presentación del módulo y curso rápido de Docker

Unidad 0 · 6 h · semanas 1-2 (1 al 8 de octubre)

Unidad transversal de presentación y arranque del curso. **No tiene prueba escrita:** el entorno de trabajo es **requisito** para el resto del módulo, y el entregable del Taller 1 se marca como **hecho / no hecho**.

1. Introducción

Esta primera unidad tiene un doble objetivo. Por una parte, **situar el curso:** qué es el curso de especialización en Inteligencia Artificial y Big Data, qué se va a aprender en el módulo 5071 *Modelos de Inteligencia Artificial* y cómo se va a evaluar. Por otra, **dejar listo el entorno de trabajo:** sin herramientas fiables y reproducibles, cualquier práctica de IA se convierte en una lucha contra configuraciones que no funcionan. Por eso el núcleo técnico de la unidad es un **curso rápido de Docker**, la tecnología que nos permitirá ejecutar el mismo entorno de Python y Jupyter en cualquier equipo.

La idea que sostiene la unidad

La IA se hace con **entornos reproducibles:** si cada alumno tiene una configuración distinta, los resultados no son comparables. Docker resuelve ese problema empaquetando el entorno completo en un contenedor.

2. Resultados de aprendizaje y objetivos

La UD00 es **transversal** (no desarrolla un RA propio del módulo): sirve de nivelador tecnológico y de presentación. Las competencias que trabaja se asocian a la **ética y el cuidado del entorno de trabajo** (RA7-i) y a los **hábitos de trabajo** del curso.

Objetivo	Descripción
O1	Describir el curso de especialización IA y Big Data y el papel del módulo 5071 en él.
O2	Explicar cómo se evalúa el módulo (criterios, calificación, recuperación).
O3	Identificar el entorno de trabajo del curso: Aules, web, Python, Jupyter y Docker.
O4	Diferenciar contenedor, imagen, volumen y red, y explicar qué ventajas aporta Docker frente a las máquinas virtuales.
O5	Crear, ejecutar, detener y eliminar contenedores con <code>docker run</code> .
O6	Escribir un <code>Dockerfile</code> sencillo y levantarlo con <code>docker build</code> .
O7	Orquestar varios servicios con Docker Compose y conservar datos con volúmenes.
O8	Levantar un contenedor de prácticas con Jupyter y las bibliotecas del curso.

¿Por qué no tiene RA propio?

El currículo del RD 279/2021 desarrolla 6 resultados de aprendizaje para el módulo 5071 (RA1-RA6). La UD00 es una **unidad de presentación y puesta en marcha** que prepara el terreno para esas unidades: sin ella, los RA1-RA6 perderían horas en configurar entornos.

3. Presentación del curso y del módulo (RA7-i)

3.1 El curso de especialización en IA y Big Data

El **curso de especialización en Inteligencia Artificial y Big Data** es un título de **Grado Superior** (familia Informática y Comunicaciones) de **600 horas y 36 ECTS** establecido por el RD 279/2021. Su competencia general es *programar y aplicar sistemas inteligentes que optimizan la gestión de la información y la explotación de datos masivos, garantizando la seguridad y la ética*.

Se organiza en **cinco módulos**:

Código	Módulo	Horas en CV
5071	Modelos de Inteligencia Artificial	90
5072	Sistemas de aprendizaje automático	90
5073	Programación de Inteligencia Artificial	210
5074	Sistemas de Big Data	90
5075	Big Data aplicado	120

El acceso se realiza desde titulaciones de grado superior de la familia de informática y otras relacionadas (ASIR, DAM, DAW, telecomunicaciones, mecatrónica y robótica industrial).



Una frase para recordar

Este curso enseña a **construir sistemas inteligentes** (5071, 5072, 5073) y a **tratar datos a gran escala** (5074, 5075). El módulo 5071 es la base conceptual de los sistemas de IA.

3.2 El módulo 5071 *Modelos de Inteligencia Artificial*

El módulo 5071 se imparte a lo largo de todo el curso (**90 horas, 3 horas semanales**, 4 ECTS) y desarrolla **6 resultados de aprendizaje** (RA1-RA6), más el **RA7 de proyecto integrador transversal**:

RA	Qué aprenderás
RA1	Caracterizar sistemas de IA y relacionarlos con la eficiencia operativa
RA2	Utilizar modelos de IA para implementar sistemas de resolución de problemas
RA3	Relacionar el procesamiento del lenguaje natural (PLN) con sus aplicaciones
RA4	Analizar sistemas robotizados y evaluar su diseño e implementación
RA5	Aplicar sistemas expertos y valorar los controladores inteligentes
RA6	Aplicar principios legales y éticos (privacidad, sesgos, normativa)
RA7	Proyecto integrador transversal de todo el curso (definición → datos → modelos → Big Data → comunicación)

3.3 El proyecto integrador (RA7)

El **RA7** es un **proyecto común a todo el curso de especialización**: se desarrolla en **equipos** durante el bloque final (semanas 25-29) y en él **confluyen todos los módulos** del título. El alumnado define un problema, obtiene datos, aplica modelos de IA, integra soluciones de Big Data, comunica resultados y evalúa críticamente el trabajo realizado.

Es transversal por dos razones: 1. Lo **imparten todos los docentes** del curso, no solo el del 5071. 2. La **nota se consensúa** entre el equipo docente, valorando la contribución de cada miembro.

**Implicación para este módulo**

El contenido del 5071 (UD00-UD06) **finaliza a finales de abril** porque las últimas semanas del curso se dedican por completo al proyecto integrador. Desde el primer día, el proyecto te permitirá aplicar lo aprendido en este módulo.

4. Cómo se evalúa el módulo

La evaluación se rige por la **Orden 8/2025** de la Comunitat Valenciana (modificada por la **Orden 5/2026**). Lo que debes saber desde el primer día:

Los **pesos de cada RA, la nota mínima y el reparto 40/60** están en la [página de información importante](#), que es la referencia única: si algún día cambian, cambian allí. Aquí van las reglas que conviene tener claras desde el primer día y que no dependen de los pesos:

Aspecto	Regla
Calificación por RA	De 1 a 10, sin decimales (art. 5.1)
Evaluación continua	Se pierde al superar el 15 % de inasistencia (asistencia $\geq$ 85 %)
Evaluaciones parciales	Dos parciales informativos (fin del 1.er y 2.º trimestre)
Recuperación	Programa individual por cada RA no superado ; en junio, solo los RA pendientes
Esta unidad (UD00)	No se califica : es requisito. Sus tres entregables (los dos talleres y el notebook) se marcan hecho / no hecho

**Asistencia**

Si superas el **15 % de faltas** perderás la evaluación continua del módulo (art. 7.3 Orden 8/2025). Es un criterio objetivo, no una decisión del profesor.

**Evaluación inicial**

Durante el primer mes realizaremos una **evaluación inicial o diagnóstica** (obligatoria antes de finalizar el segundo mes lectivo). No cuenta para la nota: sirve para conocer tus competencias previas y adaptar el ritmo del curso.

5. Entorno de trabajo del curso

Herramienta	Para qué la usaremos
Aules (Moodle de la GVA)	Entrega de tareas, pruebas, avisos y calificaciones
Web del curso (mkdocs)	Teoría, ejercicios, talleres y notebooks en el navegador
Python + Jupyter	Notebooks de prácticas con las bibliotecas de IA
Docker	Entornos reproducibles y el contenedor de prácticas
Git (opcional)	Control de versiones de tus proyectos

**Sobre Aules**

Aules es la **plataforma oficial de e-learning de la Conselleria de Educación**, basada en **Moodle**. Accederás con tu usuario (NIA) y desde ahí se gestionan las tareas del módulo.

El stack de IA del curso

Las prácticas se basan en **Python** con estas bibliotecas:

numpy, pandas, matplotlib, scikit-learn, scikit-fuzzy, nltk, spaCy, torch y experta.

Este stack se ejecutará en un **contenedor de prácticas de IA** que montaremos en el taller 2 de esta unidad.

5.2 Instalar Docker

Antes de nada, Docker tiene que funcionar en tu máquina. Estos pasos están probados en el aula.

INSTALACIÓN EN UBUNTU

<https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-20-04-es>

Requisitos previos:

- **apt-transport-https:** permite que el administrador de paquetes transfiera datos a través de https
- **ca-certificates:** permite que el navegador web y el sistema verifiquen los certificados de seguridad
- **curl:** transfiere datos (similar a wget)
- **software-properties-common:** agrega scripts para administrar el software

```
1 sudo apt-get install curl apt-transport-https ca-certificates software-properties-common
```

Agregamos repositorio

```
1 # Primero clave GPG
2 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
3
4 sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
5
6 sudo apt update
7
8 sudo apt install docker-ce
9
10 sudo systemctl status docker
```

Por defecto, el comando docker solo puede ser ejecutado por el usuario root o un usuario del grupo docker, que se crea automáticamente durante el proceso de instalación de Docker.

Para evitar escribir sudo al ejecutar el comando docker, agregue su nombre de usuario al grupo docker:

```
1 sudo usermod -aG docker ${USER}
2
3 # Cerramos y abrimos sesión de nuevo o ejecutamos
4 su - ${USER}
5
6 # Confirmamos los grupos de nuestro usuario
7 id -nG
```

DOCKER DESKTOP

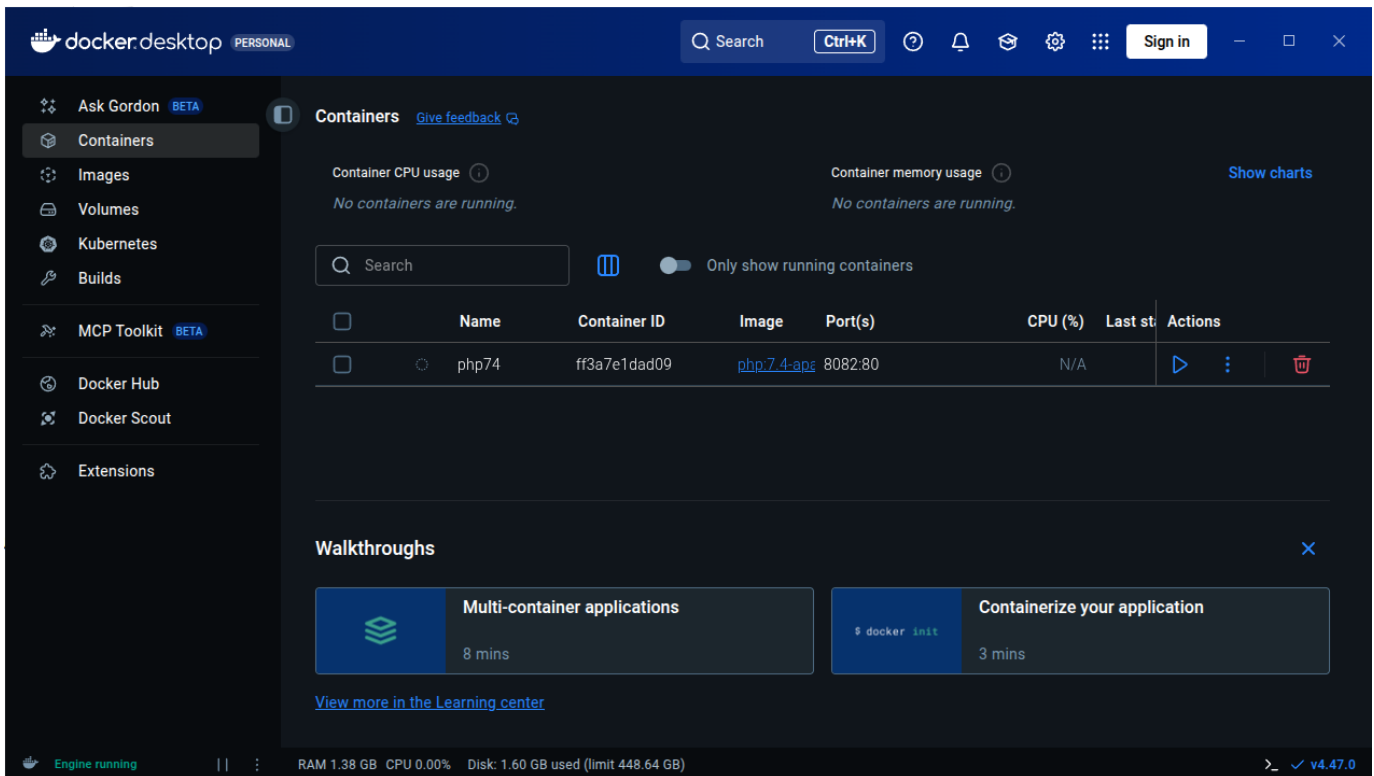
¿Qué es Docker Desktop?



Es la aplicación oficial de Docker que te da una **interfaz gráfica (GUI)** para manejar contenedores, además de la línea de comandos.

¿Para qué sirve?

- **Gestión visual:** Ver contenedores, imágenes y volúmenes de forma gráfica
- **Configuración fácil:** Ajustar recursos (CPU, RAM) con sliders
- **Monitorización:** Ver en tiempo tiempo real qué está pasando



Compatibilidad por Sistema Operativo

Windows

- **Windows 10/11** 64-bit (versiones Home, Pro, Enterprise, Education)
- **Requisitos importantes:**
 - Habilitar **WSL 2** (Windows Subsystem for Linux)
 - Virtualización activada en BIOS/UEFI
- **Windows Home** necesita WSL 2, **Pro/Enterprise** puede usar Hyper-V

macOS

- **macOS 12 Monterey** o superior
- **Tipos de chip:**
 - **Apple Silicon** (M1, M2, M3, etc.)
 - **Intel** con procesador de 2010 o más nuevo
- Necesita **macOS actualizado**

Linux (versión nativa)

- **Distribuciones compatibles:**
 - Ubuntu 20.04 LTS o superior
 - Debian 11 o superior
 - Fedora 36 o superior
 - Arch Linux (y derivados)
- **Requisitos:** kernel 5.10+, systemd, 64-bit

Guía Rápida de Instalación

Windows:

1. Descarga desde docker.com/products/docker-desktop
2. Ejecuta el instalador `.exe`
3. Sigue el asistente (marca "Use WSL 2" si tienes Windows Home)
4. Reinicia cuando termine

5. ¡Listo! Docker se inicia automáticamente

macOS:

1. Descarga desde la web oficial
2. Arrastra Docker.app a la carpeta Applications
3. Ejecuta desde Launchpad
4. Autoriza con contraseña del sistema
5. Espera a que configure todo (puede tardar unos minutos)

Linux (Ubuntu/Debian ejemplo):

```
1 # Paso 1: Descargar el .deb oficial (siempre la última versión)
2 wget https://desktop.docker.com/linux/main/amd64/docker-desktop-amd64.deb
3
4 # Paso 2: Instalar
5 sudo apt install ./docker-desktop-*.deb
6
7 # Paso 3: Iniciar
8 systemctl --user start docker-desktop
```

⚠ Problemas con Ubuntu 24.04 ▾

Si tienes problemas con Ubuntu 24.04, yo me he encontrado dos:

1. Problema de permisos

```
1 ...
2 S'estan processant els activadors per a desktop-file-utils (0.27-2build1)...
3 N: La baixada es duu a terme fora de l'entorn segur com a root ja que el fitxer «/home/ubuntu/docker-desktop-4.27.2-amd64.deb» no és accessible per l'usuari «_apt». - pkgAcquire::Run (13: Permission denied)
```

Lo he resuelto cambiando los permisos del deb:

```
1 # Change ownership of the file to make it accessible
2 sudo chown _apt:root /home/ubuntu/docker-desktop-4.27.2-amd64.deb
3 # Or alternatively, change permissions to make it readable
4 sudo chmod 644 /home/ubuntu/docker-desktop-4.27.2-amd64.deb
```

2. Lanzas Docker-desktop, aparece el icono, pero desaparece y la aplicación no inicia: Parece un problema con un cambio en Ubuntu 24.04 que he resuelto con la información de este [post](#):

```
1 sudo systemctl -w kernel.apparmor_restrict_unprivileged_userns=0
2 systemctl --user restart docker-desktop
```

En algunos casos parece que la solución anterior solo sirve hasta que reinicias, si es así, prueba esto también:

Crea un nuevo fichero:

```
1 sudo nano /etc/apparmor.d/opt.docker-desktop.bin.com.docker.backend
```

Escribe dentro el siguiente contenido:

```
1 abi <abi/4.0>,
2
3 include <tunables/global>
4
5 /opt/docker-desktop/bin/com.docker.backend flags=(default_allow) {
6   userns,
7
8   # Site-specific additions and overrides. See local/README for details.
9   include if exists <local/opt.docker-desktop.bin.com.docker.backend>
10 }
```

Reinicia el servicio `apparmor.service`:

```
1 sudo systemctl restart apparmor.service
```

Ventajas docker desktop (GUI) vs Línea de Comandos (CLI)

✓ Ventajas de Docker Desktop:

- **Más fácil para empezar** - Ideal para principiantes
- **Todo integrado** - No necesitas instalar nada más
- **Debugging visual** - Ves los logs y estados de un vistazo
- **Gestión de recursos** - Controlas CPU/RAM fácilmente

✗ Desventajas:

- **Más pesado** - Consume más recursos de tu PC
- **Menos flexible** - Algunas opciones avanzadas solo por comandos
- **Dependes de la GUI** - Si se cierra la app, pierdes la interfaz

🎯 Conclusión:

- **Empezad con Docker Desktop** para aprender sin frustraciones
- **Aprended también los comandos básicos** para ser más versátiles
- Usad **ambos**: la GUI para lo cotidiano y la terminal para lo avanzado

⚠ Comprueba la instalación antes de seguir

`docker run hello-world` debe terminar sin errores. Si te responde `permission denied while trying to connect to the Docker daemon socket`, te falta el paso de añadir tu usuario al grupo `docker` **y cerrar y abrir sesión**.

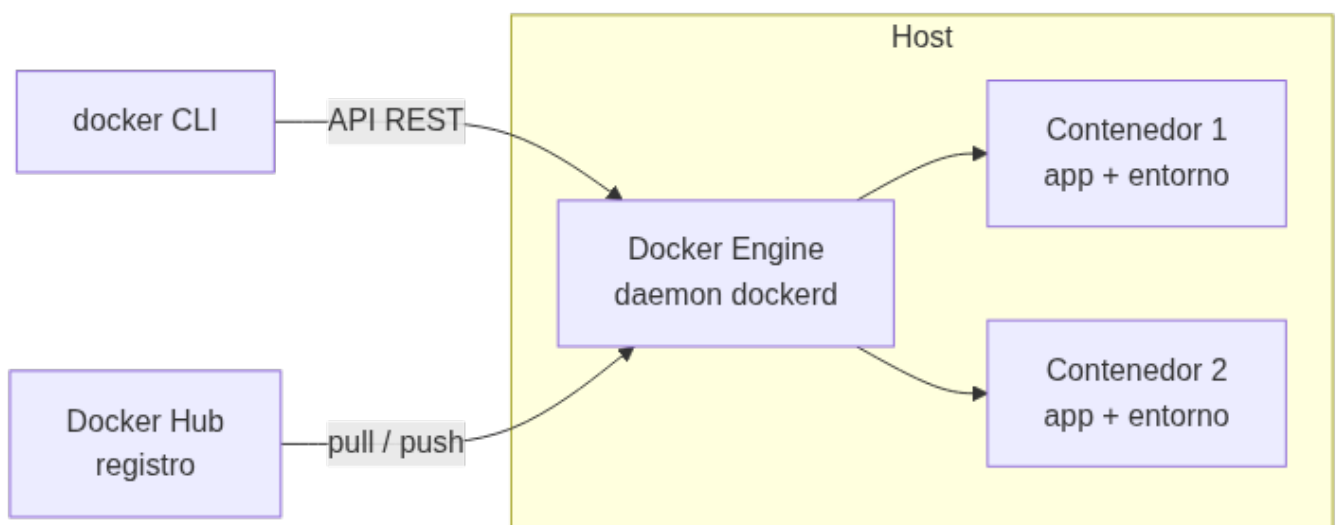
6. Curso rápido de Docker: conceptos (RA7-i)

6.1 El problema: entornos que no se reproducen

¿Cuántas veces funciona "en mi ordenador" y no en el del compañero? La causa es que los entornos difieren: versiones de Python, bibliotecas, sistema operativo, configuraciones. **Docker resuelve este problema** empaquetando la aplicación y todo lo que necesita en un **contenedor**.

✍ Definición

Docker es una plataforma abierta para desarrollar, distribuir y ejecutar aplicaciones. Permite separar la aplicación de la infraestructura mediante contenedores: entornos aislados y ligeros que incluyen todo lo necesario para ejecutar la aplicación.



6.2 Contenedores frente a máquinas virtuales

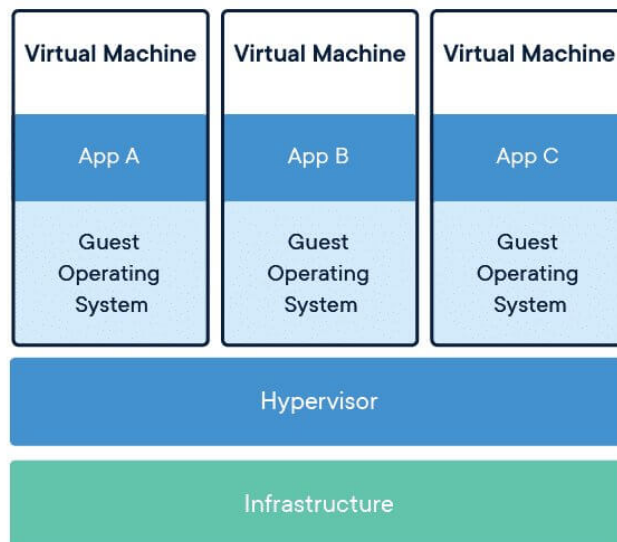
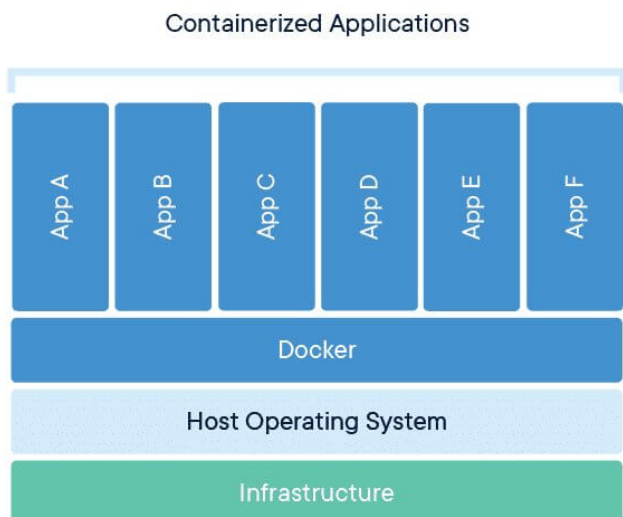
Característica	Máquina virtual	Contenedor
Aislamiento	Por hardware (hipervisor)	Por procesos (namespaces del kernel)
Sistema operativo	SO completo con su propio kernel	Comparte el kernel del host
Arranque	Minutos	Segundos
Tamaño	Gigabytes	Megabytes
Carga en un servidor	Decenas	Cientos/miles
Reproducibilidad	Media	Alta (imagen inmutable + capas)

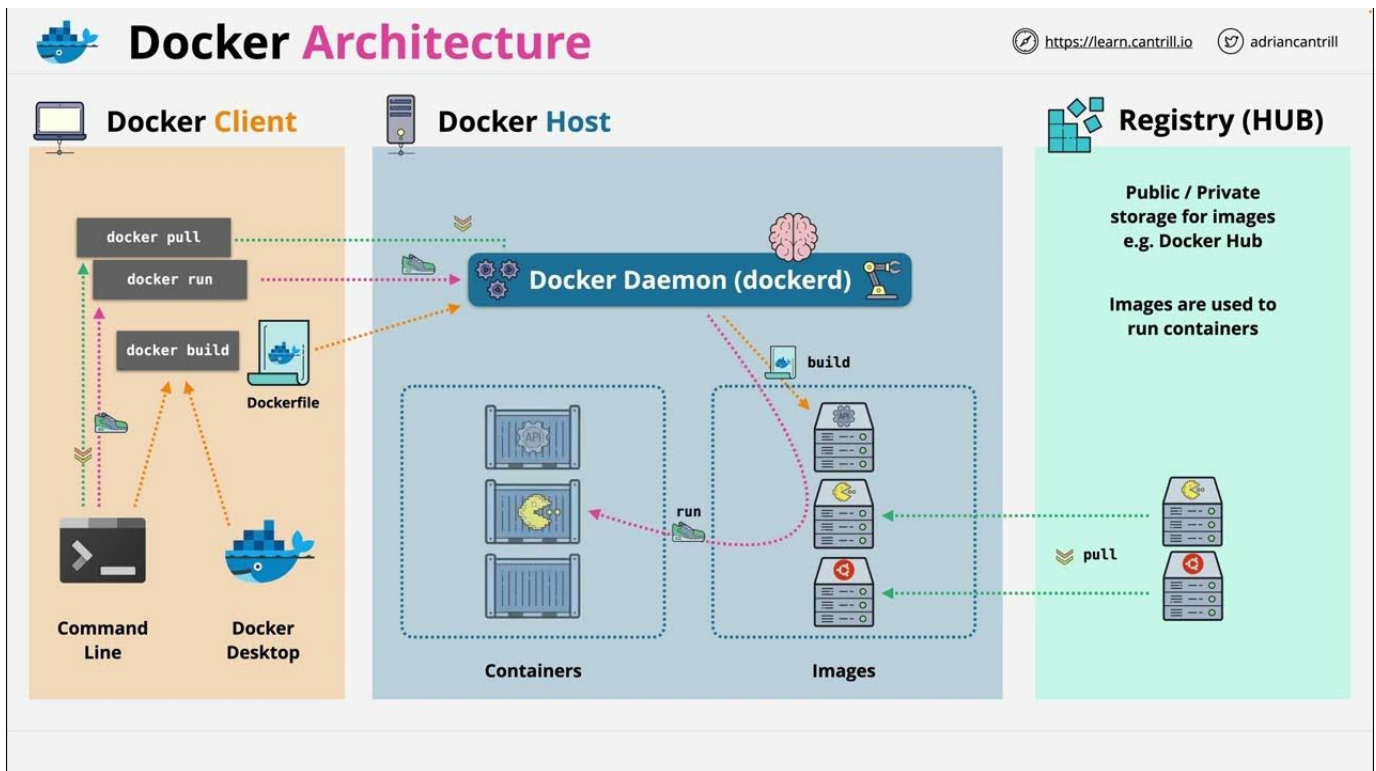


¿Y entonces las VMs son inútiles?

No. Las VMs siguen siendo necesarias cuando se requiere **aislamiento total de kernel** o se ejecutan sistemas operativos distintos. La práctica habitual es combinarlas: una VM con Docker encima. En este curso trabajaremos con contenedores porque son **ligeros y reproducibles**.

LA ARQUITECTURA DE DOCKER EN IMÁGENES





Imágenes

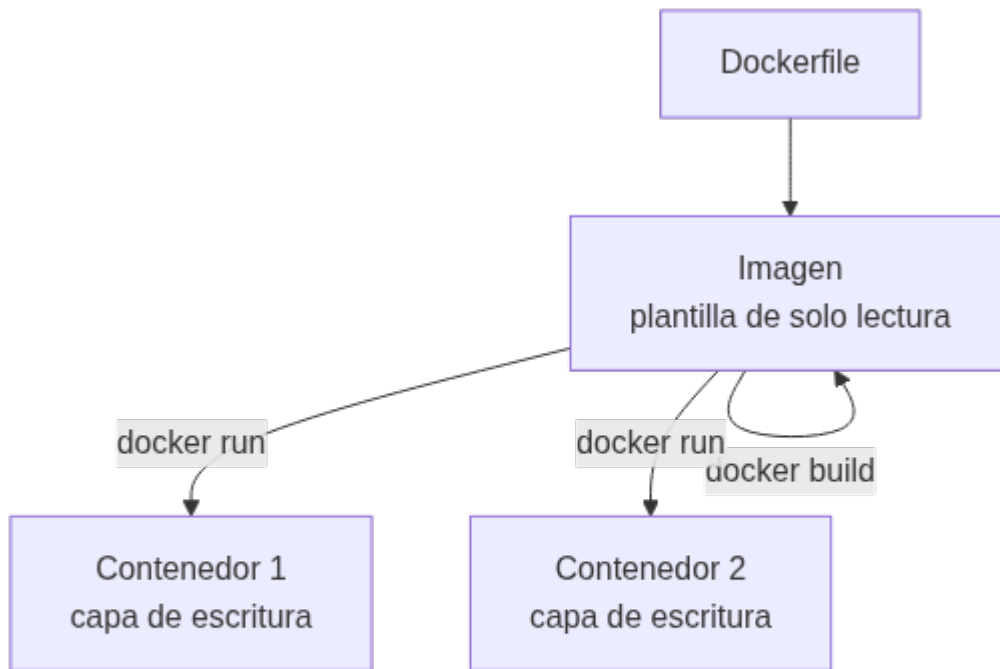
- ▶ Plantilla de solo lectura
- ▶ Sistema de ficheros y parámetros listos para ejecutar
- ▶ Basadas en sistemas operativos Linux

Contenedores

- ▶ Instancia de una imagen
- ▶ Es un directorio dentro del sistema, similar a los "jail root"
- ▶ Pueden ser ejecutados, reiniciados, parados...

6.3 Imágenes y contenedores

- **Imagen:** plantilla **de solo lectura** (un paquete inmutable con el SO base, la aplicación y las dependencias). Se construye por **capas**.
- **Contenedor:** **instancia ejecutable** creada a partir de una imagen. Añade una capa de escritura por encima de la imagen.



La regla de las capas

Cada instrucción del `Dockerfile` crea una **capa**. Al reconstruir una imagen, solo se regeneran las capas que han cambiado: por eso **el orden de las instrucciones importa** para aprovechar la caché.

6.4 El registro y el primer contenedor

Docker Hub es el registro público por defecto: se usa `pull` para **descargar** imágenes y `push` para **subirlas**. Nuestro primer comando será:

```
1 docker run hello-world
```

La primera vez, Docker descarga la imagen `hello-world`, crea un contenedor a partir de ella, ejecuta un mensaje de confirmación y lo detiene. Ese es exactamente el ciclo completo de Docker en una sola línea.



Comprueba tu instalación

Antes de seguir, verifica que Docker está instalado y el demonio en marcha:

```
1 docker --version
2 docker run hello-world
```

6.5 Mismo código, dos entornos: el caso de `experta`

El argumento definitivo a favor de los contenedores se ve mejor con una biblioteca que usaremos de verdad en este módulo: `experta`, el motor de reglas de la UD02 y la UD05.

Dato	Valor
Última versión	1.9.4 , del 16/11/2019

Dato	Valor
Python que declara soportar	3.5 a 3.8
Mantenimiento	abandonado ; fallo abierto como <i>issue</i> #34 desde julio de 2023
Causa	su dependencia <code>frozendict==1.2</code> usa <code>collections.Mapping</code> , que desapareció en Python 3.10

Monta dos contenedores con **el mismo código** y compara:

```

1 # Contenedor A: Python 3.9, experta funciona tal cual
2 docker run --rm python:3.9-slim bash -c \
3   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(1)'"
4
5 # Contenedor B: Python 3.12, el mismo código revienta
6 docker run --rm python:3.12-slim bash -c \
7   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(1)'"

```

El contenedor B falla con `AttributeError: module 'collections' has no attribute 'Mapping'`, y se arregla con tres líneas **antes** del `import`, las mismas que verás en los notebooks de la UD02:

```

1 import collections, collections.abc
2 for nombre in ("Mapping", "Iterable", "MutableMapping"):
3     if not hasattr(collections, nombre):
4         setattr(collections, nombre, getattr(collections.abc, nombre))
5
6 from experta import KnowledgeEngine # ahora sí

```



Tres lecciones en un solo ejemplo

1. **El entorno importa:** mismo código, misma biblioteca, dos resultados. Sin fijar la versión de Python, «en mi máquina funciona» no significa nada.
2. **Las dependencias se abandonan:** `experta` no publica nada desde 2019. Elegir una biblioteca es apostar también por quien la mantiene.
3. **Se puede convivir con ello:** un parche documentado y versionado, y sigues adelante. Lo que no vale es descubrirlo el día de la entrega.

7. Curso rápido de Docker: uso básico (RA7-i)

7.1 El ciclo de vida de un contenedor

Comando	Qué hace
<code>docker run</code>	Crea y arranca un contenedor nuevo (descarga la imagen si falta)
<code>docker ps</code>	Lista los contenedores en marcha
<code>docker ps -a</code>	Lista también los detenidos
<code>docker stop</code>	Detiene un contenedor (envía SIGTERM)
<code>docker start</code>	Reactiva un contenedor parado conservando sus cambios
<code>docker rm</code>	Elimina un contenedor (parado)
<code>docker rmi</code>	Elimina una imagen



`docker run` no borra el contenedor

Cuando el proceso principal termina, el contenedor **se detiene pero no se elimina**. Cada `docker run` crea un contenedor nuevo; por eso conviene usar `--rm` en los contenedores desechables y limpiar con `docker rm` los que ya no se usen.

7.2 Flags clave de `docker run`

Flag	Significado	Ejemplo
<code>-it</code>	Interactivo + terminal (para entrar a un shell)	<code>docker run -it ubuntu bash</code>
<code>-d</code>	En segundo plano (detached)	<code>docker run -d -p 8080:80 nginx</code>
<code>-p</code>	Publica un puerto (host:contenedor)	<code>docker run -p 8888:8888 ...</code>
<code>-v</code>	Monta un volumen o bind mount	<code>docker run -v "\$PWD":/app ...</code>
<code>--rm</code>	Elimina el contenedor al terminar	<code>docker run --rm hello-world</code>
<code>--name</code>	Asigna un nombre al contenedor	<code>docker run --name mi-python ...</code>

7.3 Qué ocurre dentro de `docker run`

Cuando ejecutas `docker run`, el demonio hace lo siguiente:

1. Comprueba si la imagen está en la caché local; si no, la **descarga** (`pull`).
2. **Crea el contenedor**: añade una capa de escritura sobre las capas de solo lectura.
3. **Monta el filesystem** (y los volúmenes o binds indicados) y crea la **interfaz de red**.
4. **Arranca el proceso principal** (el comando indicado, o el `CMD` de la imagen).
5. Cuando ese proceso termina, el contenedor **se detiene** (no se elimina).

7.4 Volúmenes y persistencia

Los contenedores son **efímeros**: al eliminarlos se pierden sus datos. Para conservarlos:

- **Volúmenes gestionados** (*named volumes*): los gestiona Docker (en `/var/lib/docker/volumes`). Son la opción recomendada para datos que no necesitas ver desde el host (cachés, datasets, bases de datos).
- **Bind mounts**: montan una **carpeta real del host** dentro del contenedor. Son ideales para el código y los notebooks del curso: editas en tu equipo y el contenedor ve los cambios al instante.

```
1 # bind mount: la carpeta practicas del host se ve en /app dentro del contenedor
2 docker run --rm -v "$PWD/practicas":/app -w /app python:3.12 python app.py
```



Bind mount vs volumen

Para **notebooks y código** del curso usa un **bind mount** (`-v ./practicas:/home/jovyan/work`): son ficheros reales de tu equipo. Los **volúmenes nombrados** quedan para datos que no necesitas tocar desde el host.

8. El Dockerfile: construir imágenes reproducibles (RA7-i)

8.1 Instrucciones esenciales

Un `Dockerfile` es un **guion** que describe cómo construir la imagen. Siempre empieza por `FROM` (la imagen base).

Instrucción	Qué hace
<code>FROM</code>	Define la imagen base (obligatoria)
<code>WORKDIR</code>	Establece el directorio de trabajo
<code>COPY</code>	Copia ficheros del contexto de build a la imagen
<code>RUN</code>	Ejecuta un comando durante el build (instalar, compilar)
<code>ENV</code>	Define variables de entorno
<code>EXPOSE</code>	Documenta el puerto (no lo publica; hace falta <code>-p</code>)
<code>CMD</code>	Comando por defecto en runtime (sobrescribible en <code>docker run</code>)

Instrucción	Qué hace
ENTRYPOINT	Ejecutable fijo; se puede combinar con <code>CMD</code> para los argumentos

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 CMD ["python", "app.py"]

```

8.2 La caché de capas

Cada instrucción crea una capa. Docker **reutiliza las capas en caché** mientras las instrucciones y su contenido no cambien. Por eso se copia `requirements.txt` **antes** que el código: así las dependencias se instalan una vez y se cachean.



Buenas prácticas para un curso rápido

- Anclar la versión de la imagen base (`FROM python:3.12-slim`, no `FROM python`).
- Usar `.dockerignore` para excluir del build context `.git`, `*.pyc`, `.env`, datasets grandes.
- Unir `apt-get update` && `apt-get install` en un solo `RUN`.
- Un **proceso por contenedor** y contenedores **efímeros**.

8.3 Construir y ejecutar

```

1 docker build -t mi-app . # construye la imagen desde el Dockerfile del directorio actual
2 docker run -p 8000:8000 mi-app # ejecuta la imagen publicando el puerto 8000

```

8.4 Gestionar las imágenes que acumulas

Es un instalador donde podemos incorporar nuestra aplicación. Es el punto de inicio para crear contenedores. Hay imágenes oficiales de por ejemplo Ubuntu, Apache, etc, que fueron creadas por sus creadores oficiales.

Página oficial para imágenes: <https://hub.docker.com>

Vamos a utilizar la siguiente imagen para pruebas:

https://hub.docker.com/_/hello-world

Para ejecutar este contenedor “hello-word” escribimos en la terminal:

```
1 docker run hello-world
```

Una vez ejecutado, ya dispondremos de la imagen descargada, podemos ver todas las imágenes que tenemos descargadas con:

```

1 docker images
2
3 REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
4 hello-world         latest      ee301c921b8a     9 months ago    9.14kB

```

Desde la página web de docker hub, podemos ver diferentes versiones de la misma imagen en la pestaña “TAGS”

Overview **Tags**

Sort by **Newest** Filter Tags

TAG [latest](#) Last pushed 4 days ago by [dolianky](#) `docker pull hello-world:latest`

DIGEST	OS/ARCH	VULNERABILITIES	COMPRESSED SIZE
dbbd3cf66631	linux/386	None found	2.66 KB
245fe15fbb8f	windows/amd64	None found	115.14 MB
088bdbea94d5	windows/amd64	None found	99.78 MB

+8 more...

TAG [nanoserver-ltsc2022](#) Last pushed 4 days ago by [dolianky](#) `docker pull hello-world:nanoserv...`

DIGEST	OS/ARCH	VULNERABILITIES	COMPRESSED SIZE
245fe15fbb8f	windows/amd64	None found	115.14 MB

Podemos descargar una imagen específica y ejecutarla:

```
1 docker run hello-world:linux
2
3 docker images
4 REPOSITORY          TAG          IMAGE ID      CREATED       SIZE
5 hello-world         latest      ee301c921b8a  9 months ago  9.14kB
6 hello-world         linux       ee301c921b8a  9 months ago  9.14kB
7
8 docker run hello-world:linux
```

Para eliminar una imagen utilizamos el parámetro `rmi` por ejemplo:

```
1 docker pull alpine
2
3 docker images
4 REPOSITORY          TAG          IMAGE ID      CREATED       SIZE
5 alpine              latest      ace17d5d883e  3 weeks ago   7.73MB
6 hello-world         latest      ee301c921b8a  9 months ago  9.14kB
7 hello-world         linux       ee301c921b8a  9 months ago  9.14kB
8
9 # Eliminamos, opción 1
10 docker rmi ace17d5d883e
11
12 # Eliminamos, opción 2
13 docker rmi alpine
14
15 # Eliminamos, opción 3
16 docker rmi ace1
```

También se pueden buscar imágenes desde consola.

```
1 docker search ubuntu
2
3 NAME                DESCRIPTION                STARS    OFFICIAL
4 ubuntu              Ubuntu is a Debian-based Linux operating sys... 16888    [OK]
5 websphere-liberty   WebSphere Liberty multi-architecture images ... 298      [OK]
6 open-liberty        Open Liberty multi-architecture images based... 64       [OK]
7 neurodebian         NeuroDebian provides neuroscience research s... 106      [OK]
8 ubuntu-debootstrap  DEPRECATED; use "ubuntu" instead 52       [OK]
9 ubuntu-upstart      DEPRECATED, as is Upstart (find other proces... 115      [OK]
10 ubuntu/nginx        Nginx, a high-performance reverse proxy & we... 112
11 ubuntu/squid        Squid is a caching proxy for the Web. Long-t... 83
12 ubuntu/cortex       Cortex provides storage for Prometheus. Long... 4
13 ubuntu/prometheus   Prometheus is a systems and service monitori... 56
14 ubuntu/apache2      Apache, a secure & extensible open-source HT... 70
15 ...
```

9. Docker Compose: varios servicios a la vez (RA7-i)

Compose orquesta **múltiples contenedores** (p. ej. la aplicación + una base de datos) mediante un fichero `compose.yaml`.

```

1 services:
2   jupyter:
3     image: jupyter/scipy-notebook
4     ports:
5       - "8888:8888"
6     volumes:
7       - ./practicas:/home/jovyan/work
8     environment:
9       - JUPYTER_TOKEN=cursoia

```

Comando	Qué hace
<code>docker compose up -d</code>	Crea y arranca los servicios en segundo plano
<code>docker compose down</code>	Detiene y elimina contenedores (y la red por defecto)
<code>docker compose down -v</code>	Además borra los volúmenes nombrados (¡borra datos!)
<code>docker compose config</code>	Muestra la configuración resuelta sin arrancar nada
<code>docker compose logs -f</code>	Sigue los logs de los servicios
<code>docker compose exec</code>	Ejecuta un comando dentro de un contenedor en marcha

Para ordenar el arranque se usa `depends_on` (con `condition: service_healthy` y un `healthcheck` para esperar a que un servicio esté listo).

⚠️ `down -v` borra datos

`docker compose down -v` elimina también los **volúmenes nombrados**. Úsalo solo cuando quieras empezar de cero; tus notebooks se perderían.

10. El contenedor de prácticas de IA (RA7-i)

El entorno del curso se levantará con la imagen `jupyter/scipy-notebook`, que ya incluye `numpy`, `scipy`, `scikit-learn`, `pandas`, `matplotlib` y otras bibliotecas científicas.

🔥 ¿Por qué `python:3.12-slim` y no Alpine?

La variante `-slim` usa **Debian (glibc)**; la `-alpine` usa **musl libc**. Bibliotecas como `scikit-learn` y `torch` **solo publican wheels para glibc**: en Alpine habría que compilar desde fuente (`torch` puede tardar 30-60 min). Con `-slim` todo el stack se instala con wheels precompilados. Los Jupyter Docker Stacks científicos también se construyen sobre Ubuntu (glibc).

El contenedor de prácticas:

- Publica el puerto **8888** (Jupyter).
- Monta tu carpeta de prácticas en `/home/jovyan/work` (bind mount) para que los notebooks sean ficheros reales de tu equipo.
- Fija el token con `JUPYTER_TOKEN` para que la URL sea determinista.

```

1 docker compose up -d
2 # abre http://localhost:8888 y usa el token cursoia
3 docker compose exec jupyter bash # para entrar dentro
4 docker compose down             # cuando termines

```

Lo pondrás en marcha paso a paso en el **Taller 2** de esta unidad.

10.1 Construir la imagen con un Dockerfile propio

Otra forma de levantar un contenedor con `docker-compose` es utilizar un `Dockerfile` para generar la imagen (en lugar de usar una de DockerHub)

Necesitamos una carpeta con la siguiente estructura:

```

1  .
2  ├── docker-compose.yml
3  ├── Dockerfile
4  └── notebooks
5      └── rockpaperscissors.ipynb

```

El fichero `Dockerfile` tiene el siguiente contenido:

```

1  #versión de python compatible con experta
2  FROM python:3.6
3  #librería de python para sistemas expertos
4  RUN pip3 install experta
5  #librería para usar notebooks en python
6  RUN pip3 install notebook
7  #creamos la carpeta de trabajo
8  WORKDIR /home/MIA2526
9  #ejecutamos el jupyter notebook en el puerto 8888
10 CMD jupyter notebook --allow-root --ip=0.0.0.0 --port=8888 --no-browser

```

Ahora, para el `docker-compose.yml` tendremos:

```

1  services:
2    experta:
3      container_name: experta #bautizamos a nuestro contenedor
4      build: . #con esta línea le indicamos que busque el Dockerfile, en lugar de buscar una imagen en un repositorio
5      ports:
6        - "8888:8888" #publicamos el puerto para que sea accesible desde fuera
7      volumes:
8        - ./notebooks:/home/MIA2526 #aquí asignamos la carpeta local notebooks con la carpeta de trabajo del contenedor

```

Y por último necesitamos el notebook para hacer pruebas, en nuestro caso es un juego de piedra, papel o tijeras:

[rockpaperscissors.ipynb](#) este fichero debemos guardarlo en la carpeta `notebooks`

Ahora con toda la estructura lista y desde la raíz de la carpeta (donde están el `docker-compose.yml` y el `Dockerfile`) ejecutamos:

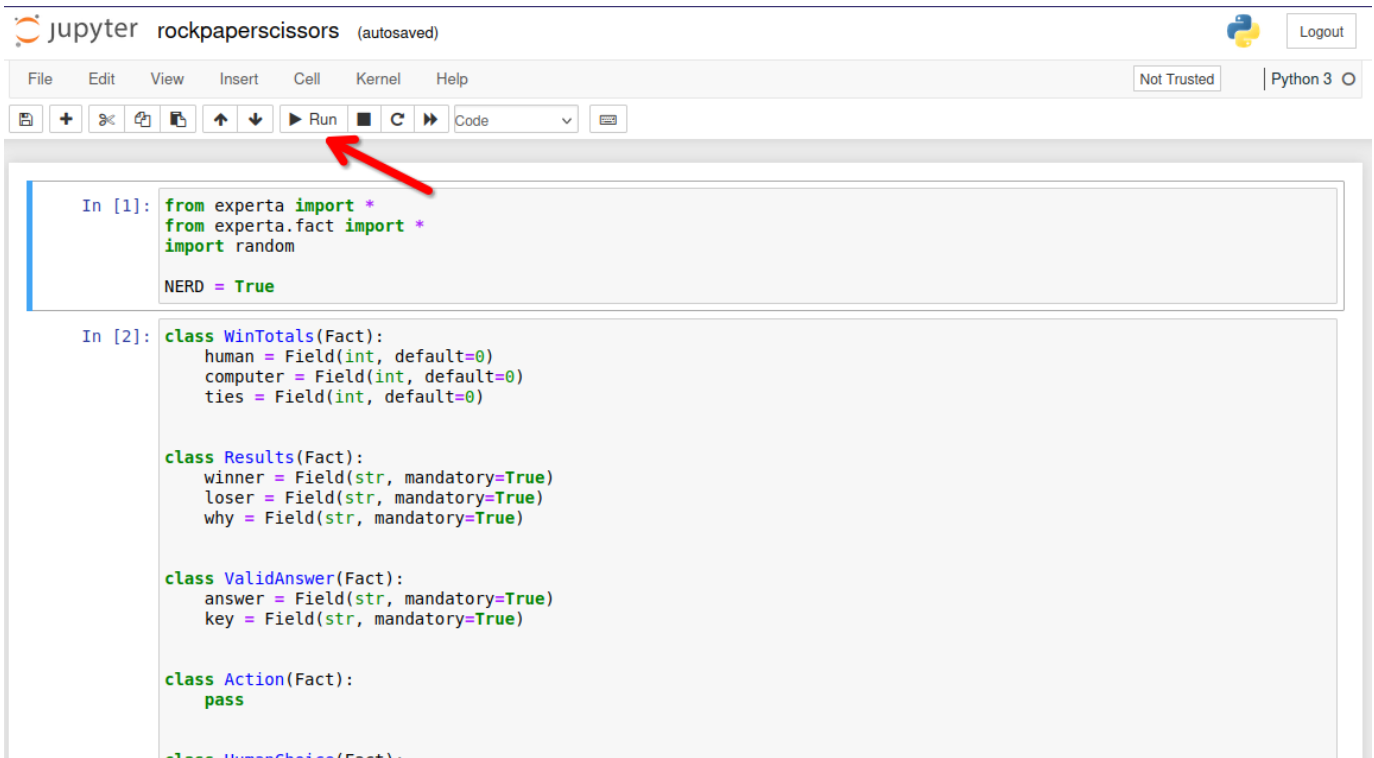
```

1  $ docker-compose up
2  [+] Running 1/1
3  ✓ Container experta Recreated                                0.1s
4  Attaching to experta
5  experta | [I 16:30:08.568 NotebookApp] Writing notebook server cookie secret to /root/.local/share/jupyter/runtime/notebook_cookie_secret
6  experta | [I 16:30:08.784 NotebookApp] Serving notebooks from local directory: /home/MIA2324
7  experta | [I 16:30:08.784 NotebookApp] Jupyter Notebook 6.4.0 is running at:
8  experta | [I 16:30:08.784 NotebookApp] http://e0ec65acd84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
9  experta | [I 16:30:08.784 NotebookApp] or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
10 experta | [I 16:30:08.784 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
11 experta | [C 16:30:08.787 NotebookApp]
12 experta |
13 experta | To access the notebook, open this file in a browser:
14 experta | file:///root/.local/share/jupyter/runtime/nbserver-7-open.html
15 experta | Or copy and paste one of these URLs:
16 experta | http://e0ec65acd84:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b
17 experta | or http://127.0.0.1:8888/?token=825b1ba76502787821bc045496e3429bca02ba09720a835b

```

Ahora podemos hacer click directamente sobre el enlace a <http://127.0.0.1:8888/?token=beb660273e8e168c7fec90978ed56fb50db6b08d915cb14> donde veremos nuestro jupyter notebook:

Si hacemos click sobre el notebook `rockpaperscissors.ipynb` podremos ver:



JupyterLab interface showing two code cells. A red arrow points to the 'Run' button in the toolbar.

```
In [1]: from experta import *
        from experta.fact import *
        import random

        NERD = True
```

```
In [2]: class WinTotals(Fact):
        human = Field(int, default=0)
        computer = Field(int, default=0)
        ties = Field(int, default=0)

        class Results(Fact):
            winner = Field(str, mandatory=True)
            loser = Field(str, mandatory=True)
            why = Field(str, mandatory=True)

        class ValidAnswer(Fact):
            answer = Field(str, mandatory=True)
            key = Field(str, mandatory=True)

        class Action(Fact):
            pass

        class HumanChoice(Fact):
```

Después de pulsar varias veces (para ir ejecutando todas las celdas) podremos ver como evoluciona nuestro juego:

```
In [*]: rps.reset()
        rps.run()
```

Lets play a game!
 You choose rock, paper, or scissors,
 and I'll do the same.
 Scissors (S), Paper (P), Rock (R)? R
 Computer wins! Paper covers rock
 Play again?Y
 Scissors (S), Paper (P), Rock (R)? S
 Tie! Ha-ha!
 Play again?Y
 Scissors (S), Paper (P), Rock (R)? P
 You win! Paper covers rock
 Play again?

Para detener el contenedor que hemos lanzado con `docker-compose`, solo hemos de pulsar `^ Ctrl + C`

Actualizado respecto al material antiguo

El `Dockerfile` de partida fijaba `python:3.6`, fuera de soporte desde 2021. La versión del curso usa `python:3.12-slim` e instala las dependencias desde `requirements-ia.txt`. Los ficheros listos para usar están en `entorno/`, junto a esta unidad. Con Python 3.12, `experta` necesita el parche del apartado 6.5.

11. Copias de seguridad de imágenes y contenedores (RA7-i)

Copias de contenedores

Ya estén encendidos o apagados, podemos realizar respaldos de seguridad de los contenedores. Utilizando la opción “*export*” empaquetará el contenido, generando un fichero con extensión “*.tar*” de la siguiente manera:

```
1 docker export -o fichero-resultante.tar nombre-contenedor
```

O

```
1 docker export nombre-contenedor > fichero-resultante.tar
```

Restauración de copias de seguridad de contenedores

Hay que tener en cuenta, antes de nada, que no es posible restaurar el contenedor directamente, de forma automática. En cambio, sí podemos crear una imagen, a partir de un respaldo de un contenedor, mediante el parámetro “*import*” de la siguiente manera:

```
1 docker import fichero-backup.tar nombre-nueva-imagen
```

Copias de imágenes

Aunque no tiene mucho sentido por que se bajan muy rápido, también tenemos la posibilidad de realizar copias de seguridad de imágenes. El proceso se realiza al utilizar el parámetro ‘*save*’, que empaquetará el contenido y generará un fichero con extensión “*.tar*”, así:

```
1 docker save nombre_imagen > imagen.tar
```

O

```
1 docker save -o imagen.tar nombre_imagen
```

Restaurar copias de seguridad de imágenes

Con el parámetro ‘*load*’, podemos restaurar copias de seguridad en formato ‘*.tar*’ y de esta manera recuperar la imagen.

```
1 docker load -i fichero.tar
```

12. Puntos clave de la unidad

- El curso de especialización IA y Big Data dura **600 horas / 36 ECTS** y el módulo 5071 se imparte en **90 horas** en la Comunitat Valenciana.
- El módulo 5071 desarrolla **6 RA** más el **RA7 de proyecto integrador transversal** (común a todo el curso, semanas 25-29, nota consensuada).
- **La normativa** exige alcanzar **todos los RA** del módulo; el centro lo concreta en **cada RA ≥ 5** . La nota de cada RA combina **40 % tareas + 60 % prueba escrita**; se pierde la evaluación continua superando el **15 % de inasistencia**.
- El entorno de trabajo usa **Aules** (Moodle), la web del curso, **Python + Jupyter** y **Docker**.
- Un **contenedor** es una instancia ejecutable de una **imagen**; Docker aporta aislamiento por procesos, ligereza y reproducibilidad frente a las máquinas virtuales.
- Los contenedores son **efímeros**: los datos se conservan con **volúmenes** o **bind mounts**.
- Un **Dockerfile** define la imagen por capas; el **orden de las instrucciones** aprovecha la caché.
- **Compose** orquesta varios servicios con un solo fichero.

13. Glosario

Término	Definición
Contenedor	Instancia ejecutable de una imagen con su propia capa de escritura
Imagen	Plantilla de solo lectura que define el entorno (SO base + app + dependencias)
Capa	Cada instrucción del Dockerfile; las capas se reutilizan en caché
Dockerfile	Fichero que describe cómo construir una imagen
Registro	Repositorio de imágenes (p. ej. Docker Hub)
Daemon (dockerd)	Proceso de Docker que hace el trabajo pesado
CLI (docker)	Cliente de línea de comandos que envía órdenes al daemon
Bind mount	Montaje de una carpeta real del host dentro del contenedor
Volumen	Almacenamiento gestionado por Docker para persistir datos
Compose	Herramienta para orquestar varios contenedores con un fichero YAML
Servicio	Un contenedor definido dentro de un <code>compose.yaml</code>
Healthcheck	Comprobación del estado de un servicio para ordenar el arranque
RA (resultado de aprendizaje)	Capacidad que el alumnado debe demostrar al terminar un módulo
CE (criterio de evaluación)	Evidencia observable que permite comprobar un RA
FE (formación en empresa)	Fase en empresa del alumnado (opcional en este curso, según centro)

14. FAQ

? ¿Docker es una máquina virtual? ▾

No. Las VMs virtualizan **hardware** (cada una con su kernel); los contenedores virtualizan el **sistema operativo** (comparten el kernel del host) y aíslan por procesos. Por eso son más ligeros y arrancan en segundos.

? ¿Pierdo mis datos si elimino un contenedor? ▾

Sí, a menos que los guardes fuera: en un **volumen** o en un **bind mount**. Por eso en las prácticas montamos tu carpeta de trabajo dentro del contenedor.

? ¿Por qué `EXPOSE` no me deja abrir el navegador? ▾

Porque `EXPOSE` solo **documenta** el puerto. Para que sea accesible hay que **publicarlo** con `-p` (o `ports:` en Compose).

? ¿Qué pasa si `docker run` me pide descargar una imagen muy grande? ▾

Solo ocurre la **primera vez**. Después, Docker reutiliza la imagen desde la caché local. Si la imagen es demasiado grande, se puede empezar con variantes más ligeras (`-slim`) y anclar la versión.

? ¿El RA7 cuenta para la nota del módulo 5071? ▾

Sí. El RA7 aporta el **20 %** de la nota final del módulo (RA1-RA6 el 80 %). Además, como es transversal, se evalúa de forma **consensuada** por todo el equipo docente del curso.

? ¿Puedo instalar Docker en cualquier equipo? ▾

Docker funciona en Linux, Windows y macOS. En este curso lo usaremos en el aula y para los talleres; si tu equipo no lo soporta, el entorno gestionado del aula será el plan B.

15. Sesiones

La unidad son **6 h en dos semanas** (1-8 de octubre), a 3 h por semana.

Semana	Fechas	Horas	Contenido	Evidencia
1	1-2 oct	3	Presentación del curso, del módulo y de la evaluación · instalación de Docker · conceptos: imagen, contenedor, registro	Docker funcionando (<code>docker run hello-world</code>)
2	5-8 oct	3	<code>docker run</code> y volúmenes · Dockerfile y Compose · contenedor de prácticas de IA	Taller 1 (hecho / no hecho) y entorno del curso levantado

✎ Si el horario del módulo son 2 h + 1 h

Cada semana se parte en dos sesiones. El **bloque de 2 h** es el único que admite trabajo con contenedores (descargas, construcción de imágenes, resolución de errores); la sesión de **1 h** se dedica a los conceptos, al repaso y a cerrar el taller. La primera semana es corta: el curso empieza el jueves 1 de octubre.

A. Anexo · chuleta de comandos

Referencia rápida para consultar en clase, no para memorizar.

GESTIÓN DE IMAGENES

```
1 docker image
2 docker history
3 docker inspect
4 docker save/load
5 docker rmi
```

GESTIÓN DE CONTENEDORES

```
1 docker attach
2 docker exec
3 docker inspect
4 docker kill
5 docker logs
6 docker pause/unpause
7 docker port
8 docker ps
9 docker rename
10 docker start/stop/restart
11 docker rm
12 docker run
13 docker stats
14 docker top
15 docker update
```

EJEMPLO

```

1 # Ver los contenedores que tenemos
2 docker ps
3 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
4
5 # Ver las imagenes que tenemos
6 docker images
7 REPOSITORY          TAG   IMAGE ID   CREATED   SIZE
8
9 # Crear un contenedor con una imagen básica de debian
10 # Como no tenemos ninguna imagen de debian, la descarga y la ejecuta
11 docker run debian
12 # Intentamos ver el contenedor en ejecución, no aparece nada porque ya se ha cerrado
13 docker ps
14 # Podemos verlo con
15 docker ps -a
16 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
17 09b14daab800   debian    "bash"    2 seconds ago    Exited (0) 1 second ago    pensive_wozniak
18
19 # Ejecutar un comando en un contenedor
20 docker run debian /bin/echo "Hello World"
21 Hello World
22
23 # Información
24 docker inspect debian

```

Crear contenedor interactivo y con nombre

<https://jolphgs.wordpress.com/2019/09/25/create-a-debian-container-in-docker-for-development/>

Para que docker no se invente un nombre como “pensive_wozniak” (comando anterior) podemos definir el nombre que queremos.

Utilizaremos una de las imágenes de: https://hub.docker.com/_/debian/tags

```

1 # Obtenemos la imagen, en el apartado anterior la hemos ejecutado directamente con "run", esto
2 # la obtiene implícitamente. En este caso la vamos a descargar.
3 $ docker pull debian:13-slim
4
5 # --name
6 # -h hostname que tendrá el contenedor
7 # -e codificación de caracteres
8 # -it modo interactivo
9 # /bin/bash -l la shell que se ejecutará
10 $ docker run --name debian-mini -h equipo1 -e LANG=C.UTF-8 -it debian:13-slim /bin/bash -l
11
12 --- Estamos dentro del contenedor ---
13 # Una vez dentro del contenedor podemos actualizarlo e instalar los paquetes que creamos necesarios
14 apt update && apt upgrade --yes && apt install sudo locales --yes
15 # Configurar timezone
16 dpkg-reconfigure tzdata
17
18 # Vamos a nuestra home y creamos un archivo
19 cd
20 echo "hola" > prueba.txt
21
22 # Salimos del contenedor
23 exit (o control + d)
24
25 --- Ahora volvemos a la consola de nuestro equipo ---
26 $ docker ps
27 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
28
29 $ docker ps -a
30 CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
31 8c6b29c818ee   debian:13-slim    "/bin/bash -l"    44 seconds ago    Exited (0) 11 seconds ago    debian-mini

```

B. Anexo · catálogo de contenedores para experimentar

MONITORIZACIÓN Y GESTIÓN

- **Netdata** - Monitorización de sistemas en tiempo real con dashboard web completo
- **Glances** - Monitorización del sistema via web que muestra información de CPU, memoria, red y procesos
- **Portainer** - Interfaz web para gestionar y administrar contenedores Docker
- **Uptime Kuma** - Monitor de disponibilidad para verificar el estado de contenedores y servicios

PROXY Y RED

- **Nginx Proxy Manager** - Proxy inverso con interfaz web para gestionar hosts virtuales y certificados SSL/TLS
- **Acestream** - Motor P2P para streaming de video, útil para integrar canales en Jellyfin

- **Libreddit** - Interfaz alternativa para Reddit libre de publicidad y tracking (modo lectura)
- **Invidious** - Interfaz alternativa para YouTube que respeta la privacidad

MULTIMEDIA Y ENTRETENIMIENTO

- **Jellyfin** - Servidor multimedia libre para organizar y streaming de películas, series y música
- **Soulseek** - Cliente para la red P2P Soulseek especializada en compartir música
- **youtube-dl** - Herramienta para descargar videos y audio de YouTube y otros sitios
- **Photoprism** - Servicio de gestión y visualización de fotos personales con funciones de IA

PRODUCTIVIDAD Y ORGANIZACIÓN

- **Nextcloud** - Plataforma de colaboración y almacenamiento en la nube auto-alojado
- **Linkding** - Marcador social para guardar y organizar enlaces web
- **GitBucket** - Plataforma Git auto-alojada similar a GitHub
- **SearXNG** - Metabuscador privado que agrega resultados de múltiples motores de búsqueda

SEGURIDAD Y CONTRASEÑAS

- **Vaultwarden** - Implementación alternativa del gestor de contraseñas Bitwarden, más ligera y eficiente

AUTOMATIZACIÓN DEL HOGAR

- **Home Assistant** - Plataforma de domótica de código abierto para automatizar y controlar dispositivos del hogar
- **Homebridge** - Puente que permite integrar dispositivos no compatibles con el ecosistema Apple HomeKit
- **Camera.UI** - Aplicación para gestionar y visualizar cámaras de seguridad con integración HomeKit

UTILIDADES Y MANTENIMIENTO

- **Homepage** - Dashboard personalizable como página de inicio para acceder a todos los servicios
- **Watchtower** - Servicio que actualiza automáticamente los contenedores Docker cuando hay nuevas versiones disponibles



Uso de este catálogo

Sirve para elegir imagen en la fase 4 del Taller 1. Antes de proponer una, comprueba en Docker Hub que sigue mantenida: hay imágenes populares con años sin actualizar.

16. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres:
 - [T01 · Verificación del entorno y primer contenedor](#) — entregable, se marca **hecho / no hecho**
 - [T02 · Contenedor de prácticas de IA](#) — entregable, se marca **hecho / no hecho**
- **Notebooks** — el N01 de la unidad, entregable que se marca **hecho / no hecho**; descarga y apertura en Colab desde el propio notebook, en el menú «Notebooks»

**Referencias de la unidad** ▾

Documentación oficial de Docker:

- [What is Docker?](#)
- [What is a container?](#)
- [Dockerfile reference](#)
- [Compose Quickstart](#)
- [Building best practices](#)

Normativa y plataforma:

- [RD 279/2021](#) — currículo del curso de especialización
- [Orden 8/2025 de la Comunitat Valenciana](#) — evaluación
- [Aules GVA](#) — plataforma del centro

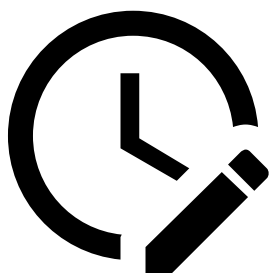
17. Evaluación

- **Tarea de la unidad** (40 % de la nota del RA al que se asocia): entregar el informe de los talleres 1 y 2 en Moodle.
- **Evaluación inicial:** diagnóstica, sin nota, antes del segundo mes lectivo.
- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

18. Recuperación

Si no superas la tarea de esta unidad, se activará un **programa de recuperación individual** (art. 14.4 Orden 8/2025) con actividades y criterios de evaluación específicos para el RA correspondiente.

[Volver al índice](#)



23 de agosto de 2026

2.2 Diapositivas de la UD00

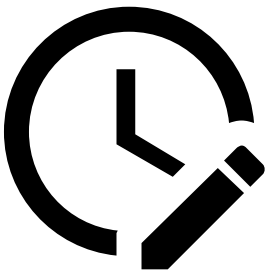


Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.



22 de agosto de 2026

2.3 UD00 — Ejercicios



Cómo se trabajan

Resuélvelos en tu cuaderno o en un documento Markdown, a tu ritmo. Si te atascas en alguno, pregunta en clase o por Moodle: el profesor te da la solución. (El **notebook** de la unidad es un entregable aparte, con su propia entrega — ver [Recursos](#).)

A. Sobre el módulo y la evaluación

1. Cita los **7 RA** del módulo y asocia cada uno con su número.
2. ¿Cuántas horas tiene el módulo? ¿Cuántas semanas y a qué ritmo semanal?
3. Explica cómo se calcula la nota final del módulo (pesos de RA1-RA6 y RA7).
4. ¿Es el RA7 un caso especial a la hora de superar el módulo? Razona tu respuesta.
5. ¿Por qué el contenido del módulo debe **finalizar a finales de abril**?
6. Indica cuándo se hacen las **evaluaciones parciales** y qué RAs cubren cada una.
7. ¿Qué herramientas usaremos como entorno de trabajo durante el curso? Describe brevemente cada una.

B. Conceptos de Docker

1. Diferencia **imagen** y **contenedor** con un ejemplo.
2. Explica el papel del **daemon**, del **cliente** y de los **registros** en la arquitectura de Docker.
3. Enumera dos ventajas de los contenedores frente a las máquinas virtuales y una situación en la que convenga usar ambas.
4. ¿Qué hace exactamente `docker run` cuando la imagen aún no está descargada? Describe los pasos.
5. ¿Qué ocurre con los datos de un contenedor cuando lo eliminas? ¿Cómo los conservas?
6. ¿Cuál es la diferencia entre `docker stop`, `docker rm` y `docker rmi`?
7. Explica con tus palabras qué ocurre al ejecutar `docker run hello-world` la primera vez.

C. Dockerfile

1. ¿Por qué un Dockerfile debe empezar por `FROM`?
2. Diferencia `RUN`, `CMD` y `ENTRYPOINT`.
3. Escribe un `Dockerfile` que parta de `python:3.12-slim`, copie `requirements.txt` y `app.py`, instale las dependencias y ejecute la app.
4. ¿Por qué se recomienda copiar `requirements.txt` antes que el resto del código? ¿Qué relación tiene con las capas?
5. ¿Qué es un `.dockerignore` y para qué sirve?
6. Dado el `Dockerfile` anterior, escribe los comandos para **construir** la imagen y para **ejecutarla** publicando el puerto 8000.

D. Docker Compose

1. ¿Qué ventaja aporta `docker-compose.yml` frente a varios `docker run`?
2. Escribe un `compose.yaml` con un servicio `jupyter` que publique el puerto 8888, monte la carpeta `./practicas` y use el token `cursoia`.
3. Explica la diferencia entre `docker compose up -d`, `docker compose down` y `docker compose down -v`.
4. ¿Para qué sirven los comandos `docker compose config`, `docker compose logs -f` y `docker compose exec`?
5. ¿Cómo se ordena el arranque de servicios en Compose? ¿Qué papel juegan los **healthchecks**?
6. ¿Cómo se conservan los datos con Compose? ¿Qué hace la clave `volumes` de nivel superior?

E. Prácticos

1. Levanta un contenedor `python:3.12` interactivo y escribe un programa que imprima `Hola desde Docker`. ¿Qué comando usas para entrar y para salir?

2. Levanta `nginx` en segundo plano en el puerto `8080` y comprueba en tu navegador que responde. ¿Cómo lo detienes y lo eliminas?
3. Crea una carpeta `practicas`, monta esa carpeta en un contenedor `python:3.12` y ejecuta desde dentro un archivo `app.py` que hayas escrito en el host.
4. Escribe el `docker-compose.yml` del taller 2, levanta Jupyter y abre la URL en el navegador.

F. Instalación, versiones y mantenimiento

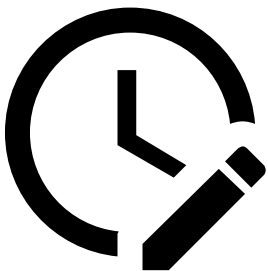
1. Tras ejecutar `sudo usermod -aG docker $USER`, ¿por qué hay que **cerrar y volver a abrir la sesión**? ¿Qué mensaje de error aparece si no lo haces?
2. El mismo `pip install experta` funciona en `python:3.9-slim` y falla al **importar** en `python:3.12-slim`. Explica la causa, qué hace el parche de tres líneas y por qué el problema no aparece al instalar sino al importar.
3. Diferencia `docker save/docker load` de `docker export/docker import`. ¿Cuál conserva el historial de capas de la imagen y por qué importa?
4. Tienes 12 GB ocupados en imágenes. Explica qué hace `docker image prune -a`, en qué se diferencia de borrar por identificador y qué riesgo tiene ejecutarlo sin mirar.

Soluciones

Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD00](#) · [Taller 1](#) · [Taller 2](#)



21 de agosto de 2026

2.4 Talleres

UD00 · Taller 1 — Verificación del entorno y primer contenedor



Unidad no calificable, pero requisito

El Taller 1 se entrega y se marca como **hecho / no hecho**: no lleva nota, pero se tiene en cuenta y **es requisito para las prácticas de la UD02** (sin entorno funcionando no se puede hacer Robocode). Los viernes no hay clase: son el momento de recopilar capturas y redactar la memoria.

Objetivo: dejar Docker funcionando en **tu** máquina y demostrarlo con evidencias.

Entrega: una memoria en PDF con las capturas y las explicaciones de las seis fases. Se marca como **hecho / no hecho**.

FASE 1 — INSTALA DOCKER Y DEMUÉSTRALO

1. Instala Docker en tu equipo siguiendo el apartado 5.2 de la teoría (Ubuntu o Docker Desktop, según tu sistema).
2. Ejecuta y captura la salida de:

```
1 docker --version
2 docker run hello-world
```



El error más común del primer día

Si aparece `permission denied while trying to connect to the Docker daemon socket`, te falta añadir tu usuario al grupo `docker` y **cerrar y volver a abrir la sesión**. Explica en la memoria por qué hace falta reiniciar la sesión.

FASE 2 — PRIMER CONTENEDOR INTERACTIVO

```
1 docker run -it --name mi-python python:3.12 bash
```

Dentro del contenedor:

```
1 python --version
2 python -c "print('Hola desde Docker!')"
3 exit
```

Comprueba que el contenedor **sigue existiendo** aunque esté parado, y vuelve a entrar:

```
1 docker ps -a
2 docker start mi-python
3 docker attach mi-python
```

En la memoria: explica la diferencia entre `docker run`, `docker start` y `docker attach`.

FASE 3 — UN SERVICIO EN SEGUNDO PLANO

```
1 docker run -d --name mi-nginx -p 8080:80 nginx
2 docker ps
```

Abre `http://localhost:8080` y captura la página de bienvenida. Después inventaría lo que llevas acumulado:

```
1 docker images      # ¿cuántas imágenes y de qué tamaño?
2 docker ps -a       # ¿cuántos contenedores y en qué estado?
```

FASE 4 — EL MISMO CÓDIGO EN DOS VERSIONES DE PYTHON

Esta fase reproduce el caso del apartado 6.5 con tus propias manos:

```
1 # Contenedor A
2 docker run --rm python:3.9-slim bash -c \
3   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(\"OK\")'"
4
5 # Contenedor B
6 docker run --rm python:3.12-slim bash -c \
7   "pip install -q experta && python -c 'from experta import KnowledgeEngine; print(\"OK\")'"
```

Captura **las dos salidas** y responde en la memoria:

- 1. ¿Qué error exacto da el contenedor B y por qué?
- 2. ¿Qué versión de `experta` se ha instalado en cada caso? ¿Y de `frozendict`?
- 3. Si tuvieras que entregar hoy un proyecto con `experta`, ¿qué imagen base elegirías y por qué?

FASE 5 — ELIGE UNA IMAGEN Y ESCRIBE SU COMPOSE

- 1. Elige una imagen del **anexo B** de la teoría (o cualquiera de [Docker Hub](#)).
- 2. Comprueba que está **mantenida**: mira la fecha de la última actualización.
- 3. Escribe su `docker-compose.yml` con puertos y, si lo necesita, un volumen para sus datos.
- 4. Levántalo con `docker compose up -d` y captura el servicio funcionando en el navegador.

FASE 6 — VOLÚMENES Y LIMPIEZA

Crea en tu equipo una carpeta `practicas` con un `app.py`:

```
1 print("Mi primera app en un contenedor")
```

Ejecútala **sin instalar Python en el host**:

```
1 cd practicas
2 docker run --rm -v "$PWD":/app -w /app python:3.12 python app.py
```

Y deja la máquina limpia:

```
1 docker compose down      # si dejaste el servicio de la fase 5
2 docker stop mi-nginx
3 docker rm mi-nginx mi-python
4 docker ps -a             # ya no deben aparecer
```

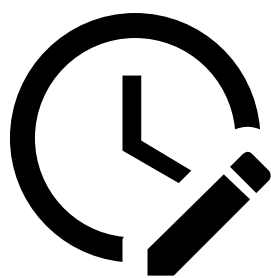
ENTREGA DEL TALLER 1

Recopila las capturas y las explicaciones de las seis fases en **una memoria en PDF**. Se valora que expliques lo que ocurre, no solo que pegues pantallazos: una captura sin explicación no demuestra que hayas entendido nada.

Fase	Evidencia mínima
1	<code>docker run hello-world</code> correcto
2	Contenedor interactivo, con la versión de Python impresa desde dentro
3	nginx respondiendo en el navegador + inventario de imágenes y contenedores
4	Las dos salidas del caso <code>experta</code> y el error del contenedor B
5	<code>docker-compose.yml</code> propio y su servicio funcionando
6	La app del host ejecutada en el contenedor y la máquina limpia

 **Soluciones**

Las soluciones no se publican: se corrigen y comentan en clase.



21 de agosto de 2026

UD00 · Taller 2 — Contenedor de prácticas de IA

**Entregable · se marca hecho / no hecho**

Se trabaja en clase, en el bloque largo de la semana 2, y **se entrega en Moodle**. No lleva nota, pero **es requisito**: al terminar tendrás el **entorno del curso levantado**, que es el que usarás en el resto de las unidades. Sin él no se pueden hacer las prácticas de la UD02.

Objetivo: levantar un entorno **Jupyter** reproducible con las bibliotecas del curso.

FASE 1 — ESTRUCTURA DE TRABAJO

```
1 practicas/
2   └─ docker/
3       └─ docker-compose.yml
4       └─ requirements-ia.txt
```

FASE 2 — requirements-ia.txt

```
1 numpy
2 pandas
3 matplotlib
4 scikit-learn
5 scikit-fuzzy
6 nltk
7 spacy
8 torch
9 experta
```

FASE 3 — docker-compose.yml

```
1 services:
2   jupyter:
3     image: jupyter/scipy-notebook
4     ports:
5       - "8888:8888"
6     volumes:
7       - ./practicas:/home/jovyan/work
8     environment:
9       - JUPYTER_TOKEN=cursoia
```

FASE 4 — VERIFICA LA CONFIGURACIÓN ANTES DE ARRANCAR

```
1 docker compose config
```

Comprueba que el puerto `8888:8888` y el token quedan resueltos correctamente.

FASE 5 — ARRANCA JUPYTER

```
1 docker compose up -d
2 docker compose ps
```

Abre en el navegador `http://localhost:8888` y usa el token `cursoia`.

Dentro de Jupyter, crea un notebook en la carpeta `work` y verifica las bibliotecas:

```
1 import numpy, pandas, matplotlib, sklearn, skfuzzy
2 import nltk, spacy, torch
3 print("Entorno de IA listo:", numpy.__version__)
```

**Nota para spacy**

Algunos modelos de `spacy` se descargan aparte (`python -m spacy download es_core_news_sm`). Lo veremos en la UD03.

FASE 6 — GESTIONA EL SERVICIO

```

1 docker compose logs -f jupyter      # sigue los logs (Ctrl+C para salir)
2 docker compose exec jupyter bash    # entra dentro del contenedor en marcha
3 docker compose down                 # detiene y elimina

```

FASE 7 — PERSISTENCIA (VOLÚMENES NOMBRADOS)

Modifica el `docker-compose.yml` para que la carpeta de Jupyter use un **volumen nombrado** y comprueba que tus notebooks sobreviven a un `down` + `up`:

```

1 services:
2   jupyter:
3     image: jupyter/scipy-notebook
4     ports:
5       - "8888:8888"
6     volumes:
7       - jupyter-work:/home/jovyan/work
8     environment:
9       - JUPYTER_TOKEN=cursoia
10
11 volumes:
12   jupyter-work:

```

ENTREGA

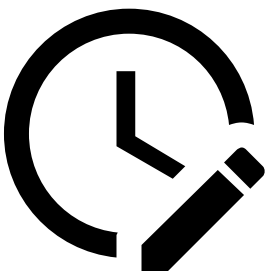
Sube a Moodle un **informe breve**, que se marca como **hecho / no hecho**, con:

1. Captura de la **fase 5 de este taller** (Jupyter abierto con el import correcto).
2. Salida de `docker compose ps` mostrando el servicio activo.
3. Respuesta a: *¿por qué este entorno es reproducible en cualquier equipo?*
4. Captura de la **fase 7** mostrando que un notebook sobrevive a `docker compose down`.

 Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD00](#) · [Ejercicios](#) · [Taller 1](#)



21 de agosto de 2026

3. UD01

3.1 Caracterización de sistemas y utilización de modelos de Inteligencia Artificial

Fundamentos de los Sistemas Inteligentes

Definición de Inteligencia Artificial (IA)

La Inteligencia Artificial es un campo de la informática y la ciencia de la computación que se enfoca en la creación de sistemas y programas capaces de realizar tareas que normalmente requieren inteligencia humana.

Una primera definición bastante común que podemos encontrar para la Inteligencia Artificial es:



Definición

"Habilidad para aprender y resolver problemas, llevada a cabo por una máquina o software"

En general, la mayoría de los expertos coinciden en que es la simulación de procesos de inteligencia humana por parte de máquinas, especialmente sistemas informáticos. Estos procesos incluyen:

1. El **aprendizaje** a través de la adquisición de información y reglas para el uso de la información.
2. El **razonamiento** usando las reglas para llegar a conclusiones aproximadas o definitivas.
3. La **autocorrección**.

Una definición más concreta y consensuada podría ser:



Definición

"La inteligencia artificial es la inteligencia llevada a cabo por máquinas. En ciencias de la computación, una máquina «inteligente» ideal es un agente flexible que percibe su entorno y lleva a cabo acciones que maximicen sus posibilidades de éxito en algún objetivo o tarea".

En realidad, cada generación de hardware y software ha asignado este término a las arquitecturas y técnicas de vanguardia en ese momento. Es por esto que la propia definición puede ir cambiando y evolucionando a medida que se van alcanzando metas más ambiciosas. Podríamos decir que cada nueva oleada de avance tecnológico en este ámbito pasa a conformar una nueva definición de inteligencia artificial, o, al menos, añade un matiz propio a ésta.

La realidad actual es que la IA despierta tanta fascinación como desconfianza. En la gran mayoría de los casos, no se conoce bien la técnica con la que se desarrolla. Ese desconocimiento es el que favorece que se mezcle la realidad con las influencias y fantasías de lo leído y visto en novelas, series y películas.

¿Qué es realmente posible con la tecnología actual y qué sigue perteneciendo al campo de la ciencia ficción? Una habilidad que debe tener cualquier profesional del campo de la IA es, precisamente, ser capaz de explicar de forma sencilla las verdaderas amenazas que esta tecnología puede representar, y saber transmitir una imagen de responsabilidad al respecto.



Recuerda

Los ordenadores (y con ellos la inteligencia artificial) no son ni buenos ni malos. Hacen lo que los humanos programamos que hagan. La inteligencia y, sobre todo, la intencionalidad que pueda tener un programa o aplicación la proporciona el humano (o equipo de humanos) que lo definen y desarrollan.

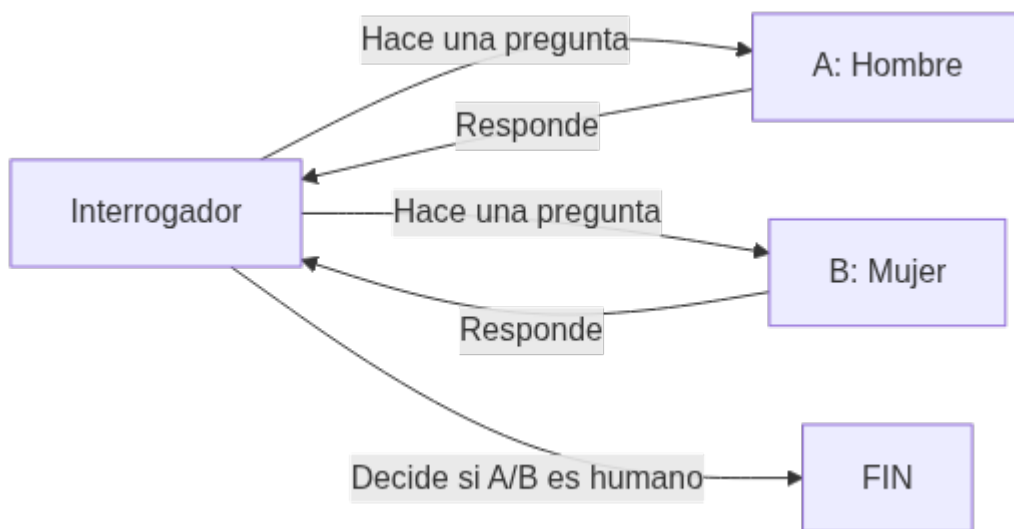
Historia de la IA

A lo largo de la historia, la IA ha pasado por diferentes etapas de desarrollo, con avances y desafíos significativos.

Prehistoria de la IA o proto-IA (Antes del 1950): En realidad, antes de 1956 ya hubo una serie de hitos científicos que podríamos considerar parte del nacimiento de la Inteligencia Artificial:

En 1943 McCulloch y Pitts presentaron un primer modelo de lo que podría ser una neurona artificial, publicándose en el Boletín de Biofísica Matemática con el título: "A logical calculus of the ideas immanent in nervous activity". Partieron de tres fuentes: conocimientos sobre la fisiología básica y funcionamiento de las neuronas en el cerebro, el análisis formal de la lógica proposicional de Russell y Whitehead y la teoría de la computación de Turing. Es por esto que se consideran los eventos más importantes en el origen de la IA (Russell, S., y Norvig, P. 2008)

En 1950, en el trabajo "Computing Machinery and Intelligence", Alan Turing define la conducta inteligente de la máquina como la capacidad de lograr eficiencia a nivel humano en todas las actividades de tipo cognoscitivo, suficiente para engañar a un evaluador humano, y da forma al famoso "Test de Turing". En este histórico artículo Turing propuso que la pregunta «¿puede pensar una máquina?» era demasiado filosófica para tener valor y, para hacerlo más concreto, propuso un «juego de imitación». En la prueba de Turing intervienen dos personas y una computadora. Una persona, el interrogador, se sienta en una sala y teclea preguntas en la terminal de una computadora. Cuando aparecen las respuestas en la terminal, el interrogador intenta determinar si fueron hechas por otra persona o por una computadora. Si la computadora actúa de manera inteligente, según Turing, es inteligente. Turing continúa, "Ahora podemos preguntar: '¿Qué sucederá cuando una máquina tome el papel de A en este juego?' ¿Decidirá el interrogador erróneamente tan a menudo cuando se juegue de esta manera como lo hace cuando el juego se juega entre un hombre y una mujer? Estas preguntas reemplazan nuestra pregunta original: '¿Pueden las máquinas pensar?' "(Turing, 1950) La configuración de la prueba puede representarse de esta manera:



Este test puede servir, como señala Turing, no solo para probar una destreza verbal superficial, sino también el conocimiento de fondo y la capacidad de razonamiento subyacente, ya que los interrogadores pueden hacer cualquier pregunta o plantear cualquier desafío verbal que elijan. Con respecto a este test, Turing predijo famosamente que "dentro de unos cincuenta años [para el año 2000] será posible programar computadoras... para hacer que jueguen el juego de imitación tan bien que un interrogador promedio tendrá no más del 70 por ciento de probabilidad de hacer la identificación correcta después de cinco minutos de interrogación" (Turing 1950); una predicción que ha fallado notoriamente. En el año 2000, las máquinas en la competición del Premio Loebner jugaron tan mal el juego que el interrogador promedio tuvo un 100 por ciento de probabilidad de hacer la identificación correcta después de cinco minutos de interrogación (ver Moor 2001).

Primeros Conceptos de IA (Décadas de 1950 y 1960): El término "Inteligencia Artificial" fue acuñado por John McCarthy en 1956, durante una conferencia (Dartmouth Summer Research Conference on Artificial Intelligence) que se considera el punto de inicio oficial del campo. En dicho encuentro, John McCarthy, Marvin Minsky, Nathaniel Rochester, Claude Shannon, Ray Solomonoff, Oliver Selfridge, Trenchard More, Arthur Samuel, Herbert Simon y Allen Newell, crearon la conjetura inicial "Every aspect of learning or any other feature of intelligence can be so precisely described that a machine can be made to simulate it". En esa época, los investigadores estaban entusiasmados por la idea de que las computadoras pudieran ser programadas para simular la capacidad de razonar y resolver problemas como lo hace el ser humano. Se realizaron esfuerzos iniciales para desarrollar programas que pudieran jugar al ajedrez, realizar cálculos matemáticos y comprender lenguaje natural. En esta conferencia se hicieron previsiones triunfalistas a diez años que jamás se cumplieron (como diríamos ahora "se vinieron muy arriba").

1956 Dartmouth Conference: The Founding Fathers of AI



John MacCarthy



Marvin Minsky



Claude Shannon



Ray Solomonoff



Alan Newell



Herbert Simon



Arthur Samuel



Oliver Selfridge



Nathaniel Rochester



Trenchard More

A partir de esta conferencia, los siguientes años, se dieron sucesivos éxitos y avances (teniendo en cuenta que entonces se contaba con computadores y herramientas de computación bastante rudimentarios) fruto de investigaciones y multitud de proyectos con grandes expectativas. Los más importantes fueron:

- Creación del LISP en 1958 por John McCarthy (será el lenguaje de programación predominante en posteriores desarrollos de Inteligencia Artificial).
- Desarrollo de Micromundos (mundo de bloques) en 1959 por Minsky y Papert en el MIT.
- Construcción del demostrador de Teoremas de Geometría en 1959 por Herbert Gelernter.
- Investigación de los "sistemas expertos" en 1965 por Santford.
- Lanzamiento de ELIZA en 1966 (será el primer chatbot que implementa lenguaje natural).

La Década del Estancamiento (1970): A finales de la década de 1960 y principios de la década de 1970, la IA experimentó una etapa de estancamiento conocida como el "invierno de la IA". Los avances prometidos no se materializaron, y las expectativas sobre lo que la IA podría lograr superaron las capacidades tecnológicas y computacionales de la época. Esto llevó a una disminución en la financiación y el interés en el campo.

El Renacimiento de la IA (Décadas de 1980 y 1990): En la década de 1980, la IA experimentó un resurgimiento significativo debido a avances en la teoría y la computación. Se desarrollaron nuevas técnicas de razonamiento, representación del conocimiento y búsqueda heurística. Los sistemas expertos, que utilizan reglas y conocimiento específico de dominio para resolver problemas, se convirtieron en una aplicación exitosa de la IA en campos como la medicina y la ingeniería. Se produjo una rivalidad entre Estados Unidos y Japón para ver quién investigaba y desarrollaba más aplicaciones de IA. El principal gran hito en esta etapa fue la creación de R1, el primer sistema experto comercial, en 1982 gracias a McDermott (en 4 años de implantación supuso un ahorro de 40 millones de dólares al año).

Aprendizaje Automático y el auge de la IA moderna (finales de los 90 y década de 2000 en adelante): El siglo XXI ha sido testigo de una revolución en la IA, impulsada en gran parte por el auge del Aprendizaje Automático. Con la disponibilidad masiva de datos y avances en algoritmos, las máquinas ahora pueden aprender patrones complejos a partir de datos y mejorar su rendimiento a través de la experiencia. Esto ha llevado a avances significativos en campos como la visión por computadora, el procesamiento del lenguaje natural y el reconocimiento de voz.

La consagración definitiva de la Inteligencia Artificial llegó a finales de los 90. Con los siguientes grandes hitos, que veremos con algo más de detalle:

- El programa **Deep Blue** desarrollado por **IBM** logró vencer en **1997** al campeón del mundo en ajedrez, **Gari Kaspárov**.



Deep Blue fue una "supercomputadora" que desarrolló la empresa IBM en los años 90 del S.XX para jugar al ajedrez.

En febrero de 1996 se enfrentaron el entonces campeón del mundo, Gary Kaspárov contra la máquina. La primera partida la ganó Deep Blue, otras tres las ganó Kaspárov y dos más quedaron en tablas. Fue la primera máquina que derrotó a un experto jugador.

En realidad, siendo fieles al concepto de Inteligencia Artificial, esta máquina no se puede considerar exactamente como tal, pues "solo" era capaz de calcular a gran velocidad millones de posiciones posibles por segundo, pero le faltaba "intuición". Es decir, contaba con una tremenda base de datos (posibles jugadas, movimientos, etc), pero sus programadores no fueron jugadores de ajedrez expertos, por lo que la máquina no siempre elegía la jugada óptima.

i Curiosidad

¿Sabes cuántas jugadas posibles hay en el ajedrez? Son cerca de 10^{120} (diez elevado a ciento veinte... 100000000000000000000... así hasta 120 ceros)

Para hacerte una idea más real de la tremenda capacidad de cálculo de Deep Blue, se estima que la cantidad de átomos que existen en el Universo es de entre 10^{80} y 10^{82}

El ajedrez es un juego complejo... ¿no crees?

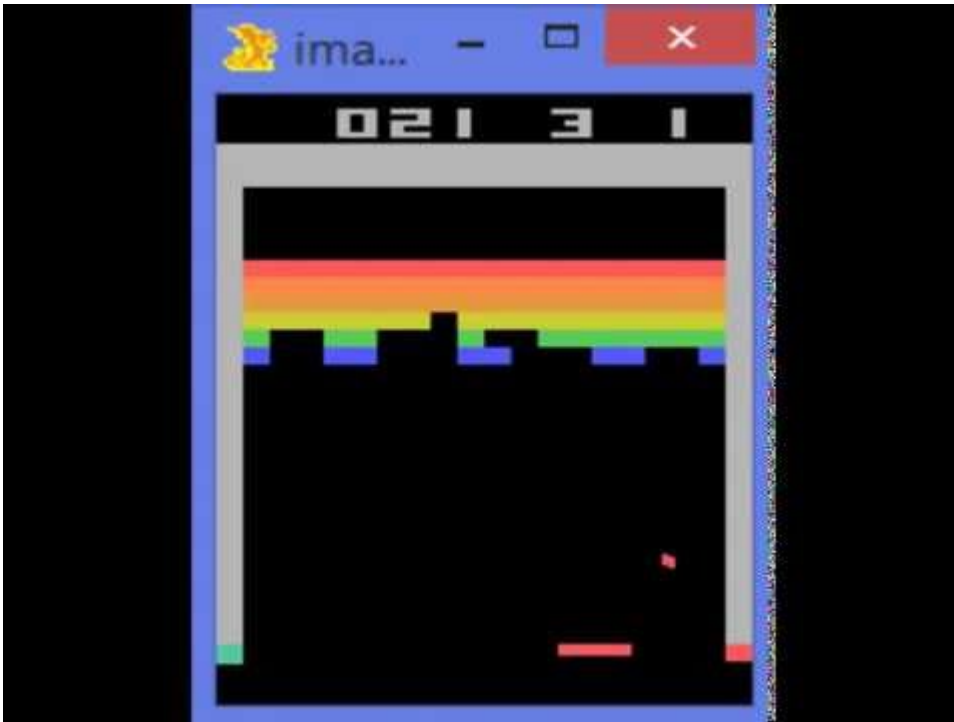
- El sistema **Watson**, también de **IBM**, logró ganar en **2011** el popular concurso televisivo **Jeopardy!** frente a los dos máximos campeones de este programa.



IBM continuó investigando en el campo de la Inteligencia Artificial (a la vez que sus ordenadores aumentaban su capacidad de cómputo) y, con lo aprendido con Deep Blue y otros desarrollos, creó Watson, una computadora que logró en 2011 ganar en Jeopardy!, uno de los concursos de conocimiento más famosos en Estados Unidos.

Watson era un sistema capaz de comprender y responder preguntas, en lenguaje natural (es decir, expresadas como habla cualquier humano, con variaciones y diferentes formas de expresar la misma idea). Contaba con una base de datos almacenada localmente (sin conexión a Internet) compuesta de enciclopedias, diccionarios, tesauros, artículos de noticias, obras literarias y otras bases de datos complementarias).

- La empresa **DeepMind** publicó "el vídeo de los 500 millones de dólares". En dicho vídeo mostraba cómo la red neuronal que había desarrollado aprende a jugar al **Arkanoid** de manera autónoma. Google acabó comprando esta empresa en **2014** por **500 millones** de dólares (de ahí el nombre del vídeo).



DeepMind, una compañía inglesa creada en 2010, publicó a los pocos años de su creación el que se denominó "El vídeo de los 500 millones de dólares". En dicho vídeo (que puedes ver más arriba) mostraba cómo su Inteligencia Artificial había aprendido a jugar gracias a la técnica de entrenamiento autónomo (Machine Learning) por refuerzo al "Arkanoid" (un juego arcade del S.XX).

La red neuronal que desarrolló "aprendía" a jugar como un humano (con memoria a corto plazo, aplicando lo aprendido en cada partida a las siguientes). Así, en el vídeo podemos ver cómo en las primeras partidas descubre cómo debe mover para evitar perder. Mas adelante aprende a ganar puntos destruyendo ladrillos, y a "ganar" la partida alcanzando la máxima puntuación. Y finalmente "descubre" que si logra colar la pelota por un lateral hasta la parte superior de la pantalla gana en mucho menos tiempo.

Al poco tiempo de publicarse este vídeo recibió oferta de compra de Facebook, que no se materializó, y acto seguido de Google, que la compró por 500 millones de dólares (de ahí el nombre del vídeo).

Ya dentro de la matriz de Google continuaron profundizando en las técnicas de Aprendizaje Automático, logrando otro hito, como veremos más adelante.

- **Google** liberó **Tensor Flow**, su librería para Machine Learning, en **2015**, permitiendo que cualquier persona pudiera acceder a sus servidores y crear su propio equipo con capacidad de autoprogramación y de aprender de forma autónoma.

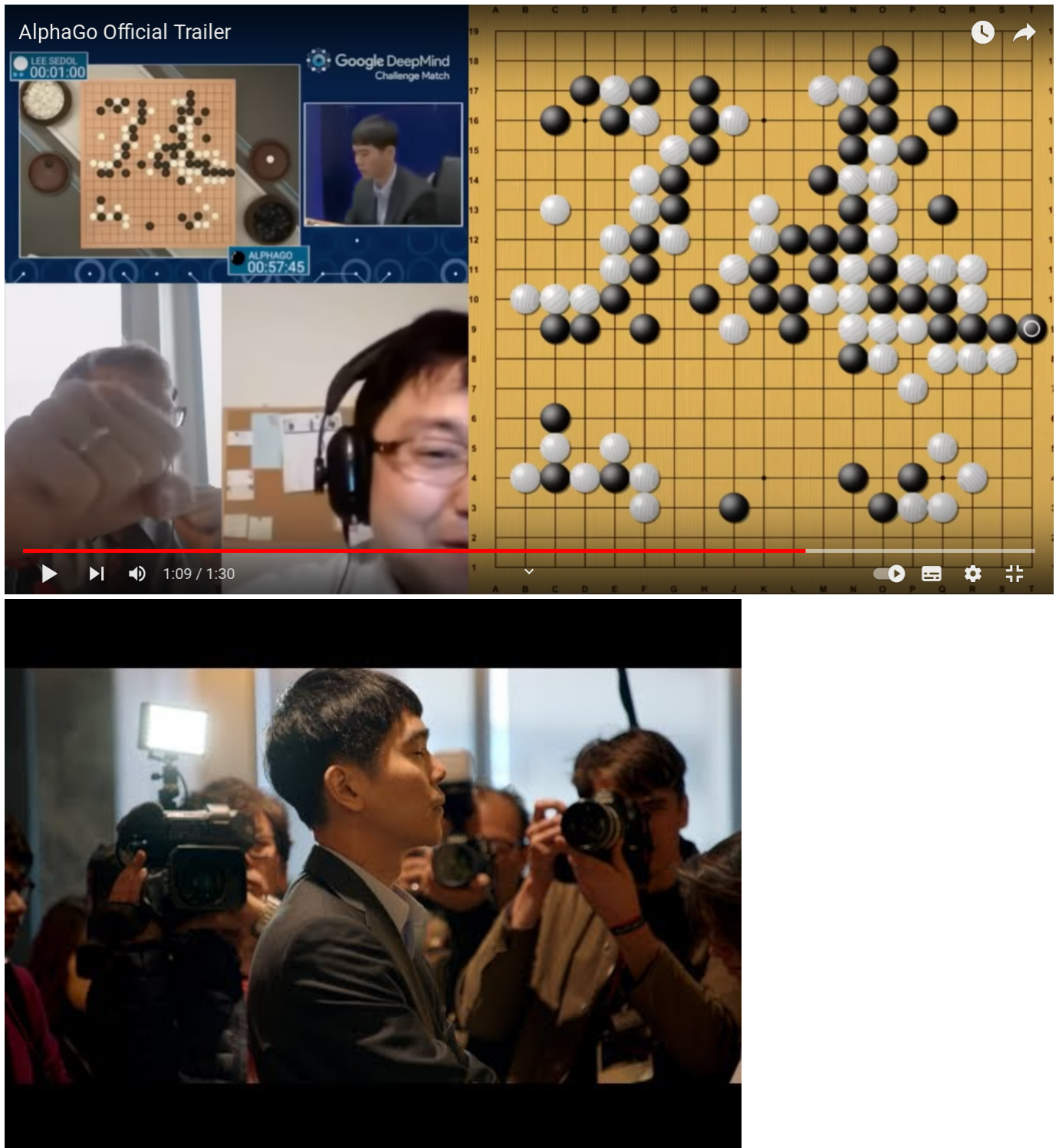


En noviembre de 2015 Google liberó TensorFlow, el programa de Inteligencia Artificial que había desarrollado mejorando un sistema de aprendizaje automático anterior conocido como DistBelief.

Fue la primera vez que se ponía a disposición de cualquier usuario, investigador o empresa interesados en realizar sus propios experimentos de Inteligencia Artificial. Supuso un gran impulso tanto en el campo de la investigación (la propia comunidad de desarrolladores ayudó y sigue ayudando a mejorar y perfeccionar la herramienta) como en el de la democratización de la Inteligencia Artificial, haciéndola accesible para todos.

En la actualidad sigue utilizándose tanto para primeras aproximaciones al universo de la IA, como para desarrollar prototipos o ejercicios más complejos.

- La IA de **AlphaGo** de **Google** sorprendió a todos proponiendo en una partida de Go una jugada que nunca hubiera hecho un experto jugador humano... que en pocos movimientos más le dio la victoria.



Trailer: https://www.youtube.com/watch?v=8tq1C8spV_g

Documental completo: <https://www.youtube.com/watch?v=WXuK6gekU1Y>

Con subtítulos en español: <https://www.youtube.com/watch?v=GfJ7zr4sYx4>

Si jugar al ajedrez es complicado... el juego Go (de origen también oriental) lo es todavía más. La división de Google Deep Mind desarrolló una Inteligencia Artificial capaz de jugar a este juego. Pero la diferencia respecto a logros previos alcanzados por programas de IA capaces de jugar (al ajedrez, a un concurso de preguntas y respuestas...) es que lo hacía "con intuición".

En marzo de **2016** AlphaGo se enfrentó a **Lee Sedol**, que era uno de los mejores jugadores del momento. Ganó AlphaGo. Pero además lo hizo con una jugada (el movimiento 37 de esa partida) completamente inesperado y que ningún experto jugador humano hubiera hecho nunca. Es decir, que aunque la máquina estaba entrenada con registros de partidas reales jugadas por

expertos, no siguió ninguna estrategia observada en su base de datos (contaba con 50 millones de partidas de Go entre humanos). Había aprendido a jugar "con intuición": aprendió de los ejemplos "humanos" pero descubrió formas de jugar nuevas (¡y eficaces!). La cara que se le puso a Lee Sedol al observar y analizar el movimiento de la máquina en ese turno 37 de la partida se hizo famosa.

- **Ian Goodfellow** presentó en **2014** su generador de imágenes basado en lo que conocemos como red **GAN**, logrando que un humano no sepa distinguir si se trata de imágenes reales o inventadas.

Las redes GAN (Generative Adversarial Networks) o generativas antagónicas presentadas por Ian Goodfellow en 2014 han permitido generar fotografías que parecen auténticas a cualquier observador humano.

Posteriormente este tipo de técnica (Aprendizaje Automático Supervisado, que veremos más adelante) también se ha aplicado a la generación de textos tal y como los escribiría un humano.

En esencia las redes GAN se componen de una red generadora (que crea la imagen, texto o diseño) y una red discriminadora (que determina si el resultado de la red generadora es aceptable o no). Ambas redes "compiten" entre ellas (la primera para "engañar a la segunda, y la segunda para detectar fallos en lo generado por la primera). El sistema se retroalimenta y perfecciona con cada iteración.



Curiosidad

En esta web se muestra, cada vez que actualizas la página, la imagen de un rostro humano generado por IA: <https://thispersondoesnotexist.com/>. En algunos casos se notan cosas raras (en las pupilas, orejas, o fondos), pero en general suelen salir rostros que bien podrían corresponder con personas reales ¡Pero en realidad esas personas no existen!

- Desarrollo de **GPT3** por **OpenAI** a través de técnicas de Deep Learning.

GPT-3 es la tercera generación del Modelo de Predicción del Lenguaje que ha sido presentada en mayo de **2020**. Se trata de una Inteligencia Artificial "educada" para escribir cualquier tipo de texto, con cualquier tipo de estilo. A partir de unas pocas palabras que le proporcionas explicando qué es lo que quieres te devuelve un texto complejo que trata sobre lo que le hayas pedido.

Lo más importante de esta tecnología son los **175 Billones de parámetros que utiliza** la para conseguir dar textos naturales (con aspecto de haber sido escritos por humanos).

- **GPT-4** representa la cuarta generación del Modelo de Predicción del Lenguaje, presentado en septiembre de **2022**. Esta versión ha experimentado avances significativos en la capacidad de generación de texto. Como una Inteligencia Artificial avanzada, GPT-4 ha sido entrenado para producir textos aún más naturales y coherentes, adaptándose a cualquier estilo y contenido requerido. Sorprendentemente, cuenta con una impresionante cantidad de **250 Billones de parámetros**, lo que le permite alcanzar un nivel de sofisticación sin precedentes en la creación de textos que parecen ser obra de escritores humanos expertos.

Aprendizaje Profundo (Deep Learning): El Aprendizaje Profundo, una rama del Aprendizaje Automático, ha sido uno de los desarrollos más destacados en la IA moderna. Las redes neuronales profundas, inspiradas en la organización del cerebro humano, han demostrado ser altamente efectivas en tareas como el reconocimiento de imágenes, la traducción automática y el juego de estrategia. La escalabilidad de los algoritmos de Aprendizaje Profundo, junto con la disponibilidad de potentes unidades de procesamiento gráfico (GPU), ha impulsado el rápido progreso en el campo.

IA en la Sociedad Actual: En la actualidad, la IA ha permeado en diversas áreas de nuestra vida cotidiana. Está presente en aplicaciones como motores de búsqueda en línea, asistentes virtuales, recomendaciones de productos y servicios, sistemas de navegación, automóviles autónomos y mucho más. La IA también está siendo utilizada en campos como la medicina para el diagnóstico y tratamiento, en finanzas para la detección de fraudes y en la industria para optimizar procesos de producción.

Desafíos Actuales y Futuros: Aunque la IA ha logrado avances impresionantes, todavía enfrenta desafíos significativos. Uno de ellos es la interpretabilidad y explicabilidad de los modelos de IA, especialmente en aplicaciones críticas donde las decisiones pueden tener un impacto importante en las vidas humanas. Otro desafío es el sesgo en los datos y la falta de diversidad, lo que puede llevar a resultados injustos o discriminatorios. A medida que la IA continúa avanzando, es esencial abordar estos desafíos y asegurar que su desarrollo y aplicación se realicen de manera ética y responsable.

**Curiosidad****¿La Inteligencia Artificial es buena o mala?**

Piensa en diferentes momentos históricos en los que la humanidad ha desarrollado alguna tecnología: El dominio del fuego, la rueda, el hormigón, la pólvora, la imprenta, la radio, Internet...

La tecnología en sí misma no es ni buena ni mala. Son las personas que la conocen y controlan quienes pueden hacer un uso beneficioso o dañino de ellas.

¿Te suena la frase "Un gran poder exige una gran responsabilidad"? La IA nos da un poder tan grande (o mayor) que el Spiderman... Hemos de ser responsables al utilizarla.

El futuro de la IA

Los campos en los que más se ha desarrollado y aplicado la IA en estos últimos años son:

- Sistemas autónomos
- Aprendizaje Autónomo (Machine Learning)
- Aprendizaje Profundo (Deep Learning)
- Redes neuronales.
- Reconocimiento de patrones
- Procesado del lenguaje natural
- Desarrollo de chatbots
- Reconocimiento de emociones

En la actualidad se está trabajando (y se esperan mejoras en los próximos años) en campos como:

- Asistentes virtuales
- Traducción simultánea universal.
- Control de juegos con el pensamiento.

Y a medio plazo se prevé que la Inteligencia Artificial proporcione soluciones y mejoras en los siguientes ámbitos:

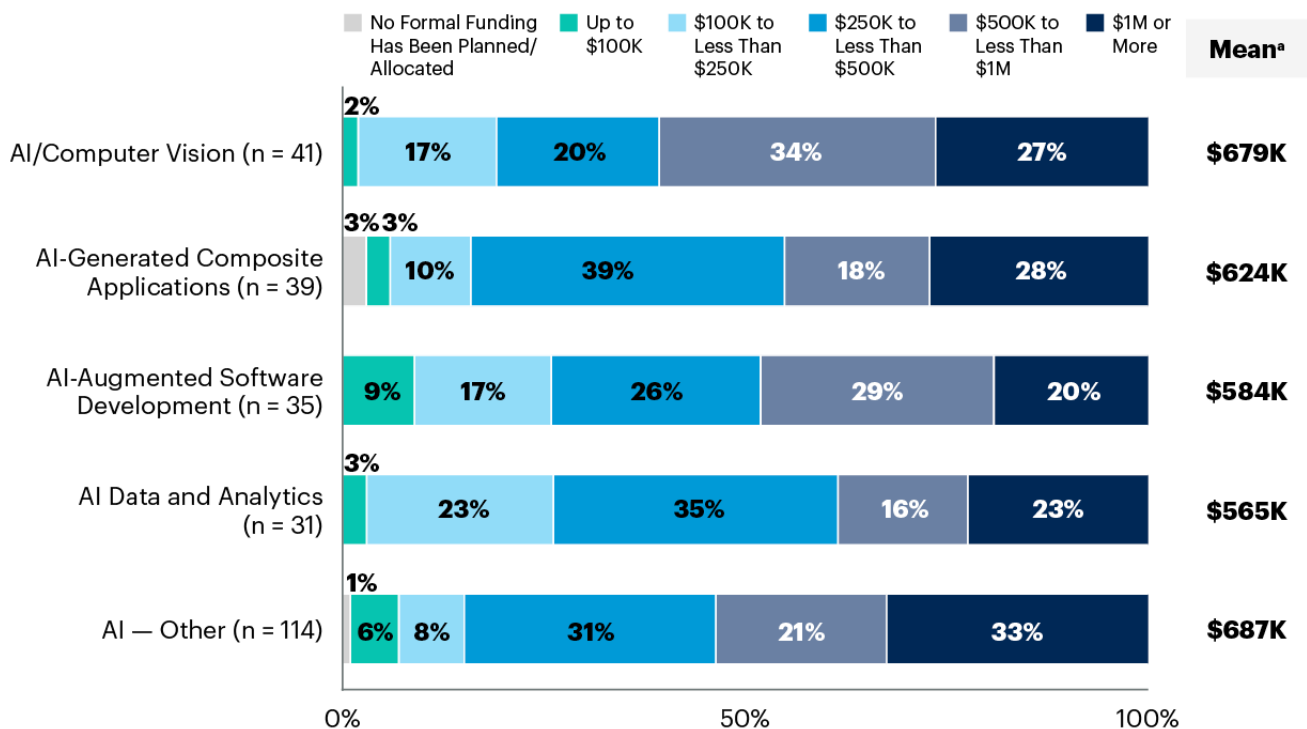
- Nueva generación de robots interconectados con la nube.
- Robots médicos autónomos.
- Asistentes personales robóticos.
- Ciber-Seguridad cognitiva.

Y a largo plazo se vislumbra que puedan llegar a desarrollarse computadoras robóticas con forma y comportamiento humano.

Inversión en proyectos de IA

Como hemos visto la Inteligencia Artificial ya está demostrando de manera práctica los beneficios que puede proporcionar a empresas e instituciones. Las empresas tecnológicas fueron pioneras hace pocos años al incorporar en sus procesos y productos estas aplicaciones. Y en la medida en que todo el entramado empresarial y social se está digitalizando, la posibilidad de incorporar tecnologías inteligentes en cualquier sector está al alcance de casi cualquiera.

De hecho, en los últimos años la Inteligencia Artificial es el primer o segundo ámbito en el que más dinero están dispuestas a invertir las empresas (por delante de otras tecnologías emergentes como Internet de las Cosas, o los datos en la nube). En el siguiente gráfico puedes ver el nivel de financiación que se preveía dedicar en 2022 a incorporar y desarrollar Inteligencia Artificial en un grupo de empresas analizado por Gartner.



fuelle: <https://www.gartner.com/en/newsroom/press-releases/2021-09-29-gartner-finds-33-percent-of-technology-providers-plan-to-invest-1-million-or-more-in-ai-within-two-years>

Limitaciones Prácticas Actuales

Aunque muchos de los argumentos filosóficos y científicos en contra de la inteligencia artificial pueden abordarse con respuestas lógicas, teóricas o basadas en la evidencia, es importante reconocer que existen limitaciones prácticas actuales en el campo de la IA que aún no se han superado por completo. Algunas de estas limitaciones incluyen:

- Complejidad de la mente humana:** La mente humana es increíblemente compleja, y aún no comprendemos completamente todos los aspectos de cómo funciona. La IA actual está lejos de replicar la complejidad y la plasticidad del cerebro humano.
- Memoria y recursos limitados:** Aunque las capacidades de almacenamiento y procesamiento de las computadoras han aumentado drásticamente, todavía estamos lejos de igualar la capacidad de almacenamiento y la eficiencia de procesamiento del cerebro humano.
- Falta de comprensión de la conciencia:** A pesar de los avances en neurociencia y cognición, aún no entendemos completamente la naturaleza de la conciencia y cómo emerge en el cerebro. Sin esta comprensión, es difícil replicarla en una máquina.
- Ética y responsabilidad:** La creación de inteligencia artificial plantea cuestiones éticas y de responsabilidad importantes, como el temor a la toma de decisiones no éticas o la falta de responsabilidad en caso de errores graves.
- IA en entornos no controlados:** La IA puede funcionar bien en entornos controlados y con datos bien estructurados, pero tiene dificultades para adaptarse a situaciones inesperadas o entornos no controlados.
- Creatividad e intuición:** Aunque se han logrado avances en la generación de contenido creativo por parte de las máquinas, la verdadera creatividad e intuición humana siguen siendo difíciles de replicar.
- Emociones y empatía:** La comprensión y expresión emocional, así como la empatía, son aspectos desafiantes de la inteligencia humana que no se han logrado de manera completa en la IA.
- Interacción social humana:** La comprensión del lenguaje natural, la interacción social y la percepción de matices emocionales en el contexto de la comunicación humana son desafíos actuales para la IA.

⚠ Importante

Es importante tener en cuenta estas limitaciones y reconocer que la IA actual está lejos de igualar la inteligencia humana en todos sus aspectos. Sin embargo, esto no implica que no haya avances significativos en el campo de la IA, ni que no se puedan superar algunas de estas limitaciones en el futuro.

Principios de Sistemas Inteligentes

Así pues, puede entenderse por sistema informático el conjunto de cosas (hardware y software) y al conjunto de reglas (procedimientos) que de manera conjunta se emplean para el fin último de adquirir, almacenar, procesar y representar la información de manera automatizada.

Definición

El **hardware** incluye computadoras o cualquier tipo de dispositivo electrónico, principalmente constituido alrededor de semiconductores como memoria, procesadores, sistemas de almacenamiento externo, etc.

El **software** incluye al sistema operativo, firmware y aplicaciones. También se puede considerar parte del sistema a quien hace uso del hardware: los programadores y los usuarios.

La potencia y eficacia de un sistema de la información radica en la correcta correlación de una gran cantidad de datos ingresados mediante procesos específicos para cada campo o tarea, con el objetivo de producir información para la posterior toma de decisiones. Un sistema informático se destaca por su diseño, facilidad de uso, flexibilidad, mantenimiento automático de los registros, apoyo en la toma de decisiones críticas y la conservación del anonimato en informaciones irrelevantes.

Por todo ello, si la inteligencia artificial es un subconjunto de la informática (en el sentido de computación o ciencia de las tecnologías de la información), un sistema de inteligencia artificial ha de ser por extensión un subconjunto dentro de los sistemas informáticos.

Definición

Se denominará, por tanto, sistema inteligente a un programa o conjunto de programas de computación que reúne características y comportamientos asimilables al de la inteligencia humana o animal.

Para que un sistema informático pueda ser considerado un sistema inteligente, habrá de tener las características que se enumeran a continuación:

1. En primer lugar y como clara obviedad el sistema debe poseer inteligencia, entendiendo como ello el **ser un sistema inteligente**.
2. El hecho de ser un sistema implica que ha de disponer de sistematización entre sus componentes, es decir, **las partes del sistema han de tener correlaciones con otros elementos del mismo sistema**.
3. El sistema ha de ser capaz de **cumplir uno o varios objetivos**, o sea, una cierta situación, condición o estado que el sistema inteligente busca lograr.
4. El sistema ha de disponer de **capacidad sensorial**, para ello ha de tener una parte que sea capaz de recibir comunicaciones del entorno: el sistema inteligente ha de poder reaccionar ante el entorno y sus variaciones, lo que se consigue normalmente mediante sensórica.
5. El sistema ha de exhibir capacidad de conceptualización (entendiendo como concepto al elemento básico del pensamiento), lo que implica la **necesidad de poder almacenar información**. Para poder conceptualizar es necesario el desarrollo de niveles de abstracción.
6. El sistema ha de disponer de **procedimientos y métodos con reglas de actuación**, por tanto, el sistema ha de ser capaz de relacionar situaciones y consecuencias de acciones.
7. El aprendizaje es la capacidad más importante y exclusiva de un sistema inteligente. **El sistema aprende nuevos conceptos a partir de la información recibida de los sentidos, las reglas conocidas y la experiencia**. El aprendizaje también es la capacidad de detectar relaciones (patrones) entre la «situación» y la parte «situación futura» de una regla de actuación.

Definición

Los primeros sistemas inteligentes, como los sistemas de expertos, no cumplen todas estas características por lo que reciben el nombre de sistemas de inteligencia incompletos.

La inteligencia artificial (IA) es un área de cambio social, que transforma rápidamente los hábitos y costumbres de las sociedades, y por ello ha de prestarse mucha atención a sus posibilidades, y marcar una serie de límites éticos. Los principios fundamentales o empleos éticos de la IA son los siguientes:

1. **La IA debe estar libre de prejuicios**, tanto en el caso inferencia como en el entrenamiento. Entrenar datos de manera equivocada puede generar discriminantes negativas que alejan la toma de decisiones de la realidad. Además, en conjunto, toda IA debe programarse de manera que no se usen conjuntos sesgados, y evitar discriminaciones algorítmicas por medio de métricas imparciales avaladas por expertos humanos. Aunque parece algo lejano, se introducen continuamente sesgos en el aprendizaje de las IA, muchas veces de manera involuntaria e inadvertida.



Ejemplo

Sistemas de ayuda médica entrenados en varones blancos de entre 30 y 50 años de edad podrán dar resultados buenos sobre el grupo para el que se ha desarrollado, pero pueden actuar no tan correctamente en otras categorías.

2. **Ayudar a ayudar.** Se debe de identificar de forma clara la responsabilidad de las decisiones tomadas por los sistemas autónomos. Los usuarios de las IA, especialmente en caso de grandes corporaciones y gobiernos, han de aprender a estimar y evaluar las consecuencias positivas y negativas de la implantación de sistemas de inteligencia artificial, en la sociedad en su conjunto, dado el gran poder de cambio que generan.
3. **Uso de algoritmos abiertos.** Para poder confiar en la respuesta de una IA es preciso tener acceso limpio al algoritmo de entrenamiento y de toma de decisiones que posee, lo que comporta acceso a todo su modelo matemático. Supone la única manera de poder explicar el funcionamiento que tendrá la misma.
4. **Seguridad, privacidad y confiabilidad.** Dado que las IA hacen uso de gran cantidad de datos, se ha de velar por la transparencia y la privacidad en el uso de los mismos.



Importante

Un asistente virtual ha de garantizar que las conversaciones escuchadas no se filtrarán ni difundirán a terceros.

5. **Bien común.** Ningún sistema de IA debería ser desplegado si al hacerlo se atenta contra el bien común.

Los agentes inteligentes son entidades capaces de percibir su entorno a través de sensores y actuar en él mediante efectores para alcanzar objetivos específicos. Un agente puede ser tan simple como un programa que juega ajedrez o tan complejo como un vehículo autónomo que navega por las calles de una ciudad. Los agentes inteligentes se basan en la idea de que una "inteligencia" puede emerger de la interacción entre un sistema y su entorno, sin necesidad de una planificación o conocimiento exhaustivo previo.

Componentes de un Agente Inteligente:

1. **Sensores (S):** Los sensores son dispositivos que permiten al agente percibir información sobre su entorno. Pueden incluir cámaras, micrófonos, sensores de temperatura, GPS, entre otros. La información que los sensores recopilan se utiliza para representar el estado actual del entorno.
2. **Actuadores (A):** Los actuadores son los medios mediante los cuales el agente interactúa con su entorno. Pueden ser ruedas en un robot, motores en un brazo robótico o simplemente salidas de datos en un sistema de software. Los actuadores permiten que el agente tome decisiones y realice acciones para alcanzar sus objetivos.
3. **Función del Agente (f):** La función del agente representa el comportamiento del agente en función de las percepciones que recibe. Toma como entrada el estado actual del entorno y devuelve una acción que el agente debe ejecutar. Esta función puede ser simple o compleja, dependiendo de la complejidad de la tarea que el agente debe realizar.
4. **Arquitectura (A):** La arquitectura del agente se refiere a cómo se organiza el agente en términos de sus componentes y cómo interactúan entre sí. Puede haber diferentes arquitecturas según la complejidad de la tarea y los requisitos de rendimiento.



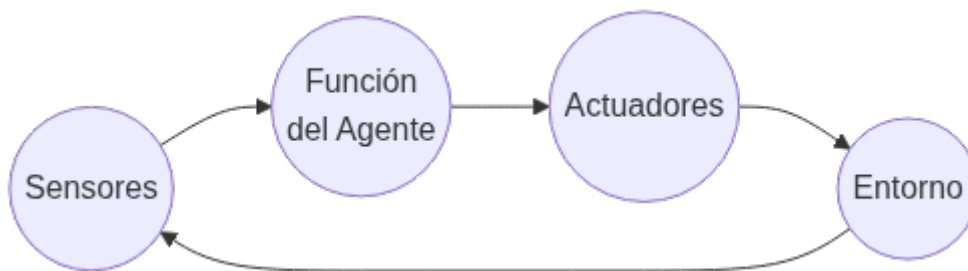
Ejemplo:

Agente Inteligente:

Un ejemplo sencillo de un agente inteligente es un sistema de navegación GPS en un automóvil. En este caso:

- **Sensores (S):** El sistema de navegación utiliza sensores GPS para recibir información sobre la ubicación actual del automóvil y sensores de velocidad para conocer su velocidad y dirección.
- **Actuadores (A):** Los actuadores son los mecanismos que permiten al sistema de navegación proporcionar instrucciones al conductor para alcanzar el destino deseado, como la pantalla de navegación o las indicaciones de voz.
- **Función del Agente (f):** La función del agente en este caso podría ser bastante simple: recibir la ubicación actual y el destino deseado, calcular la ruta más rápida y segura y guiar al conductor a lo largo del camino.
- **Arquitectura (A):** La arquitectura del sistema de navegación podría ser una combinación de algoritmos de planificación de rutas, sistemas de reconocimiento de voz para recibir comandos del conductor y sistemas de visualización para mostrar las indicaciones.

A continuación, un diagrama para ilustrar la interacción de un agente inteligente con su entorno:



En el diagrama, los sensores recopilan información del entorno, que se utiliza como entrada para la función del agente. La función del agente procesa la información y toma una decisión sobre qué acción ejecutar. Luego, los actuadores implementan la acción en el entorno, lo que puede cambiar su estado. El ciclo se repite continuamente, permitiendo al agente inteligente interactuar y adaptarse a su entorno para lograr sus objetivos.

Tipos de Inteligencia Artificial. Escuelas y clasificaciones

En la actualidad la evolución de la Inteligencia Artificial comprende un campo tan amplio, con tantas ramas, que es complicado poder atender a una única clasificación. De hecho encontramos diferentes clasificaciones según la diferente visión con la que se aborda dicha tecnología. Las hay más filosóficas, más técnicas, y según su aplicación.

Dentro de las distintas clasificaciones que vamos a ver, hay algunas tipologías que están definidas "en teoría" aunque aún no exista ninguna aplicación IA de esa clase.

Según tareas a resolver

La primera clasificación de la Inteligencia Artificial que vamos a ver se basa en qué tipo de tareas nos ayuda a resolver o ejecutar.

Cuando hablamos de tarea a resolver nos referimos, por ejemplo, a si pretendemos que la IA sea capaz de jugar al ajedrez en un ordenador o si pretendemos que sea capaz de gestionar una cocina sin intervención humana (desde el abastecimiento de alimentos, procesado, decisión de qué cocinar en cada momento, limpieza y mantenimiento de utensilios, resolver imprevistos o accidentes...).

Hay tareas sencillas, concretas, y puntuales que son relativamente sencillas de programar, mientras que hay tareas complejas en las que, además influyen muchos factores exteriores (sentimientos, contexto, moral, ética, creencia religiosa...).

Seguro que eres capaz de ir intuyendo que nuestros programas de Inteligencia Artificial actuales son más bien de los que resuelven tareas en un entorno muy concreto y acotado, mientras que hay que irse a las películas o series de ficción para poder hablar de algún caso de Inteligencia Artificial capaz de actuar en escenarios complejos, cambiantes y con "conciencia".

Esta clasificación según tareas es una aproximación bastante simple, con dos opciones:



	IA Débil	IA Fuerte
CAPACIDADES	Realiza tareas específicas .	Puede realizar cualquier tarea intelectual que un humano puede hacer.
EJEMPLOS	Siri, Alexa , Google Translate.	IA consciente de sí misma (Aún no existe).
ESTADO ACTUAL	Existe y se utiliza ampliamente .	Aún no existe y es objeto de investigación y debate .

Como veremos en la explicación más detallada de cada una de estas dos posibles categorías de Inteligencia Artificial, en la actualidad aún no hemos sido capaces de desarrollar ninguna IA Fuerte. Todo lo que conocemos en este momento quedaría dentro de la clasificación de IA Débil.

INTELIGENCIA ARTIFICIAL DÉBIL (O ESTRECHA)

Definición

IA Débil también conocida como **IA estrecha**, se define como la inteligencia artificial racional que se centra típicamente en una tarea estrecha. Es decir orientada a resolver problemas muy concretos, en un entorno perfectamente acotado.

Por tanto consideramos que este tipo de Inteligencia Artificial débil es limitada, pues no es capaz de adaptarse o asumir cambios respecto a lo que se le ha programado.

Los asistentes virtuales (Siri, Ok Google, Alexa, etc.) son un ejemplo bastante ilustrativo de hasta dónde es capaz de llegar la Inteligencia Artificial débil. Cualquiera de ellos opera dentro de un rango de respuestas limitado, definido en su base de datos. En realidad "la máquina" no tiene inteligencia genuina. No es capaz de aprender, ni de tener en cuenta el entorno o el contexto en el que se le realizan las preguntas. No tiene conciencia, ni mucho menos vida propia. Sin duda es un tipo bastante sofisticado de IA débil, pero llega un punto en el que no es capaz de responder cierta clase de preguntas, y, salvo que cambiáramos su base de datos y programación nunca sería capaz de llegar a encontrar por sí misma respuestas adecuadas a dichas preguntas.

De hecho, uno de los principales entretenimientos más habituales cuando nos ponemos "a charlar" con un asistente virtual es intentar llevarla al límite... A ver en qué tipo de pregunta, cada cual más compleja o absurda, es incapaz de responder. O, sin llegar al límite de no encontrar respuesta, puede llegar a dar respuestas molestas o inadecuadas.

Desde el punto de vista de esta clasificación (por tarea a resolver), **las características** de la Inteligencia Artificial débil son:

- **Ya existen en la vida real:** Como hemos comentado, los asistentes virtuales, programas como Watson o Alpha Go que vimos en la unidad anterior.
- Se orientan a **resolver problemas muy concretos:** El programa que "sabe" jugar al Go, no sabe hacer otra cosa. Ni tiene posibilidades de aprender a jugar a otra cosa, por muy similar que sea.
- Son **reactivas:** No tienen iniciativa, es necesario que se desencadene la acción que tienen programada para que se inicie su rutina. En el ejemplo del asistente virtual, tiene programado responder cuando le preguntas, y por tanto nunca tomará la iniciativa de ofrecerte nada sin que tú lo actives previamente.
- **No son flexibles:** Colapsan si se encuentran en un caso no previsto en su programación.
- Quedan **limitadas por lo que programa un humano:** Es el humano quien programa lo que "tiene que pensar" la máquina. Si el humano no programa deja sin considerar ciertas opciones o posibles situaciones, la IA nunca será capaz de suplirlo o aprenderlo sobre la marcha por sí misma.
- Se **programan con pocas redes neuronales:** Hablaremos más adelante sobre las redes neuronales. Por el momento es suficiente entender que el nivel de computación compleja que requieren este tipo de Inteligencias Artificiales es menor que otros casos.
- **No razonan, solo computan:** No tienen en cuenta ningún factor moral, contextual, circunstancial, emocional... que a un humano le haría reaccionar de manera diferente.
- La máquina está programada para alcanzar tal objetivo o funcionar de tal manera, y así lo hará sin "entender" lo que está haciendo. Por tanto: **no tiene conciencia**.

- **Aprenden a base de ejemplos:** Necesita conocer muchos ejemplos de lo que tiene que hacer (la base de datos), con todas las variantes posibles. Por ejemplo, en la máquina que juega al Go, se la "entrenó" con 50 millones de partidas de dicho juego.
- **Son repetitivas:** No se cansan nunca, son implacables, siempre la misma rutina. No salen de su marco de trabajo: Y esto supone que pueden ocuparse de tareas mecánicas, repetitivas, "aburridas" para sustituir al humano mejorando rendimiento y precisión. Pero necesitará siempre una supervisión humana que vaya decidiendo cómo adaptar el programa a las cambiantes circunstancias.

Según se mire la Inteligencia Artificial débil podría llegar a ser peligrosa. Porque este tipo de IA, al no tomar en consideración todo un contexto amplio, ni seguir las reglas sociales, éticas,... ejecuta las tareas para las que se le ha entrenado con eficacia y contundencia. No evalúa las consecuencias como lo hacemos los humanos, considerando un espectro amplio de efectos y relaciones. Por eso, es una opción incompleta, inestable y peligrosa si no se utiliza con prudencia, o si quien la programa pretende, precisamente, causar mal.

INTELIGENCIA ARTIFICIAL FUERTE

Definición

La **Inteligencia Artificial fuerte (IAF)** o general o (IAG), desde el punto de vista de la tarea a resolver, sería aquella que iguala o excede la inteligencia humana promedio. Sería capaz de realizar con éxito cualquier tarea intelectual del ser humano, teniendo en cuenta todos los factores y matices que pueden intervenir cuando una persona toma decisiones en cada momento mientras realiza una tarea.

En comparación con la Inteligencia Artificial débil, las características de la fuerte son:

- **No existe en la realidad:** Si quieres "ver" cómo sería puedes recurrir a personajes de ficción como T-800, Wall-E o J.A.R.V.I.S. Resolverán problemas abiertos: Deberían poder abarcar múltiples posibles tareas, distintas unas de otras (reparar una puerta, ir a recoger a los niños del colegio, regar las plantas, darte conversación...).
- **Serán proactivas:** En función de la misión u objetivo que tenga, y de las circunstancias, iniciará cualquier tipo de rutina sin esperar a que un humano se lo pida o esté pendiente.
- **Serán flexibles:** Podrán encontrar similitudes entre algo que conocen y algo que se le parezca un poco. Por ejemplo, aunque inicialmente solo haya sido programada para saber andar, será capaz de aprender a correr sin necesidad de intervenir en su programa.
- **Se autoprogramarán:** Serán capaces de detectar sus propios límites y se regularán a sí mismas para no excederlos.
- **Usarán muchas redes neuronales:** Y además podrán entrar en conflicto entre ellas en algunas ocasiones. Esto quiere decir que necesitarán una capacidad de almacenaje de información y cómputo que aún hoy no hemos llegado a alcanzar.
- **Imitarán el comportamiento humano:** Serán capaces de razonar, y por tanto, de alcanzar algún tipo de consciencia.
- **Aprenderán como las personas:** Podrán recordar datos, observar nuevas situaciones y encontrar relaciones entre diferentes acciones. Esto quiere decir que si saben jugar al ajedrez y "observan" el juego de las damas, podrán aprender a jugar a las damas basándose en lo que saben sobre jugar al ajedrez.
- **Serán capaces de aprender nuevas tareas:** Modificarán la tarea o cómo realizan la tarea para adaptarse a las circunstancias.
- **Serán capaces de adaptarse a nuevos escenarios:** Podrán adaptarse a cambios y nuevas situaciones para seguir cumpliendo su objetivo.

Este tipo de IA es la que sería capaz de analizar cualquier situación y deducir el conjunto de acciones más adecuado para dicha situación y contexto. Lo mismo sabría conducir un coche, que resolver una ecuación matemática o mantener una conversación sobre un tema concreto.

Aunque aún no existe este tipo de IA, todas las empresas e instituciones dedicadas a la investigación y desarrollo de IA están buscando formas de avanzar hacia este tipo de Inteligencia Artificial. De momento, al menos, se está trabajando en conseguir una IAF en el campo de los asistentes virtuales.

Sin duda uno de los ámbitos más ambiciosos de aplicar esta IAF (los asistentes virtuales humanoides) necesitan contar también con otras ramas científicas como son la robótica y mecatrónica.

Escuelas de Pensamiento

En el ámbito de la Inteligencia Artificial más moderna podemos encontrar dos escuelas de pensamiento:

- Inteligencia Artificial **Convencional**.
- Inteligencia Artificial **Computacional**.

Estas dos escuelas difieren en la ciencia que hay tras los procesos que siguen para llegar a los resultados esperados. Pero, con los avances que se están dando en los recursos que utiliza la segunda, muchas de las aplicaciones que tenía la primera, están siendo llevadas al campo computacional.

INTELIGENCIA ARTIFICIAL CONVENCIONAL

Se conoce también como Inteligencia Artificial **simbólico-deductiva**. Está basada en el análisis formal y estadístico del comportamiento humano ante diferentes problemas:

- **Razonamiento basado en casos:** Ayuda a tomar decisiones mientras se resuelven ciertos problemas concretos y, aparte de que son muy importantes, requieren de un buen funcionamiento.
- **Sistemas expertos:** Infieren una solución a través del conocimiento previo del contexto en que se aplica y ocupa de ciertas reglas o relaciones.
- **Redes bayesianas:** Propone soluciones mediante inferencia probabilística.
- **Inteligencia artificial basada en comportamientos:** Esta inteligencia contiene autonomía y puede auto-regularse y controlarse para mejorar.
- **Smart process management:** Facilita la toma de decisiones complejas, proponiendo una solución a un determinado problema al igual que lo haría un especialista en dicha actividad.

Esta rama de la Inteligencia Artificial ha sido la que ha proporcionado la mayoría de algoritmos que conocemos como "automatización", y básicamente se sirven de sistemas con reglas condicionales y estadística avanzada.

INTELIGENCIA ARTIFICIAL COMPUTACIONAL

La Inteligencia Computacional (también conocida como IA **subsimbólica-inductiva**) implica desarrollo o aprendizaje interactivo (por ejemplo, modificaciones interactivas de los parámetros en sistemas de conexiones). El aprendizaje se realiza basándose en datos empíricos, utilizando métodos computacionales inspirados en procesos de la naturaleza, que permiten alcanzar soluciones aptas a problemas complejos que los modelos tradicionales no pueden resolver por no existir una solución analítica, por no contar con todos los parámetros necesarios o porque el problema es en sí estocástico y precisa de una aproximación envolvente en vez de convergente.

Esta corriente ha sido la que impulsó hace pocos años lo que conocemos como "Aprendizaje Automático" o "Machine Learning", que es la técnica que más se está utilizando actualmente en desarrollos de IA.

Algunas técnicas de esta escuela son:

- **Máquina de vectores soporte:** sistemas que permiten reconocimiento de patrones genéricos de gran potencia.
- **Redes neuronales:** sistemas basados en redes de unidades de computación lineal para simular computación no lineal
- **Modelos ocultos de Markov:** aprendizaje basado en dependencia temporal de eventos probabilísticos.
- **Sistemas difusos:** técnicas para lograr el razonamiento bajo incertidumbre
- **Computación evolutiva:** también conocidos como **algoritmos genéticos**, aplica conceptos inspirados en la biología, tales como población, mutación y supervivencia del más apto para generar soluciones sucesivamente mejores para un problema.

Clasificación Stuart J. Russell y Peter Norvig

Stuart J. Russell y Peter Norvig, investigadores informáticos, publicaron en **1995** su libro "**Artificial Intelligence: A Modern Approach**", que se ha convertido en el libro de texto fundamental en cientos de universidades a nivel mundial (ya lleva varias ediciones publicadas). Plantean cuatro categorías básicas partiendo de un enfoque de génesis del acto inteligente, del origen y proceso por el cual se llega al comportamiento inteligente.

Sistemas Cognitivos: Piensan como humanos, intentan emular el proceso humano → Proceso de toma de decisiones, resolución de problemas, y el propio paradigma del aprendizaje.

Test de Turing: Actúan como humanos, intentan emular el comportamiento humano (sin pasar por el pensamiento o razonamiento que conduce a dicho comportamiento) → A nivel práctico se aplica en la robótica y sistemas de actuadores en el mundo físico.

Leyes del pensamiento: Piensan con razonamientos. Cumplimiento exacto de las leyes del razonamiento lógico, teniendo en cuenta todos los factores que afectan a la cuestión. No hemos llegado a esto aún. Sería el caso de los sistemas expertos. Solo son posibles aproximaciones para campos de investigación muy especializados y acotados.

Agentes inteligentes: Actúan racionalmente (sin pasar por el proceso de razonamiento lógico).

Los dos últimos requieren una capacidad de cómputo muy importante, a veces, aún, inaccesible.

Clasificación Hintze

En noviembre de **2016**, **Arend Hintze**, profesor de la Universidad de Michigan e investigador en el campo de la Inteligencia Artificial, escribió un artículo titulado: "**Understanding the four types of AI, from reactive robots to self-aware beings**" (Comprendiendo los cuatro tipos de IA, desde los robots reactivos a los seres auto-conscientes), en el que sintetizaba toda la evolución de los últimos desarrollos y avances en materia de Inteligencia Artificial para aportar una clasificación más realista y concreta para los tipos de entidades que existen o que se aspira a crear.

4 TIPOS DE INTELIGENCIA (ARTIFICIAL)



MÁQUINAS REACTIVAS

Los tipos más básicos de sistemas de IA son puramente reactivos. No tienen la capacidad de formar recuerdos. Tampoco pueden utilizar experiencias pasadas en las que basar las decisiones actuales.

Deep Blue (descrita en el tema anterior) fue una supercomputadora creada por IBM. Fue capaz de vencer al ajedrez al gran maestro internacional Garry Kasparov. Ocurrió a fines de la década de 1990 y es el ejemplo perfecto de este tipo de máquina.

Puede identificar las piezas en un tablero de ajedrez y saber cómo se mueve cada una.

Puede realizar predicciones sobre los mejores movimientos y elegir el mejor de todas las posibilidades.

Pero no tiene ningún concepto del pasado. Tampoco posee recuerdos de lo que ha sucedido antes. Aparte de una regla de ajedrez, Deep Blue ignora todo antes del momento presente. Todo lo que hace es enfocar las piezas del tablero en tiempo real y elegir entre los siguientes movimientos posibles.

En el caso de que se trate de una Inteligencia Artificial reactiva aplicada a una máquina capaz de "conversar" es importante que el usuario sepa que está tratando con una máquina, pues de lo contrario se suelen crear falsas expectativas sobre lo que puede esperar de dicha conversación.

MEMORIA LIMITADA

El segundo tipo de Inteligencia Artificial que contempla la clasificación de Hintze se caracteriza por que sí maneja máquinas que pueden mirar hacia el pasado. Los vehículos autónomos ya hacen algo parecido. Por ejemplo, observan la velocidad y dirección de otros autos, y en base a la memorización de dicha información toman decisiones en el futuro inmediato.

Digamos que estas observaciones se agregan a las representaciones preprogramadas para la memoria de estos coches. Se incluyen marcas de carril, semáforos y otros elementos importantes, como curvas en la carretera.

También se añaden experiencias como cuando el automóvil decide en qué momento cambiar de carril para evitar interrumpir a otro conductor o ser embestido por un automóvil cercano.

Pero estas simples piezas de información sobre el pasado son solo transitorias. No se guardan como parte de la biblioteca de experiencias del automóvil. En estos tipos de inteligencia artificial, la máquina no puede compilar la experiencia durante años, como lo hace un humano.

TEORÍA DE LA MENTE

¿Podemos construir sistemas de Inteligencia Artificial que construyan representaciones completas, recordar sus experiencias y aprender cómo manejar situaciones nuevas?

Llegamos a un punto en el que nos acercamos más a los tipos de Inteligencia Artificial que deseamos en un futuro. Las máquinas de esta clase son más avanzadas. No solo forman representaciones sobre el mundo, también sobre otros agentes o entidades.

En psicología, esto se denomina ‘teoría de la mente’. Implica la comprensión de que las personas, las criaturas y los objetos en el mundo pueden tener pensamientos y emociones que afectan a su propio comportamiento. Esto es crucial para la forma en que los humanos formamos sociedades, porque nos permite la interacción social.

Si las máquinas van a andar entre nosotros, deberán tener una comprensión sobre cómo pensamos y cómo sentimos. Además deberán llegar a saber qué esperamos y cómo queremos que nos traten. Tendrán que ajustar su comportamiento en consecuencia.

Como habrás podido intuir, este tipo de máquinas aún no existen. Igual que la IA Fuerte es aún algo que se sabe cómo funcionará pero que no hemos llegado a desarrollar todavía.

AUTOCONCIENCIA

El paso final del desarrollo de la IA es construir sistemas que puedan formar representaciones sobre sí mismos. En última instancia, los investigadores de la Inteligencia Artificial tendrán que comprender no solo la conciencia, sino también construir máquinas que la tengan.

Los seres conscientes son conscientes de sí mismos, conocen sus estados internos y pueden predecir los sentimientos de los demás. Es probable que estemos lejos de crear máquinas que sean conscientes de sí mismas. Sin embargo, los esfuerzos se enfocan hacia la comprensión de la memoria, el aprendizaje y la capacidad de basar las decisiones en experiencias pasadas.

Este es un paso importante para entender la inteligencia humana por sí misma. Es crucial para diseñar o desarrollar máquinas que sean más excepcionales para clasificar lo que ven frente a ellas.

Los cuatro tipos de inteligencia artificial dan una idea sobre las intenciones que el hombre tiene acerca del futuro de la máquina. Puede que estemos muy lejos de la Inteligencia Artificial autoconsciente. No obstante, está claro que eso es lo que se persigue en última instancia.

Utilización de modelos de Inteligencia Artificial

En esta sección, se explorarán los requisitos básicos de un sistema de resolución de problemas y los diferentes modelos de sistemas de Inteligencia Artificial, incluyendo la automatización de tareas, sistemas de razonamiento impreciso y sistemas basados en reglas.

Requisitos básicos de un sistema de resolución de problemas

Los sistemas de resolución de problemas basados en Inteligencia Artificial deben cumplir con ciertos requisitos fundamentales para ser efectivos y proporcionar soluciones precisas y útiles:

1. **Representación del Problema:** La representación adecuada del problema es crucial para la resolución exitosa. Los sistemas de IA deben seleccionar una estructura de datos y un modelo que reflejen de manera fiel el dominio del problema. Por ejemplo, para el procesamiento del lenguaje natural, se pueden utilizar modelos basados en redes neuronales que convierten el texto en representaciones vectoriales.
2. **Razonamiento y Toma de Decisiones:** Los sistemas de IA deben ser capaces de razonar sobre la información disponible y tomar decisiones informadas para llegar a una solución. Esto implica aplicar técnicas lógicas, de aprendizaje automático o de búsqueda heurística, según la naturaleza del problema.
3. **Aprendizaje y Adaptabilidad:** La capacidad de aprender de la experiencia y adaptarse es esencial para mejorar el rendimiento de un sistema de IA con el tiempo. El aprendizaje automático y el aprendizaje por refuerzo son enfoques comunes utilizados para habilitar esta funcionalidad.
4. **Eficiencia Computacional:** Los sistemas de resolución de problemas deben ser eficientes en términos computacionales para proporcionar respuestas rápidas y escalables a problemas complejos. Esto implica el uso óptimo de algoritmos y técnicas de optimización para reducir el tiempo de ejecución y los recursos necesarios.

5. **Interacción con Usuarios:** Los sistemas de Inteligencia Artificial deben permitir la interacción con los usuarios de una manera comprensible y natural. Esto implica el desarrollo de interfaces de usuario amigables que faciliten la comunicación y la comprensión mutua entre humanos y sistemas de IA.

Modelos de sistemas de Inteligencia Artificial

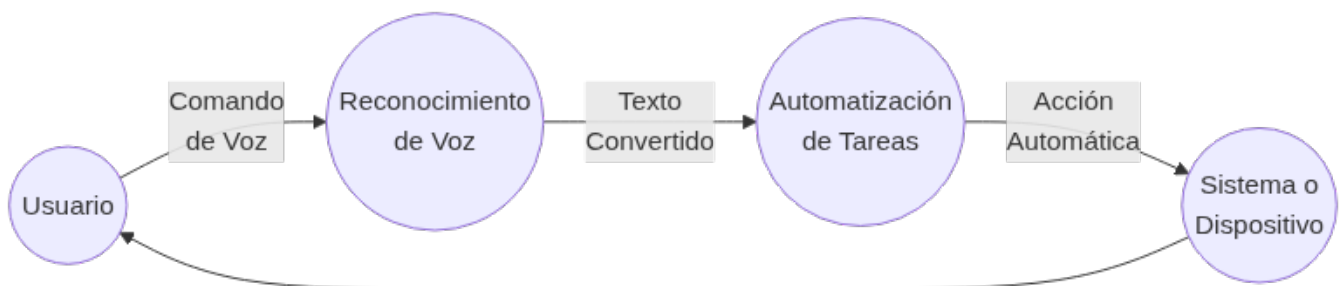
AUTOMATIZACIÓN DE TAREAS

La automatización de tareas es uno de los enfoques más comunes de la Inteligencia Artificial. En este modelo, se desarrollan sistemas que pueden realizar tareas específicas sin intervención humana directa. Algunos ejemplos de automatización de tareas son:

- **Reconocimiento de Voz:** Los sistemas de reconocimiento de voz convierten el habla en texto y pueden automatizar la transcripción de documentos o comandos de voz en dispositivos. Por ejemplo, aplicaciones de reconocimiento de voz como Google Speech-to-Text o Microsoft Azure Speech Service permiten convertir grabaciones de voz en texto escrito de manera automatizada.
- **Detección de Fraude:** Algoritmos de aprendizaje automático pueden analizar patrones de datos financieros y detectar transacciones sospechosas o fraudulentas. Por ejemplo, instituciones financieras utilizan modelos de aprendizaje automático para identificar patrones de comportamiento inusuales que podrían indicar actividades fraudulentas.

La automatización de tareas no solo mejora la eficiencia en diversas industrias, sino que también reduce la carga de trabajo manual y permite a los humanos centrarse en tareas más creativas y estratégicas.

A continuación, un diagrama para ilustrar el proceso de automatización de tareas mediante un sistema de reconocimiento de voz:



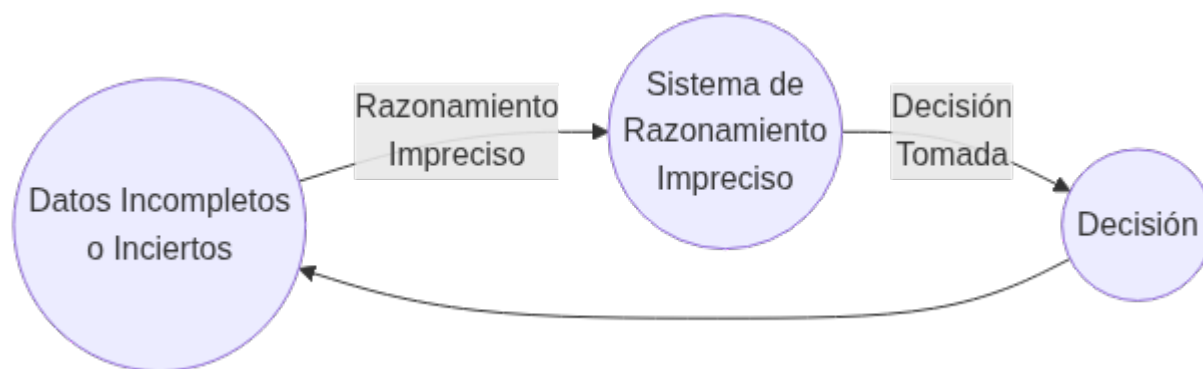
SISTEMAS DE RAZONAMIENTO IMPRECISO

Los sistemas de razonamiento impreciso permiten el manejo de incertidumbre y vaguedad en los datos y la toma de decisiones. Estos sistemas son útiles cuando los datos son incompletos o inciertos y se basan en la lógica difusa y otras técnicas de incertidumbre. Algunos ejemplos son:

- **Controladores de Tráfico:** En el control del tráfico y semáforos, se pueden utilizar sistemas de razonamiento impreciso para optimizar los tiempos de espera de los vehículos y mejorar el flujo del tráfico. La lógica difusa permite ajustar los tiempos de semáforos en función del flujo vehicular en tiempo real.
- **Diagnóstico Médico:** En medicina, se pueden aplicar sistemas de razonamiento impreciso para evaluar síntomas y proporcionar diagnósticos preliminares o sugerencias de tratamiento. Por ejemplo, en el diagnóstico de enfermedades como el cáncer, donde los resultados de las pruebas pueden no ser definitivos, los sistemas de razonamiento impreciso pueden ayudar a proporcionar una evaluación más completa y considerar múltiples factores para el diagnóstico.

El razonamiento impreciso es especialmente valioso cuando se enfrentan problemas en los que la información es vaga o incierta, lo que permite tomar decisiones más robustas y flexibles.

A continuación, un diagrama para ilustrar cómo un sistema de razonamiento impreciso maneja la incertidumbre en la toma de decisiones:



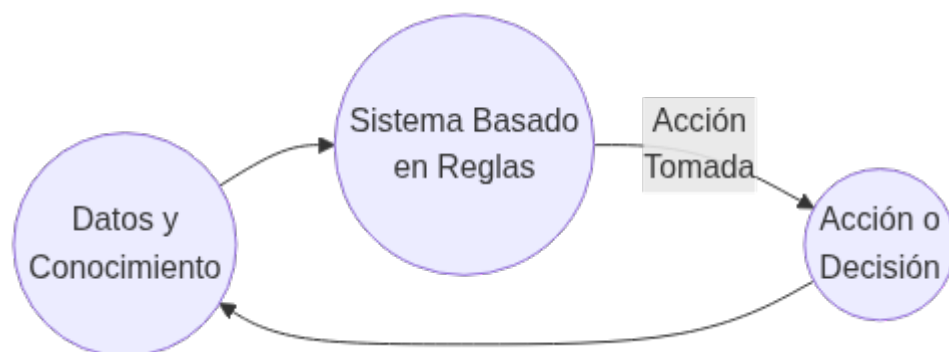
SISTEMAS BASADOS EN REGLAS

Los sistemas basados en reglas son sistemas de Inteligencia Artificial que utilizan reglas lógicas para representar el conocimiento y tomar decisiones. Cada regla consiste en una condición y una acción, y cuando se cumple la condición, se aplica la acción correspondiente. Algunos ejemplos son:

- **Sistemas de Recomendación:** Los sistemas de recomendación utilizan reglas lógicas para sugerir productos, películas, música u otros elementos en función del comportamiento del usuario y otros datos relevantes. Por ejemplo, plataformas de comercio electrónico como Amazon utilizan sistemas de recomendación basados en reglas para ofrecer productos relacionados basados en el historial de compras del usuario.
- **Diagnóstico en Sistemas de Soporte Médico:** En sistemas de soporte médico, las reglas se utilizan para evaluar síntomas y datos médicos y proporcionar diagnósticos preliminares o sugerencias de tratamiento. Por ejemplo, en sistemas de asistencia médica remota, las reglas basadas en síntomas pueden proporcionar recomendaciones iniciales antes de que el paciente sea atendido por un profesional de la salud.

Los sistemas basados en reglas son ampliamente utilizados en aplicaciones donde se requiere un razonamiento transparente y fácil de entender, ya que las reglas lógicas son explícitas y pueden ser interpretadas por humanos.

A continuación, un diagrama para ilustrar cómo un sistema basado en reglas aplica las reglas lógicas para tomar decisiones:



Estos modelos de sistemas de Inteligencia Artificial representan diferentes enfoques para resolver problemas y tomar decisiones en diversas aplicaciones. Cada uno de estos modelos tiene sus propias ventajas y desafíos, y la elección del enfoque adecuado depende del contexto y los requisitos específicos del problema a resolver.

Técnicas de la Inteligencia Artificial

A continuación veremos una taxonomía de técnicas que se usan en IA, algunas se estudiarán a fondo en este módulo (Modelos de Inteligencia Artificial - MIA) y otros en otro módulo del curso (Sistemas de Aprendizaje Automático - SAA).

Modelo Clásico. Sistemas expertos (MIA)

La Inteligencia Artificial, inicialmente, tuvo un desarrollo más teórico que práctico. Los planteamientos originarios de esta Inteligencia Artificial clásica se definieron para un tipo de trabajo informático que ignoraba en buena medida cómo se ha desarrollado en los últimos decenios y que actualmente está establecido como convencional.



Curiosidad

Recuerda que estamos hablando de los años 60 del Siglo XX, y que en esa época apenas existían ordenadores experimentales, con una memoria y capacidad de cómputo que ahora consideraríamos ridículos. Cualquier Smartwatch o controlador de aspiradora inteligente tiene más memoria y velocidad de cálculo que los ordenadores que se utilizaron o se previó que se podrían utilizar para desarrollar Inteligencia Artificial entonces.

Otro aspecto importante a tener en cuenta sobre lo que se entendía por Inteligencia Artificial en esos primeros años es que se preveía que en un plazo de tiempo razonable iba a ser posible que las máquinas "pensaran" como los humanos. Es decir, que los mecanismos de la Inteligencia Artificial imitarían la manera de aprender y reaccionar (actuar) del cerebro humano.

Se dedicó tiempo y esfuerzo, por tanto, a intentar definir de manera matemática y computacional cómo funcionaba el cerebro humano. En última instancia se intentó definir un proceso informático (basado en algoritmos matemáticos) equivalente a lo que haría una neurona humana.

Por tanto, para entender bien qué es la Inteligencia Artificial Clásica, debemos tener en cuenta que:

- Lo que ahora denominamos **Inteligencia Artificial Clásica** fue más bien un **ejercicio de creación de principios generales**, que posteriormente se emplearon para desarrollar los primeros programas informáticos prácticos de Inteligencia Artificial aplicada. Pero esta IA está bastante alejada de lo que hoy por hoy entendemos a nivel práctico como Inteligencia Artificial.
- La Inteligencia Artificial Clásica **quería desarrollar programas informáticos que replicaran el conocimiento humano**, inicialmente en casos particulares y "sencillos", con la intención de ir poco a poco abarcando procesos y casos más complejos. De tal manera que la máquina pudiera "pensar" y actuar como un humano experto en dicho caso particular.

La Inteligencia Artificial Clásica necesitaba que en el proceso de aprendizaje de dicha IA participaran "expertos" en la tarea que se pretendía que la máquina realizara por sí misma.



Ejemplo:

Si se quería que una máquina aprendiera a jugar al ajedrez, en el proceso de aprendizaje era necesario contar con expertos jugadores de ajedrez. De esa necesidad de contar con "expertos" se acabó extendiendo el término **"Sistema Experto"** para designar a los primeros programas de IA que se desarrollaron.

Siendo más concretos, la definición de Sistema Experto es:



Definición

Un sistema experto es un programa informático que se ha desarrollado a partir de nuestro conocimiento sobre una cuestión, y que consigue que el ordenador muestre un comportamiento equivalente al que tendría un experto humano sobre el mismo tema

En esencia se seguía un proceso con cuatro fases:

1. **Localizar al humano experto con conocimiento:** Según el caso particular para el que se quisiera crear esa IA, era necesario incorporar al equipo de desarrollo a una o varias personas expertas en la materia, para que aportaran todo el conocimiento en profundidad.
2. **Definir reglas:** Ese conocimiento humano había que convertirlo en reglas lo más sencillas posible, que relacionaran los diferentes casos y aspectos del conocimiento que se pretendía replicar con la IA.
3. **Informatizar:** Esas reglas había que traducirlas a lenguaje informático.
4. **Iterar:** Probar a ver si realmente la máquina se comportaba de forma "inteligente", buscar fallos, redefinir reglas, o mejorar la programación, y volver a probar. Así tantas veces como fuera necesario hasta que se pudiera considerar que la máquina actuaba igual de bien que el experto humano.

La Inteligencia Artificial Clásica quería "informatizar" modelos de conocimiento. Es decir, lograr convertir en programas informáticos capacidades humanas como "jugar al ajedrez", "detectar faltas de ortografía", "aprender un idioma"...

Pero esta manera de programar Inteligencia Artificial tiene bastantes limitaciones. **Sólo es asequible cuando el conocimiento o "inteligencia" que se quiere informatizar se basa en una relación de causalidad: Causa-Efecto.**

**Ejemplo:**

En el caso del **juego del ajedrez**:

1. Se busca a una o varias personas expertas jugadoras de ajedrez, que conozcan en profundidad el juego, sepan cuáles son las jugadas más características, etc.
2. Con la ayuda de estos expertos jugadores, los científicos definen todos los aspectos del juego de ajedrez, desde cómo es el tablero, las fichas, los movimientos, la jerarquía o relación de importancia entre las fichas, las posibles reacciones a los movimientos del contrario.
3. Los informáticos toman todas esas reglas y las traducen a lenguaje de programación.
4. Se comprueba que el ordenador sea capaz de jugar al ajedrez, sin equivocarse, y priorizando los movimientos que antes le permitan obtener la victoria. Si algo sale mal o se detectan fallos, hay que volver a revisar todo el proceso y mejorarlo (redefinir la forma de algunas normas, o la programación, etc).

Cada posible movimiento del contrario permite que la Inteligencia Artificial reaccione de diferentes maneras (moviendo tal o cual ficha). A su vez, este movimiento de la Inteligencia Artificial permite otras diferentes maneras de reaccionar por parte del contrario... Son lo que hemos mencionado más arriba: relaciones de causalidad. Cada acción tiene una serie de posibles reacciones (y la IA debe elegir una de ellas), que a su vez tienen otra serie de reacciones posibles (el contrario debe elegir una) y así sucesivamente. Gracias a la memoria de la computadora y la capacidad de cómputo es capaz de ver todas las posibles situaciones a 10, 20, 30... movimientos; e ir escogiendo los movimientos que con mayor probabilidad le puedan llevar a la victoria.

Cuando el conocimiento o "inteligencia" que se quiere informatizar se basa en una correlación (relaciones proporcionales entre todas las variables que intervienen) es prácticamente imposible definir y traducir a lenguaje informático todas esas reglas y relaciones. Para estos casos necesitamos otra manera de abordar la Inteligencia Artificial... que es la que se ha desarrollado posteriormente y veremos en los siguientes apartados.

Aprendizaje Automático (Machine Learning) (SAA)

El **Aprendizaje Automático (Machine Learning)** es una rama clave de la Inteligencia Artificial que permite a las máquinas aprender y mejorar su rendimiento en tareas específicas a través de la experiencia. En lugar de ser programadas explícitamente para realizar una tarea, las máquinas utilizan datos para aprender patrones y tomar decisiones informadas. El Aprendizaje Automático se ha convertido en una herramienta poderosa en una variedad de aplicaciones, desde reconocimiento de voz y visión por computadora hasta recomendaciones de productos y diagnósticos médicos.

**Importante**

Es importante entender que el Aprendizaje Automático es una rama de la IA, aunque en la actualidad ha adquirido mucha importancia y se utiliza en prácticamente todos los proyectos de IA. De manera que hoy cuando hablamos de Inteligencia Artificial en realidad estamos hablando de esta rama concreta (**el todo por la parte**).

El Machine Learning o Aprendizaje Autónomo (Automático) a su vez ha evolucionado en estos pocos años que lleva desarrollándose: Inicialmente se focalizaba en lograr que la máquina aprendiera basándose en datos, a través de estudiar el reconocimiento de patrones (casos similares entre el total de elementos del data set o base de datos). Actualmente se centra más bien en "resolver" problemas prácticos que en "aprender", aunque evidentemente "aprende" (pero el aprendizaje como tal ya no es el foco, sino el resultado obtenido). Al reconocimiento de patrones que ya se usaba desde el principio añade ahora lo que conocemos como el razonamiento probabilístico, la estadística y la recuperación de datos.

DEFINICIONES DE APRENDIZAJE AUTOMÁTICO

Arthur Samuel (que trabajó para IBM) en 1959 describía el Aprendizaje Automático como:

**Definición**

El campo del estudio que da a las computadoras la capacidad de aprender sin ser programadas explícitamente

Esta es una definición antigua e informal respecto a lo que hoy en día entendemos por Machine Learning.

Tom Mitchell (profesor en la Universidad de Carnegie Mellon) ha ofrecido una definición más moderna:

Definición

Se dice que un programa de computadora aprende de la experiencia E con respecto a alguna clase de tareas T y medida de rendimiento P, si su desempeño en las tareas en T medido por P mejora con la experiencia E

Ejemplo

Jugar a las damas.

- E es la experiencia de jugar muchas partidas de damas.
- T es la tarea de jugar a las damas.
- P es la probabilidad de que el programa gane la partida actual.

A medida que la máquina "observa" el desarrollo de cada partida gana experiencia. Gracias a esta experiencia acaba siendo capaz de realizar la tarea (jugar a las damas) por sí misma. Y además va comprobando el rendimiento obtenido en cada partida (si gana o no gana, en cuántos movimientos, etc), por lo que va perfeccionando su capacidad de jugar de manera eficaz.

El Aprendizaje Automático consiste en un programa informático que analiza y aprende de los datos que le proporcionamos para decidir qué hacer con ellos y proporcionar respuestas. Genera reglas para, con eso que ha "aprendido", acelerar procesos, reconocer patrones, segmentar grupos (personas, hábitos, etc). Lo fundamental es que el "cómo aprende" es automático; nosotros sólo le tenemos que dar datos o ejemplos de partida.

La definición de Aprendizaje Automático más aproximada a lo que entendemos actualmente sería:

Definición

El Aprendizaje Automático (Machine Learning) es un proceso de adquisición de conocimiento de manera automática mediante la utilización de ejemplos (experiencia) de entrenamiento.

TIPOS DE APRENDIZAJE AUTOMÁTICO

Aprendizaje Supervisado

La característica fundamental del Aprendizaje Automático Supervisado es que dicho aprendizaje se realiza **a partir de datos que ya han sido etiquetados previamente**.

¿Qué queremos decir con datos etiquetados? Pues que al programa que va a "aprender" le proporcionamos los datos indicando sus características (bien las de entrada, bien las de salida). Por ejemplo, si queremos que un programa de IA sea capaz de distinguir en qué fotos aparece un perro, al proporcionarle fotos para el aprendizaje (datos de entrada) ya le decimos en cuáles aparecen gatos, en cuáles perros y en cuáles patos... Podemos decir que "supervisamos" el aprendizaje dándole pistas al programa de Inteligencia Artificial.

En realidad el término correcto que debemos emplear es el de instancias: que son cada uno de los elementos que forman el conjunto de datos (en el ejemplo, cada foto), se componen de una serie de campos de características o atributos (en el ejemplo, aparecer gato, aparecer perro, aparecer pato...) y un campo objetivo (en el ejemplo, aparecer perro), que es el que se encuentra etiquetado en los datos de entrenamiento. El objetivo de este tipo de aprendizaje es extraer un conjunto de reglas que permitan predecir el campo objetivo para nuevos casos de estudio.

Los problemas de Aprendizaje Supervisado **se dividen en dos categorías: Regresión y Clasificación**. La diferencia entre estas dos categorías radica en el tipo de campo objetivo (lo que queremos que la Inteligencia Artificial nos dé como respuesta), que es numérico en el caso de la Regresión y categórico en el caso de la Clasificación.

Aprendizaje no Supervisado

En este tipo de aprendizaje **no se requiere un etiquetado previo** de las instancias, pues **el objetivo es encontrar relaciones de similitud, diferencia o asociación** en el conjunto de datos.

Es decir, que no "le decimos" a la Inteligencia Artificial qué estamos buscando, ni cuál es el dato concreto sobre el que queremos que haga una predicción. Asumimos que hay ciertos tipos de relación y dependencias entre los diversos datos, pero queremos que sea la Inteligencia Artificial la que encuentre esas relaciones. En muchas ocasiones nos llevamos sorpresas, cuando la IA nos muestra semejanzas entre datos que nos han pasado desapercibidas a los humanos.

Como hemos dicho, el objetivo es que la IA encuentre relaciones de tres tipos:

- Similitudes.
- Diferencias.
- Asociaciones.

Aprendizaje por Refuerzo

Existe también el Aprendizaje por Refuerzo, en el que **el objetivo es aprender cómo mapear situaciones o acciones para maximizar una cierta recompensa**. Se trata de programar agentes mediante premio y castigo sin necesidad de especificar cómo realizar la tarea.

Uno de los casos más conocidos de refuerzo automático es el cuando en la empresa Deep Mind lograron "enseñar" a jugar al Arkanoid ¿Te acuerdas? lo vimos en el punto [Historia de la IA](#). Lo hicieron con este modelo de Aprendizaje. La IA solo conocía los parámetros básicos de movimiento, y los "premios" (puntos por romper bloques, puntos por tardar lo menos posible en terminar la partida) y "castigos" (finalizar la partida sin puntos si se perdía la pelota por el extremo inferior de la pantalla).

En este tipo de problemas lo más importante es definir y programar las condiciones que deben cumplirse (las reglas del juego, qué se puede hacer y cómo interactúan unos elementos con otros). Por ejemplo, en el siguiente vídeo, podemos ver cómo una serie de personajes digitales han "aprendido" a caminar, correr y sortear obstáculos. Se ve claramente que los programadores han sido precisos para que "los brazos" se mantengan articulados al cuerpo, igual que las "patas". Pero no parece que hayan especificado mucho sobre la gravedad o sobre "el cansancio" que supone correr con los brazos hacia arriba.



Redes Neuronales Artificiales (SAA y PIA)

CONCEPTOS BÁSICOS Y FUNCIONAMIENTO

Las redes neuronales artificiales están inspiradas en la estructura y funcionamiento del cerebro humano.

i Definición

Consisten en una colección de nodos interconectados (neuronas artificiales) organizados en capas que transmiten señales entre ellas. Cada neurona recibe entradas ponderadas, las procesa mediante una función de activación y produce una salida que se envía a otras neuronas.

El proceso de aprendizaje en las redes neuronales se basa en ajustar los pesos de las conexiones entre las neuronas para que el modelo pueda realizar predicciones precisas y generalizadas en nuevos datos.

APLICACIONES Y ARQUITECTURAS COMUNES

Las redes neuronales artificiales tienen una amplia gama de aplicaciones, algunas de las cuales incluyen:

- **Reconocimiento de Patrones:** Clasificación de datos en diferentes categorías, como en reconocimiento de imágenes y diagnóstico médico.
- **Procesamiento de Lenguaje Natural:** Análisis de texto y generación de texto coherente, como en traducción automática y generación de subtítulos.
- **Juegos y Control de Robots:** Entrenamiento de agentes para jugar juegos y controlar robots mediante técnicas de aprendizaje por refuerzo.

Las arquitecturas comunes de redes neuronales incluyen:

- **Redes Neuronales Feedforward:** Son las más básicas, donde las señales solo se transmiten en una dirección, desde la entrada hasta la salida, sin ciclos.
- **Redes Neuronales Recurrentes:** Tienen conexiones cíclicas que permiten el procesamiento de secuencias de datos, lo que las hace adecuadas para tareas de procesamiento de lenguaje natural y series de tiempo.

Algoritmos Genéticos

PRINCIPIOS BÁSICOS Y FUNCIONAMIENTO

Definición

Los algoritmos genéticos son una clase de algoritmos de computación evolutiva inspirados en el proceso de selección natural. Utilizan principios biológicos como la reproducción, mutación y selección para buscar soluciones óptimas en problemas de optimización y búsqueda heurística.

En un algoritmo genético, se crea una población inicial de soluciones candidatas, y luego se evalúa su aptitud en función de una función objetivo. Las soluciones con mayor aptitud tienen una mayor probabilidad de ser seleccionadas para reproducirse y producir descendencia mediante operaciones de cruce y mutación. Este proceso se repite a lo largo de generaciones, buscando converger hacia una solución óptima.

OPTIMIZACIÓN Y BÚSQUEDA HEURÍSTICA

Los algoritmos genéticos son ampliamente utilizados en problemas de optimización y búsqueda heurística, como:

- **Problemas de Optimización:** Encontrar la mejor solución posible en un espacio de búsqueda grande, como en el diseño de redes de transporte, rutas de vehículos y programación de horarios.
- **Diseño y Aprendizaje de Parámetros:** Optimización de parámetros en modelos de aprendizaje automático y redes neuronales.

Lógica Difusa

FUNDAMENTOS Y USO EN SISTEMAS DE TOMA DE DECISIONES

La lógica difusa es una extensión de la lógica clásica que permite manejar incertidumbre y vaguedad en los datos. A diferencia de la lógica binaria (verdadero/falso), la lógica difusa utiliza grados de verdad entre 0 y 1, lo que permite representar y razonar con conceptos imprecisos.

La lógica difusa es especialmente útil en sistemas de toma de decisiones, donde las condiciones y resultados pueden ser vagos o subjetivos. Los conjuntos difusos y las reglas de inferencia difusa se utilizan para modelar y resolver problemas con datos inciertos.

VENTAJAS Y DESVENTAJAS FRENTE A LA LÓGICA CLÁSICA

Las ventajas de la lógica difusa incluyen:

- **Tratamiento de Incertidumbre:** Permite manejar datos imprecisos o ambiguos en un contexto más cercano a la forma en que los humanos toman decisiones.
- **Adaptabilidad:** Es útil para modelar sistemas complejos y no lineales donde las relaciones son difíciles de expresar con precisión.

Sin embargo, también tiene algunas desventajas:

- **Complejidad Computacional:** El procesamiento de la lógica difusa puede ser más complejo y costoso en términos computacionales en comparación con la lógica clásica.
- **Interpretación de Resultados:** La interpretación de los resultados difusos puede ser subjetiva y depender del contexto, lo que dificulta la comparación entre diferentes sistemas.

Campos de Aplicaciones de la Inteligencia Artificial

Visión por Computadora

La visión por computadora es un campo de la inteligencia artificial que se enfoca en **enseñar a las máquinas a interpretar y comprender el mundo visual**, permitiéndoles analizar y procesar imágenes y videos. Esta área ha experimentado un rápido avance en los últimos años gracias a los avances en técnicas de Aprendizaje Profundo y el aumento de la capacidad computacional. La visión por computadora tiene una amplia gama de aplicaciones en diversos campos, desde la medicina y la industria hasta la seguridad y el entretenimiento.

Los nuevos desarrollos de reconocimiento de imagen y visión artificial no han tenido un origen único y concreto, sino que se han ido conformando por la aportación de investigadores e ingenieros que han compartido sus ideas, como es el caso de **Yann Lecun**, que ideó **LeNet** usando, ya a **finales 90**, redes convolucionales para el reconocimiento de dígitos manuscritos.

El lanzamiento de **ImageNet**, abierto y gratuito, por parte de **Fei Fei Li**, que ya ha alcanzado más de 14 millones de imágenes, supuso un gran impulso al desarrollo de nuevas aplicaciones de visión artificial.



Curiosidad

Tú mismo puedes descargarte un dataset reducido (¡aunque es de 166 GB!) para probar en [Kaggle](#).

Cada vez más y mejores datasets de imágenes etiquetadas, y un mayor conocimiento y dominio de redes convolucionales, han revolucionado el campo de la visión computacional, que ha pasado de ser una cuestión de más resolución o renderizados 3D, a una cuestión más cognitiva. Se ha conseguido crear modelos que realmente entienden qué están viendo.

La visión artificial automatiza la extracción, el análisis, la clasificación y la comprensión de la información útil a partir de los datos de las imágenes. Los datos de la imagen adoptan muchas formas, como las siguientes:

- Imágenes individuales
- Secuencias de video
- Visualizaciones de varias cámaras
- Datos tridimensionales

APLICACIONES DE VISIÓN POR COMPUTADORA

Vigilancia

La utilización de cámaras para sistemas de vigilancia y seguridad, se ha extendido de forma generalizada en nuestra sociedad actual. Pero, en algunos casos, estas cámaras tienen el plus de formar parte de un sistema de inteligencia artificial.

Esta aplicación de los sistemas de reconocimiento de imagen son bastante controvertidos, pues plantean muchas dudas éticas respecto a la libertad fundamental de las personas. Es una herramienta muy útil y potente, y puede servir para beneficiar al ser humano tanto como para perjudicarlo. Lo bueno es que la comunidad en torno a la inteligencia artificial va descubriendo e ideando formas sencillas de evitar o combatir usos deshonestos de este tipo de herramientas.

En la mayoría de los casos, el software hace cálculos agregados de lo que "ve" y devuelve valores de ciertos indicadores, en vez de la imagen o fragmento de grabación con los datos personales. Solo en países donde hay regímenes autoritarios se mantienen prácticas que atentan contra los derechos de los ciudadanos.

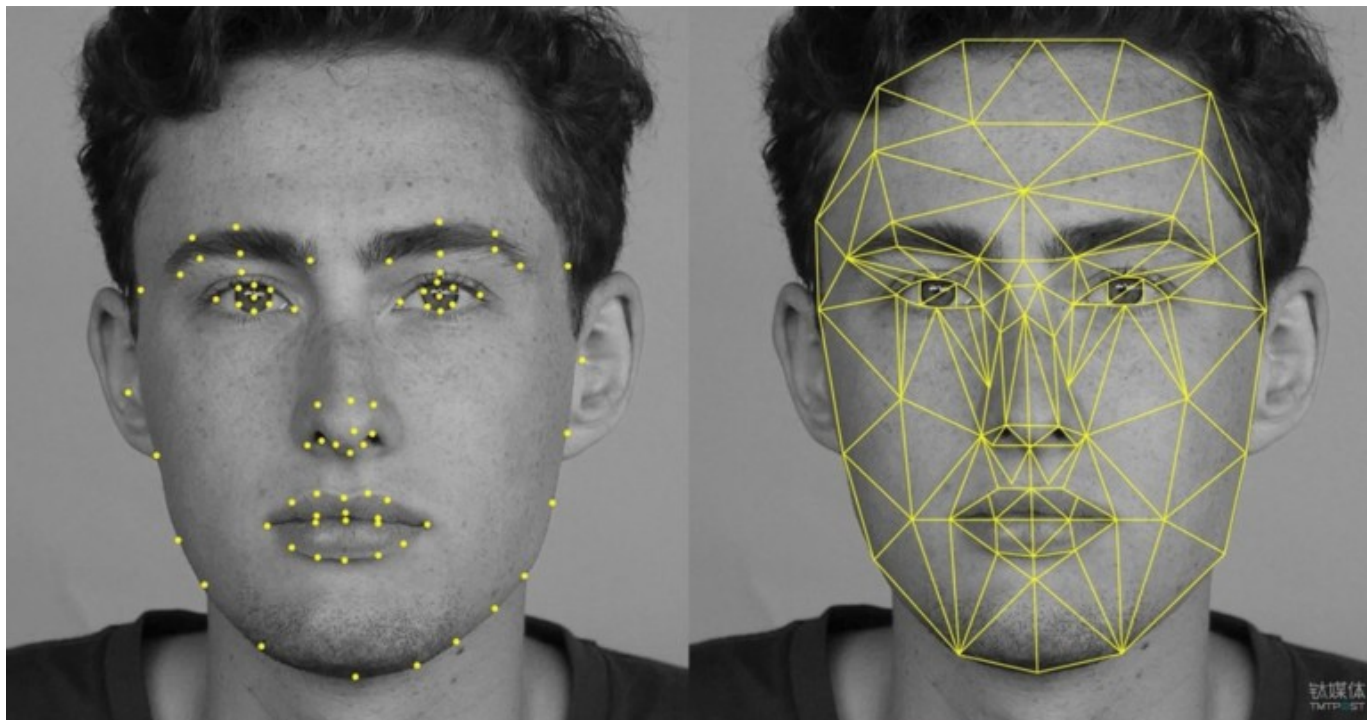
Entre las aplicaciones concretas de este tipo de sistemas, a parte de las obvias por parte de la policía o sistemas de seguridad de organizaciones, están:

- Vigilancia y control del tráfico en las ciudades.
- Cuidado de personas mayores.
- Detección de infracciones de reglas sanitarias en la industria (especialmente en la industria alimentaria).
- Monitorización del uso de infraestructuras críticas o adscritas a normas de utilización.
- Monitorización de funcionamiento y estados en líneas de producción.
- Esto son solo algunos ejemplos, pero cualquier proceso o sistema en el cual se puedan detectar anomalías a través de la imagen, sería un buen candidato para aplicar este tipo de solución.

Incluso la inteligencia artificial puede ayudar a hacer más respetuosas con la privacidad ciertas aplicaciones y herramientas que ya se estaban utilizando, como el caso del software [Cherry Home](#) de la empresa [AvantGuard](#).

Reconocimiento Facial

Un analizador facial es un software que identifica o confirma la identidad de una persona a partir del rostro. Funciona mediante la identificación y medición de los rasgos faciales en una imagen. El reconocimiento facial puede identificar rostros humanos en imágenes o videos, determinar si el rostro que aparece en dos imágenes pertenece a la misma persona o buscar un rostro entre una gran colección de imágenes existentes. Los sistemas de seguridad biométricos utilizan el reconocimiento facial para identificar de forma exclusiva a las personas durante la incorporación o el inicio de sesión de los usuarios, así como para reforzar la actividad de autenticación de estos. Los dispositivos móviles y personales también utilizan con frecuencia la tecnología de los analizadores faciales para proteger los dispositivos.



Se pueden detectar los datos faciales tanto en los perfiles frontales como en los laterales del rostro. El sistema de reconocimiento facial analiza la imagen del rostro. Asigna y lee la geometría del rostro y las expresiones faciales. Identifica los puntos de referencia faciales que son clave para distinguir un rostro de otros objetos. La tecnología de reconocimiento facial por lo general busca lo siguiente:

- Distancia entre los ojos
- Distancia de la frente a la barbilla
- Distancia entre la nariz y la boca
- Profundidad de las cuencas oculares
- Forma de los pómulos
- Contorno de los labios, las orejas y la barbilla

El sistema convierte los datos de reconocimiento facial en una cadena de números o puntos denominada huella facial. Cada persona tiene una huella facial única, de forma similar a una huella dactilar. La información utilizada por el reconocimiento facial también se puede utilizar a la inversa para reconstruir digitalmente el rostro de una persona.

El reconocimiento facial puede identificar a una persona al comparar los rostros de dos o más imágenes y evaluar la probabilidad de que coincidan. Por ejemplo, puede verificar que el rostro mostrado en una autofoto tomada con la cámara de un móvil coincide con el rostro de una imagen de un documento de identidad emitido por el gobierno, como un permiso de conducir o un pasaporte, así como verificar que el rostro que aparece en la autofoto no coincide con un rostro de un conjunto de rostros capturados previamente.



Ejemplo

Aplicación de Visión por Computadora:

Un ejemplo práctico de aplicación de visión por computadora es el reconocimiento facial utilizado en aplicaciones de seguridad y desbloqueo de dispositivos. En este caso:

- **Entrada:** La entrada es una imagen o un video que contiene rostros humanos.
- **Procesamiento de Imagen:** El sistema de visión por computadora procesa la imagen para detectar y extraer características clave del rostro, como ojos, nariz, boca, etc.
- **Aprendizaje Automático:** Las características del rostro se utilizan como entrada para un modelo de aprendizaje automático previamente entrenado. El modelo clasifica las características y compara con una base de datos de rostros previamente almacenados.
- **Salida:** Como resultado, el sistema identifica o verifica la identidad del individuo y permite el acceso o desbloqueo según los resultados.

Conducción autónoma.

El sistema de conducción autónoma de vehículos implica varias tareas y subsistemas, pero uno de los más importantes, es el de visión artificial, pues la mayoría de las decisiones de seguridad del coche se basan en lo que captan las cámaras.

La cuestión crítica en estos sistemas, es el reconocimiento de señales de tráfico u objetos/obstáculos alrededor del vehículo a una velocidad relativamente alta. Por ejemplo, si el coche debe parar porque hay una persona cruzando la carretera, el sistema de visión debe captar la imagen con antelación suficiente como para frenar a una distancia también suficiente.



Captar cómo son las líneas de la carretera para ir girando lo que corresponda, también requiere ir captando esas variaciones de trayectoria suficientemente rápido, pues en carretera es muy común ir a velocidades altas. De hecho, hay sistemas que, a partir de cierta velocidad, no permiten usar la función de conducción autónoma.



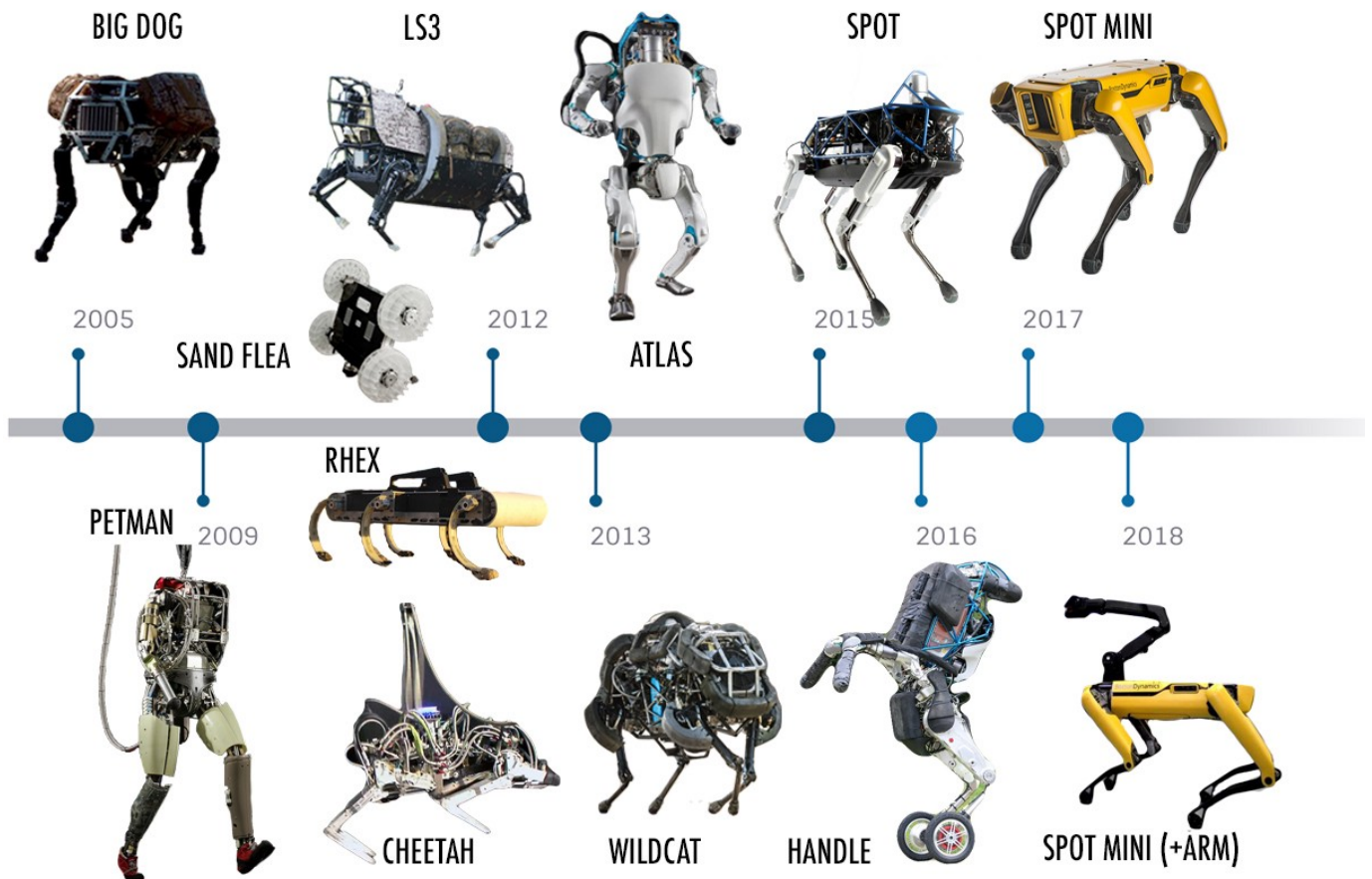
Curiosidad

Si quieres conocer mejor la clasificación de niveles de conducción autónoma y cómo se relaciona el ámbito de la visión artificial con ellos, te recomendamos leer el artículo "[Autonomous Vehicles Are Driving Computer Vision Into the Future](#)".

Sistema auxiliar en robots.

Los robots son sistemas complejos que suelen ejecutar una serie de tareas en el mundo físico en base a una secuencia programada. En la industria, se han estado utilizando sistemas robóticos desde hace muchos años. Pero este campo también ha ido evolucionando, y la inteligencia artificial está aportando grandes avances que causan un importante impacto en el alcance de estos sistemas.

BOSTON DYNAMICS



Un sistema robótico tiene tres partes fundamentales:

- Sensores o entradas.
- Sistema de control.
- Actuadores.

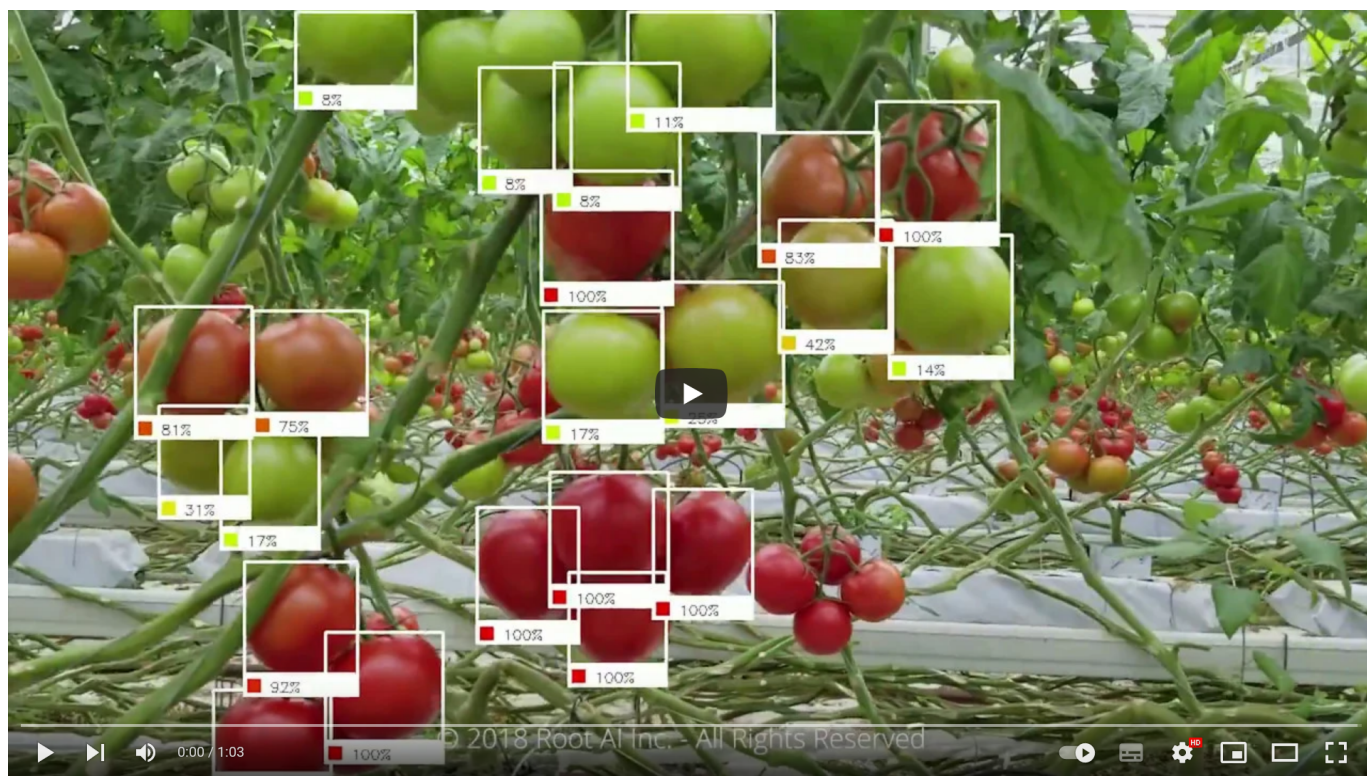
Más adelante hablaremos de cómo los sistemas de control se han beneficiado de la inteligencia artificial, pero aquí nos detenemos en el módulo de visión artificial como parte del conjunto de sensores que aportan los estímulos o la información que el sistema robótico va a necesitar para la toma de decisiones.

El sistema de visión artificial de un robot, le permite detectar objetos y posicionarse a sí mismo o a objetos que transporta en función de lo que está viendo. Esto es un gran avance respecto a otros sistemas de posicionamiento, que exigían constantes tareas de calibración y ralentizaban las tareas del robot.

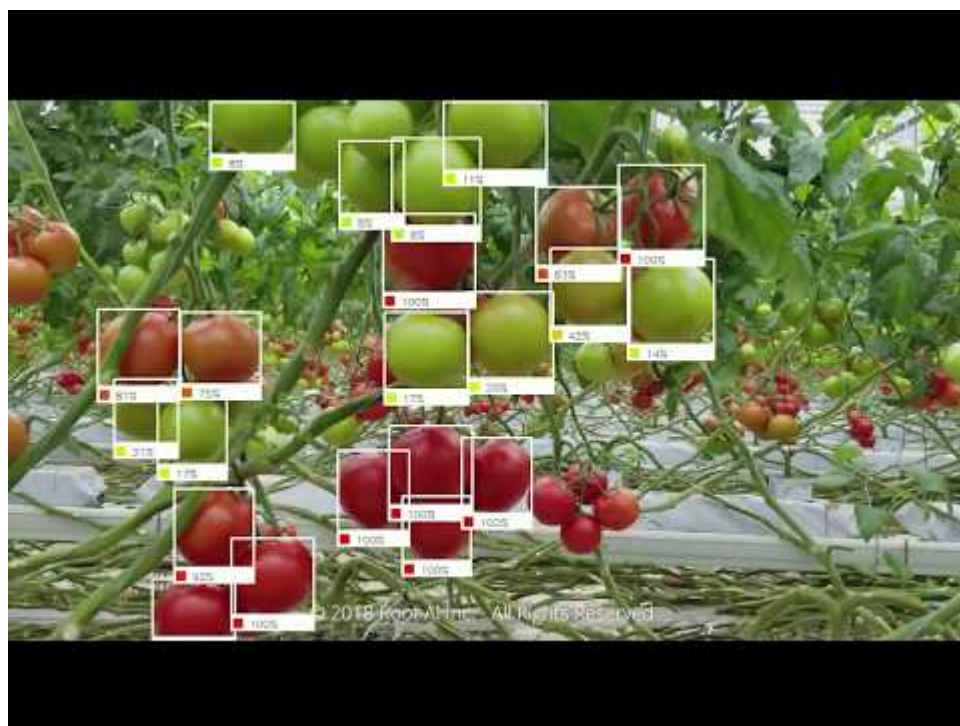
<https://bostondynamics.com/videos/>

Reconocimiento de Objetos

El reconocimiento de objetos implica identificar y localizar objetos específicos en imágenes o videos. Algunos ejemplos incluyen reconocimiento de vehículos en carreteras, detección de peatones en sistemas de asistencia al conductor y clasificación de objetos en aplicaciones de etiquetado automático.



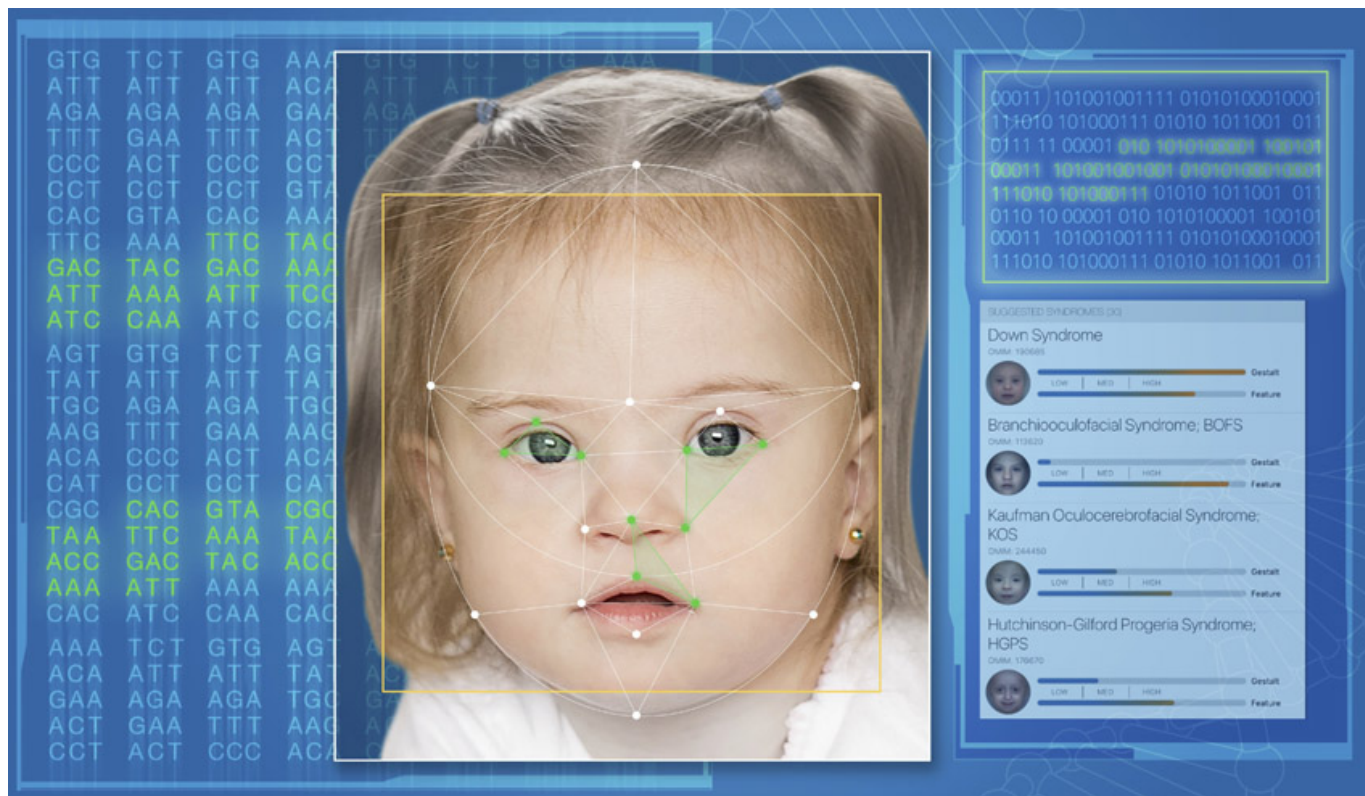
En la industria agroalimentaria, la capacidad de visión inteligente es de vital importancia, porque constituye una parte decisiva de cara a que se obtenga un buen producto o una buena cosecha. En algunos casos, el robot sabe distinguir, mejor que el humano, si una fruta está en su momento óptimo de cosecha.



Detección y diagnóstico

En el campo de la medicina, la inteligencia artificial está teniendo un impacto de muchísimo valor, porque no solo consigue mejorar la vida de las personas, es que, literalmente, en algunos casos consigue salvar vidas. Es el caso de herramientas de inteligencia artificial para la detección y diagnóstico de enfermedades a través de imágenes.

Existen muchos programas informáticos de apoyo y ayuda al diagnóstico que han ido mejorando su aprendizaje a través de su uso repetido y continuado. Actualmente existen diferentes tipos de software que se pueden aplicar a diferentes grupos de enfermedades como MYCIN/MYCIN II para enfermedades infecciosas, CASNET para oftalmología, PIP para enfermedades renales o AI/RHEUM para enfermedades reumatológicas. La empresa FDNA a través de su software de reconocimiento facial Face2Gene® es capaz de apoyar o sospechar el diagnóstico de más de 8.000 enfermedades raras, con un reciente ensayo clínico desarrollado en Japón con buenos resultados.



En el campo del procesamiento y la interpretación de imágenes para el diagnóstico, la IA ofrece algoritmos que mejoran la calidad y la precisión del diagnóstico ya que los métodos de IA son excelentes para reconocer automáticamente patrones complejos en los datos de imágenes, elimina ruido en las imágenes ofreciendo una mayor calidad y permite establecer modelos tridimensionales a partir de imágenes de pacientes concretos.

Investigadores de IBM publicaron una investigación en torno a un nuevo modelo de IA que puede predecir el desarrollo del cáncer de mama maligno, con tasas comparables a las de los radiólogos humanos. Este algoritmo aprende y toma decisiones tanto de datos de imágenes como del historial de la paciente, pudo predecir correctamente el desarrollo del cáncer de mama en el 87% de los casos analizados, y también pudo interpretar el 77% de los casos no cancerosos. Este modelo podría algún día ayudar a los radiólogos a confirmar o negar casos positivos de cáncer de mama. Si bien los falsos positivos pueden causar una enorme cantidad de estrés y ansiedad indebidos, los falsos negativos a menudo pueden obstaculizar la detección temprana y el tratamiento posterior de un cáncer. Cuando se puso a prueba frente a 71 casos diferentes que los radiólogos habían determinado originalmente como «no malignos», pero que finalmente terminaron siendo diagnosticados con cáncer de mama dentro del año, el sistema de IA pudo identificar correctamente el cáncer de mama en el 48% de las personas (48% de los 71 casos), que de lo contrario no se habrían detectado (Fuente: "[La inteligencia artificial y sus aplicaciones en medicina](#)")

Procesos creativos

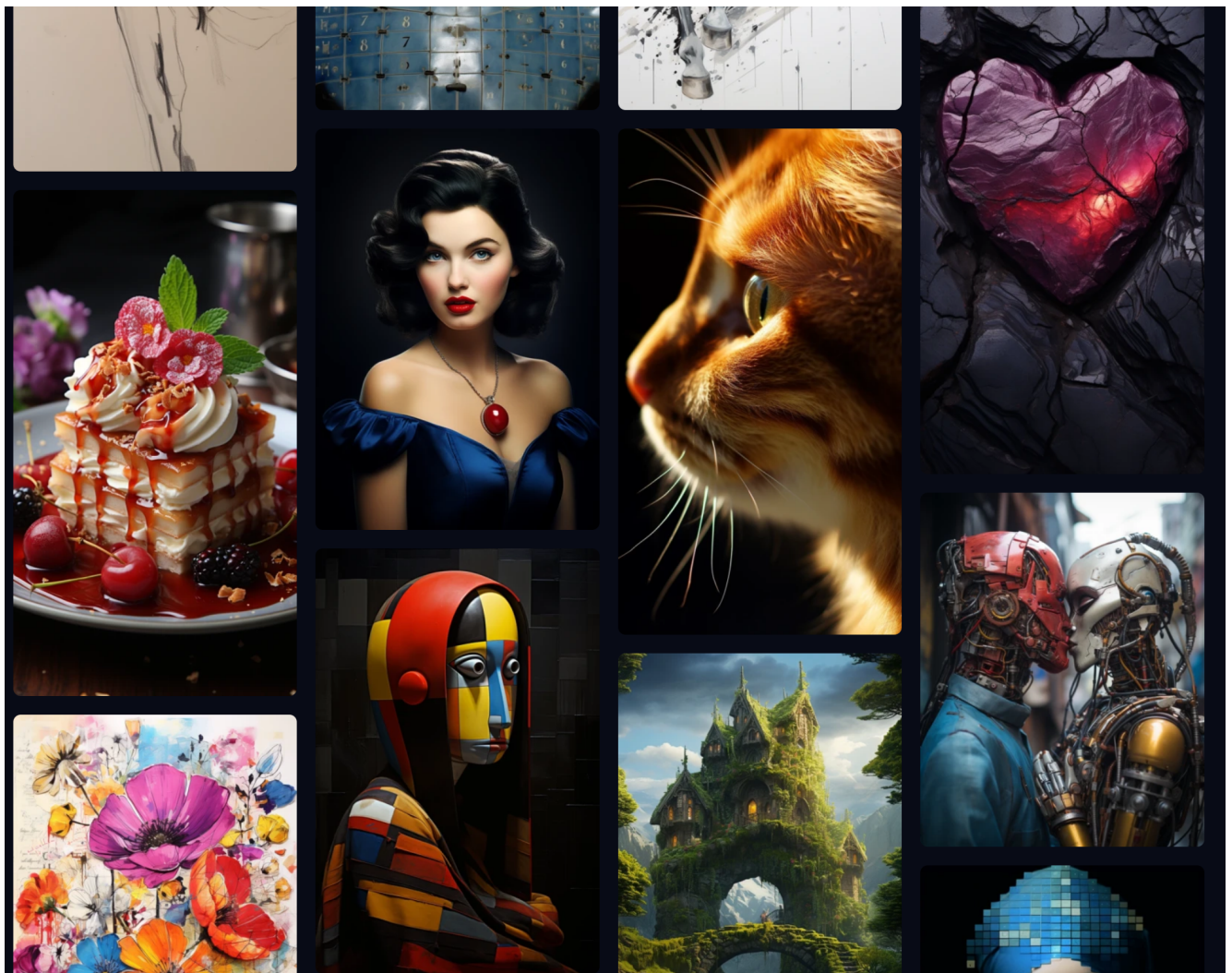
Uno de los grandes e inesperados avances de la computación de la década pasada, ha sido el de los modelos generativos: las redes GAN para el campo de la imagen y los modelos de generación de texto basados en Transformers.

- **Deep Dream:** En 2015 apareció DeepDream, un modelo de generación de imágenes creado por Google. El software Deep Dream fue desarrollado para el imageNet large scale visual recognition challenge (ILSVRC). Este era un desafío reto, propuesto a diferentes equipos de investigación, que consistió en crear un sistema de reconocimiento de objetos y su localización dentro de una misma imagen, aparte de su detección inmediata. En este Desafío se adjudicó a Google el primer premio en el año 2014, logrado gracias al uso del entrenamiento de redes neuronales. En junio de 2015 Google publicó la

investigación, y tras esto hizo su código fuente abierto utilizado para generar las imágenes en un IPython notebook. Con esto se permitió que las imágenes de la red neuronal pudiesen ser creadas por cualquiera. Actualmente, se puede utilizar la aplicación de manera online en la web [DeepDreamGenerator](https://deepdreamgenerator.com/). Básicamente, el algoritmo procesa la imagen dada identificando sus elementos, para utilizar una transferencia de estilos respetando la identidad esencial de la imagen original.

- **Gaugan:** GauGAN es una herramienta con la que se pueden crear paisajes falsos partiendo de un boceto. Este software de Nvidia, hace uso de una red de confrontación generativa (GAN), basado en una técnica denominada "normalización espacialmente adaptativa" que es capaz de generar imágenes realistas a partir de un determinado diseño semántico, controlado por el usuario con el uso de un programa de edición de imágenes, donde cada color actúa como representación de un tipo de objeto, material o ambiente. Se puede utilizar desde su interfaz web abierta.
- **DALL·E:** Una de las más recientes incorporaciones al ámbito de "cosas increíbles que la inteligencia artificial puede hacer ya" es el modelo de generación de imágenes de openAI. Se trata de una implementación multimodal de GPT3. El algoritmo interpreta una descripción escrita que se le proporciona a través de su interfaz, y genera la imagen correspondiente en base a lo que sus 12 mil millones de parámetros del modelo GPT3 han interpretado de la entrada de texto dada. En concreto, se utiliza un proceso llamado "diffusion" que parte de una imagen de ruido aleatoria y va alterando dicho esquema de puntos en función de que vaya reconociendo distintos patrones de objetos cuyas palabras clave se le han dado en la descripción.
- **MidJourney:** Midjourney es un laboratorio independiente de investigación y el nombre de un programa de inteligencia artificial con el cual sus usuarios pueden crear imágenes a partir de descripciones textuales, similar a Dall-e de OpenAI y al Stable Diffusion de código abierto.

La herramienta funcionó bajo versión de beta cerrada hasta que el 13 de julio de 2022 el laboratorio anunció el comienzo de una beta abierta. El equipo de Midjourney está dirigido por David Holz, cofundador de Leap Motion. Midjourney emplea un modelo de negocio *freemium*, con un nivel gratuito limitado y niveles de pago que ofrecen un acceso más rápido, mayor capacidad y funciones adicionales. Los usuarios pueden crear obras de arte con Midjourney dando órdenes a un bot alojado en Discord, ya sea enviando mensajes directos o invitando a dicho bot a un servidor de terceros.



Procesamiento del Lenguaje Natural (PLN)

El Procesamiento del Lenguaje Natural (PLN) es una rama de la Inteligencia Artificial que se enfoca en permitir a las máquinas entender y procesar el lenguaje humano en forma escrita o hablada. El PLN permite que las computadoras analicen, comprendan y generen texto de manera similar a como lo hacen los seres humanos. Esta tecnología ha sido fundamental en el desarrollo de asistentes virtuales, traducción automática, análisis de sentimientos y muchas otras aplicaciones útiles en el ámbito empresarial y cotidiano.

Tratar computacionalmente una lengua implica un proceso de modelización matemática. Los ordenadores solo entienden de bytes y dígitos y los informáticos codifican los programas empleando lenguajes de programación como C, Python o Java.

Los lingüistas computacionales se encargan de la tarea de "preparar" el modelo lingüístico para que los ingenieros informáticos lo implementen en un código eficiente y funcional.

Éstos son algunos de los **componentes** del procesamiento del lenguaje natural. No todos los análisis que se describen se aplican en cualquier tarea de PLN, sino que depende del objetivo de la aplicación.

- **Análisis morfológico o léxico.** Consiste en el análisis interno de las palabras que forman oraciones para extraer lemas, rasgos flexivos, unidades léxicas compuestas. Es esencial para la información básica: categoría sintáctica y significado léxico.
- **Análisis sintáctico.** Consiste en el análisis de la estructura de las oraciones de acuerdo con el modelo gramatical empleado (lógico o estadístico).
- **Análisis semántico.** Proporciona la interpretación de las oraciones, una vez eliminadas las ambigüedades morfosintácticas.
- **Análisis pragmático.** Incorpora el análisis del contexto de uso a la interpretación final. Aquí se incluye el tratamiento del lenguaje figurado (metáfora e ironía) como el conocimiento del mundo específico necesario para entender un texto especializado.

Un análisis morfológico, sintáctico, semántico o pragmático se aplicará dependiendo del objetivo de la aplicación. Por ejemplo, un conversor de texto a voz no necesita el análisis semántico o pragmático. Pero un sistema conversacional requiere información muy detallada del contexto y del dominio temático.



Curiosidad

No te recuerda a algo?



Estas son las fases de un Compilador

APLICACIONES DE PROCESAMIENTO DEL LENGUAJE NATURAL

Asistentes Virtuales y Chatbots

Los asistentes virtuales como Siri, Google Assistant y Alexa utilizan PLN para entender y responder a las consultas y comandos de voz de los usuarios. Los chatbots en aplicaciones de servicio al cliente y soporte técnico también emplean PLN para ofrecer respuestas automáticas y contextuales a las preguntas de los usuarios.

Esta generación actual de asistentes están habilitados para llevar a cabo tareas dentro del sistema que las aloja e, incluso, a través de webhooks, en otros sistemas que cuenten con las políticas de acceso correspondientes. De esta forma, los asistentes virtuales más avanzados pueden encargarse de una pizza, comprar online un producto entre varias sugerencias o incluso controlar la domótica de nuestra casa.

Generación de textos

El área del marketing y comunicación ha sido de los primeros que ha abrazado la inteligencia artificial para automatizar y mejorar muchos de sus procesos. Y entre las distintas tareas que puede llevar a cabo la IA, la generación de textos empezó a tener una aplicación comercial clara como herramienta para crear mensajes publicitarios, publicaciones de marketing de contenidos o incluso lemas de producto. Por eso, durante estos años han ido surgiendo una gran cantidad de servicios de este tipo. Siempre para generar breves fragmentos, y con una serie de requerimientos, como incluir las palabras clave.

Pero, recientemente, están surgiendo modelos mucho más generales y con una versatilidad mayor en el tipo de textos, el tema a tratar, el idioma, etc. Es el caso de los modelos BERT, GPT3 y Bloom. El primero, creado por Google en 2018, integrado en el algoritmo de búsqueda de Google y publicado con licencia de código abierto, no tiene una aplicación directa de generación de textos, pero tiene la misma arquitectura que los modelos que se están utilizando en ese campo: los Transformers.

Interpretación de textos

En el campo del análisis de lenguaje natural existen aplicaciones basadas en voz, como los asistentes virtuales que tenemos encima de la mesa, en el móvil o en el ordenador, o las aplicaciones basadas en texto. Ambas utilizan la misma base, y después, para añadir la habilitación oral, se utiliza un módulo de "Voz a Texto" y viceversa.

Las aplicaciones de PLN sirven para extraer información valiosa de los datos sin estructurar basados en textos y para acceder a la información extraída con el objetivo de generar una nueva comprensión de esos datos. Algunos ejemplos de aplicación serían:

- Traducción automática de idiomas.
- Chatbots.
- Opinión de los clientes: Se usa el análisis de entidades para identificar y etiquetar campos en documentos y canales. De esta forma, se pueden conocer mejor las opiniones de los clientes y obtener información valiosa sobre los productos y la experiencia de usuario.
- Comprender los recibos y las facturas: Extrae entidades para identificar las entradas más comunes de los recibos y facturas, como las fechas o los precios, y entiende la relación entre la solicitud y el pago.
- Análisis de documentos: Utiliza la extracción de entidades personalizada para identificar las entidades específicas de cada dominio en los documentos sin tener que invertir tiempo o dinero en análisis manuales.
- Clasificación de contenido general: Clasifica los documentos en función de las entidades más frecuentes, las entidades personalizadas de un dominio concreto o categorías generales disponibles (por ejemplo, deportes y entretenimiento).
- Análisis de tendencias: Agrega noticias con texto que permita a los profesionales del marketing extraer contenido relevante sobre sus marcas de noticias online, artículos y otras fuentes de datos.
- Sanidad: Mejora la documentación clínica, la investigación de minería de datos y los informes de registros automatizados para agilizar los ensayos clínicos.

El campo del procesamiento del lenguaje natural es considerado uno de los grandes retos de la inteligencia artificial ya que es una de las tareas más complicadas y desafiantes: ¿cómo comprender realmente el significado de un texto? ¿cómo intuir neologismos, ironías, chistes o poesía?



Ejemplos

Análisis de Sentimientos

El PLN es utilizado para analizar el contenido de opiniones, comentarios y reseñas de usuarios en línea y determinar si expresan sentimientos positivos, negativos o neutrales. Esto es útil para medir la satisfacción del cliente, realizar estudios de mercado y realizar análisis de reputación de marca.

Ejemplo de Aplicación de Procesamiento del Lenguaje Natural: Análisis de Sentimientos

Un ejemplo práctico de aplicación de Procesamiento del Lenguaje Natural es el análisis de sentimientos en comentarios de productos en línea. En este caso:

- **Entrada:** La entrada es un conjunto de comentarios escritos por usuarios sobre un producto específico.
- **Procesamiento de Lenguaje Natural:** El PLN procesa el texto para tokenizarlo (dividirlo en palabras), eliminar palabras irrelevantes (stopwords) y realizar lematización o extracción de raíces para reducir las palabras a su forma base.
- **Análisis de Sentimientos:** Se utilizan técnicas de análisis de sentimientos para asignar un valor de sentimiento (positivo, negativo o neutral) a cada comentario en función de las palabras y frases utilizadas.
- **Salida:** Como resultado, se obtiene un resumen del sentimiento general de los usuarios hacia el producto, lo que permite a las empresas identificar puntos fuertes y áreas de mejora, así como tomar decisiones basadas en la retroalimentación del cliente.

Análítica avanzada

Los Modelos Predictivos son un grupo de técnicas que, mediante los campos del aprendizaje automático, la recolección de datos históricos, el Big Data y el reconocimiento de patrones, pretende dar una predicción de resultados futuros; con el objetivo de precisar la toma de decisiones mediante técnicas de análisis de datos. En los últimos años el área predictiva ha tomado gran protagonismo en los negocios, la medicina, los servicios financieros, las políticas gubernamentales, la publicidad, la mercadotecnia, las redes sociales y gran cantidad de campos de aplicación.

Se basa, principalmente en datos organizados tabularmente. Es decir, hablamos de datos estructurados y bases de datos relacionales en la mayoría de los casos. Se busca el patrón de comportamiento y la tendencia escondida en las relaciones entre diferentes variables de un sistema, y, a través de aprendizaje supervisado, con modelos de regresión y de clasificación, se obtienen predicciones que ayudan a la toma de decisiones en la organización.

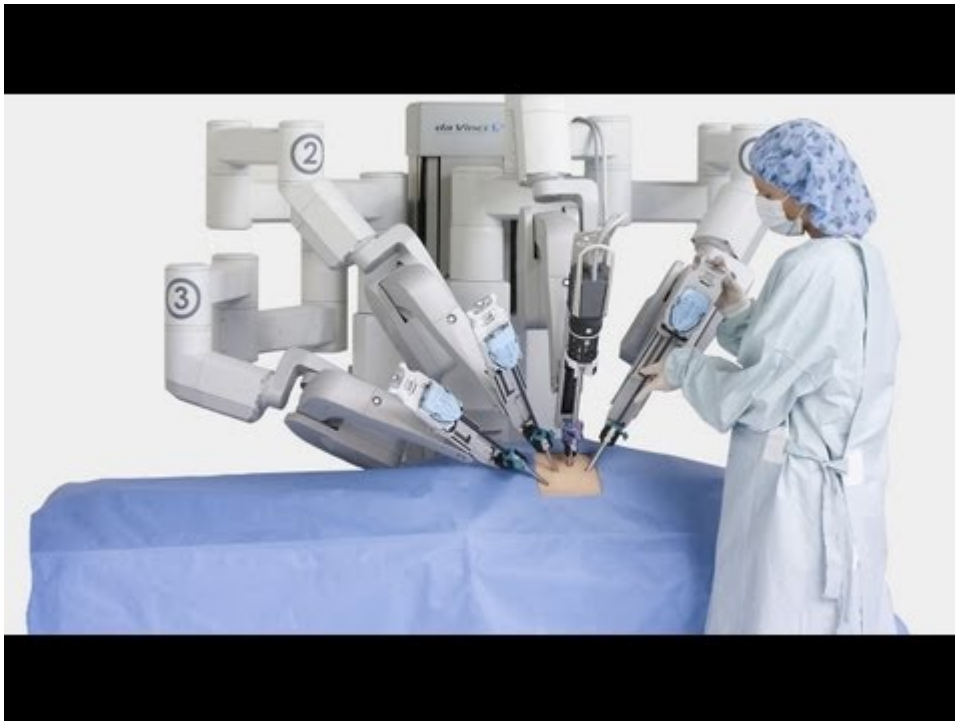
Cada vez van siendo más utilizados también los modelos de aprendizaje automático no supervisado, como el "clustering", que son el alma de sistemas de recomendación en plataformas de contenido online o comercio electrónico.

Los modelos predictivos tienen gran aplicabilidad en todos los sectores comerciales. Son capaces de resolver muchos problemas que antes eran irresolubles.

Robótica e Inteligencia Artificial

La integración de la IA en la robótica ha llevado a la creación de robots cada vez más inteligentes y autónomos, lo que ha revolucionado diversas áreas de aplicación.

Los sistemas robóticos actuales cubren una gran cantidad de campos de aplicación del entorno. En todos ellos, la inteligencia artificial mejora su desempeño y permite acometer nuevas tareas. Entre ellos, destacan muchos robots cuya principal herramienta basada en inteligencia artificial es el módulo de visión artificial, como es el caso de Davinci, el robot cirujano, o agro-bot, el robot que recoge fresas en su punto óptimo de madurez.



La inteligencia artificial tiene un impacto especialmente relevante en el sistema de control del robot.

APLICACIONES DE LA IA EN LA ROBÓTICA

Robots sociales.

Este tipo de robots tienen muy desarrollados los módulos sensoriales, es decir, el de reconocimiento de imagen, procesamiento de lenguaje natural, y su sistema de control es, básicamente, un asistente virtual que tiene ciertas opciones de movilidad y acciones remotas o conectadas, como encender la luz o hacer una llamada de emergencia.

Casas y ciudades inteligentes.

Estos sistemas robóticos cuentan con un elaborado sistema de sensores, y un hardware extendido por diversas localizaciones, lo que hace necesario contar, a menudo con microcontroladores que hagan parte del procesamiento de la información que captan y luego lo envíen al controlador principal. Por ejemplo, un sistema en el que tenemos cámaras que monitorizan las ventanas, en el propio microcontrolador de la cámara se puede hacer la tarea de reconocer la imagen de "ventana abierta", que la unidad principal recibirá junto a otros datos de interés como si está lloviendo o si hace frío, para accionar un actuador que haga saltar una alerta en el móvil de la persona propietaria o incluso que accione un motor para cerrarla.

Conducción autónoma.

La unidad de control de un vehículo autónomo es el paradigma de las técnicas más avanzadas en aprendizaje automático. Se trata de aprendizaje por refuerzo, y se entrena en simuladores virtuales hasta que el sistema tiene un comportamiento más o menos aceptable como para probarlo de forma segura en circuitos de pruebas reales.

Existen vehículos autónomos desde hace bastante tiempo, como es el caso de los drones, e incluso el sistema de control de navegación de los aviones cuenta con muchos automatismos, pero, probablemente, un coche autónomo, hoy por hoy, es el sistema más espectacular en tanto debe lidiar con muchos obstáculos y reglas de circulación.

Robots Colaborativos

Los robots colaborativos, también conocidos como cobots, son robots diseñados para trabajar de forma segura y eficiente junto a los seres humanos. Estos robots se utilizan en la industria para aumentar la productividad y mejorar la seguridad en la colaboración humano-robot. Por ejemplo:

- **Ensamblaje de Productos:** En líneas de ensamblaje de automóviles o electrónicos, los cobots pueden trabajar junto a los operadores humanos para realizar tareas repetitivas y pesadas, mejorando la eficiencia y reduciendo la fatiga de los trabajadores.
- **Embalaje y Logística:** En almacenes y centros de distribución, los cobots pueden colaborar con los trabajadores en la selección, embalaje y envío de productos, agilizando los procesos logísticos.

Robótica Médica

La robótica médica ha revolucionado la cirugía y la asistencia médica, permitiendo procedimientos más precisos y menos invasivos. Algunos ejemplos de aplicaciones de la robótica médica son:

- **Cirugía Asistida por Robot:** Los robots quirúrgicos, controlados por cirujanos, pueden realizar movimientos más precisos y estables durante procedimientos complejos, reduciendo el riesgo de errores y acelerando la recuperación del paciente.
- **Rehabilitación Robótica:** Los robots de rehabilitación se utilizan para asistir en la terapia de pacientes con discapacidades físicas, proporcionando movimientos controlados y repetitivos para mejorar la recuperación.
- **Cuidados de Pacientes:** Los robots asistenciales pueden ayudar a los pacientes en tareas diarias, como levantarse, caminar y tomar medicamentos, brindando una mayor autonomía y apoyo en el cuidado de la salud.

Robots Autónomos

Los robots autónomos son máquinas que pueden operar de manera independiente en entornos desconocidos o adversos sin intervención humana directa. Algunos ejemplos de aplicaciones de robots autónomos incluyen:

- **Exploración Espacial:** Los robots autónomos se utilizan en misiones espaciales para explorar planetas, asteroides y lunas, recopilando datos valiosos sin la necesidad de una comunicación constante con la Tierra.
- **Búsqueda y Rescate:** En situaciones de desastres naturales o emergencias, los robots autónomos pueden buscar y localizar supervivientes en áreas peligrosas o inaccesibles para los equipos de rescate humanos.

Ciencia de datos y Data Mining

La ciencia de datos es una de las disciplinas en las que la inteligencia artificial ha generado un mayor impacto, permitiendo detectar patrones y relaciones mediante métodos no supervisados, y llevar a cabo agrupaciones y heurísticos. Todo ello será visto con detalle en el módulo SAA, donde se encontrarán los algoritmos y aplicaciones más conocidos. En este campo se engloban también los heurísticos y los detectores de anomalías para planes de mantenimiento industrial.



Ejemplo

Minería de datos o Data mining

No es raro ver cómo se usan indiferentemente los conceptos minería de datos y machine learning. Son conceptos "primos hermanos", pero no son lo mismo.

La principal diferencia radica en el objetivo que tiene cada una de las disciplinas. Mientras que la minería de datos descubre patrones anteriormente desconocidos, el Machine Learning se usa para reproducir patrones conocidos y hacer predicciones basadas en los patrones.

En pocas palabras se podría decir que la minería de datos tiene una función exploratoria mientras que el machine learning se focaliza en la predicción.

Ciberseguridad

A día de hoy, la IA juega un papel fundamental en el campo de la ciberseguridad. Algunos de los usos más destacados de la IA en ciberseguridad incluyen:

1. **Detección de amenazas avanzadas:** La IA se utiliza para analizar grandes volúmenes de datos y detectar patrones de comportamiento anómalos que puedan indicar actividades maliciosas. Los sistemas de detección de intrusiones basados en IA pueden identificar amenazas avanzadas y desconocidas que los enfoques tradicionales podrían pasar por alto.
2. **Prevención de ataques de phishing:** Los algoritmos de aprendizaje automático pueden analizar correos electrónicos y sitios web en busca de indicios de phishing y ayudar a bloquear o filtrar contenido malicioso antes de que llegue a los usuarios.
3. **Identificación de malware:** Los sistemas de IA pueden analizar el comportamiento de archivos y aplicaciones para detectar malware y ransomware. Además, la IA puede mejorar la identificación y clasificación de nuevas variantes de malware desconocido.
4. **Autenticación de usuarios:** La IA se utiliza en sistemas de autenticación biométrica y de reconocimiento facial para mejorar la seguridad en el acceso a dispositivos y servicios.
5. **Análisis de logs y eventos de seguridad:** Los sistemas de IA pueden analizar y correlacionar grandes cantidades de registros y eventos de seguridad para identificar patrones de actividad sospechosa y facilitar la respuesta a incidentes.
6. **Predicción y prevención de brechas de seguridad:** Mediante el análisis de datos históricos y la identificación de vulnerabilidades conocidas, la IA puede predecir posibles brechas de seguridad y ayudar a prevenir futuros ataques.
7. **Automatización de tareas de seguridad:** La IA se utiliza para automatizar tareas repetitivas en ciberseguridad, como la gestión de parches, el análisis de vulnerabilidades y la respuesta a incidentes, lo que permite a los profesionales de seguridad centrarse en tareas más complejas.
8. **Protección de redes y sistemas IoT:** La IA puede monitorear y proteger redes empresariales y dispositivos IoT (Internet de las cosas) para detectar y prevenir actividades maliciosas.

Ejemplos de soluciones reales en ciberseguridad basadas en IA incluyen:

- **Cylance:** Una plataforma de prevención de ataques basada en IA que utiliza algoritmos de aprendizaje automático para proteger contra malware y ransomware.
- **Darktrace:** Un sistema de detección de amenazas basado en IA que utiliza algoritmos de inteligencia artificial para identificar y responder a comportamientos anómalos en tiempo real.
- **IBM Watson for Cyber Security:** Una solución de seguridad cibernética que utiliza IA para analizar grandes cantidades de datos y ayudar a identificar y responder a amenazas de manera más rápida y precisa.
- **BioCatch:** Una solución de autenticación biométrica que utiliza IA para analizar el comportamiento del usuario y detectar actividades fraudulentas.



Nuevas Formas de Interacción

Interfaces de Voz

Definición

Las interfaces de voz son una forma de interacción con sistemas de Inteligencia Artificial que permiten a los usuarios comunicarse mediante comandos de voz en lugar de texto o clics.

Estas interfaces han experimentado un crecimiento significativo en popularidad gracias a avances en el procesamiento del lenguaje natural y la tecnología de reconocimiento de voz. A continuación, se desarrollarán algunas aplicaciones y ejemplos de interfaces de voz.

ASISTENTES VIRTUALES

Los asistentes virtuales son aplicaciones de inteligencia artificial que permiten a los usuarios realizar tareas y obtener información a través de comandos de voz. Algunos ejemplos de asistentes virtuales populares incluyen:

- **Siri:** Desarrollado por Apple, Siri es un asistente virtual que se encuentra integrado en dispositivos como iPhones, iPads y Macs. Puede responder preguntas, enviar mensajes, configurar alarmas, hacer llamadas y más.
- **Google Assistant:** Desarrollado por Google, Google Assistant está disponible en dispositivos Android y en otros dispositivos como Google Home. Puede proporcionar información en tiempo real, realizar búsquedas en la web, establecer recordatorios y controlar dispositivos inteligentes del hogar.
- **Amazon Alexa:** Alexa es el asistente virtual de Amazon y está presente en dispositivos como el Amazon Echo. Permite realizar compras en línea, reproducir música, controlar luces y termostatos inteligentes, y realizar muchas otras tareas.

SISTEMAS DE NAVEGACIÓN

Las interfaces de voz también se utilizan en sistemas de navegación para proporcionar instrucciones de conducción en tiempo real. Algunos ejemplos incluyen:

- **Google Maps:** La popular aplicación de mapas y navegación utiliza el reconocimiento de voz para que los conductores reciban instrucciones mientras mantienen la vista en la carretera.

- **Sistemas de Navegación Integrados en Automóviles:** Muchos automóviles modernos están equipados con sistemas de navegación por voz que permiten a los conductores obtener direcciones y encontrar lugares de interés sin quitar las manos del volante.

APLICACIONES DE ACCESIBILIDAD

Las interfaces de voz también juegan un papel crucial en la accesibilidad tecnológica para personas con discapacidades visuales o motrices. Algunos ejemplos de aplicaciones de accesibilidad incluyen:

- **Lectores de Pantalla:** Estas aplicaciones utilizan la voz para leer en voz alta el contenido de la pantalla de un dispositivo, permitiendo que las personas con discapacidades visuales puedan interactuar con la tecnología.
- **Comandos de Voz para Controlar Dispositivos:** Las interfaces de voz permiten a personas con discapacidades motrices controlar dispositivos y realizar tareas sin la necesidad de utilizar las manos.

Interfaces Cerebro-Computadora (BCI)

Definición

Las Interfaces Cerebro-Computadora (Brain-Computer Interface BCI) son tecnologías avanzadas que permiten la comunicación directa entre el cerebro humano y dispositivos tecnológicos. A través del registro y análisis de señales cerebrales, estas interfaces posibilitan que las personas controlen dispositivos y sistemas mediante su actividad cerebral.

Algunas aplicaciones destacadas son:

ASISTENCIA MÉDICA

Las BCI han abierto nuevas posibilidades para asistir a personas con discapacidades motoras. Estas interfaces permiten a individuos con lesiones espinales, amputaciones u otras condiciones que afectan su capacidad de movimiento, controlar prótesis y dispositivos asistenciales mediante señales cerebrales. Por ejemplo, una persona con una extremidad amputada podría usar una prótesis controlada por BCI para realizar movimientos precisos y naturales, restaurando parte de su funcionalidad física.

NEUROFEEDBACK

El neurofeedback es una técnica terapéutica en la que se proporciona a los individuos información en tiempo real sobre su actividad cerebral. Las BCI se utilizan para capturar señales cerebrales y mostrar a los usuarios visualizaciones de sus patrones de actividad cerebral. Esto permite que las personas aprendan a autorregular su actividad cerebral, lo que puede ser beneficioso para tratar problemas de salud mental, como el estrés, la ansiedad y la depresión, y mejorar el rendimiento cognitivo en tareas específicas.

JUEGOS Y ENTRETENIMIENTO

Las BCI también se aplican en el campo de los juegos y el entretenimiento. Al utilizar señales cerebrales para controlar videojuegos y aplicaciones, se crea una experiencia de juego más inmersiva e interactiva. Los jugadores pueden controlar personajes y acciones dentro del juego mediante su actividad cerebral, lo que abre un mundo de posibilidades para nuevas mecánicas de juego y experiencias innovadoras.

Realidad Aumentada y Virtual

Definición

La Realidad Aumentada (AR) y la Realidad Virtual (VR) son tecnologías que mezclan el mundo digital con el mundo real o generan entornos completamente simulados para el usuario. Estas tecnologías ofrecen nuevas formas de interactuar con la IA y el mundo digital, proporcionando experiencias inmersivas y enriquecedoras.

Algunas aplicaciones destacadas son:

EDUCACIÓN Y FORMACIÓN

La AR y la VR se están convirtiendo en herramientas valiosas en el campo de la educación y la formación. Al simular entornos y escenarios de la vida real, estas tecnologías permiten a los estudiantes aprender de manera práctica y vivencial, lo que puede mejorar significativamente la retención de conocimientos y habilidades. Por ejemplo, los estudiantes de medicina pueden realizar simulaciones de cirugías en entornos de realidad virtual, lo que les proporciona una experiencia práctica sin riesgos para los pacientes.

DISEÑO Y VISUALIZACIÓN

Las AR y VR también ofrecen ventajas en el campo del diseño y la visualización. Los arquitectos, ingenieros y diseñadores pueden utilizar estas tecnologías para ver y modificar modelos 3D de edificios, productos o espacios. Esto permite una comprensión más clara y detallada de los proyectos, lo que puede conducir a un diseño más eficiente y una toma de decisiones más informada.

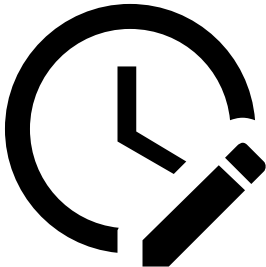
ENTRETENIMIENTO E INMERSIÓN

En el ámbito del entretenimiento, la AR y la VR proporcionan experiencias inmersivas que integran elementos del mundo real y virtual. Los videojuegos de realidad virtual permiten a los jugadores sumergirse por completo en mundos virtuales y participar en experiencias interactivas. Además, la AR se ha utilizado en aplicaciones de entretenimiento móvil, como juegos de realidad aumentada que interactúan con el entorno del usuario.



<https://sataraseguridad.com/2021/03/31/la-realidad-virtual-llega-al-entrenamiento-policial/>

Mapa conceptual



23 de agosto de 2026

3.2 Diapositivas de la UD01



Parte 1 de 5



Parte 2 de 5



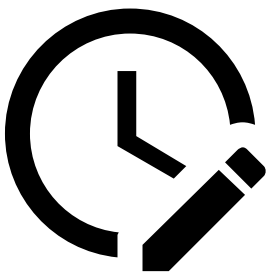
Parte 3 de 5



Parte 4 de 5



Parte 5 de 5



17 de septiembre de 2025

4. UD02

4.1 UD02 — Modelos de IA y resolución de problemas

Unidad 2 · 12 h · semanas 7-10 (9 de noviembre al 3 de diciembre)

1. Introducción

En la UD01 aprendiste a **identificar** sistemas de IA y a relacionarlos con la eficiencia operativa. En esta unidad damos un paso más: no solo reconocer la IA, sino **formalizar y resolver problemas con modelos de IA**. Para ello recorremos un camino progresivo:

1. Primero, **qué debe cumplir un sistema** que resuelve problemas con IA, y **cómo se representa un problema** para que una máquina pueda resolverlo (espacio de estados y algoritmos de búsqueda).
2. Después, **cómo se clasifican los modelos de IA** (por aprendizaje, por análisis, por conocimiento frente a datos).
3. A continuación, **qué se puede automatizar** y con qué tecnología (RPA frente a IA, agentes).
4. Luego, las dos familias de razonamiento que tocarás en el laboratorio: **lógica difusa y sistemas basados en reglas**.
5. Por último, **cómo elegir el modelo adecuado** para cada problema (CE f).

El núcleo práctico usa dos librerías de Python que ya están en el stack del curso: `scikit-fuzzy` (lógica difusa) y `experta` (sistemas basados en reglas). La unidad se cierra con **Robocode Tank Royale**: programar un bot que compite en un campo de batalla es, literalmente, implementar un sistema de resolución de problemas de principio a fin.

” Hilo conductor de la unidad

Antes de programar IA hay que saber formalizar un problema y elegir bien el modelo. Lo haremos paso a paso: modelar → clasificar → automatizar → razonar (difuso y por reglas) → decidir → implementar.

2. Resultado de aprendizaje y criterios de evaluación

RA2 — Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.

CE	Criterio de evaluación
RA2-a	Se han determinado los requisitos básicos a implementar en un sistema de resolución de problemas.
RA2-b	Se han clasificado modelos de Inteligencia Artificial.
RA2-c	Se han caracterizado los modelos de automatización de tareas.
RA2-d	Se han caracterizado los modelos de razonamiento impreciso.
RA2-e	Se han caracterizado los modelos de sistemas basados en reglas.
RA2-f	Se ha valorado la adecuación de los modelos a la implementación del sistema de resolución de problemas.

Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo para este RA dice textualmente:

- *Utilización de modelos de Inteligencia Artificial:*
- *- Requisitos básicos de un sistema de resolución de problemas.*
- *- Modelos de sistemas de Inteligencia Artificial: Automatización de tareas. Sistemas de razonamiento impreciso. Sistemas basados en reglas.*

3. Objetivos de la unidad

Objetivo	Descripción
O1	Enumerar los requisitos básicos de un sistema de resolución de problemas de IA.
O2	Formalizar un problema como espacio de estados (estado inicial, acciones, transición, objetivo).
O3	Aplicar búsqueda en anchura, en profundidad y A* y saber cuándo usar cada una.
O4	Clasificar modelos de IA por paradigma de aprendizaje, por tipo de análisis y por base de conocimiento/datos.
O5	Diferenciar RPA de IA y describir la automatización inteligente y los agentes software.
O6	Explicar la lógica difusa: conjuntos difusos, funciones de pertenencia, reglas, Mamdani/Sugeno y defuzzificación.
O7	Implementar un control difuso sencillo con <code>scikit-fuzzy</code> .
O8	Describir el ciclo reconocer-actuar de un sistema basado en reglas y el papel de CLIPS y de <code>experta</code> .
O9	Implementar un sistema basado en reglas con <code>experta</code> (con el parche de compatibilidad).
O10	Elegir el modelo adecuado a un problema según datos, explicabilidad, coste y tiempo real.
O11	Implementar un sistema de resolución de problemas completo (Robocode) dotando de comportamiento a un bot.

4. Sistema de resolución de problemas (RA2-a)

4.1 Cinco requisitos de un sistema de resolución de problemas

Antes de formalizar un problema concreto, conviene fijar **qué debe cumplir** cualquier sistema de IA que pretenda resolver problemas de forma efectiva y útil:

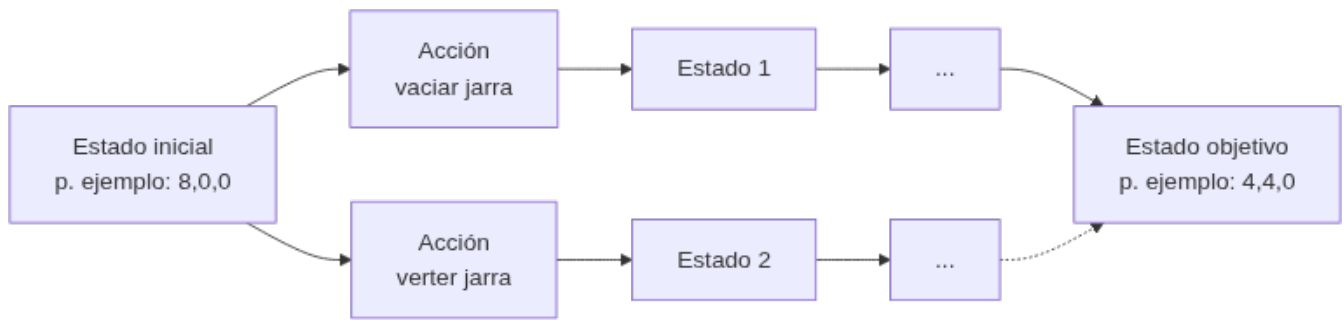
1. **Representación del problema:** elegir una estructura de datos y un modelo que reflejen fielmente el dominio. En PLN, por ejemplo, convertir texto a vectores numéricos; en un puzzle, una matriz de fichas (§4.2).
2. **Razonamiento y toma de decisiones:** aplicar técnicas lógicas, de aprendizaje o de búsqueda heurística para llegar a una solución a partir de la información disponible.
3. **Aprendizaje y adaptabilidad:** mejorar el rendimiento con la experiencia — aprendizaje automático o por refuerzo cuando el problema lo permite.
4. **Eficiencia computacional:** usar algoritmos y estructuras que den respuestas rápidas y escalables (por eso importa tanto BFS/DFS/A*, §4.3).
5. **Interacción con las personas usuarias:** una interfaz comprensible que facilite la comunicación entre el sistema y quien lo usa.

**No son independientes**

Un sistema puede cumplir muy bien 4 requisitos y fallar estrepitosamente por el quinto: un algoritmo de búsqueda óptimo (requisito 4) que nadie sabe usar porque no tiene interfaz (requisito 5) no resuelve el problema en la práctica.

4.2 El espacio de estados

Muchos problemas de IA se resuelven convirtiéndolos en una **búsqueda**: existe un conjunto de configuraciones posibles (los **estados**) y unas **acciones** que pasan de un estado a otro. Ese grafo de estados conectados por acciones es el **espacio de estados**.



Un **sistema de resolución de problemas** necesita, formalmente (base AIMA):

1. **Estado inicial:** la configuración de partida.
2. **Acciones aplicables:** qué movimientos se pueden hacer en cada estado.
3. **Modelo de transición:** función que, dado un estado y una acción, produce el siguiente estado.
4. **Test de objetivo:** cómo saber que hemos llegado a la solución.
5. **Coste de camino:** el coste de cada acción (para encontrar la solución más barata).

Además, se elige la **representación** (cómo codificar cada estado) y la **estrategia de búsqueda** (completa, óptima, memoria).



Regla para recordar

Un problema de IA bien planteado es aquel del que sabes responder: *¿cuál es el estado inicial?, ¿qué acciones puedo aplicar?, ¿cuándo he terminado?* Si no sabes responderlas, el problema no está bien formalizado.

4.3 Ejemplos clásicos de representación

Problema	Representación	Detalle
Jarras 8-5-3	Triple $[jarra8, jarra5, jarra3]$	De $[8,0,0]$ a $[4,4,0]$ en 7 pasos; las acciones son "verter hasta vaciar o llenar"
Misioneros y caníbales	Vector $\langle m, c, barca \rangle$	De $\langle 3,3,1 \rangle$ a $\langle 0,0,0 \rangle$; se descartan los estados donde los caníbales superan a los misioneros
8-reinas	Una columna por reina (permutación de 1..8)	Sin restricciones hay 4.426.165.368 arreglos; con restricciones $8! = 40.320$; solo 92 soluciones
15-puzzle	Matriz 4×4 con las fichas	$16!/2 \approx 10,46 \cdot 10^{12}$ estados (la mitad alcanzable); soluciones óptimas de hasta 80 movimientos



La explosión combinatoria

Estos problemas "de juguete" sirven para medir algo esencial: el número de estados crece **exponencialmente** con el tamaño (factor de ramificación b elevado a profundidad d). Un espacio de estados real (rutas de reparto, planificación de tareas) es enorme: por eso hace falta una **estrategia de búsqueda** y, cuando se puede, una **heurística**.

4.4 Algoritmos de búsqueda: BFS, DFS y A*

Criterio	Búsqueda en anchura (BFS)	Búsqueda en profundidad (DFS)	A*
Estructura	Cola (FIFO)	Pila (LIFO)	Cola de prioridad por $f = g + h$
Completa	Sí	No (en espacios infinitos)	Sí
Óptima	Sí (coste unitario)	No	Sí (heurística admisible)
Memoria	$O(b^d)$	$O(b \cdot d)$	$O(b^d)$

Criterio	Búsqueda en anchura (BFS)	Búsqueda en profundidad (DFS)	A*
Cuándo usar	Camino más corto en número de pasos	Exploración exhaustiva, backtracking	Ruta óptima con costes variados y buena heurística

- **BFS** explora por niveles: encuentra el camino más corto en pasos, pero consume mucha memoria (guarda todos los estados del nivel).
- **DFS** baja hasta el fondo: usa poca memoria, pero puede no encontrar la solución óptima (o no terminar en espacios infinitos). Es la base del *backtracking* (8-reinas).
- **A*** combina el coste acumulado $g(n)$ con una **heurística** $h(n)$ (una estimación de lo que falta): $f(n) = g(n) + h(n)$. Con una heurística admisible (que no sobreestime), A* es completo y óptimo. Se usa mucho en videojuegos, mapas y planificación de rutas.



Ejemplo de heurística

En un puzzle 8-puzzle (3×3), la heurística "número de fichas mal colocadas" o la "distancia de Manhattan" (suma de pasos horizontales+verticales de cada ficha hasta su posición correcta) estiman cuánto queda. Con esa h , A* guía la búsqueda hacia el objetivo en lugar de explorar a ciegas.



Regla práctica (Red Blob Games)

"Usa el algoritmo más simple que puedas": si todos los costes son iguales y solo quieres el camino más corto en pasos, BFS basta. Si los costes varían, Dijkstra. Si apuntas a un único objetivo, prefiere A* con la heurística más simple posible (p. ej. Manhattan en rejillas).

5. Clasificación de modelos de IA (RA2-b)

5.1 Por paradigma de aprendizaje

Criterio	Supervisado	No supervisado	Refuerzo
Datos	Etiquetados (ground truth)	Sin etiquetas	Estados, acciones, recompensas
Objetivo	Predecir la salida correcta	Descubrir patrones	Maximizar recompensa acumulada
Tareas	Clasificación, regresión	Clustering, asociación, reducción de dim.	Control, juegos, robótica
Ejemplos	Spam, precio de un coche	Segmentación de clientes (k-means)	Robot que aprende a andar, AlphaGo
Algoritmos (scikit-learn)	Regresión logística, SVM, árboles	k-means, DBSCAN, PCA	(fuera del stack estándar)



Recordatorio de la UD01

Todo ML es IA, pero no toda IA es ML. Los sistemas basados en reglas o en lógica difusa son IA, pero no aprenden de datos. En esta unidad estudiamos precisamente esos modelos "no-ML".

5.2 Por tipo de análisis

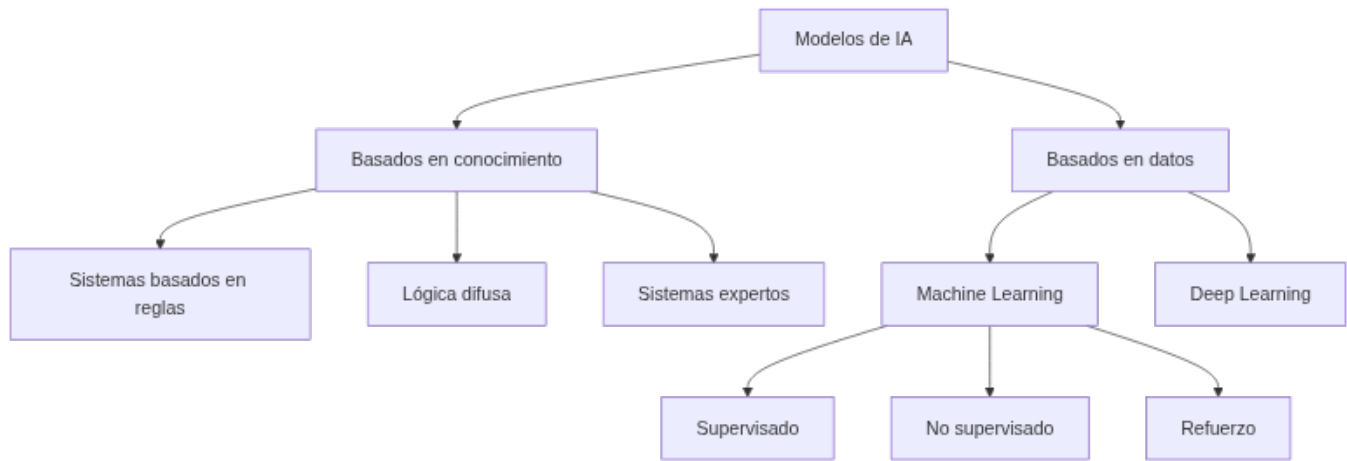
Fase	Pregunta	Ejemplo
Descriptivo	¿Qué pasó?	Informe de ventas
Predictivo	¿Qué pasará?	Previsión de demanda

Fase	Pregunta	Ejemplo
Prescriptivo	¿Qué hacer?	Qué precio fijar y qué ocurre si lo cambio

El **prescriptivo** es el que aporta más valor: no solo predice, **recomienda o ejecuta** la decisión óptima (Gartner lo llama "la última frontera" de la analítica).

5.3 Basados en conocimiento frente a basados en datos

Criterio	Basados en conocimiento	Basados en datos (ML/DL)
Conocimiento	Explícito (reglas IF-THEN)	Implícito (aprendido de los datos)
Ejemplos	Sistemas expertos, lógica difusa, termostato	Árboles, regresión, redes neuronales
Ventajas	Interpretable, fácil de mantener, explica el razonamiento	Flexible, escala a tareas complejas
Limitaciones	Frágil en tareas complejas; adquisición de conocimiento difícil	Requiere datos y cómputo; caja negra
Requisito	Experto + ingeniero del conocimiento	Datos etiquetados suficientes y representativos



Dos formas de resolver el mismo problema

Un termostato con reglas SI temperatura < 18 ENTONCES calefacción ON es IA basada en conocimiento (reglas). Un sistema que aprende de datos históricos qué temperatura poner según la hora y la estación es ML. La elección depende del problema (lo veremos en el CE f).

6. Modelos de automatización de tareas (RA2-c)

6.1 RPA frente a IA

Criterio	RPA	IA
Qué hace	Hace: replica tareas humanas repetitivas en la interfaz (GUI, formularios)	Piensa/aprende: reconoce patrones, decide, mejora con los datos
Base	Procesos (<i>process-driven</i>), reglas predefinidas	Datos (<i>data-driven</i>), modelos
Se adapta	No: los flujos hay que mantenerlos si cambia el sistema	Sí: aprende de la experiencia
Ejemplo	Un robot que copia datos de un email a un ERP	Un modelo que clasifica si el email es urgente o no



RPA no es IA

La distinción de IBM es clara: RPA **automatiza procesos** replicando acciones en pantalla; la IA **razona sobre datos**. Se complementan: la IA da decisión y el RPA ejecuta. Cuando se combinan IA + BPM (orquestración de flujos) + RPA (ejecución) hablamos de **automatización inteligente (IA)**.

6.2 Agentes software

Un **agente de IA** es un programa que percibe su entorno y actúa para lograr un objetivo. Se ordenan de más simple a más avanzado:

Tipo de agente	Cómo decide	Ejemplo
Reflejo simple	Reglas si... entonces... directas	Termostato
Reflejo con modelo	Mantiene un estado interno del mundo	Robot aspiradora que "recuerda" la habitación
Basado en objetivos	Busca y planifica para alcanzar una meta	Navegador que elige una ruta
Basado en utilidad	Maximiza una utilidad (coste, tiempo, riesgo)	Ruta que minimiza peajes y tiempo
Con aprendizaje	Aprende de la experiencia	Motor de recomendación

6.3 Tareas cognitivas automatizables

La IA automatiza **tareas cognitivas** que antes requerían juicio humano:

- **Extracción:** leer documentos (OCR), extraer entidades (NER), capturar datos de facturas.
- **Clasificación:** spam, incidencias, prioridades. Los **sistemas de reconocimiento de voz** (Google Speech-to-Text, Microsoft Azure Speech Service) son otro caso de automatización de tareas: convierten habla en texto y automatizan la transcripción o los comandos de voz.
- **Generación:** redactar respuestas, resúmenes o informes (IA generativa).



Caso con cifra (ilustrativo)

En automatización de back-office, un sistema que extrae datos de emails y mensajería puede reducir el cierre contable mensual de varios días a uno, con menor tasa de error. Las cifras concretas dependen de cada despliegue; en la práctica se mide con KPIs (tiempo de ciclo, coste por documento, tasa de error) como vimos en la UD01. Otro ejemplo habitual es la **detección de fraude**: algoritmos de aprendizaje automático que analizan patrones en transacciones financieras para señalar operaciones sospechosas, reduciendo la carga de revisión manual.

7. Razonamiento impreciso: lógica difusa (RA2-d)

7.1 De lo booleano a lo difuso

La lógica clásica solo admite **verdadero/falso** (0/1). La **lógica difusa** (Zadeh, 1965) permite grados de verdad **entre 0 y 1**: algo puede ser *parcialmente verdadero*. Modela la **vaguedad** del lenguaje humano (la probabilidad, en cambio, modela la incertidumbre). Es la familia de modelos que se usa cuando los datos son **incompletos o inciertos** y aun así hay que decidir algo razonable.

Variable	Valor clásico	Valor difuso
Temperatura	"18 °C es frío" (sí/no)	"18 °C es frío con 0,7"
Velocidad	Rápida o no	"Rápida con 0,6, media con 0,4"

7.2 Conjuntos difusos y funciones de pertenencia

Un **conjunto difuso** asigna a cada valor un **grado de pertenencia** $\mu(x) \in [0,1]$ mediante una **función de pertenencia**. Las más usadas:

Función	Forma	Uso típico
Triangular (<code>trimf</code>)	Pico en un punto	Variables simples (p. ej. "media")
Trapezoidal (<code>trapmf</code>)	Meseta central	Intervalos ("normal")
Gaussiana (<code>gaussmf</code>)	Campana suave	Variables continuas suaves
Sigmoidal (<code>sigmf</code>)	Escalón suave	Extremos ("frío", "caliente")



Diseño de variables

No importa tanto la forma exacta como el **número y la posición** de las funciones: se recomiendan entre **3 y 7 curvas** por variable, solapadas para que no existan "huecos".

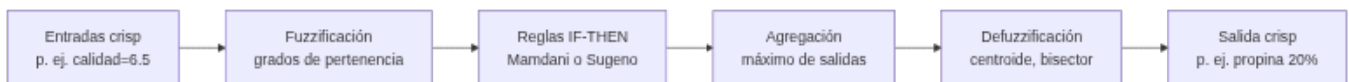
7.3 Operaciones difusas

Zadeh definió las operaciones básicas sobre grados de pertenencia:

- **AND (intersección)**: mínimo de los grados ($\mu_A \wedge \mu_B = \min(\mu_A, \mu_B)$).
- **OR (unión)**: máximo ($\mu_A \vee \mu_B = \max(\mu_A, \mu_B)$).
- **NOT (complemento)**: $1 - \mu$.

En `scikit-fuzzy`, las reglas usan por defecto `fmin / fmax` para AND/OR (se pueden cambiar a producto/suma difusa).

7.4 El sistema de inferencia difuso (FIS)



1. **Fuzzificar**: convertir cada entrada numérica en grados de pertenencia.
2. **Aplicar reglas** SI calidad es mala O servicio es pobre ENTONCES propina es baja.
3. **Agregar**: combinar la salida de todas las reglas.
4. **Defuzzificar**: obtener un número nítido. Métodos típicos: **centroide** (centro de gravedad), **bisector**, **MOM/SOM/LOM**.

Hay dos familias de sistemas: **Mamdani** (la salida es un conjunto difuso que luego se defuzzifica; la implementa `scikit-fuzzy`) y **Sugeno/TSK** (la salida de cada regla es una función $z = f(x, y)$, más eficiente para control).

7.5 Aplicaciones reales

- **Metro de Sendai (Japón, 1987)**: control difuso de aceleración y frenado.
- **Autofocus de Canon**: 12 entradas (claridad y velocidad del objetivo), **13 reglas**, solo **1,1 KB** de memoria.
- **ABS** (frenos): estima el agarre con lógica difusa sobre la temperatura del freno.
- **Electrodomésticos**: lavadoras Hitachi (peso/suciedad → programa), vacuolas Matsushita.
- **Control industrial**: hornos de cemento (Dinamarca, 1976), aires acondicionados Mitsubishi (calienta/enfría 5× más rápido con un 24 % menos de consumo).
- **Control de tráfico**: ajuste de los tiempos de los semáforos en función del flujo vehicular en tiempo real, para reducir la espera de los vehículos.
- **Apoyo al diagnóstico médico**: evaluación de síntomas cuando los resultados de las pruebas no son concluyentes, para dar una valoración preliminar que considere varios factores a la vez.

7.6 Práctica con `scikit-fuzzy`

`pip install -U scikit-fuzzy` (versión 0.5.0, 2024). Ejemplo clásico: el **problema de la propina** — la propina depende de la calidad del servicio y de la comida.

```

1 import numpy as np
2 import skfuzzy as fuzz
3 from skfuzzy import control as ctrl
4
5 # 1. Variables (universo de discurso)
6 calidad = ctrl.Antecedent(np.arange(0, 11, 1), 'calidad')
7 servicio = ctrl.Antecedent(np.arange(0, 11, 1), 'servicio')
8 propina = ctrl.Consequent(np.arange(0, 26, 1), 'propina')
9
10 # 2. Funciones de pertenencia
11 calidad.automf(3, names=['mala', 'acceptable', 'excelente'])
12 servicio.automf(3, names=['pobre', 'acceptable', 'bueno'])
13 propina['baja'] = fuzz.trimf(propina.universe, [0, 5, 10])
14 propina['media'] = fuzz.trimf(propina.universe, [10, 15, 20])
15 propina['alta'] = fuzz.trimf(propina.universe, [15, 20, 25])
16
17 # 3. Reglas
18 regla1 = ctrl.Rule(calidad['mala'] | servicio['pobre'], propina['baja'])
19 regla2 = ctrl.Rule(servicio['acceptable'], propina['media'])
20 regla3 = ctrl.Rule(servicio['bueno'] | calidad['excelente'], propina['alta'])
21
22 # 4. Sistema y simulación
23 sistema = ctrl.ControlSystem([regla1, regla2, regla3])
24 sim = ctrl.ControlSystemSimulation(sistema)
25 sim.input['calidad'] = 6.5
26 sim.input['servicio'] = 9.8
27 sim.compute()
28 print(f"Propina sugerida: {sim.output['propina']:.2f}%")
29 # Salida esperada: Propina sugerida: ~20% (el valor exacto depende del universo)

```

⚠ Compatibilidad de la librería

`scikit-fuzzy` es una librería **semi-mantenida** (última versión 2024). Para los ejercicios del curso funciona sin problemas, pero conviene fijar la versión (`scikit-fuzzy==0.5.0`) en el entorno reproducible del contenedor.

8. Sistemas basados en reglas (RA2-e)

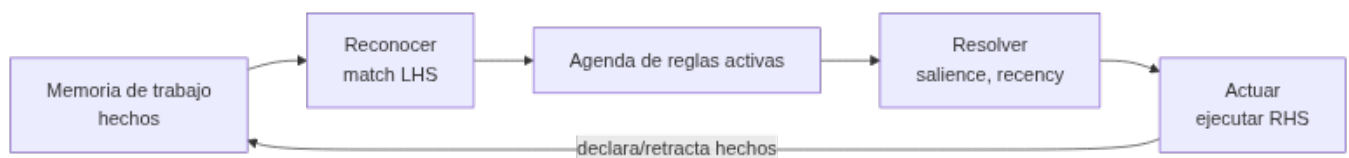
8.1 El ciclo reconocer-actuar

Un **sistema de producción** (o sistema basado en reglas) tiene:

- **Memoria de trabajo:** los hechos conocidos en cada momento.
- **Base de conocimiento:** las reglas SI <condiciones> ENTONCES <acciones>.
- **Motor de inferencia:** ejecuta el ciclo **reconocer-actuar**.

El ciclo se repite hasta que no queden reglas aplicables:

1. **Reconocer (match):** se comparan los hechos de la memoria de trabajo con las condiciones de las reglas; las reglas cuyas condiciones se cumplen van a la **agenda**.
2. **Resolver (resolve):** si hay varias reglas activas, se elige una según prioridad (*salience*), frescura de los hechos o especificidad.
3. **Actuar (act):** se ejecuta la parte ENTONCES, que modifica la memoria de trabajo (declarar/retractar hechos) y el ciclo se repite.



Los motores industriales usan el **algoritmo RETE** (Forgy, 1974) para no reevaluar todas las reglas cada vez: construye una red de filtrado en memoria.

8.2 Encadenamiento hacia delante y hacia atrás

- **Hacia delante (forward chaining, data-driven):** parte de los hechos y deduce hechos nuevos. Ideal para monitorización, planificación y entornos dinámicos. Es el que usan CLIPS y `experta`.
- **Hacia atrás (backward chaining, goal-driven):** parte de una **meta** y busca qué hechos la sustentan (preguntando al usuario solo lo necesario). Ideal para diagnóstico (MYCIN, Prolog).

8.3 CLIPS y `experta`

CLIPS (*C Language Integrated Production System*) es el motor de reglas de referencia, desarrollado por la NASA (1985-1996) y de dominio público desde 1996. `experta` es una librería de Python **inspirada en CLIPS** (fork de `pyknow`) que implementa un motor RETE con sintaxis nativa de Python.

8.4 Ejemplos cotidianos de sistemas basados en reglas

Antes de los grandes sistemas expertos históricos (§8.6), los sistemas basados en reglas están en aplicaciones que usas a diario:

- **Sistemas de recomendación:** plataformas de comercio electrónico como Amazon sugieren productos relacionados aplicando reglas sobre el historial de compras y el comportamiento del usuario.
- **Soporte al diagnóstico médico:** en asistencia remota, reglas basadas en los síntomas declarados dan una recomendación preliminar antes de que la persona sea atendida por un profesional.

Se usan mucho porque el razonamiento es **transparente**: las reglas son explícitas y una persona puede leer exactamente por qué el sistema tomó esa decisión — al contrario que una red neuronal.

8.5 Práctica con `experta`

⚠ Compatibilidad con Python 3.10+ (obligatorio)

`experta` (1.9.4, de 2019) fija `frozendict==1.2`, que usa `collections.Mapping`, eliminado en Python 3.10. Sin parche, `from experta import *` falla. Dos soluciones: - **En el código** (más didáctico): añadir el monkey-patch antes del import. - **En el entorno** (más limpio): `pip install "frozendict>=2.3" + pip install --no-deps experta + pip install schema==0.6.7`.

```
1 # PARCHE para Python 3.10+ (issue #34 de experta)
2 import collections
3 import collections.abc
4 if not hasattr(collections, 'Mapping'):
5     collections.Mapping = collections.abc.Mapping
6     collections.Iterable = collections.abc.Iterable
7     collections.MutableMapping = collections.abc.MutableMapping
8
9 from experta import *
10
11 class DiagnosticoVehiculo(KnowledgeEngine):
12     @DefFacts()
13     def inicio(self):
14         yield Fact(accion="diagnosticar")
15
16     @Rule(Fact(accion="diagnosticar"), salience=10)
17     def arrancar(self):
18         print("Iniciando diagnóstico del vehículo...")
19         self.declare(Fact(bateria="descargada"))
20         self.declare(Fact(luces="no_encienden"))
21
22     @Rule(Fact(bateria="descargada"), Fact(luces="no_encienden"))
23     def bateria(self):
24         self.declare(Fact(causa="bateria_descargada"))
25
26     @Rule(Fact(causa="bateria_descargada"))
27     def resultado(self):
28         print("CAUSA PROBABLE: Batería descargada")
29
30 engine = DiagnosticoVehiculo()
31 engine.reset() # activa DefFacts e InitialFact
32 engine.run()   # ejecuta el ciclo reconocer-actuar
```

Salida esperada:

```
1 Iniciando diagnóstico del vehículo...
2 CAUSA PROBABLE: Batería descargada
```

8.6 Sistemas expertos reales

Sistema	Origen	Qué hacía	Dato relevante
MYCIN	Stanford (1972)	Diagnóstico de infecciones y antibióticos (backward chaining + factores de certeza)	~500-600 reglas; acierto del 65-70 % frente a facultativos

Sistema	Origen	Qué hacía	Dato relevante
XCON/R1	CMU/DEC (1978)	Configuración de ordenadores VAX	Ahorró a DEC del orden de 25 M\$/año
SID	DEC (años 80)	Diseño de puertas de la CPU del VAX 9000	Generó el 93 % de las puertas lógicas
Dendral	Stanford (años 70)	Identificación de moléculas orgánicas	Primer sistema experto "completo"



Cuándo usar reglas y cuándo ML

Usa **reglas** cuando el dominio esté acotado, la explicabilidad sea exigible (auditoría, normativa) o no haya datos. Usa **ML** cuando haya datos abundantes y patrones complejos. Una práctica habitual es el **híbrido**: reglas como guardarraíl/validación y ML para la decisión fina.

9. Adecuación del modelo (RA2-f)

9.1 Criterios para elegir modelo

Criterio	Pregunta guía
Datos disponibles	¿Hay datos? ¿Etiquetados? ¿Representativos?
Explicabilidad	¿Necesito justificar la decisión (auditoría, normativa)?
Coste	¿Cuánto cuesta el cómputo, el etiquetado y el mantenimiento?
Precisión requerida	¿Cuánto mejora frente a una solución no-ML (baseline)?
Tiempo real / latencia	¿La decisión debe ser instantánea (RPA/reglas) o admite espera?



La regla de oro: empieza simple

- **Google (problem framing)**: "El ML es una herramienta especializada; no quieras una solución compleja cuando una más simple funciona". Primero optimiza la solución **no-ML** y úsala como *benchmark*.
- **scikit-learn**: su mapa de selección te hace probar primero estimadores simples y avanzar ("Try next") solo si no alcanzas el objetivo.
- **Azure ML**: "No tengas miedo a una competición en paralelo entre varios algoritmos".
- **No Free Lunch** (Wolpert y Macready, 1997): ningún algoritmo es mejor en promedio sobre todos los problemas → la elección depende de las suposiciones sobre tu problema.

9.2 Secuencia práctica recomendada

1. **Reglas o heurística** (explicable, sin datos).
2. **Árbol de decisión / regresión logística** (datos tabulares pequeños).
3. **Ensamble** (Random Forest / XGBoost) si hace falta precisión.
4. **Deep learning** solo si hay datos masivos y el problema lo exige.



Evidencia para «empezar simple»

Un estudio con 45 datasets tabulares de tamaño medio (~10.000 muestras) mostró que los modelos basados en árboles siguen siendo estado del arte, además de más rápidos de entrenar que el deep learning. No siempre hace falta una red neuronal.

10. Caso de estudio: Robocode como sistema de resolución de problemas completo

Los cinco requisitos del §4.1 y la elección de modelo del §9 dejan de ser teoría en cuanto se programa un bot de **Robocode Tank Royale** (la [actividad entregable](#) de la unidad):

- **Representación:** el estado del bot (posición, energía, rumbo del radar) y del campo de batalla se traduce a variables que el programa puede leer en cada turno.
- **Razonamiento:** decidir hacia dónde disparar o moverse es exactamente el problema de elegir modelo del §9 — puedes usar reglas fijas («si el enemigo está a menos de 100 px, dispara»), lógica difusa sobre la distancia y el ángulo, o una combinación.
- **Eficiencia:** el bot decide en tiempo real, turno a turno; una lógica demasiado costosa computacionalmente pierde el combate por lentitud, no por mala estrategia.
- **Adecuación del modelo (CE f):** no hay datos de entrenamiento previos sobre "cómo gana este bot en concreto", así que el punto de partida natural es la regla o heurística (§9.2, paso 1), no el aprendizaje automático.

11. Puntos clave de la unidad

- Un sistema de resolución de problemas de IA debe cumplir **5 requisitos**: representación, razonamiento, aprendizaje/adaptabilidad, eficiencia computacional e interacción con el usuario.
- Un problema se formaliza con **estado inicial, acciones, transición, objetivo y coste**; la búsqueda recorre el **espacio de estados**.
- **BFS** es completa y óptima en pasos pero usa mucha memoria; **DFS** usa poca memoria pero no es óptima; **A*** es óptimo con heurística admisible.
- Los modelos de IA se clasifican por **aprendizaje** (supervisado/no supervisado/refuerzo), por **análisis** (descriptivo/predictivo/prescriptivo) y por **base** (conocimiento vs datos).
- **RPA "hace" y la IA "piensa"**: la automatización inteligente combina IA + BPM + RPA.
- La **lógica difusa** modela la vaguedad con grados de pertenencia $[0,1]$, funciones de pertenencia, reglas Mamdani/Sugeno y defuzzificación por centroide.
- Los **sistemas basados en reglas** ejecutan el ciclo **reconocer-actuar** con un motor RETE; `experta` es su port Python (requiere parche en Py3.10+).
- Para elegir modelo (CE f): valora **datos, explicabilidad, coste, precisión y tiempo real** y **empieza por el modelo más simple**.

12. Glosario

Término	Definición
Espacio de estados	Grafo de configuraciones posibles conectadas por acciones
Estado	Configuración concreta de un problema en un momento dado
Búsqueda en anchura (BFS)	Algoritmo que explora el espacio por niveles (cola)
Búsqueda en profundidad (DFS)	Algoritmo que explora hasta el fondo (pila)
A*	Búsqueda con heurística: $f = g + h$ (coste acumulado + estimación)
Heurística	Estimación del coste restante hasta el objetivo
Backtracking	Exploración con retroceso (base de DFS)
RPA	Automatización robótica de procesos: replica tareas en la interfaz
Automatización inteligente	Combinación de IA (decisión) + BPM (orquestración) + RPA (ejecución)
Agente de IA	Programa que percibe y actúa para lograr un objetivo
Lógica difusa	Lógica con grados de verdad en $[0,1]$ (Zadeh, 1965)
Función de pertenencia	Asigna a cada valor su grado de pertenencia $\mu(x)$
Variable lingüística	Variable con términos difusos (frío, templado, caliente)
FIS	Sistema de inferencia difuso (fuzzificar $\rightarrow$ reglas $\rightarrow$ defuzzificar)
Mamdani	Inferencia difusa con salida difusa defuzzificada
Sugeno/TSK	Inferencia con salida funcional, más eficiente para control
Defuzzificación	Conversión del resultado difuso a número nítido (centroide...)

Término	Definición
Sistema de producción	Sistema basado en reglas con memoria de trabajo y motor de inferencia
Ciclo reconocer-actuar	match → resolve → act hasta agotar la agenda
RETE	Algoritmo de emparejamiento eficiente de reglas
Salience	Prioridad numérica de una regla
Forward chaining	Encadenamiento hacia delante (data-driven)
Backward chaining	Encadenamiento hacia atrás (goal-driven), propio del diagnóstico
CLIPS	Motor de reglas de la NASA (1985-1996), referencia histórica
No Free Lunch	Teorema: ningún algoritmo es mejor en promedio sobre todos los problemas

13. FAQ

? ¿Un problema «de juguete» como el de las jarras sirve para algo real? ▼

Sí: es la misma técnica que usan los planificadores de rutas, los motores de búsqueda de rutas en logística o los sistemas de planificación de tareas. Si aprendes a representar estados y acciones, sabes formalizar problemas reales.

? ¿A* siempre encuentra la solución más corta? ▼

Con una heurística **admisible** (que nunca sobreestime el coste restante), sí. Si la heurística sobreestima, A* pierde la garantía de optimalidad (pero suele ser más rápido).

? ¿La lógica difusa es lo mismo que la probabilidad? ▼

No. La difusa modela la **vaguedad** (pertenencia imprecisa: "hace calor") y la probabilidad modela la **incertidumbre** ("hay un 60 % de que llueva"). Son matemáticas distintas.

? ¿experta no funciona en Python moderno? ▼

La librería (2019) tiene un problema de dependencias con Python 3.10+. Con el parche `collections.Mapping = collections.abc.Mapping` (o instalando `frozendict>=2.3` y `--no-deps`) funciona bien en el entorno del curso.

? ¿Un sistema basado en reglas aprende de los datos? ▼

No: las reglas las escribe un experto. No aprende solo, pero es **totalmente explicable** (puede justificar cada decisión), algo que el ML no siempre puede.

? ¿Cuándo conviene usar RPA en lugar de un modelo de ML? ▼

Si la tarea es **repetitiva y con reglas claras** (copiar datos entre sistemas, rellenar formularios), RPA. Si exige **juicio, patrones o lenguaje**, IA/ML. Y a menudo se combinan.

? ¿Por qué el entregable de la unidad es un juego (Robocode) y no un dataset? ▼

Porque un bot de combate obliga a decidir en tiempo real con información incompleta — es un sistema de resolución de problemas completo y compacto, más cercano a control que a clasificación, y permite comparar en la práctica una solución por reglas con una por lógica difusa (\$10).

14. Planificación sesión a sesión

Semana	Horas	Contenido	CE	Evidencia / actividad
7	3	Requisitos de un SRP; espacio de estados y representación; BFS/DFS/A*	RA2-a	Ejercicios bloque A
8	3	Clasificación de modelos; automatización de tareas	RA2-b, RA2-c	Ejercicios bloques B y C
9	3	Lógica difusa (teoría + Taller 1 scikit-fuzzy)	RA2-d	Taller 1
9-10	3	Sistemas basados en reglas (teoría + Taller 2 experta); CE f	RA2-e, RA2-f	Taller 2
9-10	—	Talleres 3-5: preparar el entorno, GitHub y Markdown para Robocode	—	Talleres 3-5
10	3	Robocode Tank Royale: entrega y evaluación	RA2-a, RA2-f	Actividad entregable

15. Tabla final RA/CE

CE	Dónde se trabaja	Con qué se evalúa
RA2-a	\$4	Ejercicios bloque A, Robocode
RA2-b	\$5	Ejercicios bloque B
RA2-c	\$6	Ejercicios bloque C
RA2-d	\$7	Ejercicios bloque D, Taller 1
RA2-e	\$8	Ejercicios bloque E, Taller 2
RA2-f	\$9, \$10	Ejercicios bloque F, Robocode

16. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres: [T01](#) · [control difuso](#) · [T02](#) · [sistema de reglas](#) · [T03](#) · [preparar el entorno](#) · [T04](#) · [GitHub](#) · [T05](#) · [Markdown](#)
- Actividad entregable: [Robocode Tank Royale](#) · [comparativa Java/Python](#) · [tutorial Java](#) · [tutorial Python](#)
- **Notebooks** — Talleres 1 y 2, con descarga y apertura en Colab, en el menú «Notebooks»



Referencias de la unidad ▾

- [Red Blob Games](#) · [A*](#)
- [Wikipedia](#) · [A*](#)
- [scikit-fuzzy](#) (pythonhosted)
- [experta](#) (readthedocs)
- [CLIPS](#)
- [IBM](#) · [RPA](#)
- [IBM](#) · [Automatización inteligente](#)
- [scikit-learn](#) · [Elegir el estimador](#)
- [arXiv](#) · [Tree-based vs DL en tabulares](#)
- [Robocode Tank Royale](#) · [documentación oficial](#)

17. Evaluación

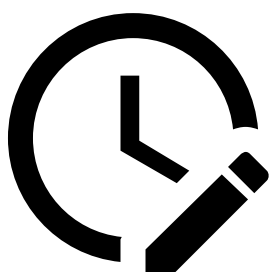
Peso	Instrumento
40 % actividades	Media de los 5 talleres y la actividad entregable (Robocode), con su rúbrica
60 % prueba escrita	Prueba del RA2 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.

18. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir la resolución de un problema de búsqueda con un caso distinto y las pruebas de autoevaluación de la unidad.

[Volver al índice](#)



24 de agosto de 2026

4.2 Diapositivas de la UD02

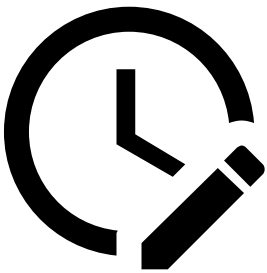


Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.



24 de agosto de 2026

4.3 UD02 — Ejercicios



Cómo se corrigen

Resuélvelos en tu cuaderno o en un documento Markdown (buen momento para practicar el [Taller 5](#)). Las soluciones no se publican: se corrigen y comentan en clase.

A. Sistema de resolución de problemas (RA2-a)

1. Define **espacio de estados** y pon un ejemplo distinto a los de la teoría (p. ej. un ascensor, una partida de ajedrez, un cajero automático).
2. Enumera los **cinco requisitos** de un sistema de resolución de problemas y aplícalos al problema de las jarras 8-5-3.
3. Para el problema de las **8 reinas**: ¿cuántos arreglos hay sin restricciones?, ¿cuántas soluciones válidas existen?
4. Explica con tus palabras la diferencia entre **búsqueda en anchura** y **búsqueda en profundidad** (estructura de datos, completitud, optimalidad, memoria).
5. ¿Por qué **DFS no es completa** en espacios infinitos? Pon un ejemplo de problema donde se atascaría.
6. ¿Qué es una **heurística**? Explica cómo usa A* la función $f(n) = g(n) + h(n)$.
7. En el 8-puzzle, propón dos heurísticas admisibles y explica por qué no sobreestiman.
8. Resuelve a mano el problema de las **jarras 8-5-3** desde $[8, 0, 0]$ hasta $[4, 4, 0]$ indicando los pasos y el contenido de cada jarra.
9. ¿En qué se diferencia A* de **Dijkstra**? ¿Y de la búsqueda **Greedy** (solo h)?
10. Dado el árbol de estados con costes: estado inicial $S \rightarrow A$ (coste 2), $S \rightarrow B$ (coste 5), $A \rightarrow G$ (coste 3), $B \rightarrow G$ (coste 1), calcula el camino que encuentra BFS, DFS y A* ($h(G)=0$).

B. Clasificación de modelos de IA (RA2-b)

1. Completa la tabla con el tipo de modelo:

Sistema	Paradigma	Base (conocimiento/datos)
Termostato con reglas		
Clasificador de spam con ML		
Sistema experto médico		
Red neuronal para imágenes		

1. Diferencia **descriptivo**, **predictivo** y **prescriptivo** con un ejemplo de comercio electrónico para cada uno.
2. ¿Por qué «todo ML es IA, pero no toda IA es ML»? Pon dos ejemplos de IA que no sea ML.
3. Clasifica por paradigma de aprendizaje: (a) agrupar clientes sin etiquetas, (b) predecir abandono de un cliente, © un agente que aprende a jugar, (d) detectar anomalías en una red.
4. ¿Qué ventaja aporta un modelo **basado en conocimiento** frente a uno **basado en datos**? ¿Y la desventaja principal?

C. Automatización de tareas (RA2-c)

1. Diferencia **RPA** de **IA** con un ejemplo concreto de cada uno.
2. Explica qué es la **automatización inteligente** y qué papel juegan la IA, el BPM y el RPA.
3. Ordena de más simple a más avanzado los **cinco tipos de agente software** y pon un ejemplo de cada uno.
4. Indica si cada tarea es candidata a RPA o a IA/ML y por qué:
 - Copiar datos de un Excel a un ERP cada mañana.
 - Clasificar correos de incidencias por urgencia.
 - Extraer la fecha y el importe de facturas escaneadas.
5. ¿Qué son las **tareas cognitivas** que automatiza la IA? Pon tres ejemplos, incluido al menos uno de reconocimiento de voz.

D. Lógica difusa (RA2-d)

1. Diferencia **lógica booleana** y **lógica difusa**. ¿Qué es la **vaguedad** y en qué se distingue de la **incertidumbre**?
2. ¿Qué es una **función de pertenencia**? Dibuja (o describe) una triangular y una trapezoidal y di qué grado de pertenencia asignan a un valor dado.
3. Dadas dos funciones con $\mu_A(x)=0,8$ y $\mu_B(x)=0,3$, calcula con las operaciones de Zadeh: AND (min), OR (max) y NOT ($1-x$) para ambos.
4. Explica las **4 fases** de un sistema de inferencia difuso (fuzzificar, reglas, agregar, defuzzificar) y los métodos de defuzzificación más comunes.
5. Diferencia **Mamdani** de **Sugeno/TSK** y di cuál implementa `scikit-fuzzy`.
6. ¿Cuántas curvas se recomiendan por variable lingüística y por qué es importante el solape?
7. Escribe el código `scikit-fuzzy` de un controlador de **velocidad del ventilador** según la temperatura: antecedentes temperatura (0-40) con términos fresca/templada/caliente y consecuente velocidad (0-100) con baja/media/alta; tres reglas y simulación con 30 °C.
8. Propón un caso real de razonamiento impreciso distinto a los de la teoría (por ejemplo, en control de tráfico o en apoyo al diagnóstico) y describe qué entradas y salidas tendría.

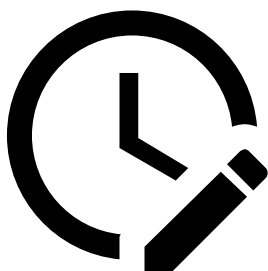
E. Sistemas basados en reglas (RA2-e)

1. Describe el **ciclo reconocer-actuar** (match, resolve, act) con tus palabras.
2. ¿Qué es el **algoritmo RETE** y por qué mejora el rendimiento de un motor de reglas?
3. Diferencia **encadenamiento hacia delante** y **hacia atrás**; indica cuál usa `experta` y cuál usaba MYCIN.
4. ¿Qué es el **salience** y para qué sirve en la resolución de conflictos?
5. Explica por qué `experta` falla en Python 3.10+ y cuáles son las **dos soluciones** posibles.
6. Escribe un sistema `experta` que determine si una **incidencia es urgente**: si `impacto=alto` o `usuarios>50`, declara `critica=True` y muestra el mensaje.
7. Un sistema de recomendación basado en reglas y un sistema de recomendación basado en ML resuelven el mismo problema. ¿Qué pierdes y qué ganas si usas reglas en vez de ML?

F. Adecuación del modelo (RA2-f)

1. Enumera los **cinco criterios** para elegir modelo y aplica cada uno a: predecir el fraude de tarjetas en tiempo real.
2. ¿Qué significa «empezar simple» y por qué el teorema **No Free Lunch** lo apoya?
3. Dado un problema de datos tabulares con 10.000 filas, ¿qué familia de modelos recomendarías primero y por qué?
4. Justifica si usarías reglas, ML o ambos en: (a) filtro de spam, (b) control de un semáforo, (c) diagnóstico de una avería con procedimiento escrito, (d) recomendación de películas.
5. Diseña un **flujo de decisión** (diagrama) para elegir entre reglas, lógica difusa, árbol de decisión y red neuronal.
6. Aplica los cinco criterios de adecuación del modelo (§9 de la teoría) al caso de un bot de Robocode: ¿qué modelo usarías para decidir el disparo, y por qué?

[Volver a la UD02](#) · [Talleres](#)



24 de agosto de 2026

4.4 Talleres

UD02 · Taller 1 — Control difuso con scikit-fuzzy



Entregable de la unidad

Cuenta en el 40 % de actividades del RA2, junto con los otros talleres y la [actividad entregable](#).



Requisitos

Necesitas el **contenedor de prácticas de IA** de la UD00 funcionando, o un entorno Python 3.12+ con `numpy` y `scikit-fuzzy`:

```
1 docker compose up -d          # si usas el contenedor del curso
2 pip install scikit-fuzzy==0.5.0
```

Objetivo: construir un sistema de control difuso tipo Mamdani que decida la **velocidad de un ventilador** según la temperatura y la humedad de una sala, recorriendo las tres fases de un sistema de razonamiento impreciso — fuzzificación, evaluación de reglas y desfuzzificación (RA2-d).



Hazlo en el notebook

Tienes este taller como **notebook con las celdas a rellenar**: [UD02_T01_control_difuso.ipynb](#). Esta página es la referencia; lo que se entrega es el notebook completado.

FASE 1 — IMPORTA LAS LIBRERÍAS Y DEFINE LOS UNIVERSOS

```
1 import numpy as np
2 import skfuzzy as fuzz
3 from skfuzzy import control as ctrl
4
5 temperatura = ctrl.Antecedent(np.arange(0, 41, 1), 'temperatura')
6 humedad = ctrl.Antecedent(np.arange(0, 101, 1), 'humedad')
7 velocidad = ctrl.Consequent(np.arange(0, 101, 1), 'velocidad')
```

FASE 2 — FUNCIONES DE PERTENENCIA (TÉRMINOS LINGÜÍSTICOS)

```
1 temperatura['fresca'] = fuzz.trimf(temperatura.universe, [0, 0, 18])
2 temperatura['agradable'] = fuzz.trimf(temperatura.universe, [15, 22, 28])
3 temperatura['caliente'] = fuzz.trapmf(temperatura.universe, [24, 30, 40, 40])
4
5 humedad['seca'] = fuzz.trimf(humedad.universe, [0, 0, 45])
6 humedad['media'] = fuzz.trimf(humedad.universe, [35, 55, 75])
7 humedad['humeda'] = fuzz.trapmf(humedad.universe, [60, 75, 100, 100])
8
9 velocidad['baja'] = fuzz.trimf(velocidad.universe, [0, 0, 40])
10 velocidad['media'] = fuzz.trimf(velocidad.universe, [25, 50, 75])
11 velocidad['alta'] = fuzz.trimf(velocidad.universe, [60, 100, 100])
```



¿Por qué trapezoidal en los extremos?

Las funciones de los extremos suelen ser trapezoidales para que «fresca» o «caliente» mantengan pertenencia plena hasta el límite del universo, y triangulares en el centro.

FASE 3 — REGLAS IF-THEN

```
1 r1 = ctrl.Rule(temperatura['fresca'] & humedad['seca'], velocidad['baja'])
2 r2 = ctrl.Rule(temperatura['agradable'] & humedad['media'], velocidad['media'])
3 r3 = ctrl.Rule(temperatura['caliente'] | humedad['humeda'], velocidad['alta'])
```

FASE 4 — SISTEMA Y SIMULACIÓN

```
1 sistema = ctrl.ControlSystem([r1, r2, r3])
2 sim = ctrl.ControlSystemSimulation(sistema)
3
4 sim.input['temperatura'] = 20
5 sim.input['humedad'] = 50
6 sim.compute()
7 print("Velocidad a 20 °C / 50 %:", f"{sim.output['velocidad']:.1f}%")
8
9 sim.input['temperatura'] = 33
10 sim.input['humedad'] = 85
11 sim.compute()
12 print("Velocidad a 33 °C / 85 %:", f"{sim.output['velocidad']:.1f}%")
```

FASE 5 — VISUALIZA LAS FUNCIONES DE PERTENENCIA

```
1 velocidad.view() # o temperatura.view()
```

Añade al informe una captura de la gráfica y explica qué ocurre en cada simulación.

FASE 6 — EXPERIMENTO (RESPUESTA RAZONADA)

Prueba estas combinaciones y anota el resultado en una tabla:

Temperatura	Humedad	Velocidad (%)	¿Coherente?
10	30		
25	50		
38	90		
18	80		

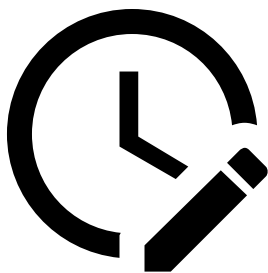
ENTREGA DEL TALLER 1

Fase	Evidencia mínima
1-2	Las dos variables de entrada y la de salida, con sus funciones de pertenencia
3	Las tres reglas IF-THEN
4	Las dos simulaciones (20 °C/50 % y 33 °C/85 %) con su velocidad resultante
5	La captura de <code>velocidad.view()</code>
6	La tabla de experimentos completa, con la columna «¿Coherente?» justificada

 Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD02](#) · [Taller 2](#) · [Ejercicios](#)



24 de agosto de 2026

UD02 · Taller 2 — Sistema basado en reglas con experta

**Entregable de la unidad**

Cuenta en el 40 % de actividades del RA2, junto con los otros talleres y la [actividad entregable](#).

**Requisitos**

```
1 pip install "frozendict>=2.3"
2 pip install --no-deps experta
3 pip install schema==0.6.7
```

**experta necesita un parche en Python 3.10+**

experta 1.9.4 importa `collections.Mapping`, retirado de la librería estándar. El parche (issue [#34](#) del proyecto) va **siempre antes** de `from experta import *`, o el import falla con `ImportError: cannot import name 'Mapping' from 'collections'`. Ver Fase 2.

Objetivo: implementar un **mini sistema experto de diagnóstico** de un PC que no arranca, con `experta`, y comprobar en código el ciclo reconocer-actuar de un sistema basado en reglas (RA2-e).

**Hazlo en el notebook**

Tienes este taller como **notebook con las celdas a rellenar**: [UD02_T02_sistema_reglas.ipynb](#). Esta página es la referencia; lo que se entrega es el notebook completado.

FASE 1 — INSTALA Y VERIFICA EL PARCHÉ

```
1 pip install "frozendict>=2.3"
2 pip install --no-deps experta
3 pip install schema==0.6.7
```

FASE 2 — PARCHÉ DE COMPATIBILIDAD E IMPORTACIÓN

```
1 import collections
2 import collections.abc
3 if not hasattr(collections, 'Mapping'):
4     collections.Mapping = collections.abc.Mapping
5     collections.Iterable = collections.abc.Iterable
6     collections.MutableMapping = collections.abc.MutableMapping
7
8 from experta import *
```

FASE 3 — DEFINE EL MOTOR DE CONOCIMIENTO

```
1 class DiagnosticoPC(KnowledgeEngine):
2     @DefFacts()
3     def inicio(self):
4         yield Fact(accion="diagnosticar")
5
6     @Rule(Fact(accion="diagnosticar"), salience=10)
7     def arrancar(self):
8         print("Diagnóstico del PC...")
9         self.declare(Fact(luz_encendida=True))
10        self.declare(Fact(sonido="pitidos_cortos"))
11        self.declare(Fact(pantalla="negra"))
12
13    @Rule(Fact(luz_encendida=True), Fact(sonido="pitidos_cortos"))
14    def ram(self):
15        self.declare(Fact(causa="problema_ram"))
16
17    @Rule(Fact(luz_encendida=True), Fact(pantalla="negra"))
18    def gpu(self):
19        self.declare(Fact(causa="problema_grafica"))
20
21    @Rule(Fact(causa="problema_ram"))
22    def resultado_ram(self):
23        print("DIAGNÓSTICO: Fallo de memoria RAM. Resitúa o sustituye los módulos.")
24
25    @Rule(Fact(causa="problema_grafica"))
26    def resultado_gpu(self):
27        print("DIAGNÓSTICO: Fallo de gráfica o cable de vídeo. Revisa la conexión.")
```

FASE 4 — EJECUTA Y OBSERVA LA AGENDA

```
1 engine = DiagnosticoPC()
2 engine.reset() # declara DefFacts + InitialFact
3 engine.run()   # ciclo reconocer-actuar
```

Registra en el informe la salida y explica qué hechos se declararon y qué reglas se dispararon.

FASE 5 — AÑADE UNA REGLA CON `salience` Y VARIABLES

Modifica el motor para que el diagnóstico tenga **prioridad** y acepte un hecho parametrizado:

```
1 @Rule(Fact(causa="problema_ram"), salience=5)
2 def prioridad(self):
3     print("> (prioridad: revisar RAM antes que cualquier otra acción)")
```

FASE 6 — AMPLÍA EL SISTEMA

Añade dos reglas nuevas: si `luz_encendida=False` → `causa="sin_alimentacion"` con su mensaje de diagnóstico, y si `pantalla="mensaje_bios"` → `causa="configuracion_bios"`. Responde en el informe: *¿cuándo usarías lógica difusa (Taller 1) y cuándo un sistema basado en reglas como este? Justifica con un ejemplo real.*

ENTREGA DEL TALLER 2

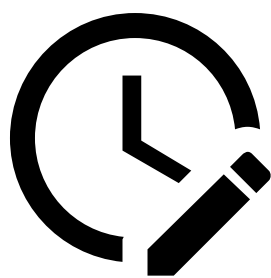
Fase	Evidencia mínima
1-2	El parche aplicado y la importación sin errores
3	El código completo del motor de conocimiento
4	La salida de <code>engine.run()</code> , con los hechos declarados y las reglas disparadas explicados
5	La regla con <code>salience</code> añadida y su efecto comprobado
6	Las dos reglas nuevas y la respuesta comparando lógica difusa frente a reglas



Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD02 · Taller 1 · Ejercicios](#)



24 de agosto de 2026

UD02 · Taller 3 — Preparar el entorno para Robocode (Java o Python)

**Entregable de la unidad**

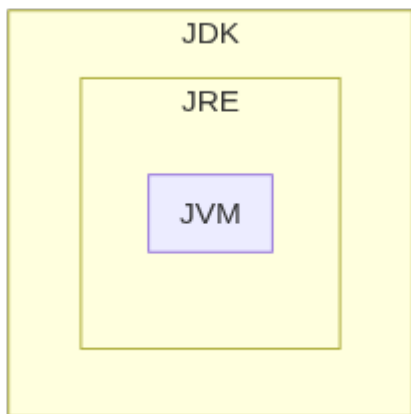
Cuenta en el 40 % de actividades del RA2, como preparación imprescindible de la [actividad entregable](#) (Robocode Tank Royale).

Objetivo: dejar listo el entorno de desarrollo del lenguaje elegido — Java con IntelliJ IDEA o Python con VS Code — y comprobarlo con un programa mínimo, antes de instalar la API de Robocode. Consulta antes la [comparativa Java vs Python](#) si aún no has elegido lenguaje. Sigue solo el bloque de tu lenguaje.

Java

FASE 1 — INSTALA EL JDK

El **JDK** (Java Development Kit) incluye el compilador y las herramientas para desarrollar en Java; contiene a su vez el **JRE** (entorno de ejecución) y este a la **JVM** (máquina virtual, la que interpreta el *bytecode*).



Usa **Java SE** (Standard Edition), la versión de este curso. Recomendado: [Adoptium](#) (antes AdoptOpenJDK, fundación Eclipse), sin las restricciones de licencia que Oracle introdujo desde el JDK 11 para uso comercial.

**En GNU/Linux**

```
1 sudo apt install default-jdk      # instala el JDK predeterminado
2 java --version                   # comprueba la versión activa
3 sudo update-alternatives --config java # si tienes varias versiones instaladas
```

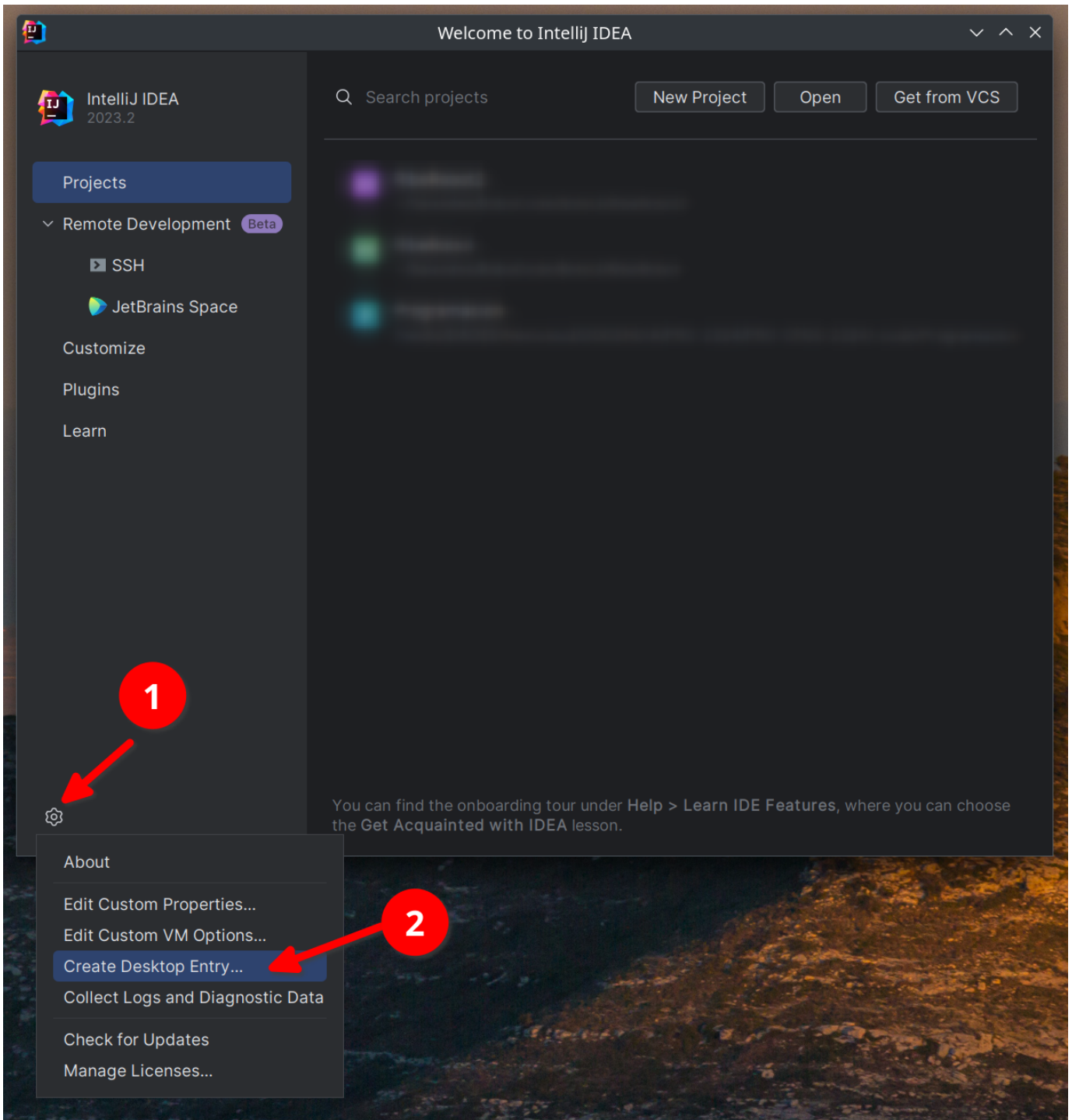
Tras instalar, comprueba que `JAVA_HOME` apunta a la carpeta de instalación y que `PATH` incluye su subcarpeta `bin` — la mayoría de instaladores lo hacen automáticamente; en GNU/Linux con `.tar.gz` hay que definirlos a mano.

FASE 2 — INSTALA Y ACTIVA INTELLIJ IDEA

Descarga la *toolbox* desde jetbrains.com/idea y sigue la [guía de instalación](#) de tu sistema operativo.

**Licencia del centro**

El instituto tiene licencia de JetBrains para el alumnado con correo `@ieseduardoprimo.es`. Actívala en *Administrar licencias* apuntando al servidor <https://iesepm.fls.jetbrains.com/> ([instrucciones](#)).



FASE 3 — PRIMER PROGRAMA Y VERIFICACIÓN

Crea un proyecto Java nuevo siguiendo la [guía oficial de tu primera aplicación](#) con una clase `HolaMundo.java` que imprima un mensaje, y ejecútala desde el propio IDE.

FASE 4 — INSTALA LA API DE ROBOCODE

Se instala más adelante, dentro del propio [tutorial de Robocode en Java](#) (usa Maven para gestionar la dependencia).

Python

FASE 1 — INSTALA PYTHON

```
1 # Linux (Debian/Ubuntu)
2 sudo apt install python3 python3-pip python3-venv
3 # macOS
4 brew install python3
5 # Windows: descarga el instalador de python.org y marca "Add Python to PATH"
```

```
1 python3 --version # en Windows: python --version
```

FASE 2 — INSTALA VS CODE Y LAS EXTENSIONES

Descarga [VS Code](#) e instala, desde el mercado de extensiones, **Python** y **Pylance** (ambas de Microsoft): dan *linting*, depuración, IntelliSense y tipado avanzado.

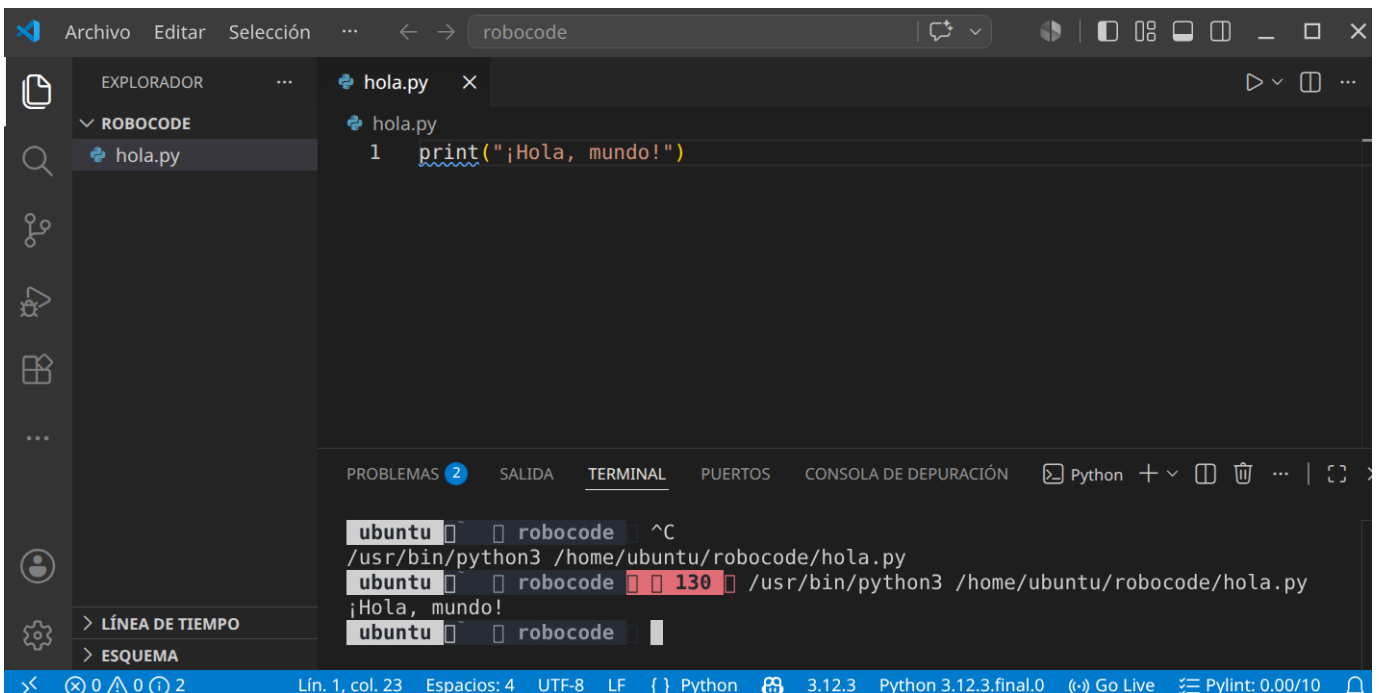
FASE 3 — ENTORNO VIRTUAL Y PRIMER PROGRAMA

```
1 python3 -m venv .venv
2 source .venv/bin/activate # Windows: .venv\Scripts\activate
```

Crea `hola.py`:

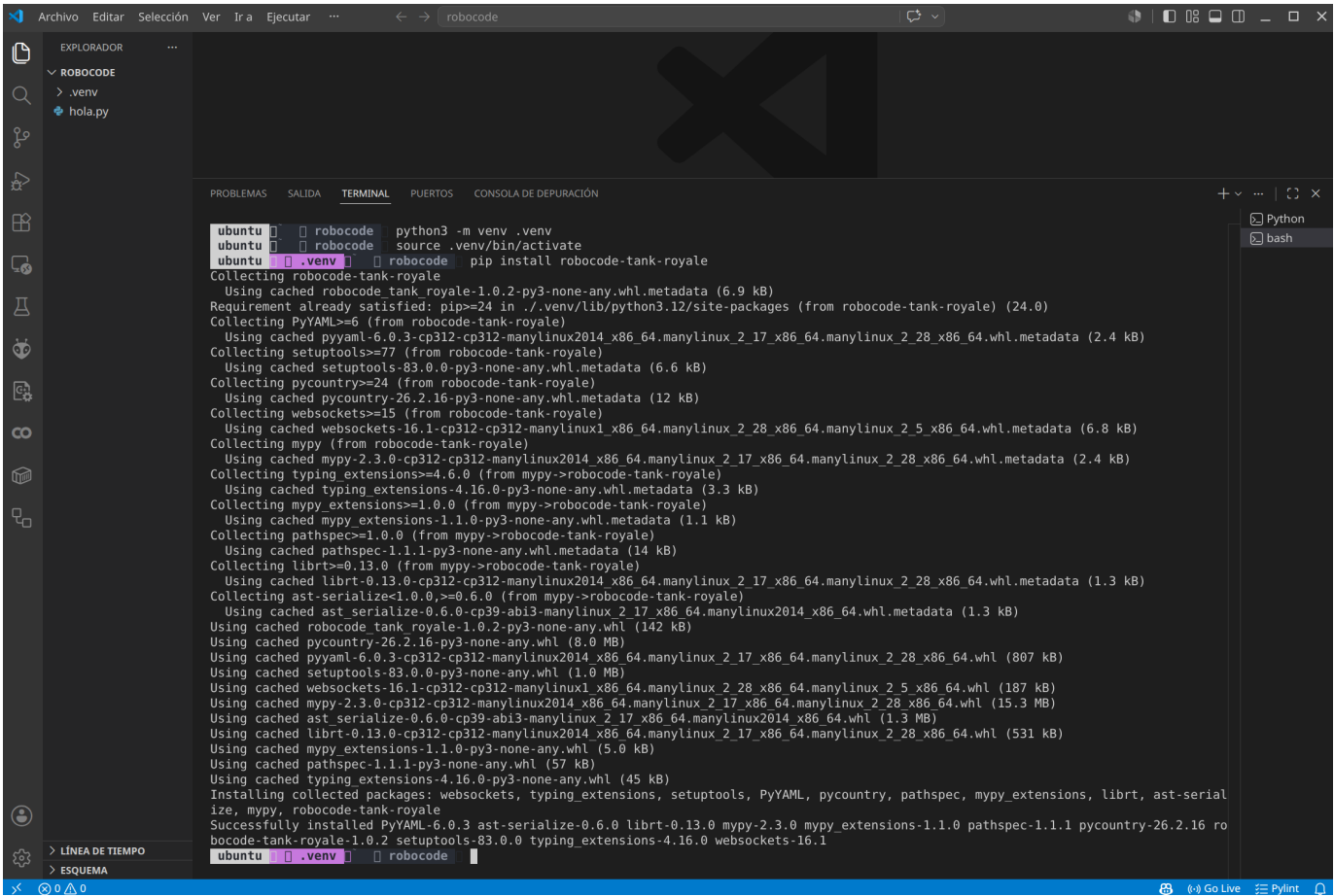
```
1 print("¡Hola, mundo!")
```

```
1 python3 hola.py
```



FASE 4 — INSTALA LA API DE ROBOCODE

```
1 pip install robocode-tank-royale
```



Entrega del Taller 3

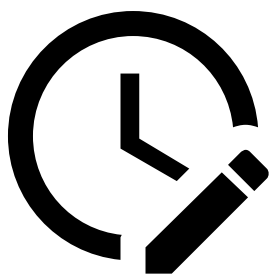
Un documento PDF con:

Si elegiste	Evidencia mínima
Java	Captura de <code>java --version</code> ; capturas de <code>HolaMundo.java</code> editado, compilado y ejecutado en IntelliJ
Python	Captura de <code>python3 --version</code> ; captura de VS Code con <code>hola.py</code> abierto, la extensión Python visible y la terminal mostrando la salida

Soluciones

No aplica: este taller es de preparación del entorno, no tiene solución que corregir — se comprueba que el entorno funciona.

Volver a la UD02 · Taller 4 · Comparativa Java/Python



24 de agosto de 2026

UD02 · Taller 4 — Control de versiones con GitHub

**Entregable de la unidad**

Cuenta en el 40 % de actividades del RA2, como preparación de la **actividad entregable**: en Robocode entregarás el código de tu bot, y saber moverte en un repositorio es parte de las herramientas del oficio.

Objetivo: crear una cuenta en GitHub y completar un ciclo real de colaboración — *fork*, edición, *commit* y *pull request* — proponiendo una mejora a un repositorio ajeno.

**Qué es GitHub**

GitHub es una plataforma en la nube basada en Git que permite almacenar, gestionar y colaborar en proyectos de código. Es el portafolio técnico habitual de cualquier programador: muestra tus proyectos y tu evolución, permite colaborar en proyectos de código abierto y es la herramienta de control de versiones y trabajo en equipo más usada en el sector.

FASE 1 — CREA TU CUENTA

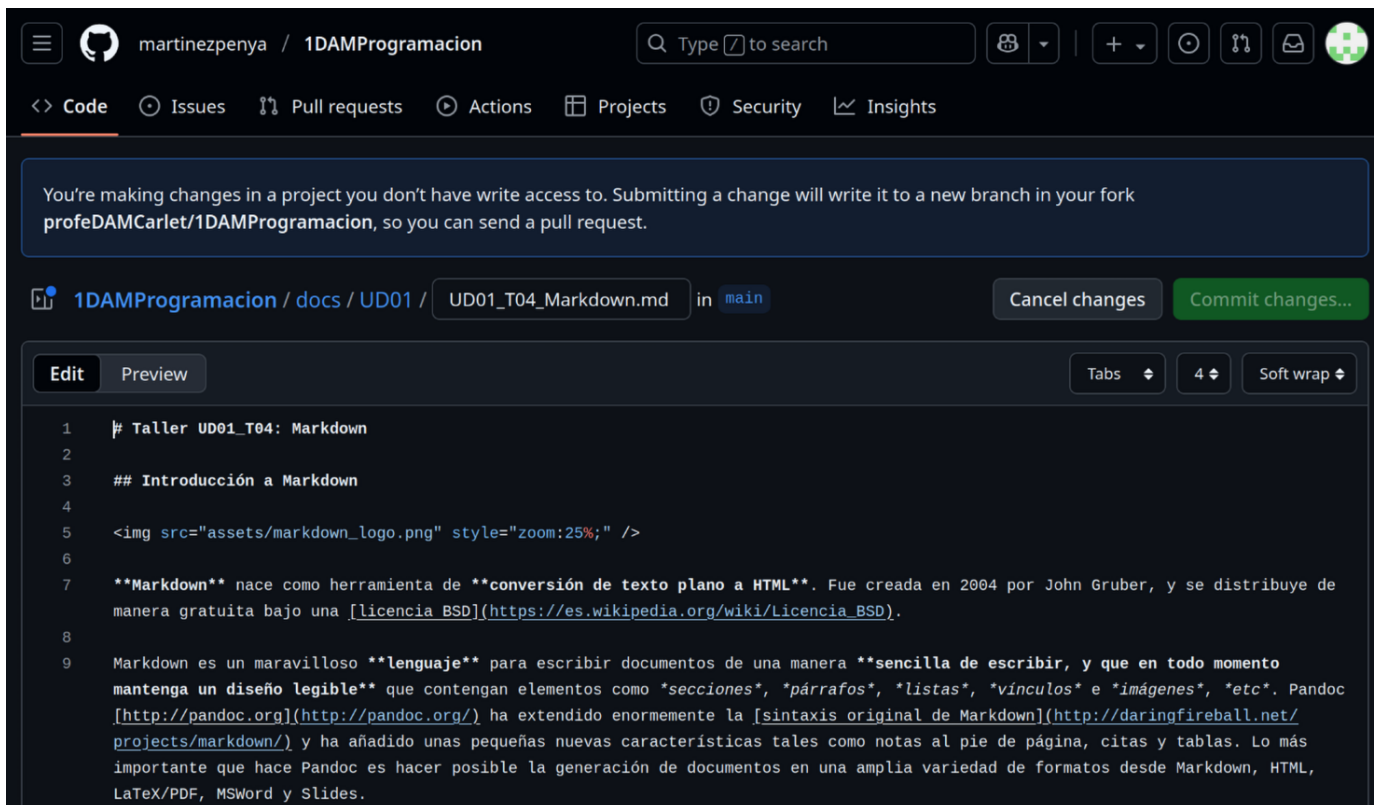
Accede a github.com, pulsa **Sign Up** y sigue las instrucciones. Entra después en tu página de perfil (github.com/tu-usuario) y haz una captura.

FASE 2 — HAZ UN FORK DEL REPOSITORIO

Busca un repositorio con documentación que quieras mejorar (por ejemplo, unos apuntes en los que hayas detectado un error) y pulsa el icono de edición:

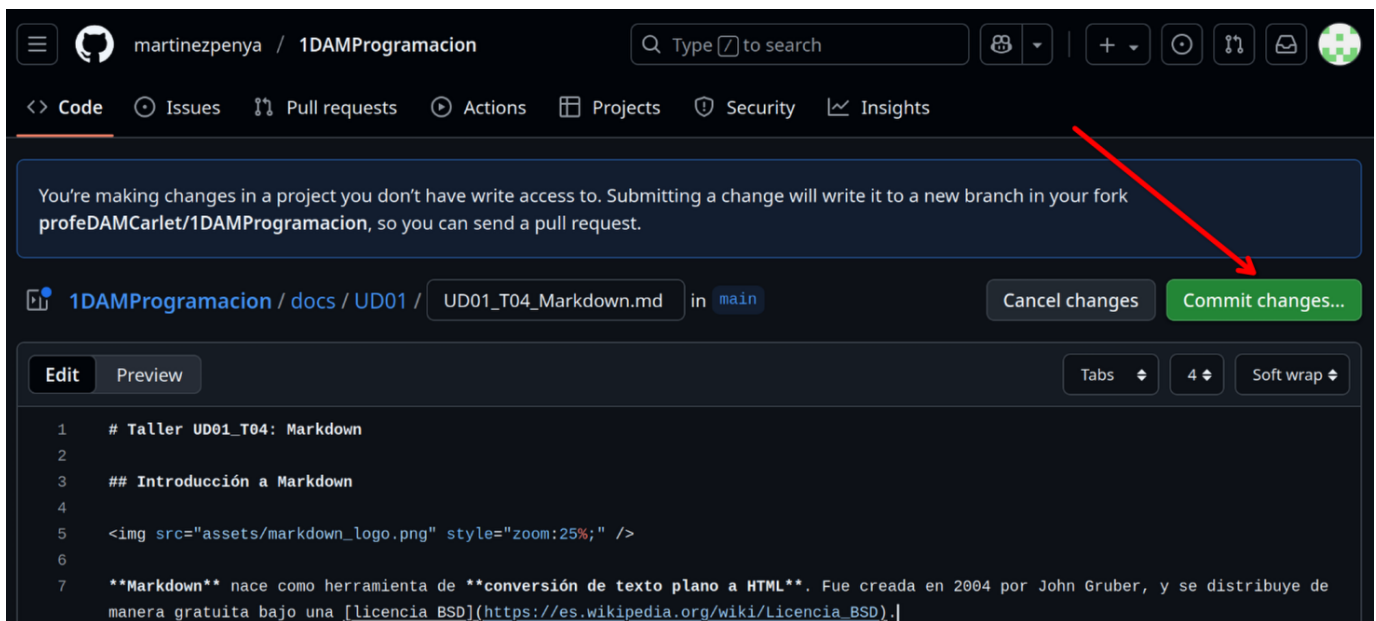
Esto crea un **fork**: una copia del repositorio bajo tu cuenta, sobre la que puedes trabajar sin tocar el original.

Pulsa **Fork this repository**. Verás el código fuente de la página, escrito en **Markdown** (lo trabajarás a fondo en el Taller 5):



FASE 3 — EDITA Y HAZ COMMIT

Localiza el texto a modificar. En cuanto cambies algo se activa el botón **Commit changes...**:



Describe brevemente qué has cambiado y pulsa **Propose changes**:

Propose changes

Commit message

Update UD01_T04_Markdown.md

Extended description

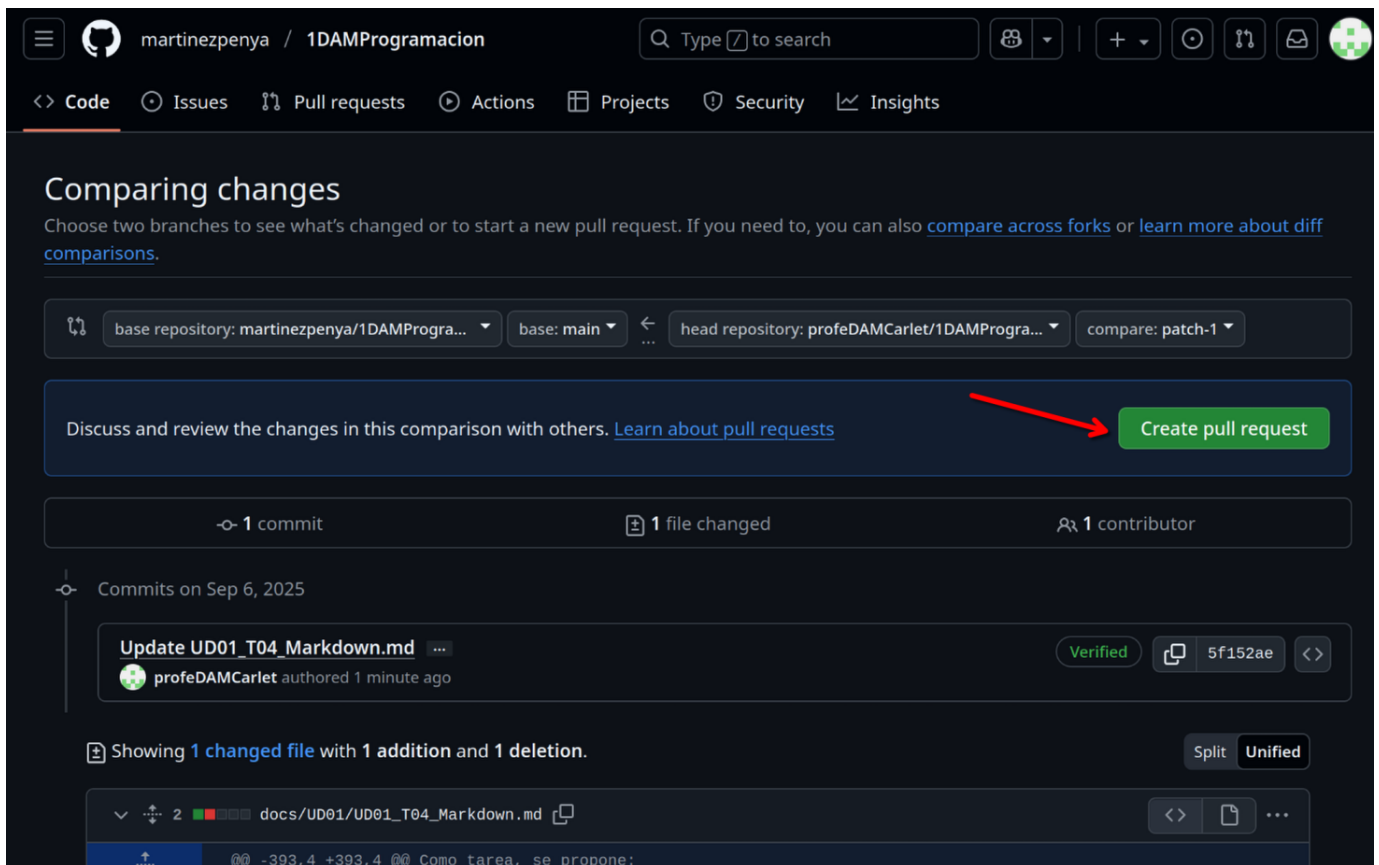
He cambiado el formato de las extensiones de los documentos
a entregar.

Cancel

Propose changes

FASE 4 — ABRE UN PULL REQUEST

Un *commit* en tu fork no cambia nada en el repositorio original: para proponer tu cambio al propietario, pulsa **Create pull request**:



martinezpenya / 1DAMProgramacion

Q Type to search

<> Code Issues Pull requests Actions Projects Security Insights

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: martinezpenya/1DAMProgra... base: main head repository: profeDAMCarlet/1DAMProgra... compare: patch-1

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) **Create pull request**

1 commit 1 file changed 1 contributor

Commits on Sep 6, 2025

Update UD01_T04_Markdown.md Verified 5f152ae

profeDAMCarlet authored 1 minute ago

Showing 1 changed file with 1 addition and 1 deletion. Split Unified

docs/UD01/UD01_T04_Markdown.md

-393,4 +393,4 Como tarea, se propone:

Puedes editar el mensaje o dejarlo tal cual; pulsa de nuevo **Create pull request**:

martinezpenya / 1DAMProgramacion

<> Code Issues Pull requests Actions Projects Security Insights

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base repository: martinezpenya/1DAMProgra... base: main head repository: profeDAMCarlet/1DAMProgra... compare: patch-1

Add a title

Update UD01_T04_Markdown.md

Add a description

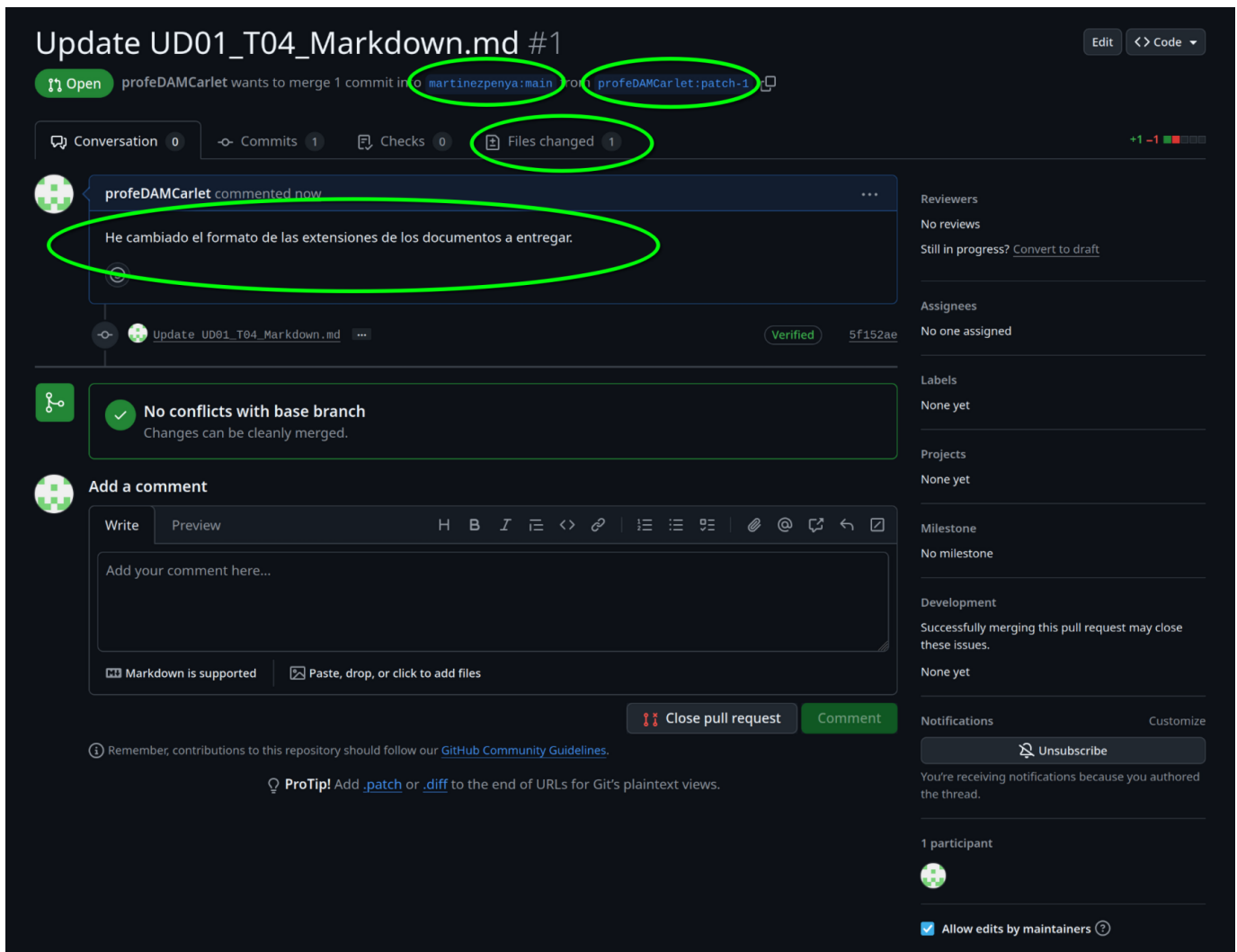
Write Preview H B I ...

He cambiado el formato de las extensiones de los documentos a entregar.

☒ Allow edits by maintainers **Create pull request**

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Deberías llegar a una página como esta. Haz una captura y explica los 4 campos señalados: qué repositorio recibe el cambio, desde qué rama de tu fork, el estado de la comparación y el estado del *pull request*.



FASE 5 — RESULTADO

En este punto has completado el ciclo: fork → edición → commit → pull request. Ahora depende de la persona propietaria del repositorio aceptar el cambio o no — **ambos resultados son válidos** para esta actividad, lo que se evalúa es el proceso, no que te lo acepten. Si se acepta, verás algo así:

Update UD01_T04_Markdown.md #1

Merged martinezpenya merged 1 commit into martinezpenya:main from profeDAMCarlet:patch-1 1 minute ago

Conversation 1

Commits 1

Checks 0

Files changed 1



profeDAMCarlet commented 10 minutes ago

Contributor ...

He cambiado el formato de las extensiones de los documentos a entregar.





Update UD01_T04_Markdown.md ...

Verified 5f152ae



martinezpenya commented 1 minute ago

Owner ...

Perfecto, gracias!





martinezpenya closed this 1 minute ago



martinezpenya merged commit 8052475 into martinezpenya:main 1 minute ago

Revert

ENTREGA DEL TALLER 4

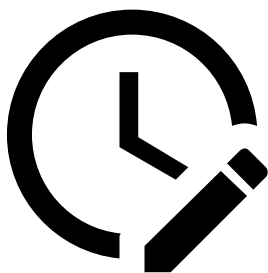
Fase	Evidencia mínima
1	Captura de tu perfil de GitHub
2-4	Captura del <i>pull request</i> creado, con los 4 campos señalados y explicados
5	Estado final del <i>pull request</i> (aceptado o pendiente; ambos son válidos)



Soluciones

No aplica: este taller se evalúa por el proceso completado, no hay una única respuesta correcta que corregir en clase.

[Volver a la UD02](#) · [Taller 3](#) · [Taller 5](#)



24 de agosto de 2026

UD02 · Taller 5 — Documentar con Markdown

**Entregable de la unidad**

Cuenta en el 40 % de actividades del RA2, como preparación de la [actividad entregable](#): la memoria de Robocode se escribe y documenta más rápido si dominas Markdown, y es el lenguaje en el que están escritos estos mismos apuntes.

Objetivo: aprender la sintaxis de Markdown lo bastante a fondo como para documentar cualquier proyecto — texto, listas, tablas, enlaces, imágenes y código — y escribir un documento propio de principio a fin.

**Qué es y para qué sirve**

Markdown es un lenguaje de marcado ligero creado en 2004 por John Gruber para convertir texto plano en HTML de forma sencilla y legible incluso sin procesar. Se usa hoy en foros (Stack Overflow), gestores de tareas (Trello), documentación técnica (GitHub, estos apuntes) y apps de notas — es **transportable**: el mismo `.md` se lee y edita en cualquier plataforma sin depender de un procesador de textos propietario. Editores habituales: **Typora**, VS Code (con previsualización integrada) o directamente el editor web de GitHub.

FASE 1 — TEXTO BÁSICO: PÁRRAFOS, ÉNFASIS Y ENCABEZADOS

Un párrafo nuevo se crea con una línea en blanco entre dos bloques de texto — Markdown, como HTML, **colapsa las líneas en blanco dobles** en una sola. Para forzar un salto de línea dentro del mismo párrafo, termina la línea con dos espacios.

```
1 Este texto es en **negrita**.
2 Este texto es en *cursiva*.
3 Este texto está tachado.
4 Este texto es en ambos ***negrita y cursiva***.
```

Los encabezados usan `#` — uno por nivel, del `#` (H1) al `#####` (H6):

```
1 # Encabezado 1
2 ## Encabezado 2
3 ### Encabezado 3
```

FASE 2 — LISTAS, TABLAS, ENLACES E IMÁGENES

Listas ordenadas (`1.`), no ordenadas (`-`, `*` o `+`, sin mezclar dentro de la misma lista) y de tareas:

```
1 1. Primer paso
2 2. Segundo paso
3
4 - Elemento A
5 - Elemento B
6
7 - [x] Tarea completada
8 - [ ] Tarea pendiente
```

Tablas, con `|` para columnas y `---` para separar el encabezado:

```
1 | Columna 1 | Columna 2 |
2 |---|---|
3 | dato 1.1 | dato 1.2 |
```

Enlaces e imágenes comparten sintaxis; la imagen añade `!` delante:

```
1 [texto del enlace](https://ejemplo.com)
2 ![texto alternativo](ruta/imagen.png)
```

FASE 3 — CÓDIGO, CITAS Y SEPARADORES

Para código **en línea**, una comilla invertida a cada lado: ``código``. Para un **bloque**, tres comillas invertidas con el nombre del lenguaje:


```
1  ```python
2  print("Hola, mundo")
3  ```
```

Las citas empiezan por `>`:

```
1  > No hay que ir para atrás, ni para darse impulso. — Lao Tsé.
```



En estos apuntes, las citas `>` no se usan para notas

Es una convención de este curso: toda nota, aviso o definición destacada va en una **admonición** (`!!! tip`, `!!! note`, `!!! warning`...), como las de esta misma página — no en una cita `>`. Reserva `>` para citar literalmente a alguien, como en el ejemplo de Lao Tsé.

Una línea horizontal de separación se crea con tres guiones en su propia línea: `---`.



Diagramas: `flow` / `sequence` frente a Mermaid

Algunos editores de Markdown (Typora entre ellos) admiten bloques ````flow` o ````sequence` para diagramas de flujo y de secuencia con su propia sintaxis. **El sitio de esta asignatura no los renderiza:** usa **Mermaid** (````mermaid`), que sí verás en la teoría de cada unidad. Si trabajas en Typora para tus propios apuntes puedes usar cualquiera de los dos; para lo que entregues en este curso, usa Mermaid.

FASE 4 — PRACTICA: DOCUMENTO SOBRE TI MISMO

Con tu editor favorito, crea un documento Markdown que:

1. Tenga un título y, si tu editor lo soporta, un índice (`[TOC]`).
2. Incluya **4 encabezados principales** — por ejemplo *Datos*, *Currículum*, *Aficiones* y *Otros datos de interés* (no hace falta que sea información real; puedes inventarla).
3. Use al menos: negrita, cursiva, una lista ordenada, una lista no ordenada, un enlace, una imagen, una cita y un bloque de código.
4. *Opcional, para nota extra:* un diagrama Mermaid con los pasos de una mañana de sábado.

Exporta el resultado a PDF (la mayoría de editores Markdown lo hacen con un clic, o usa `pandoc documento.md -o documento.pdf`).

ENTREGA DEL TALLER 5

Fase	Evidencia mínima
1-3	El documento <code>.md</code> usa correctamente encabezados, énfasis, listas, tabla, enlace, imagen, cita y bloque de código
4	El documento tiene 4 encabezados principales con contenido propio
Extra	Diagrama Mermaid incluido (no obligatorio)

Sube a Moodle el documento `.md` original y su exportación a `.pdf`.



Soluciones

No aplica: es un documento de creación libre, se comenta en clase con ejemplos del alumnado.

Volver a la UD02 · Taller 4 · Ejercicios



24 de agosto de 2026

4.5 Actividades Entregables

UD02 — Actividad entregable: Robocode Tank Royale



Cierre de la unidad

Robocode es la práctica de cierre del RA2, del **23 de noviembre al 3 de diciembre**. Cuenta dentro del 40 % de actividades de la unidad, junto con los 5 talleres.

Introducción

Robocode es un juego de programación donde el objetivo es codificar un bot en forma de tanque virtual para competir contra otros bots en un campo de batalla. Quien juega es quien programa el bot: no hay control directo durante la partida. El programa dice cómo debe comportarse y reaccionar el bot ante los eventos del campo de batalla — es, en la práctica, el sistema de resolución de problemas del §10 de la teoría hecho código.

El nombre viene de una versión anterior del juego ("Código de robot"); esta versión usa "bot" en vez de "robot". Las batallas tienen lugar en un campo donde varios bots luchan hasta que solo queda uno, como en un *Battle Royale* — de ahí Tank **Royale**.

- Documentación del juego: <https://robocode-dev.github.io/tank-royale/>
- Repositorio en GitHub: <https://github.com/robocode-dev/tank-royale>
- API Java: [overview](#) · [Bot API](#)



Tank Royale no es el Robocode clásico

RCTR se basa en una versión más antigua de Robocode, pero con un API distinto: los bots antiguos y el funcionamiento del campo de batalla han cambiado sustancialmente. Mucha documentación que encontrarás en internet es del sistema antiguo — la estrategia sigue siendo válida, pero el código no. Hay un puente entre las dos versiones en [robocode-api-bridge](#), la documentación y estrategias de la API antigua siguen en [robocode.sourceforge.io](#), y un archivo con mucha información histórica sobre estrategias y robots está cacheado en [Wayback Machine](#).

Antes de empezar

1. Elige lenguaje con la [comparativa Java vs Python](#).
2. Completa el [Taller 3](#) para dejar listo tu entorno.
3. Sigue el tutorial completo: [Robocode en Java](#) o [Robocode en Python](#), según el lenguaje elegido.

Objetivo de la práctica

Usando Robocode Tank Royale, te pones en el papel de los primeros programadores que dotaban de «inteligencia» a los primeros sistemas: reaccionan solo ante estímulos que su programador previó, no tienen intuición y el resultado es tan bueno como lo sea su programación — el mismo debate del §9 de la teoría sobre cuándo usar reglas y cuándo aprendizaje.

El profesor establecerá unos bots simples que tu bot deberá derrotar para superar la práctica: no sirve ganar por azar, tiene que estar razonado y documentado. Después habrá una competición entre todo el alumnado.

Pasos a seguir

1. **Preparación del entorno** — en una sesión conjunta, instalación y primer combate de pruebas.
2. **Mi primer bot** — con la guía de la documentación y el profesor, generar el primer bot, añadirlo a la batalla y depurar su funcionamiento.
3. **¿Cómo mejoro mi bot?** — estudiar estrategias y mejoras para dotarlo de comportamiento inteligente (radar, movimiento, disparo — ver los tutoriales).
4. **Investigación y desarrollo propio** — trabajo individual: prever a los adversarios (los conocidos y los de tus compañeros) y aplicar las técnicas que consideres más útiles para subir en la clasificación.

Qué debes entregar

A través de Moodle, un archivo **ZIP** que contenga el código fuente de tu bot (nombre: tu nombre más los 4 últimos dígitos de tu DNI/NIE, sin letras) y la memoria justificativa en PDF.

- `TuNombreNNNN.java` (Java) o `TuNombreNNNN.py` (Python) — clase del bot
- `TuNombreNNNN.json` — información del autor
- `TuNombreNNNN.cmd` y `TuNombreNNNN.sh` — arranque en Windows y Linux

La memoria en PDF debe incluir, al menos: datos del alumnado, descripción del funcionamiento (estructura, sistema basado en reglas/casos, análisis y evolución de la solución...), descripción detallada de los métodos definidos o usados, conclusiones y webgrafía/bibliografía.

Requisitos mínimos de la competición

- **Versión 0.34.0 de la API** (multi-idioma y con escalado)
- Modo **classic**, 10 asaltos, tamaño de campo 2000×2000
- Adversarios de referencia: `RamFire`, `Walls`, `SpinBot`
- Gana quien acumule más puntuación; velocidad de enfriamiento 0.1; tiempo máximo de inactividad 450
- No se permite llamar a métodos prohibidos ni ganar por azar: **hay que demostrar IA**
- Los combates se graban y se devuelven como feedback

Rúbrica

Rúbrica real de la tarea en Moodle (`assign_8288613`, método `rubric`). Suma **10 puntos**: el peso del resultado en combate (criterios 3 y 4) es de **8 sobre 10** — se premia el comportamiento del bot, no solo la documentación.

CRITERIO 1 · ENTREGA DE FICHEROS (0-1 PUNTO)

Puntos	Nivel
0	No entregados
0,2	Faltan archivos o es imposible hacer funcionar el bot
0,4	Se entregan todos o casi todos los ficheros, pero el docente no consigue hacer funcionar el bot
0,6	Ficheros con el formato correcto; el bot funciona con grandes correcciones del docente
0,8	Ficheros con el formato correcto; el bot funciona con pequeñas correcciones del docente
1	Se entregan únicamente los ficheros solicitados, con el formato correcto, y el bot funciona sin que el docente deba modificarlos

CRITERIO 2 · MEMORIA (0-1 PUNTO)

Puntos	Nivel
0	No entregada
0,25	Insuficiente
0,5	Suficiente
0,75	Bien

Puntos	Nivel
1	Muy bien

CRITERIO 3 · RESULTADO DE LA MELÉ FINAL, TODOS CONTRA TODOS (0-3 PUNTOS)

Puntos	Nivel
0	No entregado
0,5	Ni gana ni se acerca en ninguna de las rondas
1	No gana ninguna ronda, pero queda relativamente cerca de la cabeza en las 3
2	Gana al menos una de las 3 rondas
3	Gana las 3 rondas

CRITERIO 4 · RESULTADO CONTRA RAMFIRE, WALLS Y SPINBOT (0-5 PUNTOS)

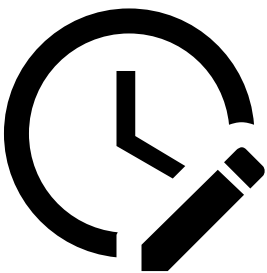
Configuración: 3 rondas × 10 asaltos.

Puntos	Nivel
0	No entregado
1	Queda al final de la clasificación y muy alejado en puntos
2	Queda al final, pero relativamente cerca en puntuación
3	No gana ninguna ronda, pero queda cerca de la cabeza en alguna
4	Gana al menos una de las 3 rondas
5	Gana las 3 rondas

**Soluciones**

Las soluciones no se publican: se corrigen en clase y en la competición final.

[Volver a la UD02](#) · [Comparativa Java/Python](#)



24 de agosto de 2026

UD02 · Comparativa Java vs Python para Robocode

Robocode Tank Royale (RCTR) ofrece API oficial para múltiples lenguajes. En esta unidad podéis elegir entre **Java (JVM)** y **Python**. A continuación se detallan las diferencias, ventajas e inconvenientes de cada opción.

¿Por qué dos lenguajes?

RCTR v0.34.0 introdujo la API multilingüe. El proyecto Robocode nació en Java, pero la comunidad y la evolución del software han traído soporte para Python, .NET y TypeScript. Esto permite elegir el lenguaje que mejor se adapte a vuestro perfil y a los objetivos del curso.

Tabla comparativa

Aspecto	Java	Python
Entorno requerido	JDK 11-23 + IntelliJ IDEA + Maven	Python 3.10+ + VS Code + pip
Dependencias	<code>pom.xml</code> (Maven) o JAR manual	<code>pip install robocode-tank-royale</code>
Sintaxis API	<code>camelCase()</code> — <code>setTurnLeft()</code> , <code>onScannedBot()</code>	<code>snake_case()</code> — <code>set_turn_left()</code> , <code>on_scanned_bot()</code>
Tipado	Estático (obligatorio)	Dinámico (opcional, con type hints)
Proyecto starter	ZIP preempaquetado (<code>IABDBotMaven.zip</code>)	Manual: 4 ficheros (<code>.py</code> + <code>.json</code> + <code>.cmd</code> + <code>.sh</code>)
Conf. servidor	Variables de entorno en configuración de ejecución de IntelliJ	Variables de entorno <code>SERVER_SECRET</code> y <code>SERVER_URL</code> en terminal
Curva aprendizaje	Más pronunciada (JDK, Maven, IDE pesado)	Más suave (pip, editor ligero)
Rendimiento batalla	Compilado JIT (mayor velocidad)	Interpretado (suficiente para RCTR, pero menor rendimiento)
Ecosistema IA/ML	Limitado para IA aplicada	Rico: scikit-learn, PyTorch, TensorFlow, etc.
Documentación comunitaria	RoboWiki mayoritariamente en Java	RoboWiki en Java (traducción a Python necesaria)
Uso en CE IA&BD	Transversal (programación genérica)	Alineado (Python es el lenguaje de referencia en IA)
API RCTR	Madura (existe desde el origen)	Estable (v0.34.0+, en evolución)

Ejemplos de código comparativos

CLASE BOT E IMPORTS

Java

Python

```

1  import dev.robocode.tankroyale.botapi.*;
2  import dev.robocode.tankroyale.botapi.events.*;
3
4  public class MiBot extends Bot {
5      MiBot() {
6          super(BotInfo.fromFile(getConfigPath()));
7      }
8
9      private static String getConfigPath() {
10         String configPath = "src/main/java/MiBot.json";
11         java.io.File configFile = new java.io.File(configPath);
12         if (!configFile.exists()) {
13             configPath = "MiBot.json";
14         }
15         return configPath;
16     }
17 }

```

```

1  from robocode_tank_royale.bot_api import Bot, BotInfo
2
3  class MiBot(Bot):
4      def __init__(self):
5          super().__init__(BotInfo.from_file("MiBot.json"))

```

MÉTODO run() Y BUCLE PRINCIPAL

Java

Python

```

1  public void run() {
2      setAdjustRadarForBodyTurn(true);
3      setAdjustGunForBodyTurn(true);
4      setAdjustRadarForGunTurn(true);
5
6      while (isRunning()) {
7          setTurnRadarLeft(Double.POSITIVE_INFINITY);
8          forward(100);
9          turnGunLeft(360);
10         back(100);
11         turnGunLeft(360);
12     }
13 }

```

```

1  def run(self):
2      self.set_adjust_radar_for_body_turn(True)
3      self.set_adjust_gun_for_body_turn(True)
4      self.set_adjust_radar_for_gun_turn(True)
5
6      while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
7          self.set_turn_radar_left(float("inf"))
8          self.forward(100)
9          self.turn_gun_left(360)
10         self.back(100)
11         self.turn_gun_left(360)

```

EVENT HANDLER `onScannedBot`

Java

Python

```
1 public void onScannedBot(ScannedBotEvent e) {
2     fire(1);
3 }
```

```
1 def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
2     del scanned_bot_event
3     self.fire(1)
```

DISPARO PREDICTIVO (TRIGONOMÉTRICO)

Java

Python

```
1 public void onScannedBot(ScannedBotEvent e) {
2     double enemyDistance = distanceTo(e.getX(), e.getY());
3     double firePower = Math.min(500 / enemyDistance, 3);
4     double bulletSpeed = 20 - firePower * 3;
5     long time = (long) (enemyDistance / bulletSpeed);
6
7     double enemyVelocity = e.getSpeed();
8     double enemyDirection = e.getDirection();
9     double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
10    double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));
11    double futureX = e.getX() + (deltaX * time);
12    double futureY = e.getY() + (deltaY * time);
13
14    setTurnGunLeft(gunBearingTo(futureX, futureY));
15    if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {
16        fire(firePower);
17    }
18 }
```

```
1 def on_scanned_bot(self, e: ScannedBotEvent):
2     enemy_distance = self.distance_to(e.x, e.y)
3     fire_power = min(500 / enemy_distance, 3)
4     bullet_speed = 20 - fire_power * 3
5     time = int(enemy_distance / bullet_speed)
6
7     enemy_velocity = e.speed
8     enemy_direction = e.direction
9     delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
10    delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))
11    future_x = e.x + (delta_x * time)
12    future_y = e.y + (delta_y * time)
13
14    self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))
15    if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
16        self.fire(fire_power)
```

Ventajas e inconvenientes

JAVA

Ventajas:

- Documentación histórica amplia (RoboWiki, ejemplos clásicos)
- Proyecto starter preempaquetado (descomprimir y abrir en IntelliJ)
- Rendimiento compilado mayor en cálculos intensivos
- API muy madura y estable

Inconvenientes:

- Requerimientos de entorno pesados (JDK + IntelliJ)
- Curva de aprendizaje mayor si no se conoce Java/Maven
- Menos alineado con el ecosistema de IA y Big Data del curso
- Configuración de ejecución dependiente del IDE

PYTHON

Ventajas:

- Instalación sencilla (`pip install`)
- Editor ligero (VS Code) y rápido de configurar
- Sintaxis limpia y legible
- Alineado con el resto del curso (ecosistema IA/BD)
- Mejor integración con librerías de IA si se quiere ampliar el bot
- Variables de entorno simples (terminal o script)

Inconvenientes:

- Documentación histórica en Java (hay que traducir conceptualmente)
- Proyecto starter manual (4 ficheros) en lugar de ZIP preempaquetado
- Rendimiento interpretado (normalmente no perceptible en RCTR)

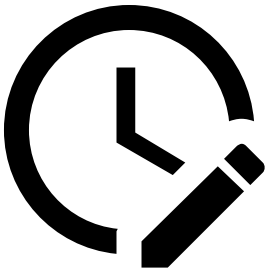
Recomendación

Para la naturaleza de este curso de especialización en **Inteligencia Artificial y Big Data**, se recomienda el uso de **Python** como lenguaje principal. Python es la herramienta estándar de la industria en IA/ML, y su sintaxis clara permite centrarse en las estrategias del bot en lugar de la infraestructura del lenguaje.

La opción Java queda disponible para alumnado con experiencia previa en Java o interés particular, pero la vía Python será la que reciba soporte preferente por parte del profesorado.

**Elegid vuestro camino**

- **Noveles en programación o provenientes de Python** → seguid la guía Python
- **Experiencia previa en Java o interés en JVM** → seguid la guía Java
- **Dudas** → consultad al profesorado



15 de julio de 2026

UD02 · Robocode Tank Royale en Java

Preparación del entorno

- Al menos java 11 (Yo estoy usando Java 23 (OpenJDK) y la versión 0.34.0 de Tank Royale)

```
1 java -version
```

- Descargar la última versión desde releases: <https://github.com/robocode-dev/tank-royale/releases>
- Ejecutar el `jar` descargado:

```
1 java -jar robocode-tankroyale-gui-x.y.z.jar
```

- Descarga y descomprime los Bots de ejemplo: <https://github.com/robocode-dev/tank-royale/releases>
- Configura la gui para acceder a la carpeta de robots: Config -> Bot Root Directories (del menú)
- [opcional] Añadir sonidos al gui. Descarga y descomprime la carpeta `sounds.zip` de: <https://github.com/robocode-dev/sounds/releases>

```
1 [directorio padre]
2 └─ robocode-tankroyale-gui-x.y.z.jar
3 └─ sounds/ <-- carpeta de sonidos
4     └─ bots_collision.wav
5     ...
6     └─ wall_collision.wav
```

- Necesitaras IntelliJ para poner todo en funcionamiento y para programar tu Bot.



Consejo

Juega un poco por tu cuenta con el entorno GUI, explora las opciones, tipos de batallas, crea alguna ronda de las mismas, inicializa Bots, añádelos, juega con la velocidad de juego, etc.

Robocode con Maven

Robocode tankroyale está disponible como artefacto del repositorio de maven, si estas familiarizado con el uso de maven puede simplificar bastante el trabajo y actualizaciones de dependencias, te dejo el enlace por si quieres investigar por tu cuenta, ya que escapa del objetivo de esta asignatura: <https://central.sonatype.com/search?q=g:dev.robocode.tankroyale&smo=true>

Mi primer Bot

PROYECTO INTELLIJ

Para acelerar el proceso, he preparado un proyecto en IntelliJ con toda la arquitectura básica que necesitará tu Bot. Descarga y descomprime [este paquete](#) y abre el proyecto con el IDE IntelliJ.



Sin maven

Si no quieres usar maven y deseas montar el proyecto por tu cuenta sigue estas indicaciones:

También necesitaras la librería (API) de Robocode Tank Royale que puedes descargar desde aquí: <https://github.com/robocode-dev/tank-royale/releases>. Descarga el archivo: `robocode-tankroyale-bot-api-x.y.z.jar`

La estructura debería quedar de la siguiente manera:

```
1 [directorio padre]
2 └─ lib
3     └─ robocode-tankroyale-gui-x.y.z.jar
4 └─ IABDBot <-- Proyecto de IntelliJ
5     └─ src
6     ...
7     └─ IABDBot.iml
```

 Atención

Asegúrate de entender el funcionamiento del Bot IABDBot, tiene comentarios donde se explican partes importante del Bot y que daremos por conocidas en los siguientes puntos.

Es muy importante que entiendas la diferencia entre movimientos bloqueantes y simultáneos, así como las condiciones y los eventos disponibles.

 Ojo!

Gracias a este fragmento, tu bot funcionará correctamente desde IntelliJ y desde el gui de RoboCode:

```

1 // Constructor, que carga el fichero de configuración
2 IABDBot() {
3     super(BotInfo.fromFile(getConfigPath()));
4 }
5
6 // Este método obtiene la ruta del fichero de configuración y se asegura que funcione desde IntelliJ y desde la gui de Robocode
7 private static String getConfigPath() {
8     String miBot = "IABDBot";
9     String configPath = "src/main/java/" + miBot + ".json";
10    java.io.File configFile = new java.io.File(configPath);
11    if (!configFile.exists()) {
12        configPath = miBot + ".json";
13    }
14    return configPath;
15 }
```

ÚLTIMAS NOVEDADES DEL PROYECTO

- Se pueden grabar los combates en un fichero dentro de la carpeta `recordings` con la extensión `battle.gz` que luego podemos reproducir en diferido. Esto nos permite revisar situaciones y estudiar estrategias de mejora.

CONFIGURACIÓN DEL SERVIDOR (GUI) Y EL PROYECTO INTELIJ PARA EJECUTAR EN LOCAL O REMOTO

El servidor permite conexiones en remoto/local a través de un password que se genera automáticamente la primera vez que lanzas la GUI. Este password lo puedes encontrar/modificar en el archivo `server.properties`, en el ejemplo siguiente la clave de acceso es **CEIABDEPM2025**

```

1 bots-secrets=CEIABDEPM2025
2 [...]
```

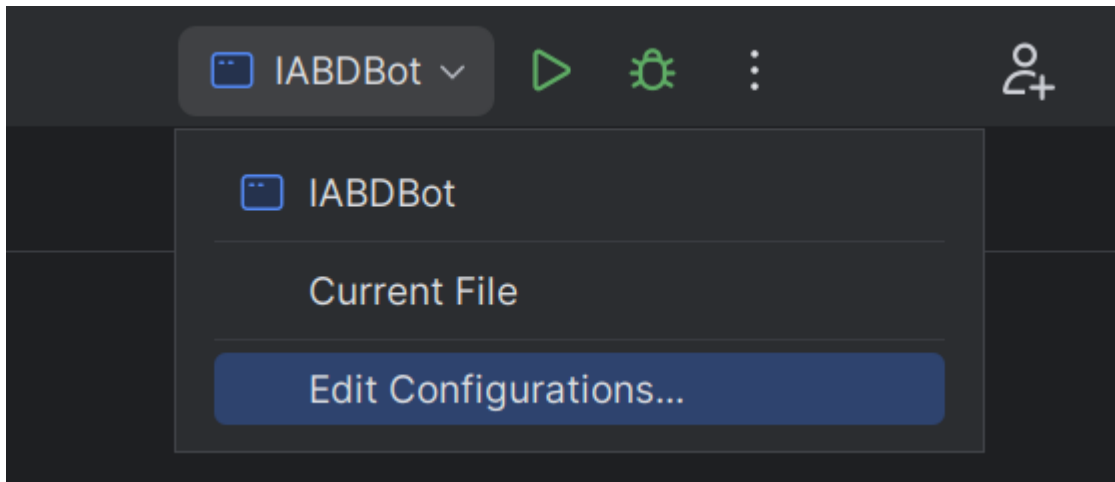
 Ojo

Con las últimas versiones de la gui el uso de contraseñas en el servidor está inicialmente deshabilitado, si queremos habilitarlo lo podemos hacer desde el propio interfaz: `Config/Server Options/Enable server secrets` o bien desde el fichero de configuración `server.properties` añadiendo:

```
1 server-secrets-enabled=true
```

Otra información que necesitaras será la IP del servidor al que quieres conectar, normalmente será `localhost` (tu host local), y cuando sea necesario hacer pruebas el profesor te facilitará su IP.

Ahora solo queda configurar nuestro proyecto de IntelliJ para configurar estas variables en el momento de ejecutar el Bot. Accede a la configuración de las ejecuciones en la parte superior derecha (una vez abierta la clase `IABDBot.java`):



Puedes ver que en el apartado `Environment Variables` tienes el siguiente valor:

```
1 SERVER_SECRET=CEIABDEPM2025;SERVER_URL=ws://localhost:7654
```

Como puedes imaginar **SERVER_SECRET** hace referencia a la clave del servidor, y **SERVER_URL** a la dirección del servidor y es donde deberás reemplazar (según el caso) `localhost` por la IP que te indique el profesor. Así podrás ejecutar tu Bot directamente desde IntelliJ y aparecerá en el Servidor para poder añadirlo a las batallas.

⚡ Atención

Ojo, el servidor debe estar inicializado antes de lanzar el bot de lo contrario obendrás un error similar a este:

```
1 Exception in thread "main" dev.robocode.tankroyale.botapi.BotException: Could not create web socket for URL: ws://localhost:7654
2   at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.connect(BaseBotInternals.java:268)
3   at dev.robocode.tankroyale.botapi.internal.BaseBotInternals.start(BaseBotInternals.java:254)
4   at dev.robocode.tankroyale.botapi.BaseBot.start(BaseBot.java:114)
5   at IABDBot.main(IABDBot.java:11)
6
7 Process finished with exit code 1
```

Si el Bot encuentra el servidor y la contraseña es correcta debería aparecer esto al inicializarlo en IntelliJ:

```
1 Connected to: ws://localhost:7654
```

🔥 Consejo

Prueba por tu cuenta mezclando en las batallas Bots de ejemplo, con tu Bot o incluso el de algún compañero/a.

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la [anatomía de los Bots](#)

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)
- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las [coordenadas y ángulos](#)

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las **físicas**

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfrie para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último te en cuenta las **puntuaciones**

Puntos importantes:

- Consigues puntos si:
- tu disparo golpea a otro Bot (sino, te resta)
- eres el que mata al Bot
- cada vez que otro Bot muere, pero tu sigues vivo
- eres el último robot vivo
- atropellas a otro Bot
- si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos propios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```

1 // Esta condición se dispara cuando el bot completa su giro
2 public static class TurnCompleteCondition extends Condition {
3
4     private final IBot bot;
5
6     public TurnCompleteCondition(IBot bot) {
7         this.bot = bot;
8     }
9
10    @Override
11    public boolean test() {
12        // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
13        return bot.getTurnRemaining() == 0;
14    }
15 }
```

Podríamos usar esta condición en un fragmento similar a este:

```

1 waitFor(new TurnCompleteCondition());
```

Podríamos usar funciones Lambda de la siguiente manera:

```

1 waitFor(new Condition() -> getTurnRemaining() == 0);
```

El resultado de los dos fragmentos sería el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

```

1  Color verde = Color.fromRgb(0,255,0);
2  setBodyColor(Color.KHAKI);
3  setTurretColor(Color.KHAKI);
4  setRadarColor(Color.KHAKI);
5  setScanColor(Color.KHAKI);
6  setBulletColor(Color.KHAKI);

```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`onHitWall`),
- es alcanzado por un disparo (`onHitByBullet`),
- es alcanzado por otro Bot (`onHitBot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`onScannedBot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`onDeath`), cuándo ha muerto otro robot (`onRobotDeath`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`onBulletFired`) ha alcanzado a un oponente (`onBulletHit`), cuando una bala golpea una pared (`onBulletWall`) o cuando una bala golpea a otra bala (`onBulletHitBullet`).

Configurar correctamente tu Bot con:

```

1  setAdjustRadarForBodyTurn(true); //true if the radar must adjust/compensate for the body's turn;
2                                     //false if the radar must turn with the body turning (default).
3  setAdjustGunForBodyTurn(true);    //true if the gun must adjust/compensate for the bot's turn;
4                                     //false if the gun must turn with the bot's turn.
5  setAdjustRadarForGunTurn(true);   //true if the radar must adjust/compensate for the gun's turn;
6                                     //false if the radar must turn with the gun's turn.

```

Que el radar siempre de vueltas:

```

1  setTurnRadarLeft(Double.POSITIVE_INFINITY); //degrees - is the amount of degrees to turn left. If negative, the radar will turn right.

```

Revisa el API (`rescan()` i `setRescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```

1  public void onScannedBot(ScannedBotEvent e) {
2      double factor=1.9;
3      setTurnRadarLeft(factor * radarBearingTo(e.getX(), e.getY()));
4  }

```

Segun el factor:

- 1.0 - Bloqueo de radar delgado. Debe llamar a `scan()` para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- 1.9 - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- 2.0 : el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

```

1 public void onScannedBot(ScannedBotEvent e) {
2     //set the wide to add to the scan.
3     double wide=36.0;
4     // Absolute angle towards target
5     double angleToEnemy = radarBearingTo(e.getX(), e.getY());
6     // Distance we want to scan from middle of enemy to either side
7     // The 36.0 is how many units from the center of the enemy robot it scans.
8     double extraTurn = Math.min(Math.toDegrees(Math.atan(36.0 / distanceTo(e.getX(), e.getY()))), getMaxRadarTurnRate());
9
10    // Adjust the radar turn so it goes that much further in the direction it is going to turn
11    // Basically if we were going to turn it left, turn it even more left, if right, turn more right.
12    // This allows us to overshoot our enemy so that we get a good sweep that will not slip.
13    if (angleToEnemy < 0)
14        angleToEnemy -= extraTurn;
15    else
16        angleToEnemy += extraTurn;
17
18    //Turn the radar
19    setTurnRadarLeft(angleToEnemy);
20 }

```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?) ...
- Nos interesa mantener una lista de enemigos? `e.getScannedBotId()` ? y por ejemplo fijar el radar en el más débil?...

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```

1 turnLeft(bearingTo(e.getX(), e.getY()) + 90);

```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declare una variable como hicimos para oscilar el radar.

```

1 class MyRobot extends Bot {
2     private byte moveDirection = 1;

```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```

1 setForward(100 * moveDirection);

```

Puedes cambiar de dirección cambiando el valor de `moveDirection` de 1 a -1 así:

```

1 moveDirection *= -1;

```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```

1 public void onHitWall(HitWallEvent e) { moveDirection *= -1; }
2 public void onHitBot(HitBotEvent e) { moveDirection *= -1; }

```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `onHitRobot()` tantas veces que `moveDirection` sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si su robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:

```

1  if (getSpeed() == 0)
2      moveDirection *= -1;

```

Ponlo en tu método `doMove()` (o en cualquier otro lugar donde estés manejando el movimiento) y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```

1  public class IABDBot extends Bot {
2      double moveDirection=1;
3      private double ultimoEnemigoVisto;
4      ...
5      @Override
6      public void run() {
7          while (isRunning()) {
8              doMove();
9              go();
10         }
11
12         public void onScannedBot(ScannedBotEvent e) {
13             ...
14             ultimoEnemigoVisto=bearingTo(e.getX(), e.getY());
15             ...
16         }
17
18         public void doMove() {
19             // switch directions if we've stopped
20             if (getSpeed() == 0)
21                 moveDirection *= -1;
22             // circle our enemy
23             setTurnLeft(ultimoEnemigoVisto+90);
24             setForward(1000 * moveDirection);
25         }

```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```

1  public void doMove() {
2      // switch directions if we've stopped
3      if (getSpeed() == 0)
4          moveDirection *= -1;
5      // circle our enemy
6      setTurnLeft(ultimoEnemigoVisto+90);
7      if (getTurnNumber() % 20 == 0) {
8          moveDirection *= -1;
9          setForward(1000 * moveDirection);
10     }
11 }

```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquivas las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si su enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```

1  setTurnLeft(lastScannedBearing + (90 - (15 * moveDirection)));

```

15 es un factor en grados que puedes modificar y ajustar. Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
1 public class WallAvoider extends Bot {
2     ...
3     private int wallMargin = 60;
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
1 public void run() {
2     ...
3     // No te acerques mucho a las paredes
4     addCustomEvent(new Condition("TooCloseToWalls") {
5         public boolean test() {
6             // El método test() es el que debemos reescribir para definir el resultado de nuestra condición.
7             return (
8                 // cerca de la pared izquierda
9                 (getX() <= wallMargin ||
10                // cerca de la pared derecha
11                getX() >= getArenaWidth() - wallMargin ||
12                // cerca de la pared inferior
13                getY() <= wallMargin ||
14                // cerca de la pared superior
15                getY() >= getArenaHeight() - wallMargin)
16            );
17         }
18     });
19     ...
20 }
```

Ten en cuenta que estamos creando una clase interna anónima con esta llamada. Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
1 public void onCustomEvent(CustomEvent e) {
2     if (e.getCondition().getName().equals("TooCloseToWalls")) {
3         // switch directions and move away
4         moveDirection *= -1;
5         forward(100 * moveDirection);
6     }
7 }
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento. Podemos declarar otro así:

```
1 public class WallAvoider extends Bot {
2     ...
3     private int tooCloseToWall = 0;
```

Luego maneje el evento de manera un poco más inteligente:

```

1 public void onCustomEvent(CustomEvent e) {
2     if (e.getCondition().getName().equals("TooCloseToWalls")) {
3         if (tooCloseToWall <= 0) {
4             // Si no estábamos ya cerca de las paredes, ahora lo estamos
5             tooCloseToWall += wallMargin;
6             setMaxSpeed(0); // Para!!!
7         }
8     }
9 }

```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `doMove()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `tooCloseToWall`.

Podemos resolver ambos problemas con la siguiente implementación de `doMove()`:

```

1 public void doMove() {
2     ...
3
4     // if we're close to the wall, eventually, we'll move away
5     if (tooCloseToWall > 0) tooCloseToWall--;
6
7     // switch directions if we've stopped
8     if (getSpeed() == 0) {
9         setMaxSpeed(8);
10        moveDirection *= -1;
11        setForward(1000 * moveDirection);
12    }
13 }

```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```

1 public class MultiModeBot extends Bot {
2     private final static int MOD0 Rondar=0;
3     private final static int MOD0 Esquivar=1;
4     private final static int MOD0 Asesino=2;
5     private int modoRobot=MOD0 Rondar;
6     ...
7 }
8
9 public void onBotDeath(BotDeathEvent e) {
10    ...
11    if (getEnemyCount() > 10) {
12        // Un gran número de enemigos sugiere movimientos fluidos
13        modoRobot=MOD0 Rondar;
14    } else if (getEnemyCount() > 1) {
15        // esquivar es la mejor táctica para grupos pequeños
16        modoRobot=MOD0 Esquivar;
17    } else if (getEnemyCount() == 1) {
18        // Si solo queda un robot, persíguelo
19        modoRobot=MOD0 Asesino;
20    }
21    ...
22 }
23
24 public void doMove() {
25     switch (modoRobot){
26     case MOD0 Rondar:
27         //ronda
28         break;
29     case MOD0 Esquivar:
30         //esquiva
31         break;
32     case MOD0 Asesino:
33         //asesina
34         break;
35     }
36     ...
37 }

```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`setAdjustGunForBodyTurn`)
- Técnica de apuntado básica: `setTurnGunLeft` o `setTurnGunRight` junto con `gunBearingTo`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de `0.1` a `3.0`. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```
1 public void smartFire(double distance){
2     if ((distance >200) || (getEnergy() <15)){
3         fire(1);
4     } else if (distance > 50){
5         fire(2);
6     } else {
7         fire(3);
8     }
9 }
```

Podrías usar una serie de declaraciones if-else-if-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```
1 setFire(400/distanceTo(e.getX(), e.getY()));
```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```
1 setFire(Math.min(500/distanceTo(e.getX(), e.getY()), 3));
```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa el método `getGunTurnRemaining()` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `setFire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `getGunHeat()`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

\$\$ Distancia = Velocidad * Tiempo \$\$ Usando la anterior formula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia:** se puede encontrar llamando a `distanceTo(e.getX(), e.getY())`
- **Velocidad:** según la documentación de RCTR, una bala viaja a una velocidad de: `20 - potenciaDeFuego * 3`
- **Tiempo:** podemos calcular el tiempo resolviendo: `Tiempo = {Distancia \over Velocidad}`

El siguiente código lo hace:

```
1 // calcular la potencia basado en la distancia
2 double enemyDistance = distanceTo(e.getX(), e.getY());
3 double firePower = Math.min(500 / enemyDistance, 3);
4 // calcular la velocidad de la bala
5 double bulletSpeed = 20 - firePower * 3;
6 // calculamos el tiempo que necesita la bala para impactar al enemigo
7 long time = (long) (enemyDistance / bulletSpeed);
```

Obteniendo futuras coordenadas X,Y

A continuación, debemos calcular la posición futura de nuestro enemigo:

```

1 // Calcular la velocidad actual de tu enemigo
2 double enemyVelocity = e.getSpeed();
3 // Calcular la dirección actual del enemigo
4 double enemyDirection = e.getDirection();
5 // Descomponer el desplazamiento en las coordenadas X e Y
6 // Componente X del desplazamiento COSENO
7 double deltaX = enemyVelocity * Math.cos(Math.toRadians(enemyDirection));
8 // Componente Y del desplazamiento SENO
9 double deltaY = enemyVelocity * Math.sin(Math.toRadians(enemyDirection));
10
11 // Calcular las coordenadas futuras
12 double futureX = e.getX() + (deltaX * time);
13 double futureY = e.getY() + (deltaY * time);

```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañon al lugar previsto de impacto:

```

1 setTurnGunLeft(gunBearingTo(futureX, futureY));

```

Disparando!

Por último disparamos nuestro cañon

```

1 if (getGunTurnRemaining() <= 0 && getGunHeat() == 0) {
2     fire(firePower);
3 }

```

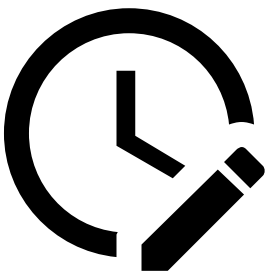
Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quien disparo si mi previsión de impacto falla mucho?

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.



19 de octubre de 2025

UD02 · Robocode Tank Royale en Python

Antes de empezar

Leed primero la [Comparativa Java vs Python](#) para entender las diferencias y elegir vuestro camino.

Esta guía es la versión **Python** de la actividad Robocode. Si buscáis la versión Java, id a [Robocode en Java](#).

Preparación del entorno

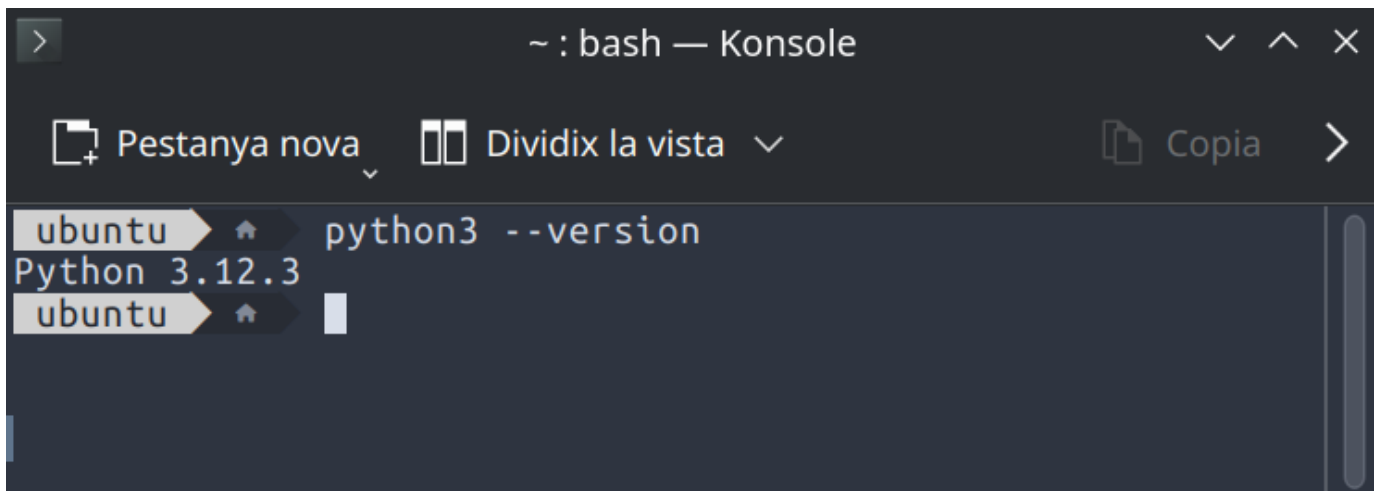
REQUISITOS PREVIOS

- **Python 3.10 o superior** instalado en el sistema
- **VS Code** (Visual Studio Code) u otro editor de código
- Conexión a Internet para instalar dependencias

COMPROBACIÓN DE PYTHON

Abrid un terminal y ejecutad:

```
1 python3 --version
```



The screenshot shows a terminal window with a dark background. At the top, the title bar reads '~ : bash — Konsole'. Below the title bar, there are icons for 'Pestanya nova', 'Dividix la vista', and 'Copia'. The terminal content shows a prompt 'ubuntu' followed by the command 'python3 --version' and the output 'Python 3.12.3'. A second prompt 'ubuntu' is visible below the output.

 Windows

Si usas Windows, el comando es `python --version` en lugar de `python3 --version`.

Si no tenéis Python instalado, descargadlo desde <https://www.python.org/downloads/> o usad el gestor de paquetes de vuestro sistema:

Linux (Debian/Ubuntu)

Linux (Fedora)

macOS

Windows

```
1 sudo apt install python3 python3-pip python3-venv
```

```
1 sudo dnf install python3 python3-pip
```

```
1 brew install python3
```

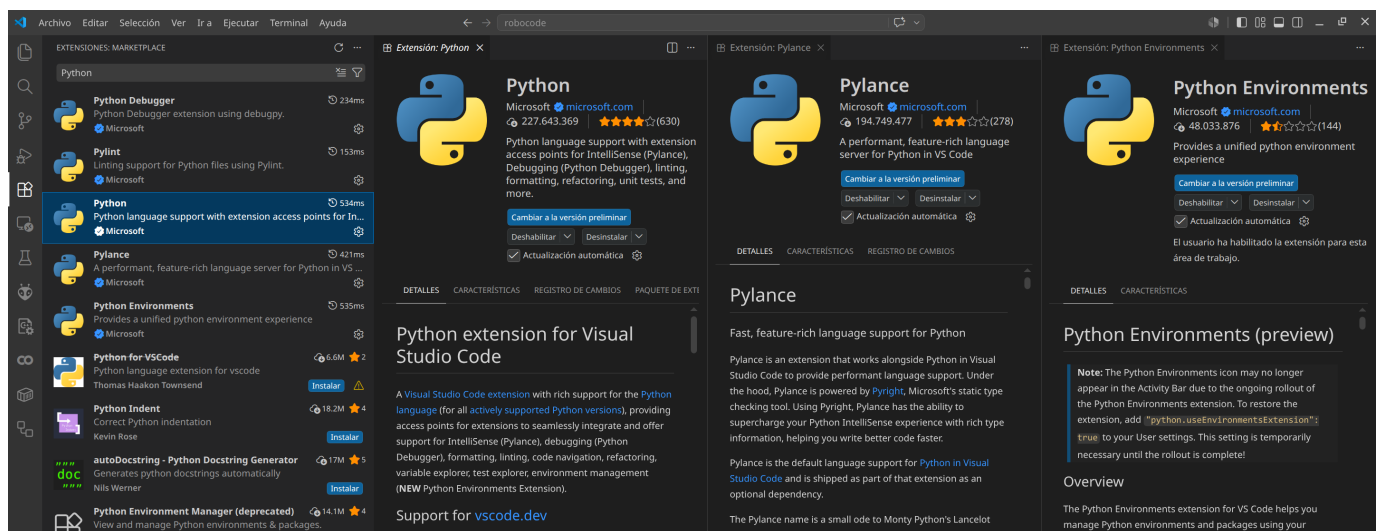
Descargad el instalador desde <https://www.python.org/downloads/> y marcad la opción "Add Python to PATH"

VS CODE Y EXTENSIONES

Si aún no tenéis VS Code, descargadlo desde <https://code.visualstudio.com/>

Una vez instalado, añadid las siguientes extensiones:

1. **Python** (de Microsoft) — todo el soporte para Python: linting, debugging, IntelliSense
2. **Pylance** (de Microsoft) — completado de código y tipado avanzado
3. *Opcional:* **Python Environments** (de Microsoft)



INSTALACIÓN DE LA API DE ROBOCODE

La API de Python para Robocode Tank Royale se distribuye como paquete pip:

```
1 pip install robocode-tank-royale
```

O si trabajáis con entornos virtuales:

```
1 python3 -m venv .venv
2 # Linux/macOS:
3 source .venv/bin/activate
4 # Windows PowerShell:
5 # .venv\Scripts\Activate.ps1
6 # Windows CMD:
7 # .venv\Scripts\activate.bat
8
9 pip install robocode-tank-royale
```

```

ubuntu | robocode | python3 -m venv .venv
ubuntu | robocode | source .venv/bin/activate
ubuntu | .venv | pip install robocode-tank-royale
Collecting robocode-tank-royale
  Using cached robocode_tank_royale-1.0.2-py3-none-any.whl.metadata (6.9 kB)
Requirement already satisfied: pip>=24 in ./.venv/lib/python3.12/site-packages (from robocode-tank-royale) (24.0)
Collecting PyYAML>=6 (from robocode-tank-royale)
  Using cached pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting setuptools>=77 (from robocode-tank-royale)
  Using cached setuptools-83.0.0-py3-none-any.whl.metadata (6.6 kB)
Collecting pycountry>=24 (from robocode-tank-royale)
  Using cached pycountry-26.2.16-py3-none-any.whl.metadata (12 kB)
Collecting websockets>=15 (from robocode-tank-royale)
  Using cached websockets-16.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)
Collecting mpyy (from robocode-tank-royale)
  Using cached mpyy-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting typing_extensions>=4.6.0 (from mpyy->robocode-tank-royale)
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting mpyy_extensions>=1.0.0 (from mpyy->robocode-tank-royale)
  Using cached mpyy_extensions-1.1.0-py3-none-any.whl.metadata (1.1 kB)
Collecting pathspec>=1.0.0 (from mpyy->robocode-tank-royale)
  Using cached pathspec-1.1.1-py3-none-any.whl.metadata (14 kB)
Collecting librt>=0.13.0 (from mpyy->robocode-tank-royale)
  Using cached librt-0.13.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (1.3 kB)
Collecting ast_serialize>=0.6.0 (from mpyy->robocode-tank-royale)
  Using cached ast_serialize-0.6.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.3 kB)
Using cached robocode_tank_royale-1.0.2-py3-none-any.whl (142 kB)
Using cached pycountry-26.2.16-py3-none-any.whl (8.0 MB)
Using cached pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (807 kB)
Using cached setuptools-83.0.0-py3-none-any.whl (1.0 MB)
Using cached websockets-16.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (187 kB)
Using cached mpyy-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (15.3 MB)
Using cached ast_serialize-0.6.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.3 MB)
Using cached librt-0.13.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (531 kB)
Using cached mpyy_extensions-1.1.0-py3-none-any.whl (5.0 kB)
Using cached pathspec-1.1.1-py3-none-any.whl (57 kB)
Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Installing collected packages: websockets, typing_extensions, setuptools, PyYAML, pycountry, pathspec, mpyy_extensions, librt, ast-serial
ize, mpyy, robocode-tank-royale
Successfully installed PyYAML-6.0.3 ast-serialize-0.6.0 librt-0.13.0 mpyy-2.3.0 mpyy_extensions-1.1.0 pathspec-1.1.1 pycountry-26.2.16 ro
bocode-tank-royale-1.0.2 setuptools-83.0.0 typing_extensions-4.16.0 websockets-16.1
ubuntu | .venv | robocode |

```

La GUI de Robocode Tank Royale

DESCARGA Y EJECUCIÓN DE LA GUI

La GUI (interfaz gráfica) de Robocode es **independiente del lenguaje**. El proceso es idéntico tanto si usáis Java como Python:

1. Id a <https://github.com/robocode-dev/tank-royale/releases>
2. Descargad la última versión del fichero `robocode-tankroyale-gui-x.y.z.jar`
3. Ejecutadlo:

```
1 java -jar robocode-tankroyale-gui-x.y.z.jar
```

⚠ Java necesario para la GUI

La GUI requiere Java 11 o superior para ejecutarse, independientemente de que vuestro bot esté escrito en Python. Si no tenéis Java, ved el [Taller 3: preparar el entorno](#) o instalad OpenJDK:

```
1 sudo apt install default-jdk # Linux
```

CONFIGURACIÓN INICIAL DE LA GUI

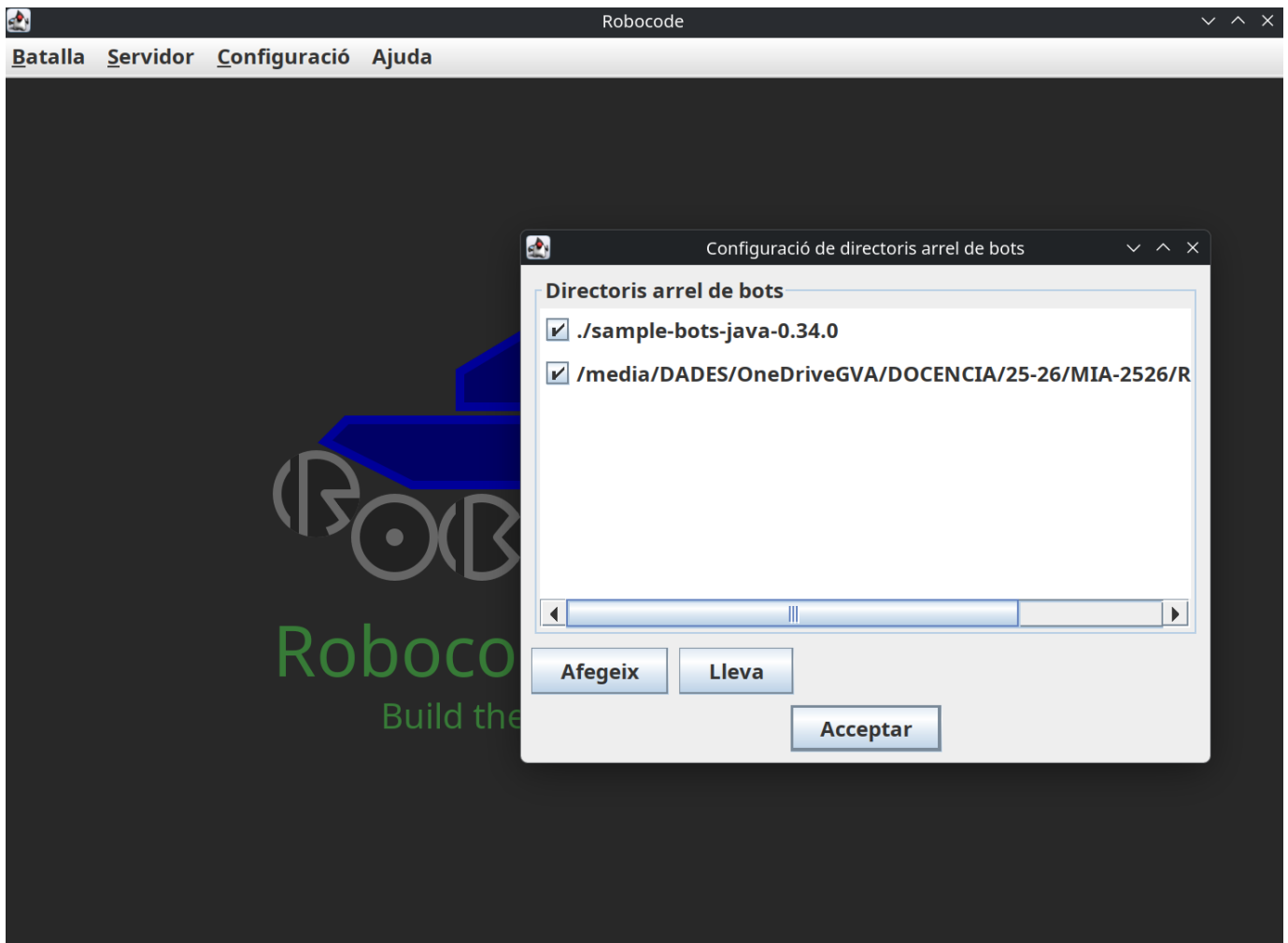
Una vez abierta la GUI:

1. Descargad y descomprimid los **Bots de ejemplo** desde la misma página de releases
2. Id a Config → Bot Root Directories y añadid la carpeta donde habéis descomprimido los bots de ejemplo
3. *Opcional*: descargad y descomprimid `sounds.zip` de los [releases de sonidos](#) en el mismo directorio que el JAR

```

1 [directorio padre]
2 |─ robocode-tankroyale-gui-x.y.z.jar
3 |─ sounds/
4 |   └─ bots_collision.wav
5 |   ...
6 |   └─ wall_collision.wav
7 |─ bots/      <-- carpeta de bots de ejemplo

```



Consejo

Jugad un poco con el entorno GUI, explorad las opciones, tipos de batallas, cread alguna ronda, inicializad bots, añadidlos, ajustad la velocidad de juego, etc.

Vuestro primer Bot en Python

ESTRUCTURA DEL PROYECTO

Cread una carpeta para vuestro bot (por ejemplo, `MiBot`) con la siguiente estructura:

```

1 MiBot/
2 |─ MiBot.py
3 |─ MiBot.json
4 |─ MiBot.cmd      (Windows)
5 |─ MiBot.sh       (Linux/macOS)

```

FICHERO DE CONFIGURACIÓN JSON

El bot necesita un fichero `.json` con la información del bot. Cread `MiBot.json`:


```

1 {
2     "name": "MiBot",
3     "version": "1.0",
4     "authors": ["Vuestro Nombre"],
5     "description": "Mi primer bot con Python",
6     "homepage": "",
7     "countryCodes": ["es"],
8     "platform": "Python",
9     "programmingLang": "Python 3.x"
10 }

```

CÓDIGO BÁSICO DEL BOT

Cread `MiBot.py` con el siguiente contenido:

```

1 import os
2 from robocode_tank_royale.bot_api import Bot, BotInfo
3 from robocode_tank_royale.bot_api.events import ScannedBotEvent, HitByBulletEvent
4
5 class MiBot(Bot):
6     def __init__(self):
7         config_path = os.path.join(os.path.dirname(__file__), "MiBot.json")
8         super().__init__(BotInfo.from_file(config_path))
9
10
11     def run(self):
12         while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
13             self.forward(100)
14             self.turn_gun_left(360)
15             self.back(100)
16             self.turn_gun_left(360)
17
18     def on_scanned_bot(self, scanned_bot_event: ScannedBotEvent):
19         del scanned_bot_event
20         self.fire(1)
21
22     def on_hit_by_bullet(self, hit_by_bullet_event: HitByBulletEvent):
23         bearing = self.calc_bearing(hit_by_bullet_event.bullet.direction)
24         self.turn_right(90 - bearing)
25
26
27 def main():
28     bot = MiBot()
29     bot.start()
30
31
32 if __name__ == "__main__":
33     main()

```



VS Code: Pylint/Pylance no encuentran el paquete

Si después de seleccionar el intérprete del venv (`Ctrl+Shift+P` → "Python: Select Interpreter" → `./venv/bin/python`) los errores persisten, es porque Pylint usa su propio intérprete y Pylance necesita el fichero `.vscode/settings.json`.

Cread la carpeta `.vscode/` dentro de la carpeta de vuestro bot (`/home/ubuntu/robocode/MiBot/`) y añadid el siguiente fichero:

```

1 {
2     "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
3     "python.linting.pylintEnabled": false,
4     "python.linting.enabled": false,
5     "python.analysis.autoImportCompletions": true
6 }

```

Si no tenéis Pylint instalado en el venv (solo lo tiene el sistema), VS Code puede ejecutar el Pylint del sistema que no ve los paquetes instalados en el venv. Opciones: (1) desactivar Pylint (`"python.linting.enabled": false`) y confiar solo en Pylance, o (2) instalar Pylint en el venv: `pip install pylint`.

Los subrayados rojos desaparecerán al recargar la ventana (`Ctrl+Shift+P` → "Developer: Reload Window").

SCRIPTS DE INICIO

Windows (`MiBot.cmd`)

```

1 @echo off
2 python MiBot.py %*

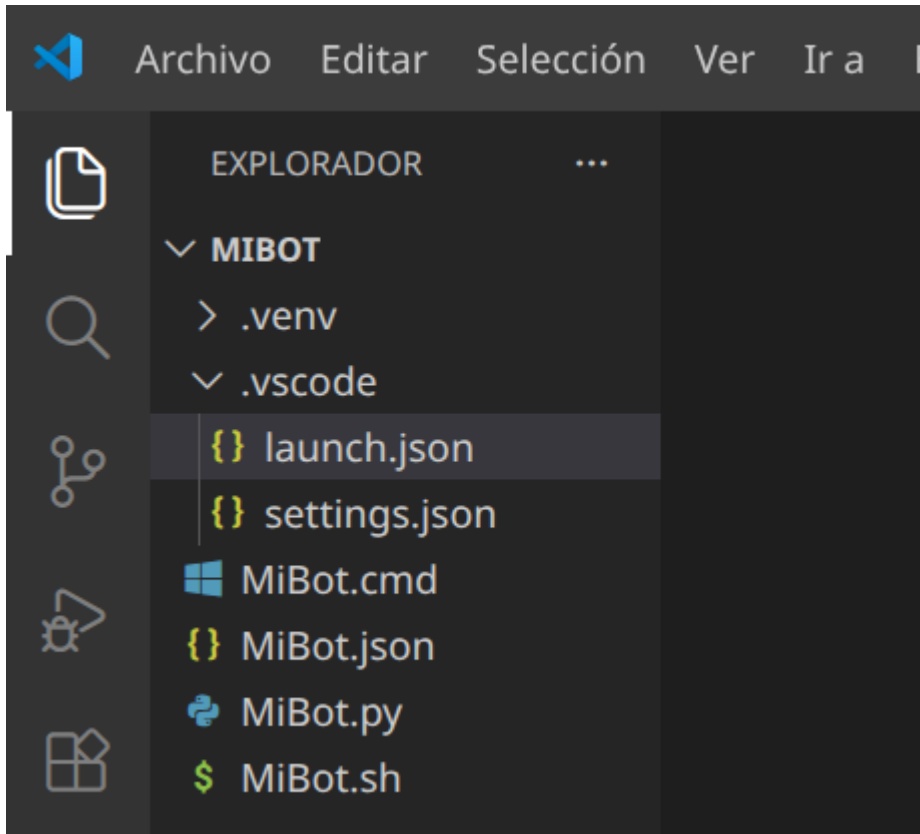
```

Linux/macOS (MiBot.sh)

```
1 #!/usr/bin/env sh
2 python3 "$(dirname "$0")/MiBot.py" "$@"
```

Hacedlo ejecutable:

```
1 chmod 755 MiBot.sh
```



PRUEBA DEL BOT

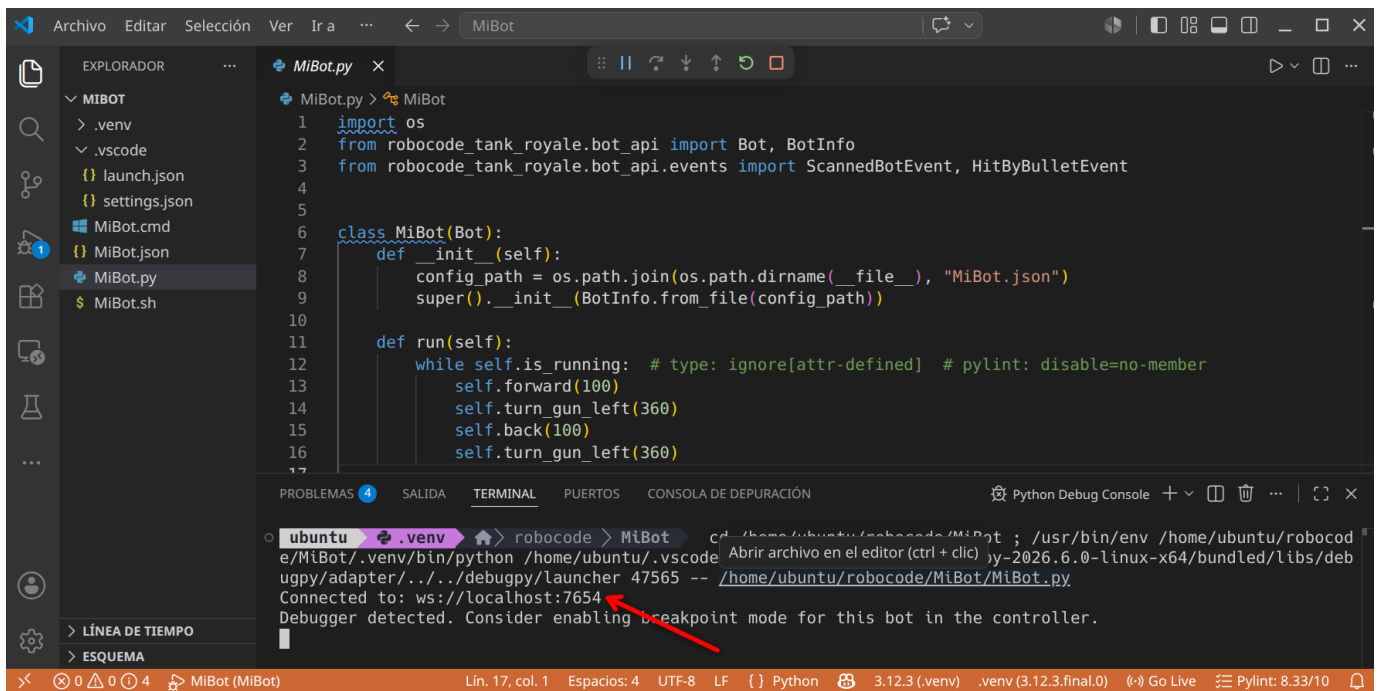
1. Inicial la GUI de Robocode (`java -jar robocode-tankroyale-gui-x.y.z.jar`)
2. Abrid un terminal en el directorio de vuestro bot
3. Ejecutad:

```
1 python3 MiBot.py
```

Si todo va bien, veréis un mensaje similar a:

```
1 Connected to: ws://localhost:7654
```

Y vuestro bot aparecerá en la lista de bots disponibles en la GUI.



Configuración del servidor y secrets

SECRETS DEL SERVIDOR

El servidor de Robocode puede requerir una contraseña (secret) para conectarse. Esta contraseña se genera automáticamente la primera vez que lanzáis la GUI.

Para habilitar los secrets:

1. Id a Config → Server Options → Enable server secrets en la GUI
2. O editad `server.properties` y añadid:

```

1 server-secrets-enabled=true
2 bots-secrets=CEIABDEPM2025

```

CONECTAR EL BOT CON SECRET

Estableced la variable de entorno `SERVER_SECRET` antes de ejecutar el bot:

Linux/macOS

Windows CMD

Windows PowerShell

```

1 export SERVER_SECRET=CEIABDEPM2025
2 python3 MiBot.py

```

```

1 set SERVER_SECRET=CEIABDEPM2025
2 python MiBot.py

```

```

1 $Env:SERVER_SECRET = "CEIABDEPM2025"
2 python MiBot.py

```

CONECTARSE A UN SERVIDOR REMOTO

Si el profesor indica una IP remota, podéis especificarla con `SERVER_URL` :

Linux/macOS

Windows CMD

```
1 export SERVER_URL=ws://192.168.1.100:7654
2 export SERVER_SECRET=CEIABDEPM2025
3 python3 MiBot.py
```

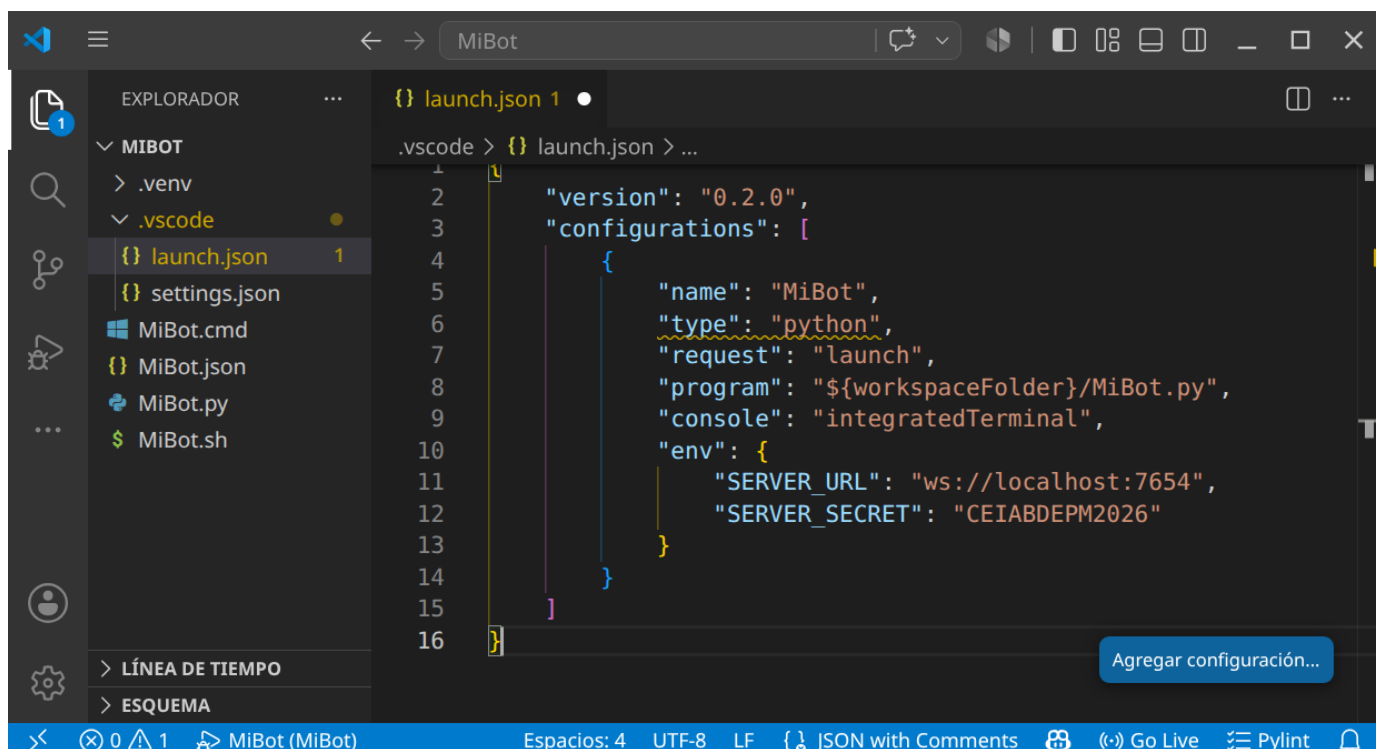
```
1 set SERVER_URL=ws://192.168.1.100:7654
2 set SERVER_SECRET=CEIABDEPM2025
3 python MiBot.py
```

USO CON VS CODE (LAUNCH.JSON)

Para mayor comodidad, podéis configurar VS Code para ejecutar el bot con las variables de entorno definidas en el fichero `.vscode/launch.json` :

Cread la carpeta `.vscode/` dentro del proyecto y el fichero `launch.json` :

```
1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "name": "MiBot",
6       "type": "python",
7       "request": "launch",
8       "program": "${workspaceFolder}/MiBot.py",
9       "console": "integratedTerminal",
10      "env": {
11        "SERVER_URL": "ws://localhost:7654",
12        "SERVER_SECRET": "CEIABDEPM2025"
13      }
14    }
15  ]
16 }
```



Ahora podéis ejecutar el bot directamente desde VS Code pulsando `F5` o yendo a `Run` → `Start Debugging` .

**El servidor debe estar en marcha**

El servidor (la GUI) debe estar inicializado **antes** de lanzar el bot. Si no, obtendréis un error:

```
1 Could not create web socket for URL: ws://localhost:7654
```

API de Python: diferencias clave con Java

La API de Python utiliza **snake_case** en lugar de **camelCase**. Aquí tenéis una tabla de correspondencia:

Java	Python
<code>setTurnLeft(degrees)</code>	<code>set_turn_left(degrees)</code>
<code>setForward(dist)</code>	<code>set_forward(dist)</code>
<code>setTurnRadarLeft(degrees)</code>	<code>set_turn_radar_left(degrees)</code>
<code>setAdjustRadarForBodyTurn(bool)</code>	<code>set_adjust_radar_for_body_turn(bool)</code>
<code>distanceTo(x, y)</code>	<code>distance_to(x, y)</code>
<code>bearingTo(x, y)</code>	<code>bearing_to(x, y)</code>
<code>gunBearingTo(x, y)</code>	<code>gun_bearing_to(x, y)</code>
<code>getX()</code>	<code>self.x</code> (propiedad)
<code>getY()</code>	<code>self.y</code> (propiedad)
<code>getSpeed()</code>	<code>self.speed</code> (propiedad)
<code>getEnergy()</code>	<code>self.energy</code> (propiedad)
<code>getTurnRemaining()</code>	<code>self.turn_remaining</code> (propiedad)
<code>getGunHeat()</code>	<code>self.gun_heat</code> (propiedad)
<code>isRunning()</code>	<code>self.is_running</code> (propiedad)
<code>onScannedBot(e)</code>	<code>on_scanned_bot(self, e)</code>
<code>e.getX()</code>	<code>e.x</code> (propiedad)
<code>e.getY()</code>	<code>e.y</code> (propiedad)
<code>e.getSpeed()</code>	<code>e.speed</code> (propiedad)
<code>e.getDirection()</code>	<code>e.direction</code> (propiedad)
<code>Double.POSITIVE_INFINITY</code>	<code>float("inf")</code>
<code>Math.min(a, b)</code>	<code>min(a, b)</code>
<code>Math.cos(angle)</code>	<code>math.cos(angle)</code>
<code>Math.toRadians(degrees)</code>	<code>math.radians(degrees)</code>

¿Cómo mejoro mi Bot?

Dividiremos todo lo que debemos conocer en 4 apartados:

CONOCIENDO EL CAMPO DE BATALLA

Sigue atentamente la documentación de RCTR sobre la [anatomía de los Bots](#)

Puntos importantes:

- Si el radar no se mueve, no detecta
- Inicialmente el robot, el cañón y el radar se mueven conjuntamente (pero se puede cambiar)

- El Bot se simplifica a un cuadrado de 36x36 unidades.
- Para las colisiones (con Bots o paredes) se simula como un círculo de 18 unidades de radio.

A continuación revisa las **coordenadas y ángulos**

Puntos importantes:

- Este apartado es totalmente diferente al de Robocode Original.

Estudia también las **físicas**

Puntos importantes:

- Se frena más rápido que se acelera.
- Cuanto más rápido vas, más lento giras.
- El cañón gira máximo 20º por turno.
- El radar gira máximo 45º por turno.
- La potencia de tiro influye en el daño provocado y los puntos conseguidos, pero también en el calentamiento del cañón.
- Inicialmente el cañón está caliente (3 unidades), al principio has de esperar a que se enfríe para disparar.
- El atropello da más puntos que los disparos (pero te resta energía)
- Si chocas con una pared, pierdes puntos.
- La energía que te sobra en una ronda no da puntos (modo kamikaze con el último robot?)

Por último ten en cuenta las **puntuaciones**

Puntos importantes:

- Consigues puntos si:
- tu disparo golpea a otro Bot (sino, te resta)
- eres el que mata al Bot
- cada vez que otro Bot muere, pero tu sigues vivo
- eres el último robot vivo
- atropellas a otro Bot
- si matas por atropello a otro Bot
- El último Bot que quede vivo no tiene porqué ser el ganador.

Eventos propios

Definimos una condición, y cuando esta se cumple se dispara un evento:

```
1 from robocode_tank_royale.bot_api.events import Condition
2
3 class TurnCompleteCondition(Condition):
4     def __init__(self, bot):
5         super().__init__()
6         self.bot = bot
7
8     def test(self):
9         return self.bot.turn_remaining == 0
```

Podríamos usar esta condición en un fragmento similar a este:

```
1 self.wait_for(TurnCompleteCondition(self))
```

Podríamos usar funciones lambda de la siguiente manera:

```
1 self.wait_for(Condition(lambda: self.turn_remaining == 0))
```

El resultado de los dos fragmentos sería el mismo.

PERSONALIZAR MI BOT

Podemos establecer colores para el cuerpo, torreta de cañón, el radar, el arco de escaneo y de la bala:

```

1 verde = Color.from_rgb(0, 255, 0)
2 self.set_body_color(Color.from_rgb(0, 0, 0))
3 self.set_turret_color(Color.from_rgb(255, 0, 0))
4 self.set_radar_color(Color.from_rgb(255, 0, 0))
5 self.set_scan_color(Color.from_rgb(255, 255, 0))
6 self.set_bullet_color(Color.from_rgb(255, 255, 255))

```

Puedes encontrar la lista completa de colores disponibles en la API de Robocode TankRoyale.

TÉCNICAS DE ESCANEO (RADAR)

Sentidos de nuestro Bot:

Sentido del **tacto**, tu Bot sabe cuando:

- golpea una pared (`on_hit_wall`),
- es alcanzado por un disparo (`on_hit_by_bullet`),
- es alcanzado por otro Bot (`on_hit_bot`).

Sentido de la **vista**, tu Bot sabe cuándo ha visto otro robot, pero sólo si lo escanea (`on_scanned_bot`)

Otros sentidos, tu robot también sabe cuándo ha muerto (`on_death`), cuándo ha muerto otro robot (`on_bot_death`).

Además también es consciente de sus balas y sabe cuando una bala ha sido disparada (`on_bullet_fired`) ha alcanzado a un oponente (`on_bullet_hit`), cuando una bala golpea una pared (`on_bullet_wall_hit`) o cuando una bala golpea a otra bala (`on_bullet_hit_bullet`).

Configurar correctamente tu Bot con:

```

1 self.set_adjust_radar_for_body_turn(True) # True si el radar debe ajustar/compensar el giro del cuerpo;
2                                           # False si el radar gira con el cuerpo (por defecto).
3 self.set_adjust_gun_for_body_turn(True)   # True si el cañón debe ajustar/compensar el giro del cuerpo;
4                                           # False si el cañón gira con el cuerpo (por defecto).
5 self.set_adjust_radar_for_gun_turn(True)  # True si el radar debe ajustar/compensar el giro del cañón;
6                                           # False si el radar gira con el cañón (por defecto).

```

Que el radar siempre de vueltas:

```

1 self.set_turn_radar_left(float("inf")) # grados - cantidad de grados a girar a la izquierda. Si es negativo, gira a la derecha.

```

Revisa el API (`rescan()` y `set_rescan()`)

Fijar el radar en un enemigo (one on one radar):

- Escaneo con multiplicador

```

1 def on_scanned_bot(self, e: ScannedBotEvent):
2     factor = 1.9
3     self.set_turn_radar_left(factor * self.radar_bearing_to(e.x, e.y))

```

Según el factor:

- **1.0** - Bloqueo de radar delgado. Debe llamar a `scan()` para evitar perder el bloqueo. Que Dios te ayude si alguna vez te saltas un turno.
- **1.9** - El arco del radar comienza amplio y se estrecha lentamente tanto como sea posible mientras se mantiene en el objetivo.
- **2.0** : el arco del radar recorre un ángulo fijo. El ángulo exacto elegido depende de las posiciones del enemigo y del radar cuando se detecta al enemigo por primera vez. El ángulo se incrementará si es necesario para mantener el bloqueo.
- Arco de radar Ancho

```

1  def on_scanned_bot(self, e: ScannedBotEvent):
2      wide = 36.0
3      # Ángulo absoluto hacia el objetivo
4      angle_to_enemy = self.radar_bearing_to(e.x, e.y)
5      # Distancia que queremos escanear desde el centro del enemigo a cada lado
6      extra_turn = min(
7          math.degrees(math.atan(wide / self.distance_to(e.x, e.y))),
8          self.max_radar_turn_rate
9      )
10     # Ajustamos el giro del radar para que vaya más allá en la dirección que ya gira
11     if angle_to_enemy < 0:
12         angle_to_enemy -= extra_turn
13     else:
14         angle_to_enemy += extra_turn
15     # Giramos el radar
16     self.set_turn_radar_left(angle_to_enemy)

```

- Radar Oscilante (variable global que sepa la dirección y nos ayude a cambiarla de vez en cuando)
- Escaneo más inteligente (seguir al cercano? según la situación?)
- Nos interesa mantener una lista de enemigos? `e.scanned_bot_id`? y por ejemplo fijar el radar en el más débil?

TÉCNICAS DE DESPLAZAMIENTO (MOVIMIENTO)

Pensamiento lateral

Si has jugado baloncesto antes, sabes que si quieres defender a alguien que sostiene el balón, debes maximizar tu movimiento lateral enfrentándote siempre a él (de frente). Lo mismo ocurre con tu robot.

Para conseguir esta posición debemos hacer algo similar a esto:

```

1  self.turn_left(self.bearing_to(e.x, e.y) + 90)

```

que siempre colocará tu robot perpendicular (90 grados) a tu enemigo.

¿Hacia adelante o hacia atrás?

Cuando te enfrentas a un oponente, la idea de "adelante" y "atrás" (del primer Bot) se vuelven algo obsoletas. Probablemente estés pensando más en términos de "atacar a la izquierda" o "atacar a la derecha". Para realizar un seguimiento de la dirección del movimiento, simplemente declara una variable como hicimos para oscilar el radar.

```

1  class MiBot(Bot):
2      def __init__(self):
3          super().__init__(BotInfo.from_file("MiBot.json"))
4          self.move_direction = 1

```

luego, cuando quieras mover tu robot, simplemente puedes decir:

```

1  self.set_forward(100 * self.move_direction)

```

Puedes cambiar de dirección cambiando el valor de `move_direction` de 1 a -1 así:

```

1  self.move_direction *= -1

```

Cambiar de dirección

El enfoque más intuitivo para cambiar de dirección es simplemente cambiar la dirección del movimiento cada vez que golpeas una pared o golpeas a otro robot de esta manera:

```

1  def on_hit_wall(self, e: HitWallEvent):
2      self.move_direction *= -1
3
4  def on_hit_bot(self, e: HitBotEvent):
5      self.move_direction *= -1

```

Sin embargo, descubrirás que si haces eso, terminarás presionando obstinadamente contra un robot que te embiste desde un costado (como un perro en celo). Esto se debe a que se llama a `on_hit_bot` tantas veces que `move_direction` sigue cambiando y nunca te alejas.

Un mejor enfoque es simplemente probar para ver si tu robot se ha detenido. Si es así, probablemente significa que has golpeado algo y querrás cambiar de dirección. Puedes hacerlo con el código:


```

1  if self.speed == 0:
2      self.move_direction *= -1

```

Ponlo en tu método que maneje el movimiento y podrás manejar todos los eventos de impacto con las paredes u otros Bots.

¿Bailamos?

Dando vueltas (Círculos)

Puedes rodear a tu enemigo simplemente usando las técnicas anteriores:

```

1  class MiBot(Bot):
2      def __init__(self):
3          super().__init__(BotInfo.from_file("MiBot.json"))
4          self.move_direction = 1
5          self.ultimo_enemigo_visto = 0
6
7      def run(self):
8          while self.is_running: # type: ignore[attr-defined] # pylint: disable=no-member
9              self.do_move()
10             self.go()
11
12     def on_scanned_bot(self, e: ScannedBotEvent):
13         self.ultimo_enemigo_visto = self.bearing_to(e.x, e.y)
14
15     def do_move(self):
16         if self.speed == 0:
17             self.move_direction *= -1
18         self.set_turn_left(self.ultimo_enemigo_visto + 90)
19         self.set_forward(1000 * self.move_direction)

```

Objetivo: rodea a tu enemigo usando el código de movimiento anterior, como un tiburón rodeando a su presa en el agua.

Evita Ametrallamiento

Un problema que puedes notar con el anterior tipo de movimiento es que es presa fácil para los objetivos predictivos porque sus movimientos son tan... predecibles.

Para evadir las balas de manera más efectiva, debes moverte de lado a lado o "atacar". Una buena forma de hacerlo es cambiar de dirección después de un cierto número de "tics", así:

```

1  def do_move(self):
2      if self.speed == 0:
3          self.move_direction *= -1
4          self.set_turn_left(self.ultimo_enemigo_visto + 90)
5      if self.turn_number % 20 == 0:
6          self.move_direction *= -1
7          self.set_forward(1000 * self.move_direction)

```

Mira como se balancea hacia adelante y hacia atrás usando el código de movimiento anterior. Observa lo bien que esquiva las balas. Pero ojo, porque puede afectar a tu precisión de disparo.

Cada vez más cerca

Notarás que tanto el primero como este último tipo de movimientos tienen otro problema: se atascan fácilmente en las esquinas y terminan golpeándose contra las paredes. Un problema adicional es que si tu enemigo está lejos, disparan mucho pero no aciertan mucho.

Para hacer que tu robot se acerque a tu enemigo, simplemente modifica el código para que se gire ligeramente hacia su enemigo, así:

```

1  self.set_turn_left(self.ultimo_enemigo_visto + (90 - (15 * self.move_direction)))

```

15 es un factor en grados que puedes modificar y ajustar. ¡Prueba!

Modificando el primero conseguimos una variación que utiliza el código anterior para lanzarse en espiral hacia su enemigo.

Mientras que con el segundo que utiliza el código anterior para acercarse cada vez más. También es bastante bueno para evadir balas.

Fíjate que ninguno de los Bots anteriores queda atrapado en una esquina por mucho tiempo.

Evitando las paredes

Un problema con todos los Bots anteriores es que golpean mucho las paredes y golpearlas agota tu energía. Una mejor estrategia sería detenerse antes de chocar contra las paredes. ¿Pero cómo?

Agregar un evento personalizado

Lo primero que debemos hacer es decidir qué tan cerca permitiremos que nuestro robot llegue a las paredes:

```
1 class WallAvoider(Bot):
2     def __init__(self):
3         super().__init__(BotInfo.from_file("WallAvoider.json"))
4         self.wall_margin = 60
5         self.too_close_to_wall = 0
6         self.move_direction = 1
```

A continuación, agregamos un evento personalizado que se activará cuando se cumpla una determinada condición dentro del método `run()`:

```
1 def run(self):
2     # ... inicialización ...
3     # No te acerques mucho a las paredes
4     self.add_custom_event(Condition("TooCloseToWalls", lambda: (
5         self.x <= self.wall_margin or
6         self.x >= self.arena_width - self.wall_margin or
7         self.y <= self.wall_margin or
8         self.y >= self.arena_height - self.wall_margin
9     )))
10    # ... resto del bucle ...
```

Necesitamos hacer override del método `test()` para devolver un valor booleano cuando ocurra nuestro evento personalizado.

Gestionar el evento personalizado

Lo siguiente que debemos hacer es manejar el evento, que se puede hacer así:

```
1 def on_custom_event(self, e: CustomEvent):
2     if e.condition.name == "TooCloseToWalls":
3         self.move_direction *= -1
4         self.set_forward(100 * self.move_direction)
```

Sin embargo, el problema con ese enfoque es que este evento podría dispararse una y otra vez, provocando que cambiemos rápidamente de un lado a otro, sin alejarnos nunca. O si ya aparecemos cerca de la pared quedamos atrapados.

Para evitar este "sacudida de la muerte" deberíamos tener una variable que indique que estamos manejando el evento:

```
1 def on_custom_event(self, e: CustomEvent):
2     if e.condition.name == "TooCloseToWalls":
3         if self.too_close_to_wall <= 0:
4             self.too_close_to_wall += self.wall_margin
5             self.set_max_speed(0)
```

Manejo de los dos modos

Hay dos últimos problemas que debemos resolver. En primer lugar, tenemos un método `do_move()` donde colocamos todo nuestro código de movimiento normal. Si estamos tratando de alejarnos de una pared, no queremos que se llame a nuestro código de movimiento normal, creando (una vez más) la "sacudida de la muerte". En segundo lugar, queremos volver eventualmente al movimiento "normal", por lo que eventualmente deberíamos tener el "tiempo de espera" de la variable `too_close_to_wall`.

Podemos resolver ambos problemas con la siguiente implementación de `do_move()`:

```
1 def do_move(self):
2     if self.too_close_to_wall > 0:
3         self.too_close_to_wall -= 1
4     if self.speed == 0:
5         self.set_max_speed(8)
6         self.move_direction *= -1
7         self.set_forward(1000 * self.move_direction)
```

Con todo el código anterior evitamos chocar contra las paredes. Observa cómo se desliza suavemente hacia los lados pero nunca (bueno, rara vez) los golpea.

Bot multimodo

Además de los colores que elijas, la mayor parte de la personalidad de tu robot está en su código de movimiento. Por otra parte, situaciones diferentes requieren tácticas diferentes. Usando el ejemplo de evitar paredes, es posible que desees codificar tu bot para que cambie los "modos" según ciertos criterios. Podrías pensar en algo como esto:

```

1  MODO Rondar = 0
2  MODO Esquivar = 1
3  MODO Asesino = 2
4
5  class MultiModeBot(Bot):
6      def __init__(self):
7          super().__init__(BotInfo.from_file("MultiModeBot.json"))
8          self.modo_robot = MODO Rondar
9
10     def on_bot_death(self, e: BotDeathEvent):
11         if self.enemy_count > 10:
12             self.modo_robot = MODO Rondar
13         elif self.enemy_count > 1:
14             self.modo_robot = MODO Esquivar
15         elif self.enemy_count == 1:
16             self.modo_robot = MODO Asesino
17
18     def do_move(self):
19         if self.modo_robot == MODO Rondar:
20             # ronda
21             pass
22         elif self.modo_robot == MODO Esquivar:
23             # esquivo
24             pass
25         elif self.modo_robot == MODO Asesino:
26             # asesina
27             pass

```

Los detalles se dejan (como siempre) como ejercicio para el alumnado.

TÉCNICAS DE ATAQUE (DISPARO)

Técnicas básicas

- Separa el movimiento del cañón del movimiento del Bot (`set_adjust_gun_for_body_turn`)
- Técnica de apuntado básica: `set_turn_gun_left` o `set_turn_gun_right` junto con `gun_bearing_to`

Fórmula de cálculo de potencia de fuego

Otro aspecto importante al disparar es calcular la potencia de fuego de tu bala. La documentación del método `fire()` explica que puedes disparar una bala en el rango de 0.1 a 3.0. Como probablemente ya habrás concluido, es una buena idea disparar balas de baja potencia cuando tu enemigo está lejos y balas de alta resistencia cuando está cerca.

```

1  def smart_fire(self, distance):
2      if distance > 200 or self.energy < 15:
3          self.fire(1)
4      elif distance > 50:
5          self.fire(2)
6      else:
7          self.fire(3)

```

Podrías usar una serie de declaraciones if-elif-else para determinar la potencia de fuego, en función de si el enemigo está a 50 unidades, a 200, etc (como en el fragmento anterior). Pero tales construcciones son demasiado rígidas. Después de todo, el rango de posibles valores de potencia de fuego cae a lo largo de un continuo, no de bloques discretos. Un mejor enfoque es utilizar una fórmula. He aquí un ejemplo:

```

1  self.set_fire(400 / self.distance_to(e.x, e.y))

```

Con esta fórmula, a medida que aumenta la distancia del enemigo, la potencia de fuego disminuye. Asimismo, a medida que el enemigo se acerca, la potencia de fuego aumenta. Los valores superiores a 3 se reducen a 3, por lo que nunca dispararemos una bala mayor que 3, pero probablemente deberíamos reducir el valor de todos modos (solo para estar seguros) de esta manera:

```

1  self.set_fire(min(500 / self.distance_to(e.x, e.y), 3))

```

Evitar disparos prematuros

Una situación que encontraras es que tu Bot dispare antes de haber girado el arma hacia el objetivo. Para evitar disparos prematuros, usa la propiedad `gun_turn_remaining` para ver qué tan lejos está tu cañón del objetivo y no dispare hasta que esté cerca.

Además, no puedes disparar si el arma está "caliente" desde el último disparo y llamar a `fire()` o `set_fire()` solo desperdiciará un turno. Podemos probar si el arma está fría llamando a `gun_heat`.

Apuntando de manera predictiva

O... "Usar la trigonometría para impresionar a tus amigos y destruir a tus enemigos".

Si quisiéramos poder golpear a un robot que recorre las paredes siempre fallaríamos, necesitamos poder predecir dónde estará en el futuro, pero ¿cómo podemos hacerlo?

$\backslash$ [Distancia = Velocidad $\times$ Tiempo]

Usando la anterior fórmula podemos calcular cuánto tiempo tardará una bala en llegar allí.

- **Distancia:** se puede encontrar llamando a `distance_to(e.x, e.y)`
- **Velocidad:** según la documentación de RCTR, una bala viaja a una velocidad de: $\$ \$ 20 - potenciaDeFuego \times 3 \$ \$$
- **Tiempo:** podemos calcular el tiempo resolviendo: $\$ \$ Tiempo = \{Distancia \over Velocidad\} \$ \$$

El siguiente código lo hace:

```
1 # calcular la potencia basado en la distancia
2 enemy_distance = self.distance_to(e.x, e.y)
3 fire_power = min(500 / enemy_distance, 3)
4 # calcular la velocidad de la bala
5 bullet_speed = 20 - fire_power * 3
6 # calculamos el tiempo que necesita la bala para impactar al enemigo
7 time = int(enemy_distance / bullet_speed)
```

Obteniendo futuras coordenadas X, Y

A continuación, debemos calcular la posición futura de nuestro enemigo:

```
1 import math
2
3 # Calcular la velocidad actual de tu enemigo
4 enemy_velocity = e.speed
5 # Calcular la dirección actual del enemigo
6 enemy_direction = e.direction
7 # Descomponer el desplazamiento en las coordenadas X e Y
8 # Componente X del desplazamiento COSENO
9 delta_x = enemy_velocity * math.cos(math.radians(enemy_direction))
10 # Componente Y del desplazamiento SENO
11 delta_y = enemy_velocity * math.sin(math.radians(enemy_direction))
12
13 # Calcular las coordenadas futuras
14 future_x = e.x + (delta_x * time)
15 future_y = e.y + (delta_y * time)
```

Girando el arma al punto previsto

Ahora debemos apuntar nuestro cañón al lugar previsto de impacto:

```
1 self.set_turn_gun_left(self.gun_bearing_to(future_x, future_y))
```

¡Disparando!

Por último disparamos nuestro cañón:

```
1 if self.gun_turn_remaining <= 0 and self.gun_heat == 0:
2     self.fire(fire_power)
```

Mejor aún...!

Aquí te dejo algunas mejoras para que las valores:

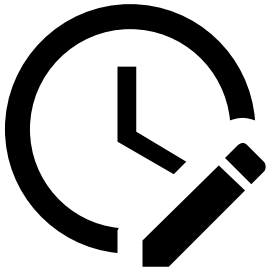
- ¿Debemos esperar siempre a que el arma esté completamente en la dirección calculada?
- Cuando nuestro enemigo se acerca a una pared, nuestros disparos impactan en ella.
- ¿Puedo saber a quién disparo si mi previsión de impacto falla mucho?

Investigación y desarrollo propio

A partir de aquí el trabajo será individual de cada alumno (puede solaparse con el paso 3). Deberéis investigar/prever a vuestros adversarios (los conocidos, y los de vuestros compañeros). Aplicar las técnicas que consideréis más útiles para intentar quedar lo más arriba posible en la tabla de clasificación.

Algunos recursos adicionales:

- [Documentación oficial de la API Python](#)
- [Tutorial oficial "My First Bot for Python"](#)
- [The Book of Robocode](#) — estrategias avanzadas
- [RoboWiki](#) — conocimiento comunitario (código en Java, conceptos universales)



15 de julio de 2026

5. UD05

5.1 UD05 — Sistemas expertos y controladores inteligentes



Unidad 5 · 12 h · semanas 11-15 (7 de diciembre al 14 de enero)

Continúa las reglas y la lógica difusa de RA2, justo antes de que la fragmentación de Navidad parta el curso. Se evalúa con **cinco entregables prácticos** y la prueba escrita del RA5.

1. Introducción

Todo el curso ha sido **cómo construir** sistemas de IA. Esta unidad da un paso más: los **sistemas expertos**, que codifican el conocimiento de un experto humano en reglas y lo utilizan para **diagnosticar, asesorar y controlar**. Y lo conectamos con el **control**, porque un sistema experto puede actuar como **controlador inteligente** de un proceso físico, compitiendo con los controladores clásicos (PID).

Antes de construir nada hay que situar **qué es el conocimiento** y **cómo se representa** para que una máquina pueda usarlo — esa es la primera pregunta de la inteligencia artificial simbólica, y la respondemos con la jerarquía DIKW y el continuo de representaciones del §5.

El recorrido de la unidad:

1. **Arquitectura y dinámica** de los sistemas expertos (base de conocimiento, motor de inferencia, ciclo reconocer-actuar).
2. **Estructuras elementales** de representación del conocimiento (reglas, marcos, lógica, ontologías, redes semánticas).
3. **Representar y simular comportamientos** con la librería `experta`, en dominios muy diversos: diagnóstico médico, clasificación, control de procesos.
4. **Sistemas híbridos reglas/datos**: cuando las reglas se combinan o se extraen del aprendizaje automático.
5. **Sistemas de razonamiento impreciso**: la lógica difusa, para trabajar con lo que no es blanco o negro.
6. **Cómo influye la variación de las características** en la dinámica del sistema (sensibilidad y robustez).
7. **Estrategias de control** de un sistema experto.
8. **Controladores inteligentes** (difuso, por reglas, redes neuronales, MPC) y su influencia en el comportamiento del sistema.
9. **Aplicaciones y tendencias** (MYCIN, XCON, BRMS, neuro-simbólico, IA explicable).



Hilo conductor de la unidad

Un sistema experto convierte el conocimiento de un experto en reglas que pueden diagnosticar, asesorar y controlar. Primero situamos qué es ese conocimiento y cómo se representa; después lo construimos y simulamos, con reglas puras, híbridas o difusas; luego estudiamos por qué a veces falla; y por último lo usamos como controlador, comparándolo con el PID clásico.

2. Resultado de aprendizaje y criterios de evaluación

RA5 — Aplica sistemas expertos evaluando la influencia de los controladores inteligentes en el comportamiento del sistema.

CE	Criterio de evaluación	Bloque
RA5-a	Se ha descrito la dinámica y las estructuras elementales de los sistemas expertos.	\$4-5
RA5-b	Se han determinado las destrezas necesarias para representar y simular comportamientos básicos de sistemas de muy diversos ámbitos.	\$6-8
RA5-c	Se ha razonado cómo influye la variación de las características de los sistemas en su dinámica de actuación.	\$9
RA5-d	Se han desarrollado estrategias de control definiendo los objetivos y las especificaciones de la respuesta del sistema.	\$10
RA5-e		\$11

CE	Criterio de evaluación	Bloque
	Se han relacionado los controladores inteligentes con el comportamiento del sistema.	



Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, «*Sistemas Expertos*», y le asigna estos contenidos:

- *Dinámica de los sistemas expertos.*
- *Estructuras elementales de los sistemas expertos.*
- *Representar y simular comportamientos básicos.*
- *Estrategias de control de un sistema experto.*
- *Aplicaciones de sistemas expertos.*
- *Tendencias en sistemas expertos.*

Son **seis contenidos para cinco criterios de evaluación**, y no encajan uno a uno: los dos últimos (aplicaciones y tendencias) no tienen CE propio y se reparten entre todos, mientras que RA5-c (variación de las características) **no tiene contenido explícito** en el anexo y lo desarrolla el centro (art. 3.3 del Decreto 95/2026).

3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de...
O1	Situ ar el conocimiento en la jerarquía DIKW y describ ir la arquitectura de un sistema experto.
O2	Diferenciar los modos de representación del conocimiento (reglas, frames, lógica, ontologías, redes semánticas).
O3	Representar y simular un comportamiento básico con <i>experta</i> , en un dominio real.
O4	Combinar reglas con datos cuando el conocimiento experto no basta por sí solo.
O5	Aplicar lógica difusa a un problema con incertidumbre.
O6	Explicar cómo influye la variación de reglas, hechos y umbrales en la dinámica (sensibilidad/robustez).
O7	Desarrollar estrategias de control (salience, control de meta) con sus especificaciones.
O8	Relacionar los controladores inteligentes (difuso, por reglas, ANN, MPC) con el comportamiento del sistema y compararlos con un PID.
O9	Conocer aplicaciones reales y tendencias de los sistemas expertos.

4. Arquitectura y dinámica de los sistemas expertos (RA5-a)

4.1 Del dato al conocimiento: la jerarquía DIKW

Antes de representar el conocimiento hay que distinguirlo de sus vecinos. La **jerarquía DIKW** (*Data, Information, Knowledge, Wisdom*) los ordena:



Nivel	Qué es	Ejemplo
Datos	Hechos o valores registrados, independientes de quien los lee	«Un reloj inteligente registra la temperatura corporal»
Información	Los datos interpretados por un agente; subjetiva	«La temperatura corporal es 37 °C»
Conocimiento	Información integrada en un modelo del mundo	«Si la temperatura supera 37 °C, la persona tiene fiebre»
Sabiduría	Meta-conocimiento: cuándo y cómo aplicar el conocimiento	«Si tiene fiebre, debe tomar paracetamol»

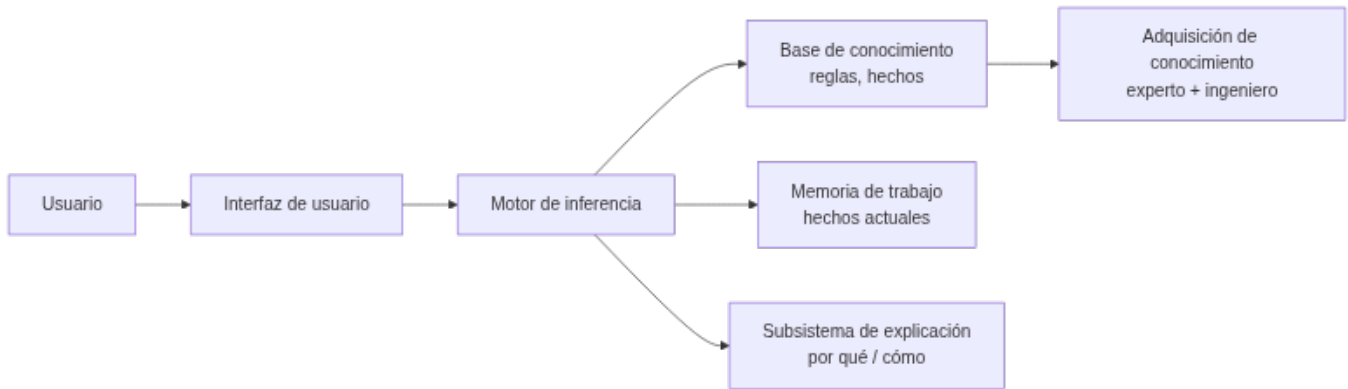


Por qué importa esta distinción

Un sistema experto no almacena datos ni información: almacena **conocimiento** (reglas) y, en los más avanzados, un poco de **sabiduría** (metarreglas que deciden cuándo aplicar otras reglas — el «control de meta» del §10.1). Confundir los niveles es el error más común al empezar a diseñar uno: se acumulan datos en vez de codificar reglas.

4.2 ¿Qué es un sistema experto?

Un **sistema experto** es un programa de IA que **emula el razonamiento de un experto humano** en un dominio concreto: codifica su conocimiento en una **base de conocimiento** y utiliza un **motor de inferencia** para aplicar ese conocimiento a los hechos del problema. Comenzaron su desarrollo en los años 70 y fueron muy populares esa década y la siguiente: se consideran los primeros sistemas de IA capaces de obtener resultados con utilidad práctica real.



4.3 Componentes

Componente	Función
Interfaz de usuario	Vía de las consultas: pregunta datos, muestra resultados y alerta de datos erróneos. Incluye un módulo de comunicaciones (con otros sistemas, útil en automatización industrial)
Base de conocimiento (KB)	Reglas y hechos del dominio, formalizados por el ingeniero de conocimiento junto al experto. Puede organizarse como listas, objetos relacionados, cálculo de predicados o redes semánticas
Memoria de trabajo (base de hechos)	Hechos actuales: los que aporta el usuario o los sensores, más los deducidos durante el razonamiento. Guarda el histórico de estados de la consulta
Motor de inferencia	Evalúa qué reglas se cumplen, resuelve conflictos y ejecuta las acciones. Determina el orden de las acciones y la interacción entre las partes del sistema
Subsistema de explicación	Justifica el razonamiento («por qué» y «cómo» llegó a esa conclusión), mostrando las reglas usadas. Ayuda también al ingeniero de conocimiento a depurar el motor y al experto a verificar la coherencia de la base
Adquisición de conocimiento	Herramientas para incorporar nuevo conocimiento sin perfil técnico (el famoso <i>bottleneck</i> : conseguir que el experto vuelque lo que sabe es la parte más lenta de construir el sistema)



La explicación marca la diferencia

La capacidad de **explicar** su razonamiento es lo que distingue a un sistema experto de un modelo de ML: en dominios regulados (medicina, finanzas) la justificación es obligatoria. MYCIN fue el pionero de esta *explicabilidad*, mostrando las reglas de inferencia que empleó — aunque, para consultas complejas, enumerar todas las reglas puede resultar tedioso para el usuario.

4.4 El ciclo de ejecución (dinámica)

El motor de inferencia opera en un **bucle reconocer-actuar**:

1. **Reconocer (match)**: se comparan los hechos de la memoria de trabajo con las condiciones (LHS) de las reglas; las reglas activas van a la **agenda**.
2. **Resolver (resolve)**: si hay varias reglas activas, se elige una según la estrategia de control (saliencia, recency, especificidad — ver §10.1).
3. **Actuar (act)**: se ejecuta el consecuente (RHS), que declara/modifica/retira hechos, y el ciclo se repite hasta agotar la agenda.

El objetivo de fondo es hacer que la lógica sea **explícita** en vez de estar enterrada en código que solo revisa un informático: con reglas legibles, el propio experto del dominio puede revisar y mantener el sistema.

4.5 Encadenamiento

- **Hacia delante (forward, data-driven)**: parte de los hechos y deduce conclusiones. Ideal para monitorización, control de procesos en tiempo real y planificación.

- **Hacia atrás (backward, goal-driven)**: parte de una **meta** y busca qué hechos la sustentan, preguntando al usuario solo lo necesario. Ideal para diagnóstico (MYCIN, Prolog).
- **Encadenamiento mixto**: combina los dos anteriores.
- Otros mecanismos de razonamiento: **búsqueda heurística** (el proceso de inferencia recorre un árbol) y **herencia** (en entornos orientados a objetos, un objeto hijo hereda propiedades del padre).



Las dos reglas de inferencia clásicas

Antes de forward/backward chaining está la lógica que los sustenta: **Modus Ponens** («si P implica Q , y P es verdad, entonces Q es verdad») y **Modus Tollens** («si P implica Q , y Q no es cierto, entonces P no es cierto»). El encadenamiento hacia delante aplica Modus Ponens repetidamente; el encadenamiento hacia atrás busca qué P sustentaría un Q dado.

4.6 Incertidumbre: factores de certeza

MYCIN introdujo los **factores de certeza (CF)** para manejar la incertidumbre: cada regla tiene un CF y se combinan al encadenar. Alternativas modernas: lógica difusa (§8), teoría de Dempster-Shafer o redes bayesianas (Heckerman & Shortliffe, 1992).



Combinar factores de certeza

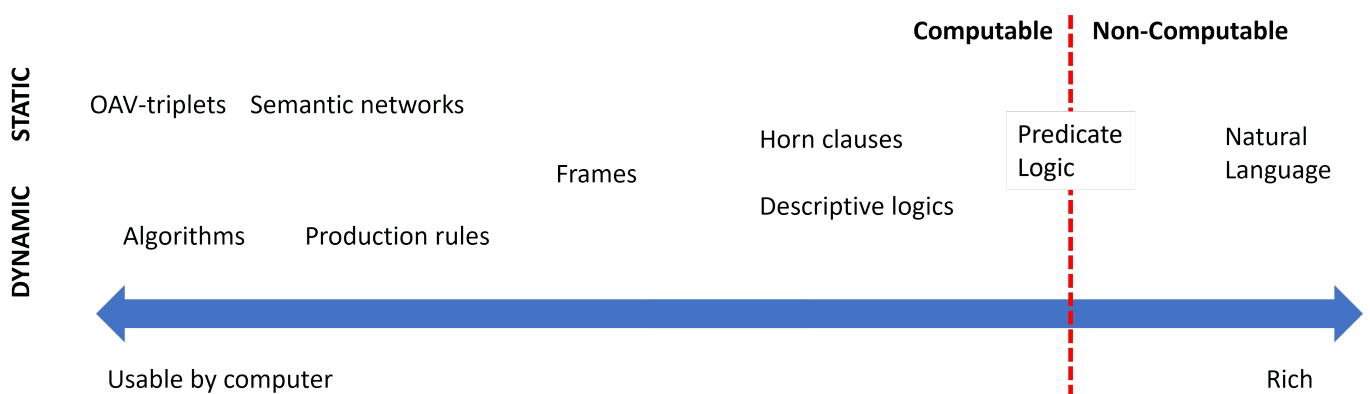
Una regla dice «SI hay fiebre alta ENTONCES sospecha de meningitis **con CF = 0,6**». Otra regla independiente aporta «SI rigidez de nuca ENTONCES sospecha de meningitis **con CF = 0,4**». MYCIN combina ambos apoyos para dar una confianza conjunta mayor que cada una por separado (combinación de evidencia). Si en cambio hubiera evidencia en contra, los CF se descuentan. De ahí nace el concepto de **acumulación de evidencia**, que luego se formalizó con redes bayesianas.

Esta gestión explícita de la **incertidumbre** es una diferencia clave frente a un sistema de reglas «duro» (todo verdadero o falso): permite razonar con conocimiento parcial y explicar el nivel de confianza de una conclusión. La lógica difusa del §8 resuelve el mismo problema de otra forma: en vez de un factor de certeza sobre una regla booleana, deja que el propio hecho sea **parcialmente verdadero**.

5. Estructuras elementales de representación (RA5-a/b)

5.1 Un continuo, no una lista cerrada

Representar el conocimiento es uno de los problemas fundamentales de la IA: hay que hacerlo de forma **entendible** para las máquinas, **útil** para resolver problemas y **eficiente** de procesar. Las representaciones forman un **continuo**:



En un extremo están las representaciones **simples** (algoritmos): eficientes para el ordenador pero poco flexibles. En el otro, las **flexibles** (texto natural): muy expresivas pero no directamente utilizables por una máquina. Entre medias viven las que usamos en esta unidad.

5.2 Las representaciones, con sus ventajas y límites

Representación	Estructura	Inferencia	Ventajas	Limitaciones
Pares atributo-valor (tripletes objeto-atributo-valor)	Lista de nodos y aristas de un grafo: «el perro es un animal, tiene cuatro patas...»	Recorrido y coincidencia de atributos	Muy simple de construir a partir de descripciones	Poco expresiva para relaciones complejas
Reglas de producción	SI ... ENTONCES ...	Forward/backward + resolución de conflictos	Modular, legible, explicable	Difícil modelar jerarquías; lenta con bases grandes
Representaciones jerárquicas	Árbol: nodos = conceptos, aristas = relaciones (Animales → Vertebrados → Mamíferos → Perros)	Recorrido del árbol, herencia	Natural para taxonomías	Rígida si un concepto pertenece a varias ramas
Marcos (frames)	Registros con ranuras (slots) y valores por defecto	Herencia de clases, disparadores	Taxonomías y conocimiento estructurado	Conflicto con herencia múltiple
Lógica formal	Predicados de primer orden (propuesta por Aristóteles hace más de 2000 años)	Resolución, unificación, deducción	Rigor matemático	Explosión combinatoria; poco utilizable directamente por máquinas (salvo un subconjunto, como en Prolog)
Redes semánticas	Grafos dirigidos con etiquetas	Búsqueda en grafos, propagación	Intuitiva para relaciones	Semántica ambigua
Ontologías	Clases, propiedades, axiomas (OWL)	Razonadores (Pellet, HermiT)	Interoperabilidad, reutilización	Curva de aprendizaje alta



Lo mismo en cuatro formas distintas

El hecho «si la temperatura supera 37 °C, la persona tiene fiebre» se puede escribir como **regla de producción** (SI temperatura > 37 ENTONCES fiebre), como **lógica proposicional** ($(p \rightarrow q)$, con (p) : «tiene fiebre», (q) : «debe tomar paracetamol»), como **jerarquía** (Síntomas → Fiebre → Paracetamol) o como **par atributo-valor** (persona.temperatura = 38.2, evaluado por una regla externa). Elegir la representación es elegir **qué es fácil de hacer** con ese conocimiento después.



Regla práctica de esta unidad

Usaremos sobre todo **reglas de producción** (con `experta`) y, en el §8, **lógica difusa** (con `scikit-fuzzy`). Las ontologías y las redes semánticas se mencionan como alternativas; los **frames** y la **lógica formal** se tratan a nivel conceptual.

6. Representar y simular comportamientos con `experta` (RA5-b)

Para **simular un comportamiento** se escribe la base de conocimiento en `experta` y se ejecuta el ciclo de inferencia. Recordamos el parche obligatorio para Python 3.10+ (la librería es de 2019 y `collections.Mapping` desapareció en esa versión):

```
1 import collections, collections.abc
2 if not hasattr(collections, 'Mapping'):
3     collections.Mapping = collections.abc.Mapping
4     collections.Iterable = collections.abc.Iterable
5     collections.MutableMapping = collections.abc.MutableMapping
6
7 from experta import *
8
9 class DiagnosticoPc(KnowledgeEngine):
10     @DefFacts()
11     def inicio(self):
12         yield Fact(accion="diagnosticar")
13
14     @Rule(Fact(accion="diagnosticar"), salience=10)
15     def arrancar(self):
16         print("Diagnóstico del PC...")
17         self.declare(Fact(luz_encendida=True))
18         self.declare(Fact(sonido="pitidos_cortos"))
19
20     @Rule(Fact(luz_encendida=True), Fact(sonido="pitidos_cortos"))
21     def ram(self):
22         self.declare(Fact(causa="problema_ram"))
23
24     @Rule(Fact(causa="problema_ram"))
25     def resultado(self):
26         print("DIAGNÓSTICO: Fallo de memoria RAM.")
27
28 engine = DiagnosticoPc()
29 engine.reset()
30 engine.run()
```

Salida esperada:

```
1 Diagnóstico del PC...
2 DIAGNÓSTICO: Fallo de memoria RAM.
```

 **Otro ámbito: clasificación de animales (sistema de producción clásico)**

El mismo motor `experta` simula un sistema de clasificación con reglas de un zoólogo:

```
1 class Animales(KnowledgeEngine):
2     @DefFacts()
3     def inicio(self):
4         yield Fact(analizar=True)
5
6     @Rule(Fact(analizar=True), salience=10)
7     def datos(self):
8         self.declare(Fact(pelo=True), Fact(carnivoro=True), Fact(color="leonado"))
9         self.declare(Fact(manchas="oscuras"))
10
11     @Rule(Fact(pelo=True))
12     def mamifero(self):
13         self.declare(Fact(mamifero=True))
14
15     @Rule(Fact(mamifero=True), Fact(carnivoro=True), Fact(color="leonado"),
16           Fact(manchas="oscuras"))
17     def guepardo(self):
18         print("IDENTIFICADO: guepardo")
```

Así se comprueba que un sistema experto **representa y simula el comportamiento** de un dominio sin programar la lógica imperativamente: el motor aplica las reglas hasta llegar a la conclusión.

6.1 «Muy diversos ámbitos»: los notebooks de la unidad

El CE b exige simular comportamientos en **dominios distintos**, no repetir el mismo ejemplo. Esta unidad trae varios notebooks ejecutados de verdad, en dominios reales (la lista completa está en las [actividades guiadas](#) y las [entregables](#)):

Notebook	Dominio	Qué simula
UD05_N01_experta_primeros_pasos.ipynb	Introducción	Primeros pasos con <code>experta</code> : hechos, reglas, <code>DefFacts</code>
UD05_N02_piedra_papel_tijera.ipynb	Juego	Piedra, papel o tijera decidido por reglas
UD05_N03_clasificacion_animales.ipynb	Zoología	El ejemplo de arriba, completo y ejecutado
EX1 · lesión de rodilla	Medicina	Diagnóstico de una lesión a partir de síntomas, con <code>experta</code>

Notebook	Dominio	Qué simula
UD05_N04_reglas_desde_datos_titanic.ipynb	Datos históricos	Reglas extraídas de datos, no escritas a mano (§7)
EX2 · valor de mercado	Deporte	Estimación híbrida reglas + aprendizaje automático (§7)
EX3 · centrocampistas · EX4 · quemador de gas	Deporte · industria	Lógica difusa (§8) y control de un proceso real (§10-11)



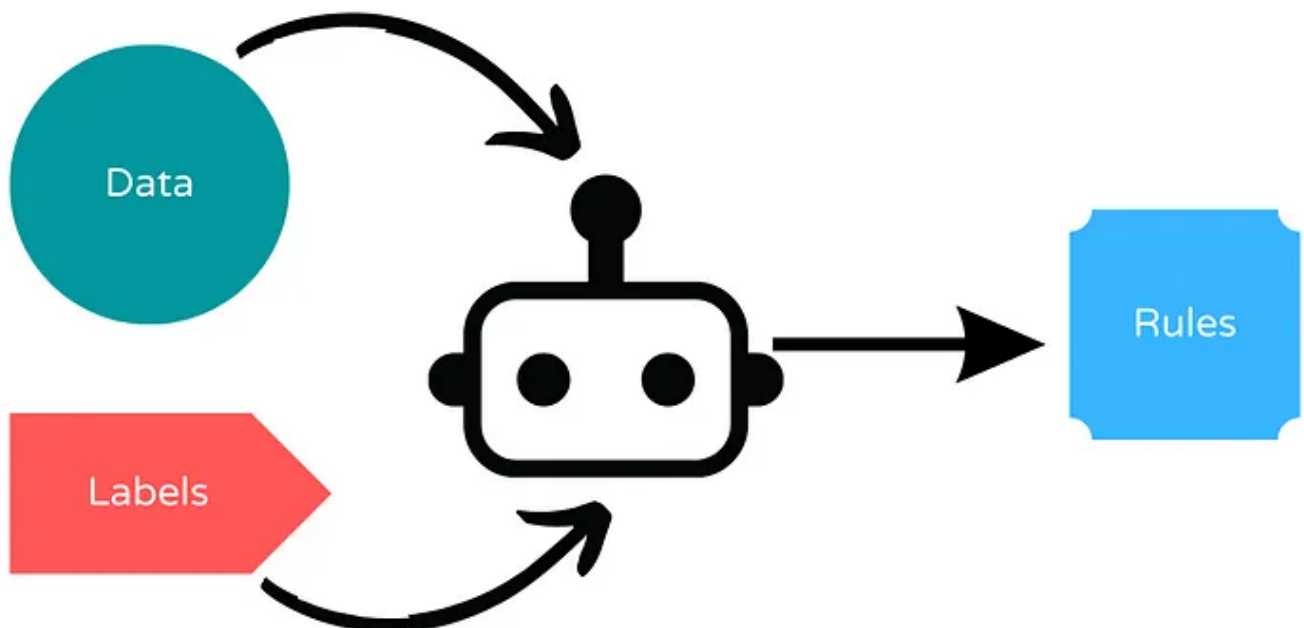
Por qué importa la diversidad

Un sistema experto no es una técnica de un solo dominio: la **misma arquitectura** (base de conocimiento + motor de inferencia) sirve para diagnosticar una rodilla, valorar a un futbolista o regular un quemador de gas. Lo que cambia es el conocimiento que se codifica, no el motor que lo aplica.

7. Sistemas híbridos reglas/datos (RA5-b)

Un sistema experto puro necesita que **alguien escriba las reglas a mano**. Pero a veces el conocimiento del dominio ya está en los datos, o conviene mezclar las dos fuentes. Hay dos enfoques:

- **Deducir reglas a partir de datos:** un algoritmo genera reglas legibles a partir de un conjunto de entrenamiento, en vez de que las escriba una persona. Facilita la **interpretación** del razonamiento, porque el resultado sigue siendo **SI ... ENTONCES ...** en vez de una caja negra.
- **Integrar reglas definidas por el usuario con aprendizaje automático:** el experto pone las reglas de partida y el aprendizaje automático las **mejora** con datos.



7.1 Bibliotecas del segundo enfoque

Biblioteca	Qué hace
Human-Learn	Permite definir y dibujar reglas propias como si fueran un clasificador de scikit-learn (<code>FunctionClassifier</code>), y combinarlas con aprendizaje automático

Biblioteca	Qué hace
skope-rules	Analiza los datos y deduce reglas de clasificación; permite auditarlas e interpretarlas
FIGS (<code>imodels</code>)	Genera reglas fáciles de interpretar combinando varios árboles pequeños («sumas de árboles»), en vez de un único árbol grande difícil de leer
spaCy	Reglas para extraer información de texto : útil cuando no hay suficientes datos etiquetados o para casos muy específicos

UD05_N04_reglas_desde_datos_titanic.ipynb : reglas que salen de los datos, no de un experto

En vez de que alguien escriba a mano «si viajas en primera clase y eres mujer, sobrevives», **FIGS** analiza los datos históricos del Titanic y **genera esa regla por sí solo**, junto con otras. El resultado sigue siendo legible —un árbol de reglas pequeño—, pero nadie lo escribió a mano: se dedujo de los datos. Es el primer enfoque de la tabla de arriba.

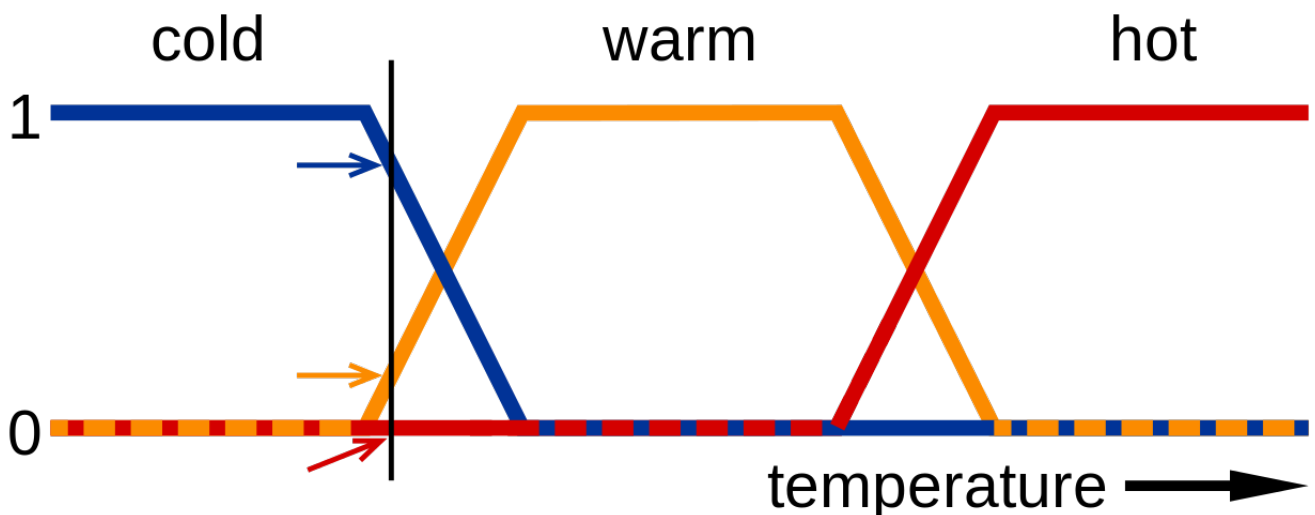
EX2 : valor de mercado de futbolistas con Human-Learn + FIGS

El entregable **EX2** combina las dos ideas sobre los mismos datos (FIFA 22): una `FunctionClassifier` de Human-Learn (reglas que **tú** defines sobre atributos del jugador) y un `FIGSClassifier` que **deduce** sus propias reglas del conjunto de entrenamiento. Comparar los dos resultados sobre el mismo problema es la mejor forma de sentir la diferencia entre los dos enfoques híbridos.

Esto no es una técnica aislada: es una tendencia

Los sistemas híbridos reglas/datos son la puerta de entrada a los **sistemas neuro-simbólicos** y a las **reglas como guardarraíl del ML** que se tratan en el §12.2. No son un tema aparte de los sistemas expertos: son su evolución natural cuando el conocimiento no cabe entero en la cabeza de un experto.

8. Sistemas de razonamiento impreciso: lógica difusa (RA5-b/d)



8.1 De lo binario a lo continuo

La lógica proposicional clásica es **binaria**: un enunciado es cierto o falso. Pero los humanos no razonamos así — «hace frío» no es verdadero o falso de forma tajante, es una cuestión de grado. La **lógica difusa** (o borrosa) extiende la lógica proposicional para trabajar con esa incertidumbre:

- Los valores de verdad son **números reales en el intervalo $[0, 1]$** : 0 es falso, 1 es cierto, 0,5 es «cierto al 50 %».
- La pertenencia de un elemento a un conjunto viene dada por una **función de pertenencia**, $\mu_A(x)$: el grado de pertenencia de (x) al conjunto (A) .

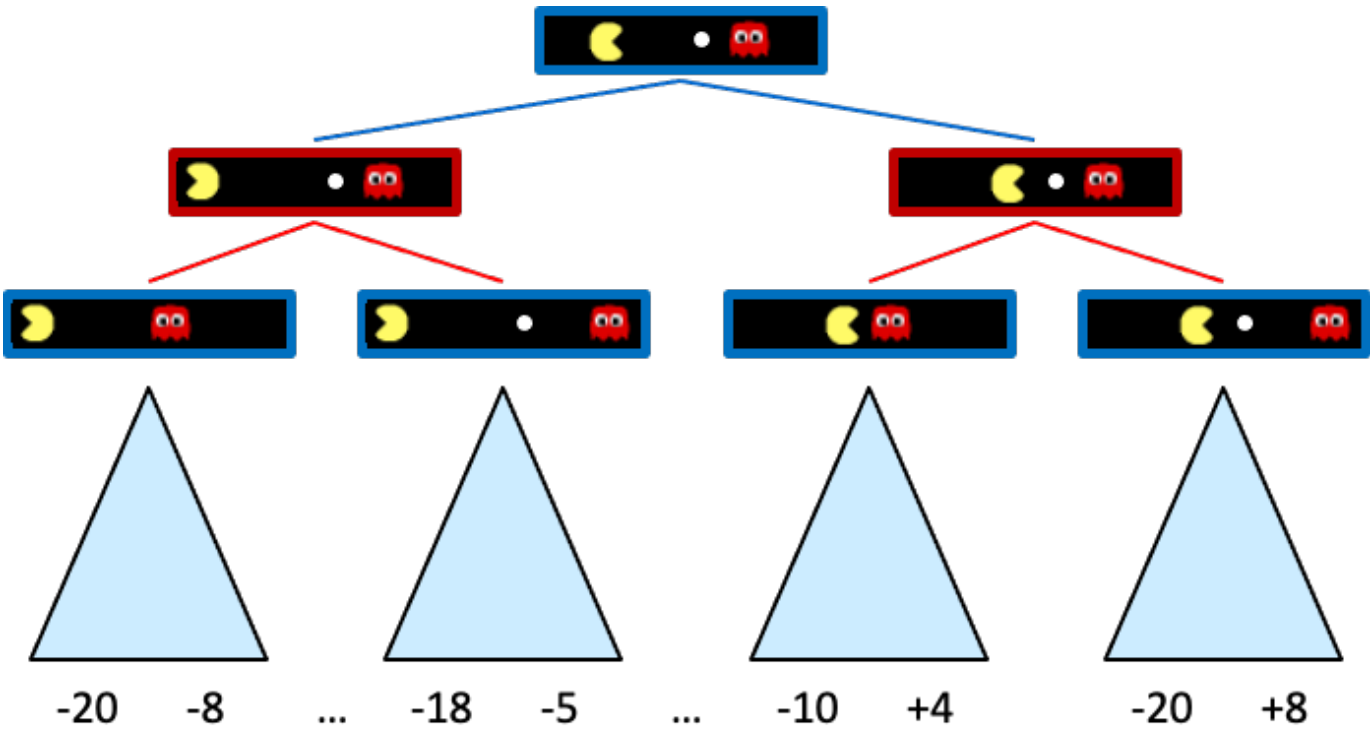
- Conceptos como *húmedo* o *frío* son difíciles de definir con precisión, pero la lógica difusa los define con funciones de pertenencia — lo que facilita crear dispositivos como termostatos: «*si la temperatura es fría, entonces enciende la calefacción*».
- Un **sistema de razonamiento impreciso** es un sistema basado en reglas que usa lógica difusa. Permite trabajar con valores **continuos**, modela mejor el conocimiento humano y es muy apropiado para **sistemas de control** — por eso enlaza directamente con el §10 y el §11.

8.2 Conceptos básicos

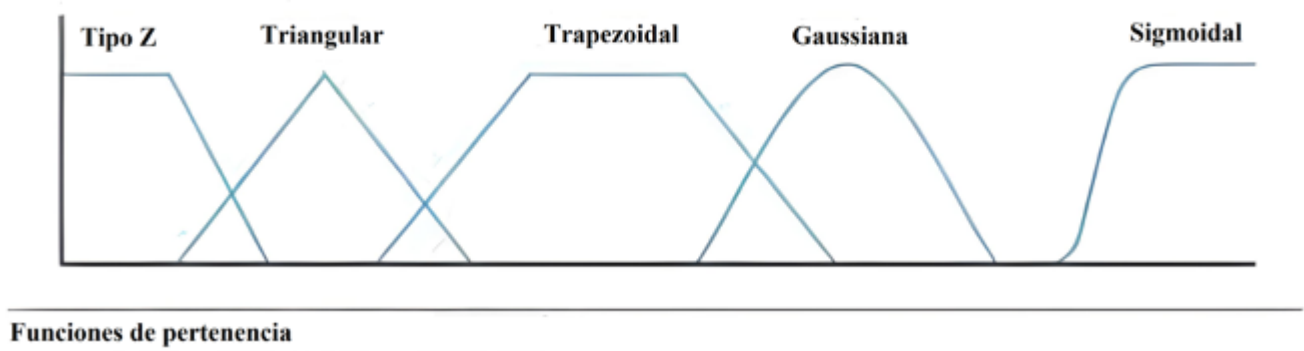
Concepto	Qué es	Ejemplo
Variable lingüística	Variable que toma valores lingüísticos	Temperatura
Valores lingüísticos	Los valores que puede tomar esa variable	Frío, Calor
Función de pertenencia	Asigna a cada valor un grado de pertenencia a un valor lingüístico	$\mu_{\text{Calor}}(27^{\circ}\text{C}) = 0.8$
Regla difusa	Regla que usa valores difusos	«Si la temperatura es fría , entonces calefacción alta »
Función de agregación	Combina los valores difusos de varias reglas en una conclusión	$\mu_{\text{calefacción}} = \min(\mu_{\text{fría}}, \mu_{\text{calefacción alta}})$

8.3 El funcionamiento en tres pasos

1. **Fuzzificación:** convierte los datos de entrada precisos en valores difusos, usando las funciones de pertenencia. $\mu_{\text{Calor}}(27^{\circ}\text{C}) = 0.8$, $\mu_{\text{MuchoCalor}}(27^{\circ}\text{C}) = 0.2$.
2. **Evaluación de las reglas:** se aplican las reglas del sistema, combinando las funciones de pertenencia de las variables de entrada para deducir la relevancia de la salida. Por ejemplo: «si la temperatura es **alta** y la humedad es **baja**, la velocidad del ventilador debe ser **alta**».
3. **Desfuzzificación:** convierte la conclusión difusa de vuelta en un valor preciso, con una función de agregación (normalmente **centro de gravedad** o **máximo**).



Las funciones de pertenencia más usadas son **trapezoidales** y **triangulares**; las sinusoidales sirven para representar periodos, y las sigmoidales, probabilidades.

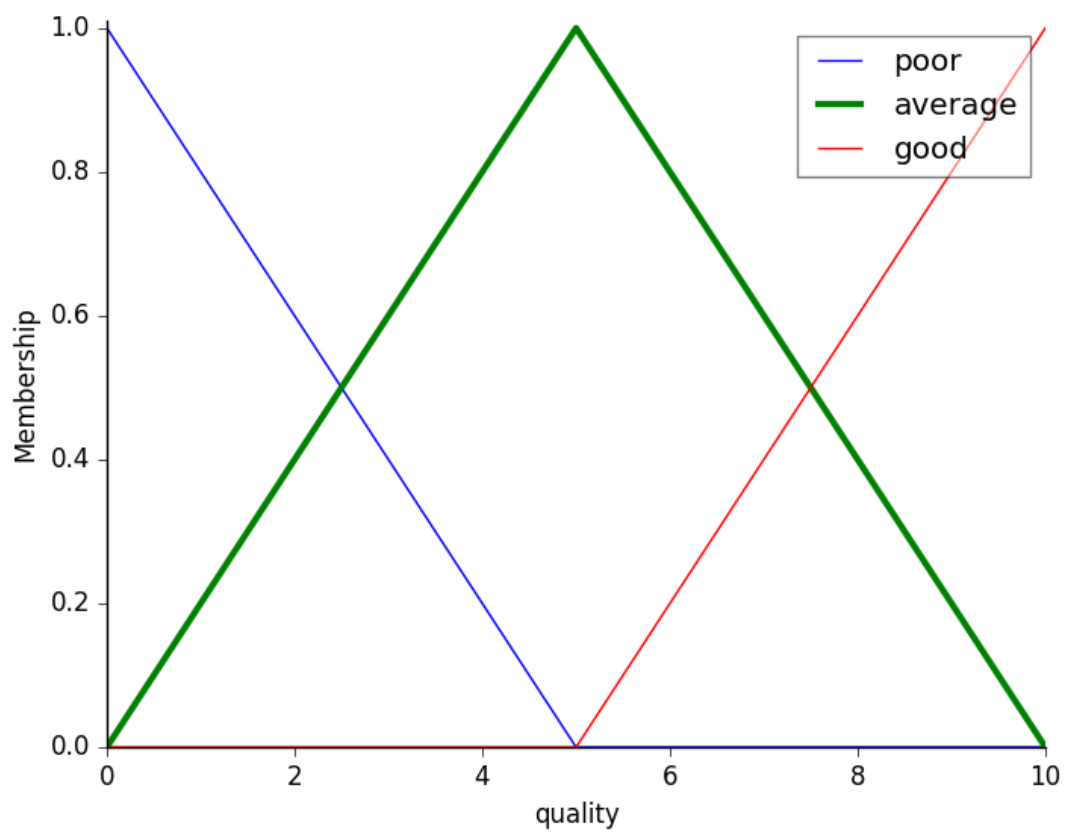
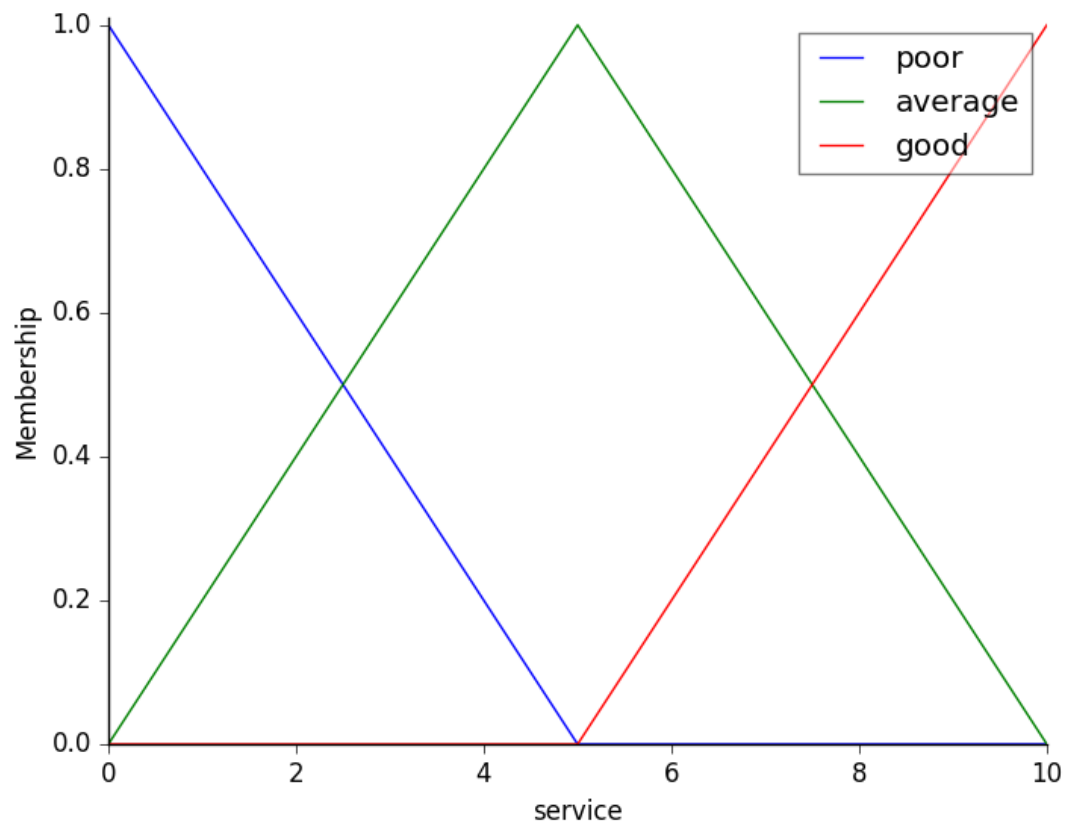


8.4 Ejemplo trabajado: la propina del restaurante

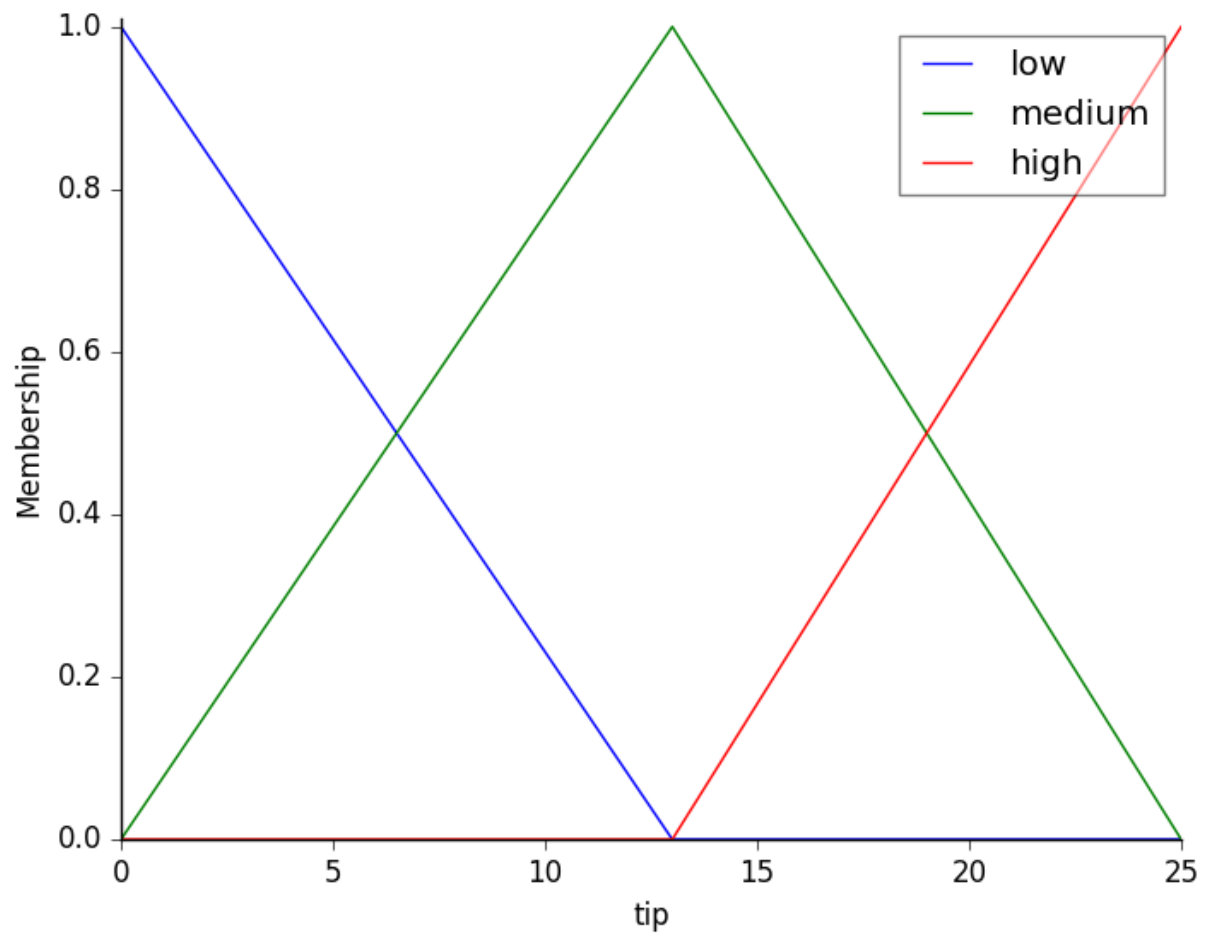
Es el ejemplo canónico de `scikit-fuzzy`, y el que verás resuelto paso a paso en el [Taller 2](#).

Variables de entrada (funciones triangulares):

- **Calidad del servicio:** baja $\setminus([0,5])$ · media $\setminus([0,10])$ · alta $\setminus([5,10])$
- **Calidad de la comida:** mala $\setminus([0,5])$ · media $\setminus([0,10])$ · buena $\setminus([5,10])$

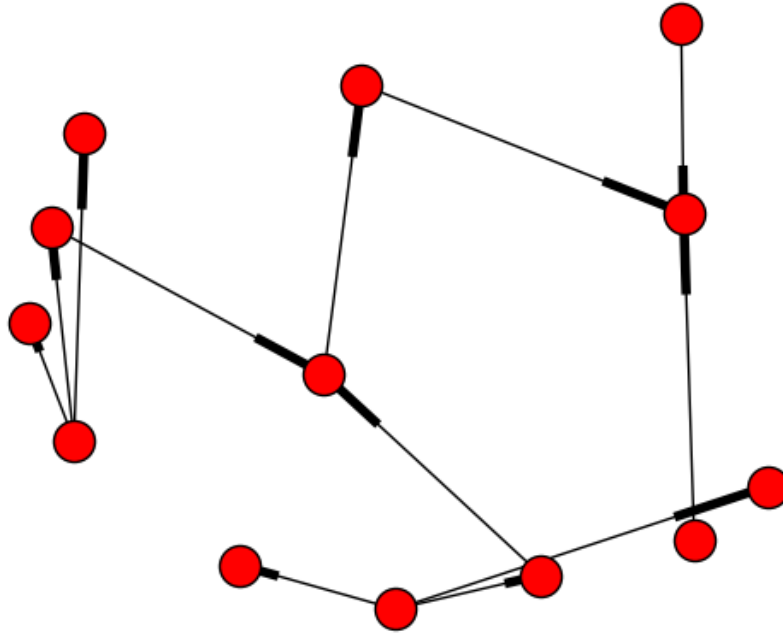


Variable de salida: propina — baja $\setminus([0,13])$ · media $\setminus([0,25])$ · alta $\setminus([13,25])$

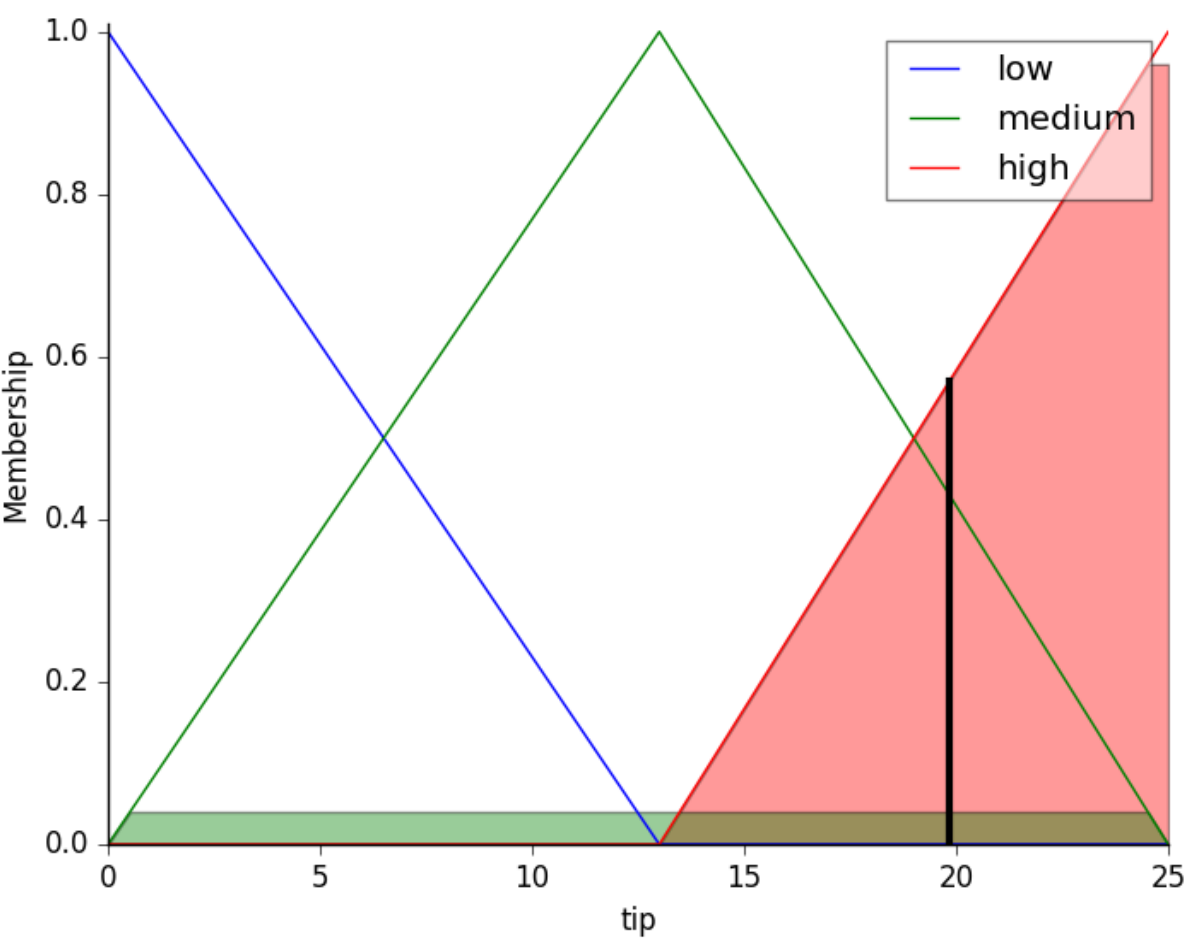


Reglas:

- SI (servicio es **baja** O comida es **mala**) ENTONCES propina es **baja**
- SI (servicio es **media**) ENTONCES propina es **media**
- SI (servicio es **alta** O comida es **buena**) ENTONCES propina es **alta**



Inferencia: con un servicio de **9,8** y una comida de **6,5**, el sistema desfuzzifica una propina de **19,24 €** — en la banda alta, coherente con un servicio casi perfecto y una comida notable.



De la propina al quemador de gas

Este ejemplo es idéntico en estructura al **Taller 3** (control de un quemador de gas) y al entregable **EX4**: variables de entrada difusas, reglas lingüísticas, una salida desfuzzificada. La diferencia es que ahí la salida no es una propina, es la **potencia de un actuador real** — es el paso del §8 al §11.

9. Variación de características y dinámica (RA5-c)

9.1 Sensibilidad y robustez

- **Sensibilidad:** cómo cambian las conclusiones ante pequeñas desviaciones en los parámetros o datos de entrada. Un sistema muy sensible detecta pronto anomalías, pero **falsas alarmas** ante ruido.
- **Robustez:** capacidad de mantener conclusiones estables y seguras ante perturbaciones, ruido o fallos parciales.

9.2 Tres mecanismos de variación

Qué varía	Efecto en la dinámica
Las reglas (estructura lógica)	Añadir condiciones al antecedente → más inercia (la regla se dispara menos); simplificar → sobreactivación (reglas se disparan ante transitorios sin riesgo)
Los hechos (perturbaciones)	Ruido en un sensor (p. ej. CO en un túnel oscilando alrededor del umbral) → el sistema conmuta actuadores sin parar
Los umbrales de certeza	Subir el umbral de un diagnóstico → menos falsos positivos pero puede ignorar alertas tempranas



El problema de la histéresis

Un sensor de CO en un túnel oscila entre 28 y 32 ppm con umbral en 30. Sin histéresis, los extractores se encienden y apagan en cada ciclo. Soluciones: una **regla de histéresis temporal** (la alerta debe mantenerse 30 s) o un **controlador difuso** que suavice la transición (enlaza con el §8).

10. Estrategias de control de un sistema experto (RA5-d)

10.1 Control de la agenda (resolución de conflictos)

Estrategia	Qué hace
Salience	Prioridad numérica explícita de cada regla (las de emergencia van antes)
Recency	Prefiere las reglas con hechos más recientes (time-tags)
Specificity	Prefiere la regla con más condiciones en el antecedente
Control de meta (metarrazonamiento)	Reglas de nivel superior que modifican prioridades, activan grupos de reglas según el modo (arranque, operación, parada) o cambian la estrategia. Es la sabiduría de la jerarquía DIKW del §4.1, aplicada al propio motor

10.2 Especificaciones de respuesta

Para controlar un proceso, el sistema experto debe cumplir **especificaciones** sobre la respuesta de la variable controlada:

Especificación	Qué mide
Precisión	Error en régimen permanente (diferencia entre la variable y el setpoint al estabilizarse)
Tiempo de respuesta	Tiempo de subida (t_r) y de asentamiento (t_s)
Estabilidad	Ausencia de oscilaciones; sobreimpulso (overshoot) máximo aceptable
Alcance	Rango de operación de la planta



Nivel FP

En este módulo se trabajan estas especificaciones de forma **conceptual** (qué miden y por qué importan), sin las fórmulas de la teoría de control de ingeniería. El foco está en *definir el objetivo y saber interpretar si la respuesta lo cumple*.

10.3 Ejemplo guiado: definir las especificaciones de un controlador de climatización

Recorremos juntos el razonamiento que repetirás en el Taller 3, antes de aplicarlo al quemador de gas real de [EX4](#).

Problema: queremos controlar la temperatura de una sala para que llegue a 21 °C.

Paso 1 · Setpoint y precisión. La variable controlada (temperatura) debe estabilizarse en 21 °C con un **error en régimen permanente** de $\pm 0,5$ °C como máximo.

Paso 2 · Tiempo de respuesta. La sala debe alcanzar la banda aceptable (20,5-21,5 °C) en menos de **15 minutos** desde el arranque (tiempo de asentamiento).

Paso 3 · Estabilidad y sobreimpulso. Se tolera un sobreimpulso máximo de **1 °C** por encima del setpoint (sin oscilaciones que dañen el equipo).

Paso 4 · Elegir el controlador. Con un PID bien sintonizado se cumple en un sistema lineal, pero la sala tiene inercia térmica y perturbaciones (puertas, sol). Un **controlador experto o difuso** con reglas del operador («si la diferencia es grande → máxima potencia, y al acercarse → reducir para no sobrepasar») consigue la respuesta con **menos sobreimpulso y más robustez** ante las perturbaciones.

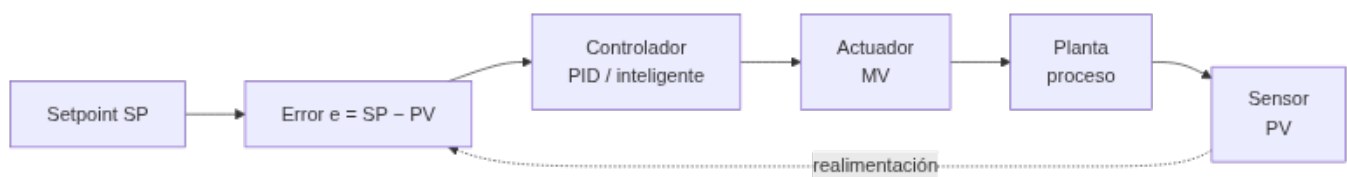
Paso 5 · Comprobar y ajustar. Se simula la respuesta y se mide t_s y el sobreimpulso; si no se cumple la especificación, se ajustan reglas o ganancias (análisis de sensibilidad, §9).

**Conclusión del ejemplo guiado**

Controlar no es «conectar un motor»: es **definir el objetivo** (precisión), **acotar el tiempo**, **limitar el sobreimpulso** y **elegir el controlador** que lo cumpla, verificándolo con una simulación. Ese es el sentido del CE d y del CE e — y exactamente lo que se te pide en EX4, con un quemador de gas real en vez de una sala.

11. Controladores inteligentes (RA5-e)**11.1 Del control clásico al control inteligente**

Un **lazo de control** clásico compara el setpoint (SP) con la variable medida (PV), calcula el error y aplica una acción (MV) con un **controlador PID** (proporcional-integral-derivativo). El PID es eficaz en sistemas lineales, pero sufre con **retrasos**, **no linealidades** y **perturbaciones multivariable**.



La acción del PID es $u = K_p \cdot e + K_i \cdot \int e + K_d \cdot de/dt$: la parte proporcional responde al error; la integral elimina el error residual y la derivada reduce el sobreimpulso. Sus límites: no predice el futuro, se degrada con retrasos y no linealidades, y hay que **re-sintonizarlo** cuando la planta envejece (ganancias fijas).

Los **controladores inteligentes** sustituyen o complementan al PID usando técnicas de IA:

Controlador	Cómo funciona	Ventaja frente al PID
Control difuso	Reglas lingüísticas + funciones de pertenencia (Mamdani/Sugeno) — el §8 aplicado al control	Robusto ante ruido y no linealidad; no necesita modelo
Control por reglas (experto)	Motor de inferencia con reglas del operador experto	Captura heurísticas de operadores; fácil de entender
Redes neuronales	Aprenden la dinámica inversa de la planta	Adaptación a sistemas no lineales
Control predictivo (MPC)	Predice la trayectoria futura y optimiza la acción	Anticipa restricciones; proactivo

11.2 Ejemplos con datos

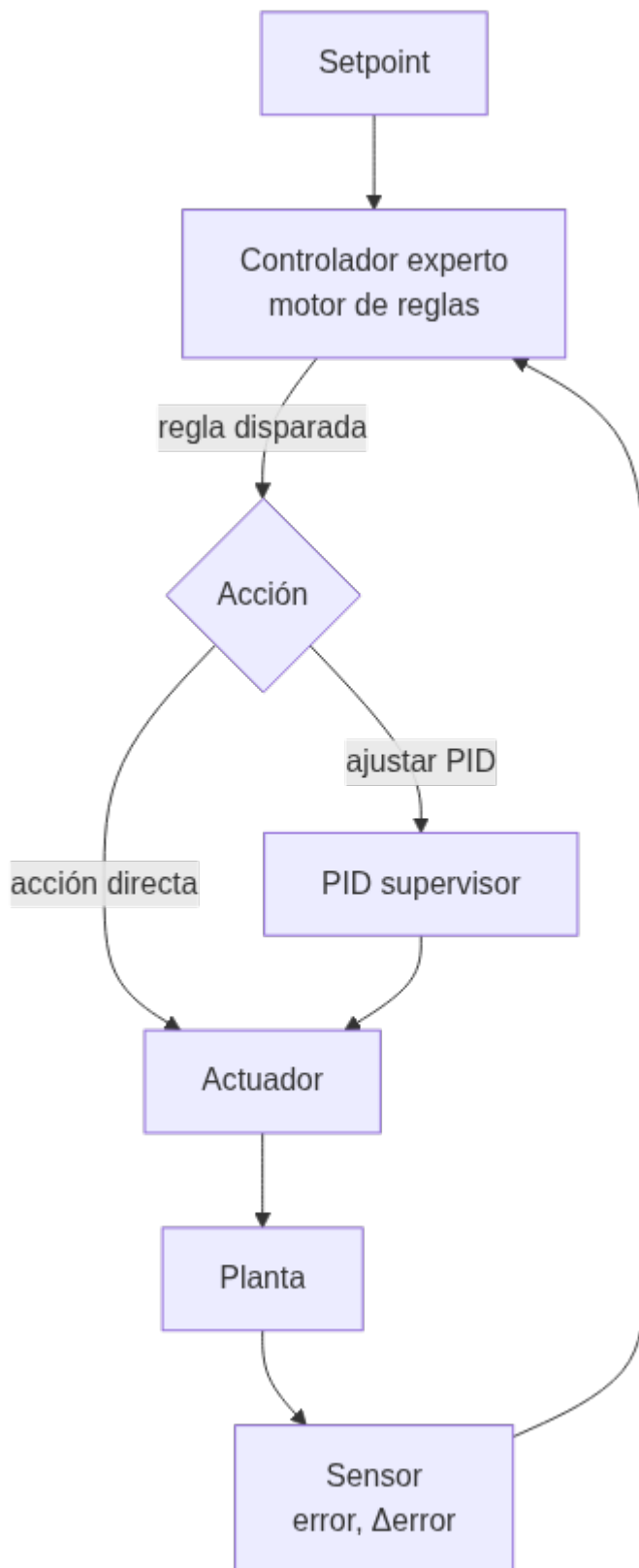
Caso	Mejora frente a PID
Control térmico difuso	Tiempo de asentamiento –76 % (416 s → 100 s); sobreimpulso 5,64 % → 0 %
Horno de fundición (ANN)	MSE 132,75 frente a 134,13 del PID (ligera mejora)
Mitsubishi (aire acondicionado)	Calienta/enfría 5× más rápido, –24 % consumo
DeepMind centros de datos	–40 % en energía de refrigeración (–15 % PUE overhead)

**¿Controlador inteligente siempre gana?**

No siempre. El PID es simple, barato y fiable en sistemas bien modelados. El controlador inteligente aporta cuando hay **no linealidad**, **retraso o ruido fuerte**. La decisión es *empezar simple* y añadir inteligencia solo si se justifica — es lo que comprobarás tú mismo en EX4, que pide **dos enfoques distintos** para el mismo quemador.

11.3 El sistema experto como controlador

El patrón del **control experto** (Åström, Anton y Årén, 1986) usa un motor de inferencia dentro del lazo: el sistema evalúa el error y su variación, decide la acción por reglas (supervisando y ajustando incluso un PID) y explica la decisión. Es la base de sistemas industriales como **Gensym G2** (refinerías, energía, farmacia) desde los años 80.



El controlador experto puede actuar de dos formas: **directamente** (decide la acción por reglas, por ejemplo «si el error es negativo grande y aumenta → máxima potencia») o **supervisando** un PID (ajusta sus ganancias cuando detecta que la respuesta se degrada). En ambos casos, la decisión es **explicable**: se puede consultar qué regla se disparó y por qué.



Regla de un controlador experto de climatización

```

1 SI error de temperatura es NEGATIVO_GRANDE
2 Y variación del error es POSITIVA_PEQUENA
3 ENTONCES potencia del quemador es MEDIA_ALTA

```

Estas reglas capturan la heurística de un operador humano y son fáciles de auditar, algo que un PID o una red neuronal no ofrecen de forma directa.

12. Aplicaciones y tendencias (contenidos del RA5)

12.1 Aplicaciones por sector

Sector	Uso
Industria	Control de procesos, diagnóstico de máquinas, mantenimiento
Salud	Diagnóstico asistido (MYCIN, GARVAN-ES1), dosificación
Finanzas	Asesoramiento, detección de fraude por reglas
Telecomunicaciones	Gestión de redes, diagnóstico de averías



Ejemplos clásicos con datos

- **MYCIN** (Stanford, 1972): ~500-600 reglas de diagnóstico clínico; precisión 69-70 % (equiparable a especialistas). Pionero de la explicación y los factores de certeza (§4.6).
- **XCON/R1** (CMU/DEC, 1978): configuración de ordenadores VAX; pasó de 250 a más de 6.200 reglas en 1986; precisión 95-98 % y ahorro de ~25 M\$/año para DEC.
- **DENDRAL** (1965): primer sistema experto; acotaba millones de isómeros moleculares a un conjunto manejable.
- **SID** (DEC): generó el 93 % de las puertas lógicas del VAX 9000.
- **Cyc** (1984): proyecto de enciclopedia el sentido común (millones de hechos y reglas).

12.2 Tendencias

- **BRMS** (*Business Rule Management Systems*): motores de reglas empresariales como **Drools** (Apache KIE) con estándares DMN/JSR-94. Permiten a las empresas gestionar miles de reglas de negocio separadas del código, con motores optimizados (PHREAK) que escalan linealmente.
- **Sistemas neuro-simbólicos** (3.ª ola de la IA): combinan redes neuronales (aprendizaje) con reglas y razonamiento simbólico (lógica), reduciendo alucinaciones y aportando explicabilidad — es la generalización de los **sistemas híbridos reglas/datos** del §7. Ejemplo: Amazon utiliza arquitecturas neuro-simbólicas en sus motores de compra para contrastar las salidas generativas con hechos y reglas.
- **IA explicable (XAI)**: MYCIN fue el origen de la explicación; hoy se usan SHAP/LIME para explicar modelos y la normativa exige el **derecho a explicación** del RGPD (enlaza UD06).
- **Agentes y razonamiento declarativo**: agentes que planifican con reglas y guardas lógicas de seguridad, garantizando que las decisiones distribuidas cumplen restricciones.
- **Reglas como guardarraíl del ML**: usar reglas para validar y acotar las salidas de los modelos (p. ej. un LLM no puede emitir una orden fuera de los rangos seguros definidos por reglas). Es la misma idea del §7, llevada a los modelos generativos.



Del pasado al presente

Los sistemas expertos «clásicos» (MYCIN, XCON) demostraron que el conocimiento declarativo y la explicación importan. Hoy esa idea **no ha muerto**: vive en los BRMS, en los sistemas neuro-simbólicos y en los guardarraíles que aseguran las salidas del ML. Saber construir y auditar sistemas basados en reglas —puras, híbridas o difusas— sigue siendo una competencia valiosa.

13. Puntos clave de la unidad

- La jerarquía **DIKW** distingue dato, información, conocimiento y sabiduría: un sistema experto almacena conocimiento (reglas) y, en su forma más avanzada, sabiduría (metarreglas).
- Un **sistema experto** codifica el conocimiento de un experto en una **base de conocimiento** y lo aplica con un **motor de inferencia** (ciclo reconocer-actuar, forward/backward).
- La **explicación** del razonamiento es su gran ventaja frente al ML (obligatoria en dominios regulados).
- La **representación del conocimiento** forma un continuo: pares atributo-valor, reglas, jerarquías, frames, lógica, redes semánticas u ontologías.
- Con `experta` (parche Py3.10+) se **simulan comportamientos** de ámbitos muy diversos: medicina, zoología, deporte, industria.
- Los **sistemas híbridos reglas/datos** (Human-Learn, FIGS, skope-rules) combinan el conocimiento experto con lo que aprenden los datos, cuando las reglas escritas a mano no bastan.
- La **lógica difusa** extiende la lógica a valores continuos $\in [0,1]$, con fuzzificación, evaluación de reglas y desfuzzificación — clave para el control de procesos reales.
- **Sensibilidad vs robustez**: variar reglas, hechos o umbrales cambia la dinámica (inanición, sobreactivación, falsas alarmas); la **histéresis** o el control difuso la estabilizan.
- Las **estrategias de control** (salience, control de meta) y las **especificaciones** (precisión, tiempo, estabilidad) definen la calidad de la respuesta.
- Los **controladores inteligentes** (difuso, por reglas, ANN, MPC) superan al PID en sistemas no lineales, con datos como -76% de asentamiento o 0% de sobreimpulso.
- **Tendencias**: BRMS/Drools, neuro-simbólico, XAI, reglas como guardarraíl del ML.

14. Glosario

Término	Definición
Dato	Hecho o valor registrado, independiente de quien lo interpreta
Información	Un dato interpretado por un agente
Conocimiento	Información integrada en un modelo del mundo
Sabiduría	Meta-conocimiento: cuándo y cómo aplicar el conocimiento
Sistema experto	Programa de IA que emula el razonamiento de un experto en un dominio
Base de conocimiento	Reglas y hechos del dominio
Memoria de trabajo	Hechos actuales del problema
Motor de inferencia	Ejecuta el ciclo reconocer-actuar
Ingeniero de conocimiento	Profesional que formaliza el conocimiento del experto
Adquisición de conocimiento	Proceso de capturar el conocimiento (bottleneck)
Subsistema de explicación	Justifica el razonamiento del sistema
Forward chaining	Encadenamiento hacia delante (data-driven)
Backward chaining	Encadenamiento hacia atrás (goal-driven)
Modus Ponens	Si P implica Q y P es verdad, Q es verdad
Modus Tollens	Si P implica Q y Q es falso, P es falso
Factor de certeza	Medida de confianza de una conclusión (MYCIN)
Frame	Estructura con ranuras y valores por defecto
Red semántica	Grafo de conceptos y relaciones
Ontología	Esquema formal de clases y propiedades (OWL)
Sistema híbrido reglas/datos	Combina reglas escritas o deducidas con aprendizaje automático

Término	Definición
FIGS	Algoritmo que genera reglas interpretables como suma de árboles pequeños
Lógica difusa	Extensión de la lógica a valores continuos en $\mathbb{[0,1]}$
Variable lingüística	Variable que toma valores lingüísticos (p. ej. «temperatura»)
Función de pertenencia	Grado de pertenencia de un valor a un conjunto difuso
Fuzzificación	Conversión de un valor preciso a valores difusos
Desfuzzificación	Conversión de una conclusión difusa a un valor preciso
Salience	Prioridad numérica de una regla
Control de meta	Metarreglas que gestionan la propia inferencia
Sensibilidad	Variación de las conclusiones ante perturbaciones pequeñas
Robustez	Estabilidad de las conclusiones ante ruido o fallos
Histéresis	Retardo en el cambio de estado para evitar conmutaciones
PID	Controlador proporcional-integral-derivativo
Lazo de control	SP → error → controlador → planta → PV
Error en régimen permanente	Diferencia residual entre variable y setpoint
Sobreimpulso (overshoot)	Exceso máximo sobre el setpoint durante el transitorio
Tiempo de asentamiento	Tiempo hasta que la respuesta entra en la banda aceptable
Control difuso	Controlador basado en reglas lingüísticas y pertenencia
Control predictivo (MPC)	Optimiza la acción prediciendo la trayectoria futura
BRMS	Sistema de gestión de reglas de negocio (Drools)
Neuro-simbólico	Combinación de redes neuronales y razonamiento simbólico
XAI	IA explicable

15. FAQ

? ¿Un sistema experto puede aprender de los datos? ▾

No por sí mismo: las reglas las escribe un experto. Por eso es totalmente **explicable**, pero requiere que el conocimiento esté disponible y formalizado (el famoso *bottleneck*). Los **sistemas híbridos** del §7 son justo la respuesta a esta limitación: dejan que el aprendizaje automático deduzca o mejore las reglas.

? ¿Los sistemas expertos están anticuados? ▾

No: los motores de reglas siguen en **BRMS empresariales** (Drools, SAP) y resurgen en los **sistemas neuro-simbólicos** y como **guardarrail** del ML. MYCIN/XCON fueron los pioneros, pero la tecnología evolucionó (RETE → PHREAK, estándares DMN).

? ¿Por qué un sistema experto puede dar falsas alarmas? ▾

Por **sensibilidad excesiva**: si un sensor tiene ruido y el umbral de la regla es muy justo, el sistema conmuta sin parar. Se corrige con **histéresis**, suavizado o un controlador difuso.

? ¿Qué es mejor, un PID o un controlador inteligente? ▾

Depende del sistema. El PID es simple y fiable en sistemas lineales; el controlador inteligente (difuso, ANN, MPC) gana en no linealidades, retrasos o ruido. Se empieza simple y se añade inteligencia si hace falta — es lo que comprobarás con las dos versiones que pide [EX4](#).

? ¿`experta` funciona en Python moderno? ▾

La librería (2019) falla en Python 3.10+ por `frozendict`. Con el parche `collections.Mapping = collections.abc.Mapping` funciona igual de bien. Existe también un fork (`om-experta`) que lo resuelve de otra forma, pero está archivado desde 2023: en este módulo usamos siempre el parche, para no tener dos soluciones al mismo problema.

? ¿Cuándo uso reglas puras y cuándo un sistema híbrido? ▾

Si el experto puede formalizar el conocimiento completo, con reglas puras basta y son más explicables. Si el conocimiento es parcial, cambia con el tiempo, o hay muchos datos disponibles, un sistema híbrido (§7) aprovecha lo mejor de las dos fuentes.

? ¿Los sistemas expertos pueden explicar sus decisiones? ▾

Sí, y es su gran ventaja: el subsistema de **explicación** puede responder «¿por qué?» y «¿cómo?» mostrando las reglas usadas. MYCIN fue el primero. Hoy esto se llama **IA explicable (XAI)** y es obligatorio en algunos dominios (derecho a explicación del RGPD).

16. Sesiones

Semana	Horas	Contenido	CE
11	3	DIKW, arquitectura y dinámica; estructuras de representación	RA5-a
12	3	Taller 1 (<code>experta</code>) + guiadas; EX1 ; sistemas híbridos reglas/datos (EX2)	RA5-b
13	3	Lógica difusa: Taller 2 (propinas); variación y dinámica	RA5-b, RA5-c
14	3	Estrategias de control; controladores inteligentes; Taller 3	RA5-d
15	3	EX3 , EX4 ; aplicaciones y tendencias; evaluación	RA5-d, RA5-e

17. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres: [T01](#) · [simular un sistema experto](#) · [T02](#) · [lógica difusa](#) · [T03](#) · [controlador experto](#)
- [Actividades guiadas](#) — 6 notebooks de introducción y práctica
- [Actividades entregables](#) — 5 notebooks evaluables ([EX0](#) - [EX4](#)), con sus rúbricas
- **Notebooks** — todos los de la unidad, con descarga y apertura en Colab, en el menú «Notebooks»



Referencias de la unidad ▾

- [Wikipedia · Expert system](#)
- [Wikipedia · MYCIN](#)
- [Wikipedia · XCON](#)
- [experta · readthedocs](#)
- [CLIPS · clipsrules.net](#)
- [Drools / Apache KIE](#)
- [Human-Learn](#)
- [imodels \(FIGS\)](#)
- [skope-rules](#)
- [Wikipedia · Lógica difusa](#)
- [Wikipedia · Fuzzy control system](#)
- [Wikipedia · PID controller](#)
- [Modus Ponens · Modus Tollens](#)

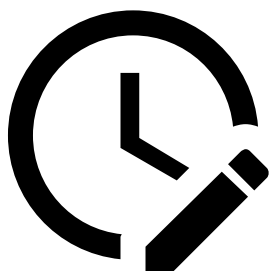
18. Evaluación

Peso	Instrumento
40 % actividades	Media de los cinco entregables (EX0 - EX4); cada uno con su rúbrica sobre 10
60 % prueba escrita	Prueba del RA5 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «*en función de la consecución de los RA*»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- Los entregables EX1 - EX4 cubren un dominio fijo cada uno (medicina, deporte × 2, industria); EX0 es de libre elección y premia la originalidad.

19. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 Orden 8/2025): repetir la construcción de un sistema experto con un problema distinto y las pruebas de autoevaluación de la unidad.



23 de agosto de 2026

5.2 Diapositivas de la UD05

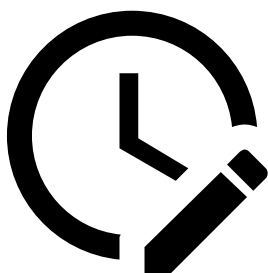


Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.



23 de agosto de 2026

5.3 UD05 · Ejercicios



Cómo se trabajan

Resuélvelos en tu cuaderno o en un documento Markdown. **No se publican las soluciones:** se corrigen y comentan en clase. Los cinco entregables (EX0 - EX4) son la evaluación práctica de la unidad y se describen en las [actividades entregables](#).

A. Del conocimiento a la arquitectura (RA5-a)

1. Sitúa en la jerarquía **DIKW** estos cuatro enunciados: «37 °C», «la temperatura corporal es 37 °C», «si supera 37 °C hay fiebre» y «si hay fiebre, toma paracetamol».
2. Define qué es un **sistema experto** y en qué se diferencia de un programa tradicional.
3. Enumera los **componentes** de un sistema experto y la función de cada uno.
4. Explica el **ciclo reconocer-actuar** (match, resolve, act) con tus palabras.
5. Diferencia **encadenamiento hacia delante** y **hacia atrás**. Pon un ejemplo de problema para cada uno.
6. Enuncia **Modus Ponens** y **Modus Tollens** y relaciona cada uno con un tipo de encadenamiento.
7. ¿Qué son los **factores de certeza** y por qué los introdujo MYCIN?
8. ¿Por qué la **explicación** es una ventaja clave de los sistemas expertos frente al ML?
9. Dado el ejemplo de la teoría (fiebre CF 0,6 + rigidez CF 0,4), explica cómo se acumula la evidencia y qué diferencia hay frente a una regla binaria.

B. Representación del conocimiento (RA5-a/b)

1. Completa la tabla con la representación del conocimiento:

Representación	Estructura base	Inferencia típica
Pares atributo-valor		
Reglas de producción		
Representaciones jerárquicas		
Marcos (frames)		
Lógica formal		
Redes semánticas		
Ontologías		

2. Escribe el mismo hecho («si llueve, coge el paraguas») en tres representaciones distintas de la tabla anterior.
3. ¿Cuándo conviene usar **reglas** y cuándo una **ontología**?
4. ¿Qué es el *bottleneck* de la adquisición de conocimiento y cómo afecta al desarrollo de un sistema experto?
5. Enumera dos sistemas expertos históricos con sus datos (reglas y precisión).

C. Simular comportamientos con `experta` (RA5-b)

1. ¿Por qué `experta` falla en Python 3.10+ y cuál es la solución que usamos en este módulo?
2. Escribe el código de un sistema `experta` que clasifique una incidencia como crítica si `impacto=alto` o `usuarios>50`.
3. Escribe el código de un sistema de **clasificación de animales** (guepardo) como el de la teoría.
4. Explica qué hacen `engine.reset()` y `engine.run()`.
5. Propón un ámbito distinto a los vistos en clase (p. ej. hipoteca, semáforo, cultivo) y describe 3 reglas de su sistema experto.
6. De los siete notebooks de la unidad (§6.1 de la teoría), elige dos de dominios distintos y explica en qué se parece su arquitectura y en qué difiere el conocimiento que codifican.

D. Sistemas híbridos reglas/datos (RA5-b)

1. Diferencia los dos enfoques híbridos de la teoría: deducir reglas de los datos frente a mejorar reglas propias con aprendizaje automático.
2. ¿Qué hace **Human-Learn** que no hace un sistema experto puro?
3. ¿Qué ventaja tiene **FIGS** frente a un árbol de decisión grande, en términos de interpretabilidad?
4. En `UD05_N04_reglas_desde_datos_titanic.ipynb`, ¿quién escribe las reglas: el experto o el algoritmo? Justifica con lo que hace `FIGSClassifier`.
5. Propón un caso donde usarías **skope-rules** en vez de escribir las reglas a mano.

E. Lógica difusa (RA5-b/d)

1. Explica la diferencia entre lógica proposicional y lógica difusa con el ejemplo de «hace frío».
2. Define **variable lingüística**, **valor lingüístico** y **función de pertenencia**, con un ejemplo de cada una.
3. Describe los tres pasos del funcionamiento de un sistema de razonamiento impreciso (fuzzificación, evaluación de reglas, desfuzzificación).
4. Con el ejemplo de las propinas: si el servicio es 9,8 y la comida 6,5, ¿por qué la propina resultante (19,24 €) está en la banda alta y no en la media?
5. ¿Qué tipo de función de pertenencia usarías para representar un ciclo (p. ej. la hora del día) y por qué las triangulares no sirven ahí?

F. Variación de características y dinámica (RA5-c)

1. Diferencia **sensibilidad** y **robustez** de un sistema experto.
2. ¿Qué ocurre si se añaden demasiadas condiciones al antecedente de una regla crítica? ¿Y si se simplifica en exceso?
3. Explica el problema de la **histéresis** en un sistema de ventilación con sensor de CO y la solución.
4. ¿Cómo afecta **subir el umbral de certeza** de un diagnóstico a los falsos positivos y a las alertas tempranas?
5. ¿Qué le ocurre a la dinámica del sistema si los sensores aportan ruido? ¿Cómo lo mitigarías?

G. Estrategias de control (RA5-d)

1. Explica qué es la **salience** y por qué se usa para reglas de emergencia.
2. ¿Qué es el **control de meta** (metarazonamiento)? Relaciónalo con el nivel de «sabiduría» de la jerarquía DIKW.
3. Define las **especificaciones de respuesta** de un sistema de control: precisión, tiempo de asentamiento, sobreimpulso y estabilidad.
4. Resuelve el ejemplo guiado: para controlar una sala a 21 °C con error $\pm 0,5$ °C y sobreimpulso máximo 1 °C, ¿qué controlador elegirías (PID sintonizado, experto o difuso) y por qué?

H. Controladores inteligentes (RA5-e)

1. Describe el **laço de control** (SP, error, controlador, actuador, planta, PV) y la fórmula del PID.
2. ¿Qué limitaciones tiene el PID frente a sistemas no lineales o con retraso?
3. Enumera los **cuatro tipos** de controladores inteligentes y la ventaja de cada uno sobre el PID.
4. Con los datos de la teoría: ¿qué mejora aportó el control difuso frente al PID en el control térmico (asentamiento y sobreimpulso)?
5. Explica cómo un **sistema experto puede actuar como controlador** (directo y supervisor) y qué ventaja aporta la explicación.
6. ¿Qué es el **MPC** (control predictivo) y por qué es proactivo?

I. Aplicaciones y tendencias (contenidos RA5)

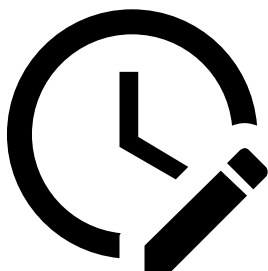
1. Enumera **cuatro sectores** donde se aplican sistemas expertos y un uso en cada uno.
2. ¿Qué es un **BRMS** y qué herramienta es la más conocida (Drools)?
3. ¿Qué son los **sistemas neuro-simbólicos** y qué relación tienen con los sistemas híbridos del bloque D?
4. ¿Qué es la **IA explicable (XAI)** y qué relación tiene con MYCIN y con el RGPD?

5. Explica el uso de **reglas como guardarraíl** del ML con un ejemplo (p. ej. LLM acotado).

J. Caso práctico

1. Elige uno de los cinco entregables (EX0 - EX4) y, antes de programarlo, redacta en una tabla: qué representación de conocimiento usarás, si es puro/híbrido/difuso, y qué especificación de respuesta tendría si actuara como controlador.
-

[Volver a la UD05](#) · [Taller 1](#) · [Taller 2](#) · [Taller 3](#)



23 de agosto de 2026

5.4 Talleres

UD05 · Taller 1 — Simular un sistema experto con `experta`



Entregable de la unidad

Cuenta en el 40 % de actividades del RA5, junto con los [notebooks entregables](#). Trabaja en parejas si lo indica el profesor.



Requisitos

Python 3.10+ con `experta`. Recuerda el **parche de compatibilidad** (`collections.Mapping` desapareció en Python 3.10):

```
1 import collections, collections.abc
2 if not hasattr(collections, 'Mapping'):
3     collections.Mapping = collections.abc.Mapping
4     collections.Iterable = collections.abc.Iterable
5     collections.MutableMapping = collections.abc.MutableMapping
```

Objetivo: construir un sistema experto que **diagnostique** un problema y lo compare con uno de **clasificación**, demostrando que un mismo motor simula comportamientos de ámbitos distintos (CE b).



Hazlo en el notebook

Tienes este taller como **notebook con las celdas a rellenar**: [UD05_T01_simular_sistema_experto.ipynb](#). Esta página es la referencia; lo que se entrega es el notebook completado.

FASE 1 — PARCHE E IMPORTACIÓN

```
1 import collections, collections.abc
2 if not hasattr(collections, 'Mapping'):
3     collections.Mapping = collections.abc.Mapping
4     collections.Iterable = collections.abc.Iterable
5     collections.MutableMapping = collections.abc.MutableMapping
6
7 from experta import *
```

FASE 2 — SISTEMA DE DIAGNÓSTICO DE UN COCHE QUE NO ARRANCA

```
1 class DiagnosticoCocher(KnowledgeEngine):
2     @DefFacts()
3     def inicio(self):
4         yield Fact(accion="diagnosticar")
5
6     @Rule(Fact(accion="diagnosticar"), salience=10)
7     def arrancar(self):
8         print("Diagnóstico del vehículo...")
9         self.declare(Fact(luces="no_encienden"), Fact(sonido="ninguno"))
10        self.declare(Fact(bateria="dudosa"))
11
12    @Rule(Fact(luces="no_encienden"), Fact(sonido="ninguno"))
13    def sin_corriente(self):
14        self.declare(Fact(causa="bateria_descargada"))
15
16    @Rule(Fact(causa="bateria_descargada"))
17    def resultado(self):
18        print("CAUSA PROBABLE: Batería descargada. Recarga o sustituye.")
19
20    engine = DiagnosticoCocher()
21    engine.reset()
22    engine.run()
```

Salida esperada:

```
1 Diagnóstico del vehículo...
2 CAUSA PROBABLE: Batería descargada. Recarga o sustituye.
```

Anota la salida y explica qué reglas se dispararon y en qué orden.

FASE 3 — SISTEMA DE CLASIFICACIÓN (OTRO ÁMBITO)

```
1 class Animales(KnowledgeEngine):
2     @DefFacts()
3     def inicio(self):
4         yield Fact(analizar=True)
5
6     @Rule(Fact(analizar=True), salience=10)
7     def datos(self):
8         self.declare(Fact(pelo=True), Fact(carnivoro=True),
9                     Fact(color="leonado"), Fact(manchas="oscuras"))
10
11    @Rule(Fact(pelo=True))
12    def mamifero(self):
13        self.declare(Fact(mamifero=True))
14
15    @Rule(Fact(mamifero=True), Fact(carnivoro=True), Fact(color="leonado"),
16        Fact(manchas="oscuras"))
17    def guepardo(self):
18        print("IDENTIFICADO: guepardo")
19
20 engine = Animales()
21 engine.reset()
22 engine.run()
```



El fallo más habitual de esta fase

Si llamas a `Animales().run()` sin `reset()` antes, no se dispara ninguna regla: `run()` solo ejecuta el ciclo de inferencia sobre los hechos ya cargados, y es `reset()` quien invoca `@DefFacts()` para cargarlos. Sin ese paso, `engine.facts` queda casi vacío y no hay nada que coincida con las reglas.

FASE 4 — ANALIZA EL ORDEN DE DISPARO

Modifica la regla `mamifero` para que tenga `salience=5` y vuelve a ejecutar. ¿Qué cambia? Explica la **resolución de conflictos**.

FASE 5 — AÑADE UNA REGLA CON NOT Y PREGUNTA AL USUARIO

```
1 class DiagnosticoConPregunta(KnowledgeEngine):
2     @DefFacts()
3     def inicio(self):
4         yield Fact(accion="diagnosticar")
5
6     @Rule(Fact(accion="diagnosticar"), salience=10)
7     def preguntar(self):
8         r = input("¿Encienden las luces? (s/n): ").strip().lower()
9         self.declare(Fact(luces=(r == "s")))
10
11    @Rule(Fact(luces=True))
12    def con_luces(self):
13        print("Hay corriente: revisa arranque y combustible.")
14
15    @Rule(Fact(luces=False))
16    def sin_luces(self):
17        print("Sin corriente: revisa batería y conexiones.")
18
19 motor = DiagnosticoConPregunta()
20 motor.reset() # sin reset() los DefFacts no se cargan y NO se dispara nada
21 motor.run()
```

FASE 6 — ANALIZA LA SENSIBILIDAD

Añade una regla con umbral (`usuarios > 50 → critica`) y prueba con distintos valores (40, 55, 50). ¿Qué ocurre justo en el umbral? ¿Cómo lo harías robusto (histéresis, \$9)?

ENTREGA DEL TALLER 1

Recopila las capturas y las explicaciones de las seis fases en **una memoria en PDF**. Se valora que expliques **qué reglas se dispararon y en qué orden**, no solo la salida final.

Fase	Evidencia mínima
1	El parche aplicado y <code>experta</code> importado sin error
2	El diagnóstico del coche funcionando, con las reglas que se dispararon
3	La clasificación del guepardo, y por qué el mismo motor sirve para otro ámbito

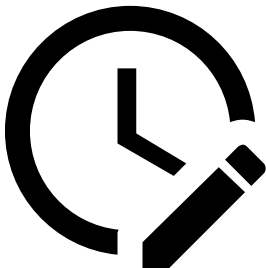
Fase	Evidencia mínima
4	El cambio de <code>saliency</code> y su efecto en el orden de disparo
5	La regla con <code>NOT</code> y la versión que pregunta al usuario
6	El umbral probado con tres valores, y tu propuesta de histéresis



Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD05](#) · [Taller 2](#) · [Taller 3](#) · [Ejercicios](#)



23 de agosto de 2026

UD05 · Taller 2 — Lógica difusa: el problema de las propinas

**Entregable de la unidad**

Cuenta en el 40 % de actividades del RA5, junto con los [notebooks entregables](#). Trabaja en parejas si lo indica el profesor.

**Requisitos**

```
1 pip install scikit-fuzzy networkx
```

**scikit-fuzzy : dos versiones y una trampa**

La versión **0.4.2** ya **no arranca** en Python 3.12 o posterior: importa `imp` y `distutils`, eliminados del lenguaje. Instala la **0.5.0** (es lo que hace `pip install scikit-fuzzy`).

En la 0.5.0 falla solo `ControlSystem.view()`, el dibujo del grafo del sistema, por un error de la librería. Los dibujos de cada variable (`servicio.view()`, `propina.view(sim=sim)`) no están afectados, y son los que se usan aquí. Si necesitas el grafo, párchalo:

```
1 from skfuzzy.control.visualization import ControlSystemVisualizer
2
3 _init_original = ControlSystemVisualizer.__init__
4
5 def _init_parcheado(self, control_system):
6     _init_original(self, control_system)
7     self.ctrl = control_system
8
9 ControlSystemVisualizer.__init__ = _init_parcheado
```

Objetivo: implementar de principio a fin el ejemplo difuso de la teoría (§8.4) y comprobar en código las tres fases de un sistema de razonamiento impreciso: fuzzificación, evaluación de reglas y desfuzzificación (CE b).

**Hazlo en el notebook**

Tienes este taller como **notebook con las celdas a rellenar**: [UD05_T02_logica_difusa.ipynb](#). Esta página es la referencia; lo que se entrega es el notebook completado.

FASE 1 — INSTALACIÓN Y VARIABLES DE ENTRADA

```
1 pip install scikit-fuzzy networkx
```

```
1 import numpy as np
2 import skfuzzy as fuzz
3 from skfuzzy import control as ctrl
4
5 servicio = ctrl.Antecedent(np.arange(0, 11, 1), 'servicio')
6 comida = ctrl.Antecedent(np.arange(0, 11, 1), 'comida')
7 propina = ctrl.Consequent(np.arange(0, 26, 1), 'propina')
8
9 servicio['baja'] = fuzz.trimf(servicio.universe, [0, 0, 5])
10 servicio['media'] = fuzz.trimf(servicio.universe, [0, 5, 10])
11 servicio['alta'] = fuzz.trimf(servicio.universe, [5, 10, 10])
12
13 comida['mala'] = fuzz.trimf(comida.universe, [0, 0, 5])
14 comida['media'] = fuzz.trimf(comida.universe, [0, 5, 10])
15 comida['buena'] = fuzz.trimf(comida.universe, [5, 10, 10])
```

FASE 2 — VARIABLE DE SALIDA Y REGLAS

```

1 propina['baja'] = fuzz.trimf(propina.universe, [0, 0, 13])
2 propina['media'] = fuzz.trimf(propina.universe, [0, 13, 25])
3 propina['alta'] = fuzz.trimf(propina.universe, [13, 25, 25])
4
5 regla1 = ctrl.Rule(servicio['baja'] | comida['mala'], propina['baja'])
6 regla2 = ctrl.Rule(servicio['media'], propina['media'])
7 regla3 = ctrl.Rule(servicio['alta'] | comida['buena'], propina['alta'])
8
9 sistema = ctrl.ControlSystem([regla1, regla2, regla3])
10 sim = ctrl.ControlSystemSimulation(sistema)

```

FASE 3 — INFERENCIA CON EL CASO DE LA TEORÍA

```

1 sim.input['servicio'] = 9.8
2 sim.input['comida'] = 6.5
3 sim.compute()
4 print(f"Propina calculada: {sim.output['propina']:.2f} €")

```



Tu número puede no ser exactamente el de la teoría

Con esta discretización del universo (`np.arange(0, 26, 1)`), el resultado ronda **19-20 €**, cerca de los 19,24 € de la teoría pero no idéntico: la **resolución de los universos** de entrada y salida afecta al resultado de la desfuzzificación por centro de gravedad. Es el mismo fenómeno de **sensibilidad** del §9: un parámetro que parece un detalle técnico (cuántos puntos tiene el universo discreto) cambia la respuesta del sistema.

FASE 4 — VISUALIZA LAS FUNCIONES DE PERTENENCIA

```

1 servicio.view()
2 comida.view()
3 propina.view()
4 sim.compute()
5 propina.view(sim=sim)

```

Guarda las cuatro figuras: son la evidencia de las fases de fuzzificación (dos primeras) y desfuzzificación (última, con la línea vertical del resultado).

FASE 5 — CAMBIA EL ESCENARIO

Repite la Fase 3 con un servicio de 3 (malo) y una comida de 8 (buena). ¿Qué regla domina? ¿Es coherente el resultado con lo que esperarías de un cliente que recibió un trato mediocre pero comió muy bien?

FASE 6 — COMPARA CON UN SISTEMA DE REGLAS «DURO»

Escribe en `experta` una versión sin lógica difusa del mismo problema (reglas con umbrales fijos, p. ej. `servicio >= 8 → propina alta`). Ejecuta los mismos dos escenarios de las Fases 3 y 5 y compara los resultados. ¿En qué casos da lo mismo la versión difusa y la de reglas duras? ¿En cuáles no?

ENTREGA DEL TALLER 2

Recopila las figuras y las explicaciones de las seis fases en **una memoria en PDF**.

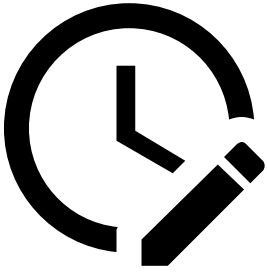
Fase	Evidencia mínima
1	Las dos variables de entrada con sus funciones de pertenencia
2	La variable de salida y las tres reglas
3	La propina calculada para el caso de la teoría, y por qué no sale exactamente la misma cifra
4	Las figuras de las funciones de pertenencia y del resultado
5	Un escenario propio, con su resultado interpretado
6	La comparación con un sistema de reglas «duro»: qué cambia en la respuesta



Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD05](#) · [Taller 1](#) · [Taller 3](#) · [Ejercicios](#)



23 de agosto de 2026

UD05 · Taller 3 — Sistema experto como controlador de un proceso

**Entregable de la unidad**

Cuenta en el 40 % de actividades del RA5, junto con los [notebooks entregables](#). Trabaja en parejas si lo indica el profesor.

**Requisitos**

Python 3.10+ con `experta`. Recuerda el **parche de compatibilidad** (`collections.Mapping` desapareció en Python 3.10):

```
1 import collections, collections.abc
2 if not hasattr(collections, 'Mapping'):
3     collections.Mapping = collections.abc.Mapping
4     collections.Iterable = collections.abc.Iterable
5     collections.MutableMapping = collections.abc.MutableMapping
```

Objetivo: implementar un **controlador experto** que regule la temperatura de una sala, definiendo las especificaciones de respuesta (CE d) y observando cómo influye en el comportamiento del sistema (CE e).

**Hazlo en el notebook**

Tienes este taller como **notebook con las celdas a rellenar**: [UD05_T03_controlador_experto.ipynb](#). Esta página es la referencia; lo que se entrega es el notebook completado.

FASE 1 — LA PLANTA (MODELO SIMPLE)

```
1 def simular_planta(temp, potencia, dt=1.6, inercia=0.05, exterior=15.0):
2     """Modelo simple de una sala con inercia térmica."""
3     return temp + inercia * (exterior - temp) + potencia * dt
```

FASE 2 — EL CONTROLADOR EXPERTO

```

1  from experta import P # constraint de predicado: envuelve el lambda
2
3  class ControladorClima(KnowledgeEngine):
4      def __init__(self, setpoint=21.0):
5          super().__init__()
6          self.setpoint = setpoint
7          self.potencia = 0.0
8
9          @DefFacts()
10         def inicio(self):
11             yield Fact(controlar=True)
12
13         @Rule(Fact(controlar=True), salience=10)
14         def leer(self):
15             # simula la lectura del sensor
16             error = self.setpoint - self._temp
17             self.declare(Fact(error=round(error, 2)))
18
19         @Rule(Fact(error=P(lambda e: e > 3)))
20         def alta(self):
21             self.potencia = 1.0
22
23         @Rule(Fact(error=P(lambda e: 0 < e <= 3)))
24         def media(self):
25             self.potencia = 0.5
26
27         @Rule(Fact(error=P(lambda e: -1 <= e <= 0)))
28         def baja(self):
29             self.potencia = 0.2
30
31         @Rule(Fact(error=P(lambda e: e < -1)))
32         def apagar(self):
33             self.potencia = 0.0
34
35         def paso(self, temp):
36             self._temp = temp
37             self.reset()
38             self.run()
39             return self.potencia
40
41     ctrl = ControladorClima(setpoint=21.0)
42     temp = 15.0
43     for i in range(80):
44         potencia = ctrl.paso(temp)
45         temp = simular_planta(temp, potencia)
46         if i % 5 == 0:
47             print(f"t={i:3d}: temp={temp:.2f} potencia={potencia:.2f}")

```

⚠ Por qué el condicional va envuelto en P(...)

`experta` compara el valor de un campo de un `Fact` por **igualdad**, no lo evalúa: si escribes `Fact(error=lambda e: e > 3)`, la regla busca un hecho cuyo `error` sea literalmente esa función, y nunca lo encuentra. `P(...)` es el **constraint de predicado** de `experta`: le dice al motor «aplica esta función al valor del campo y compara el resultado con verdadero». Sin `P()`, el controlador de esta fase se queda **congelado con potencia 0** para siempre y no parece un error evidente, porque no lanza ninguna excepción.

Con estos parámetros, la temperatura entra en la banda 20,5-21,5 °C hacia el paso 5 y se estabiliza en torno a **21,4 °C**, dentro de la especificación.

FASE 3 — MIDE LAS ESPECIFICACIONES DE RESPUESTA

Añade código para medir:

- **Error en régimen permanente** (temperatura final – setpoint).
- **Tiempo de asentamiento** (en qué paso entra en la banda 20,5-21,5 °C, y no vuelve a salir).
- **Sobreimpulso** (¿supera alguna vez 21,5 °C?).

FASE 4 — AÑADE UNA PERTURBACIÓN

En el paso 40, cambia la temperatura exterior a 5 °C (ventana abierta) y observa cómo responde el controlador. ¿Mantiene la temperatura en la banda?

FASE 5 — ANALIZA LA SENSIBILIDAD

Cambia el umbral de la regla `media` (de 3 a 1,5) y compara la respuesta. ¿Qué pasa con el sobreimpulso? ¿Y con el tiempo de asentamiento? Registra los valores en una tabla.

FASE 6 — COMPARA CON UN PID (CONCEPTUAL)

Dibuja (o describe) cómo respondería un PID sintonizado frente al controlador experto y explica qué ventaja tiene cada uno. ¿Cuándo usarías cada uno?

ENTREGA DEL TALLER 3

Recopila la simulación y las explicaciones de las seis fases en **una memoria en PDF**.

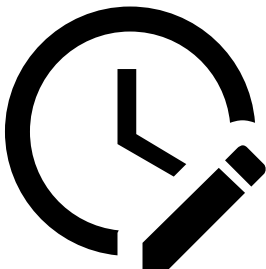
Fase	Evidencia mínima
1	La planta simulada y qué representa cada parámetro
2	El controlador experto funcionando, con las reglas que se disparan en cada tramo
3	La tabla de especificaciones : error en régimen permanente, tiempo de asentamiento y sobreimpulso
4	La respuesta ante la perturbación, y si se mantiene en la banda
5	La misma tabla antes y después de cambiar el umbral
6	La comparación razonada con un PID: cuándo usarías cada uno

Y la respuesta a: *¿qué relación tiene un sistema experto con los sistemas híbridos reglas/datos y la lógica difusa? ¿Cuándo usarías cada uno?*

**Soluciones**

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD05](#) · [Taller 1](#) · [Taller 2](#) · [Ejercicios](#)



23 de agosto de 2026

5.5 UD05 · Actividades guiadas

**Se trabajan en clase**

Seis notebooks de introducción y práctica guiada, en dominios distintos (§6.1 de la teoría). No son evaluables por sí solos, pero preparan los cinco entregables de la [página siguiente](#).

Notebook	Dominio	Descargar	Ejecutar
0 · Sistema experto con Python y Experta	Introducción a <code>experta</code>	 Descargar <code>.ipynb</code>	 Open in Colab
0 · Piedra, papel o tijera	Juego decidido por reglas	 Descargar <code>.ipynb</code>	 Open in Colab
1 · Sistema experto: clasificación de animales	Zoología	 Descargar <code>.ipynb</code>	 Open in Colab
2 · Sistema basado en reglas: Titanic	Reglas extraídas de datos (§7)	 Descargar <code>.ipynb</code>	 Open in Colab
3 · Sistema de control: regar el césped	Control difuso (§8, §10)	 Descargar <code>.ipynb</code>	 Open in Colab
4 · Lógica difusa: quién jugará	El clásico «jugar al tenis» (§8)	 Descargar <code>.ipynb</code>	 Open in Colab

**Si Google Colab da un error de versión**

Los notebooks 3 y 4 necesitan una versión concreta del entorno de ejecución de Colab ([Editar] → [Configuración del cuaderno] → **2025.7** en vez de «Última (recomendada)»), o puedes usar el `docker-compose.yml` de la unidad para ejecutarlos en local.

Referencia: un sistema experto grande, resuelto

[EX0 · Tasación de vehículos usados](#) ([descargar](#) · [Colab](#)) es un ejemplo **extenso ya resuelto**, portado de una práctica original en CLIPS: no es un enunciado que se te pida entregar, es una referencia para ver cómo se estructura un sistema experto grande de principio a fin, con más reglas y más profundidad que los ejemplos guiados de arriba.

[Volver a la UD05](#) · [Ejercicios](#) · [Taller 1](#) · [Taller 2](#) · [Taller 3](#) · [Actividades entregables](#)



5.6 UD05 · Actividades entregables



Cinco entregables, promediados, 40 % de la nota de RA5

Los cinco cuentan para la nota (decisión del profesor, 2026-08-22): se promedian sobre 10. Cuatro tienen dominio fijo (EX1 - EX4); EX0 es de libre elección.

Notebook	Dominio	Descargar	Ejecutar
EX0 · Sistema experto de libre elección	El que tú elijas	 Descargar .ipynb	 Open in Colab
EX1 · Sistema experto: detectar lesiones de rodilla	Medicina	 Descargar .ipynb	 Open in Colab
EX2 · Sistema basado en reglas: previsión del valor de mercado	Deporte · sistema híbrido (\$7)	 Descargar .ipynb	 Open in Colab
CSV de jugadores del FIFA 22 (para EX2 y EX3)	—	 CSV players_22.csv	—
EX3 · Lógica difusa: centrocampistas con potencial	Deporte · lógica difusa (\$8)	 Descargar .ipynb	 Open in Colab
EX4 · Sistema de control: simulador de quemador de gas	Industria · control real (\$10-11)	 Descargar .ipynb	 Open in Colab



EX2 y EX3 comparten el mismo fichero de datos

Descarga EX2.-players_22.csv una vez y súbelo a la carpeta de archivos de Colab (o colócalo junto al notebook si trabajas en local) antes de ejecutar cualquiera de los dos.

Rúbricas

Las cinco rúbricas son las mismas que ya se usaban en Moodle. Cada una puntúa sobre el máximo de su tabla, y esa suma se escala a la nota final de la tarea sobre 10.

EX0 · Sistema experto de libre elección

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Variabilidad de las reglas	No entregado	No ha usado reglas (2) · fuera de plazo (2,5) · todas iguales (3) · dos tipos (4)	Tres o más tipos de reglas (5)
Memoria	No entregada	Insuficiente (2) · fuera de plazo (2,5) · suficiente (3) · bien (4)	Muy bien (5)
Hechos iniciales (@DefFacts)	No entregado	Ninguno (2) · fuera de plazo (2,5) · uno (3) · dos (4)	Tres o más (5)
Reglas (@Rule)	No entregado	Ninguna (2) · fuera de plazo (2,5) · una (3) · dos (4)	Tres o más (5)
Originalidad	No entregado	Nada original (2) · fuera de plazo (2,5) · poco original (3) · original (4)	Muy original (5)
Funciones extra (preguntas, respuestas, matemáticas...)	No entregado	Ninguna (2) · fuera de plazo (2,5) · una (3) · dos (4)	Tres o más (5)

EX1 · Detectar lesiones de rodilla

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Regla de diagnóstico	No entregada	No la ha definido, pero muestra algo (1)	Correctamente definida (2)
Método para añadir hechos	No entregado o con <code>declare</code>	—	Definido correctamente (1)
Pruebas	No entregado	Fallan algunas celdas (1)	Todas las celdas de prueba funcionan (2)
Reglas (<code>@Rule</code>)	No entregado	Sin reglas (1) · casi todas (8)	Todas las reglas correctamente definidas (10)

EX2 · Previsión del valor de mercado

Criterio	Mínimo (0)	Niveles intermedios	Máximo
<code>FunctionClassifier</code> de Human-Learn (<code>score</code>)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
<code>FIGSClassifier</code> (<code>fit</code> , predicción, <code>score</code>)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
<code>FIGS</code> sin campos evidentes	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)

EX3 · Centrocampistas con potencial

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Antecedentes y consecuentes	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Funciones de pertenencia (×3)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Reglas + <code>ControlSystem</code> + <code>view</code>	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Pruebas del potencial (×4)	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Columna de potencial en el <code>DataFrame</code>	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)
Conclusiones	No entregado	Muy mal (1) · mal (2) · fuera de plazo (2,5) · neutral (3) · bien (4)	Muy bien (5)

EX4 · Simulador de quemador de gas



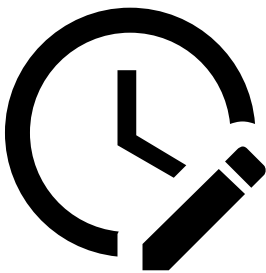
Pide dos enfoques, no uno

Esta rúbrica evalúa **dos soluciones distintas** para el mismo quemador: un controlador simple y un enfoque alternativo. No es un error del enunciado entregar dos veces «lo mismo» con matices: es lo que se pide.

Criterio	Mínimo (0)	Niveles intermedios	Máximo
Controlador simple	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)
Antecedentes, consecuente y conjuntos difusos	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)
4 reglas, controlador y <code>view</code>	No entregado		Muy bien (5)

Criterio	Mínimo (0)	Niveles intermedios Muy mal (1) · mal (2) · neutral (3) · bien (4)	Máximo
Enfoque alternativo: antecedentes, consecuente, conjuntos difusos	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)
3 reglas, controlador y <code>view</code> (del enfoque alternativo)	No entregado	Muy mal (1) · mal (2) · neutral (3) · bien (4)	Muy bien (5)

[Volver a la UD05](#) · [Ejercicios](#) · [Taller 1](#) · [Taller 2](#) · [Taller 3](#) · [Actividades guiadas](#)



23 de agosto de 2026

6. UD03

6.1 UD03 — Procesamiento del Lenguaje Natural



Unidad 3 · 12 h · semanas 16-19

Es el RA con **más criterios de evaluación** de todo el módulo: **siete**. Se evalúa con **cuatro entregables prácticos** y la prueba escrita del RA3.

1. Introducción

De todas las formas de inteligencia, el **lenguaje** es la que mejor nos permite comunicar ideas, pedir ayuda, decidir y persuadir. Conseguir que las máquinas entiendan y generen lenguaje humano es el objetivo del **procesamiento del lenguaje natural (PLN)**: el campo de la IA que está detrás de los buscadores, los traductores, los asistentes de voz, los correctores y los chatbots.

Esta unidad tiene una particularidad que conviene saber desde el principio: de sus **siete criterios**, tres no se demuestran escribiendo código. Hablan del **papel del lingüista**, del **trabajo cooperativo** entre perfiles y de la **formación** que necesita quien investiga en PLN. No son relleno: son la mitad del RA, y en esta unidad se evalúan con una parte escrita dentro de los entregables.

El recorrido:

1. **Qué es el PLN** y qué tareas comprende, del análisis de una palabra a la generación de texto.
2. **Qué se consigue hoy** — el potencial, con cifras.
3. **La ambigüedad**: por qué es *el* problema del lenguaje, en sus seis formas.
4. **Desambiguación y etiquetado morfológico (POS)**: cómo se ataca ese problema.
5. **Las demás limitaciones**: falta de comprensión, sesgos, lenguas con pocos recursos, ironía.
6. **Cuándo es factible** aplicar PLN a un problema, con la lupa del AI Act.
7. **Quién participa**: el lingüista, la cooperación con informáticos y la formación necesaria.
8. **Las herramientas** en Python: `nltk`, `spaCy`, `scikit-learn` y `transformers`.
9. **Cómo construir un sistema** de PLN orientado a una tarea concreta.



Hilo conductor de la unidad

El lenguaje es la interfaz más natural entre personas y máquinas, y también la más ambigua. Primero vemos qué es el PLN y hasta dónde llega; luego por qué falla —la ambigüedad— y cómo se ataca; después quién hace falta en un proyecto y cuándo merece la pena; y por último construimos un sistema que resuelva una tarea concreta.



De dónde sale el material de esta unidad

Los bloques de **ambigüedad**, **desambiguación y etiquetado morfológico**, con sus ejemplos en español, y el itinerario de formación provienen del material del profesor, adaptados aquí. Las prácticas usan `nltk`, `spaCy`, `PyTorch` y modelos de **Hugging Face**. La referencia teórica de fondo es *Speech and Language Processing* de **Jurafsky y Martin**, de acceso libre.

2. Resultado de aprendizaje y criterios de evaluación

RA3 — Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.

CE	Criterio de evaluación	Bloque
RA3-a	Se ha caracterizado el procesamiento de lenguaje natural.	\$4, \$7
RA3-b	Se ha justificado el papel del lingüista en un proyecto de inteligencia artificial.	\$9

CE	Criterio de evaluación	Bloque
RA3-c	Se ha determinado el potencial de las técnicas existentes de procesamiento de lenguaje, así como sus limitaciones.	\$5-8
RA3-d	Se ha considerado en qué casos es factible aplicar estas técnicas en la resolución de un problema.	\$9
RA3-e	Se ha evaluado el trabajo cooperativo entre lingüistas e informáticos en el campo del procesamiento del lenguaje natural.	\$10
RA3-f	Se ha descrito la formación teórica que precisa el investigador en procesamiento del lenguaje natural.	\$10
RA3-g	Se ha elaborado un sistema de procesamiento de lenguaje orientado a una tarea específica.	\$11-12



Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, «*Procesamiento del Lenguaje Natural*», y le asigna estos contenidos:

- *Procesamiento del lenguaje natural: Potencial y limitaciones.*
- *Aplicaciones del procesamiento del lenguaje natural.*

Son **dos contenidos para siete criterios de evaluación**: el desajuste **más grande de todo el módulo**. Ni el papel del lingüista, ni el trabajo cooperativo, ni la formación del investigador —tres de los siete CE— tienen contenido propio en el anexo. Los desarrolla el centro (art. 3.3 del Decreto 95/2026), y es lo que hace esta unidad.

3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de...
O1	Explicar qué es el PLN y situarlo entre la lingüística y la informática.
O2	Describir las tareas del PLN y el <i>pipeline</i> típico de un sistema.
O3	Valorar el potencial de las técnicas actuales con ejemplos y cifras.
O4	Distinguir los seis tipos de ambigüedad y reconocerlos en frases reales.
O5	Explicar cómo el etiquetado morfológico (POS) desambigua, y qué cuesta elegir el conjunto de etiquetas.
O6	Identificar las demás limitaciones: falta de comprensión, sesgos, lenguas con pocos recursos, ironía.
O7	Determinar cuándo es factible aplicar PLN, incluidas las restricciones del AI Act.
O8	Justificar el papel del lingüista, evaluar la cooperación con informáticos y describir la formación necesaria.
O9	Usar <code>nltk</code> y <code>spaCy</code> para tareas básicas de PLN en español.
O10	Construir y evaluar un sistema de PLN orientado a una tarea específica.

4. Qué es el PLN (RA3-a)

4.1 Definición

El **procesamiento del lenguaje natural** es el subcampo de la informática y de la IA que busca que las máquinas **comprendan y se comuniquen con el lenguaje humano**. Combina tres disciplinas:

Disciplina	Qué aporta
Lingüística computacional	Los modelos del lenguaje: morfología, sintaxis, semántica, pragmática
Estadística y aprendizaje automático	Aprender los patrones de los textos, en vez de programar todas las reglas a mano
Aprendizaje profundo	Redes que aprenden representaciones del lenguaje (<i>transformers</i>)



No es magia, y la distinción importa

Un buscador o un traductor no «entienden» como una persona: **procesan estadísticamente** el texto. Esa diferencia no es filosófica, es operativa: explica por qué fallan donde fallan, y es el hilo de todo el §6.

4.2 Los niveles del lenguaje

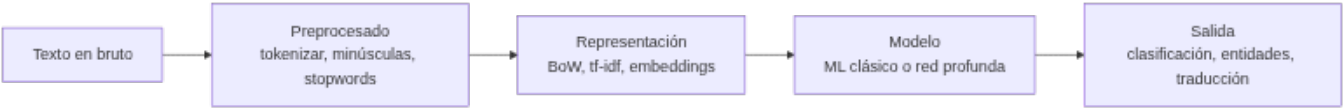
El lenguaje se analiza por capas, y cada una tiene sus tareas:

Nivel	Qué analiza	Tarea de PLN
Fonética y fonología	Los sonidos	Reconocimiento de voz (ASR), síntesis (TTS)
Morfología	La estructura de las palabras	Stemming, lematización, etiquetado POS
Sintaxis	La estructura de las oraciones	Análisis sintáctico, dependencias
Semántica	El significado de palabras y frases	Desambiguación (WSD), entidades (NER)
Pragmática y discurso	El significado según el contexto	Correferencia, actos de habla, sentimiento

Esta tabla vale también como mapa de la ambigüedad: **hay ambigüedad en todos los niveles**, y el §6 recorre uno a uno.

4.3 Las tareas del PLN

Tarea	Qué hace	Ejemplo
Tokenización	Divide el texto en unidades	«Hola mundo» → [«Hola», «mundo»]
Etiquetado POS	Asigna categoría gramatical	«correr» → verbo
Entidades (NER)	Identifica nombres, lugares, fechas	«María vive en Madrid» → María: PER, Madrid: LOC
Análisis de sentimiento	Positivo, negativo o neutro	«¡Genial!» → positivo
Traducción automática	Traduce entre idiomas	ES ↔ EN
Resumen	Condensa documentos	Resumir un informe
Generación	Produce texto nuevo	Chatbots, redacción asistida
Respuesta a preguntas	Responde a partir de un texto	Asistentes, buscadores



4.4 El pipeline, paso a paso

1. **Preprocesado**: tokenizar, pasar a minúsculas, quitar palabras vacías (*stopwords*), lematizar.
2. **Representación**: convertir el texto en números — bolsa de palabras, tf-idf, *embeddings*.
3. **Modelo**: aplicar un modelo clásico o una red neuronal.
4. **Salida**: la tarea concreta.



Esto lo usas todos los días

El corrector del móvil usa morfología y un modelo de lenguaje. El buscador clasifica tu consulta y le extrae entidades. El asistente de voz primero **transcribe** (ASR), luego interpreta y responde. Los tres son *pipelines* de PLN, con los mismos cuatro pasos.

5. El potencial (RA3-c)

5.1 Qué se consigue hoy

Aplicación	Estado actual
Buscadores	Entienden la consulta, extraen entidades y responden directamente
Asistentes de voz	Transcriben, interpretan y ejecutan órdenes
Traducción automática	Documentos y conversación en tiempo real, con calidad funcional
Análisis de opinión	Valorar miles de reseñas o publicaciones automáticamente
Extracción de información	Leer facturas, contratos e historiales y sacar los campos (OCR + NER)
Chatbots	Resolver consultas frecuentes sin horario
Modelos generativos	Redactar, resumir, traducir y generar contenido



Tres cifras para calibrar

- En **SQuAD 2.0** (respuesta a preguntas), los mejores sistemas **superan la precisión humana**: EM ~91 frente a ~87.
- El análisis de sentimiento en reseñas de cine (IMDb) pasó de ~89 % en 2011 a **95-97 %** con *transformers*.
- El ecosistema **Hugging Face** supera los **2 millones de modelos** y 500.000 *datasets* públicos. Es el «almacén» del que salen los modelos de esta unidad.

5.2 Los grandes modelos de lenguaje

El **AI Act** (Reglamento UE 2024/1689) define los *modelos de IA de uso general*: modelos con al menos **mil millones de parámetros**, entrenados con autosupervisión a gran escala, que realizan con competencia una amplia variedad de tareas. Los **LLM** son el ejemplo típico.



Modelos de gran impacto

El AI Act presume que un modelo de uso general tiene **capacidades de gran impacto** cuando el cómputo acumulado de su entrenamiento supera los **10²⁵ FLOPS** (art. 51.2), y entonces le exige documentación y evaluación adicionales. Se trata a fondo en la UD06.

6. La ambigüedad, el problema de fondo (RA3-c)

Si hay que quedarse con una sola limitación del PLN, es esta: **el lenguaje es ambiguo en todos sus niveles**. Múltiples interpretaciones de una misma palabra pueden arruinar la capacidad de un sistema para hacer su trabajo. Y no es un caso raro de laboratorio: aparece en frases que decimos sin darnos cuenta.

6.1 Los seis tipos de ambigüedad

Tipo	Cuándo aparece
Sintáctica	La oración admite más de un análisis sintáctico : más de una regla gramatical la representa
Léxica y morfológica	La léxica, cuando un término aislado admite varias interpretaciones; la morfológica, cuando una palabra puede tener más de un rol o categoría dentro de la frase
Semántica	Un elemento de la frase se puede interpretar de varios modos
Pragmática	El sentido depende del contexto y de quién habla, en ese momento
Fonológica	Una cadena de sonidos resulta confusa
Funcional	Un término se usa con doble función gramatical



Los seis, con frases reales

- **Sintáctica** — «*Los perros y los gatos enfermos son recogidos por el servicio municipal.*» ¿Se recogen todos los perros y solo los gatos enfermos, o solo los enfermos de las dos especies? Y «*Compro los libros baratos*»: ¿compro **los baratos** (adjetivo) o compro libros, **que además son baratos** (complemento predicativo)?
- **Léxica y morfológica** — «*Usted aquí no pinta nada.*» ¿No tiene mando o no pinta paredes? Y «*Pedro y yo escribimos un cuento*»: ¿ya lo escribimos o lo estamos escribiendo? La forma conjugada vale para presente y para pretérito.
- **Semántica** — «*Pedro quiere pelearse con un italiano.*» ¿Con cualquier italiano o con uno concreto?
- **Pragmática** — «*Golpeó el armario con el bastón y lo rompió.*» ¿Se rompió el bastón o el armario?
- **Fonológica** — «*es-conde*»: ¿el verbo *esconder* conjugado, o *es + conde*?
- **Funcional** — «*He vuelto a oler.*» ¿He recuperado el olfato, o he regresado a un sitio para oler algo?

6.2 Un banco de frases para probar cualquier sistema

Estas frases son un buen test rápido: si un sistema las resuelve, es que usa el contexto de verdad.

- Vino de la Rioja.
- Compré unos zapatos de piel de señora.
- La policía observó al sospechoso con unos prismáticos.
- El pescado está listo para comer.
- El cura recibió una cura en su habitación.
- Juan vio a Pedro enfurecido.
- Antonio no nada nada.
- No puedo ir a la fiesta porque no traje traje.
- Estaré de vacaciones solo unos días.
- El Villarreal le ganó al Valencia en su campo.
- Me quedé esperándote en el banco.



Fíjate en el patrón

Casi todas se resuelven **con el contexto**, no con la frase. «*Vino de la Rioja*» es un sustantivo o un verbo según lo que venga antes; «*en su campo*» depende de quién sea «su». Esa es exactamente la información que un modelo estadístico intenta capturar — y la razón de que el tamaño del contexto sea tan determinante en los *transformers*.

6.3 Por qué la ambigüedad no se «arregla»

La ambigüedad no es un error del lenguaje que se pueda corregir: es una **propiedad** suya, y además útil —permite ser breve, irónico o cortés—. Lo que hace un sistema de PLN es **elegir la interpretación más probable** dado el contexto, y por eso siempre puede equivocarse.



Forma frente a significado

Emily Bender y Alexander Koller argumentaron que un modelo entrenado **solo con la forma** del texto no tiene manera *a priori* de aprender el **significado**: aprende correlaciones estadísticas, no comprensión. De ahí que un modelo generativo pueda «sonar» impecable y estar equivocado. Es la limitación que hay que tener presente al leer las cifras del \$5.

7. Desambiguación y etiquetado morfológico (RA3-a, RA3-c)

7.1 Qué es el POS *tagging*

El **etiquetado morfológico** —*POS tagging*, etiquetado léxico, desambiguación morfosintáctica— es el proceso de asignar a **cada palabra de un texto su categoría gramatical**, sin entrar en las relaciones sintácticas.

Y aquí está la clave: **una palabra aislada suele ser ambigua respecto a su categoría; dentro de un contexto, casi siempre se puede desambiguar.**



La misma palabra, tres categorías

- «*Mételo en ese **sobre***» → **nombre**
- «*Déjalo **sobre** la mesa*» → **preposición**
- «*Dame lo que te **sobre***» → **verbo**

Tres frases, la misma forma, tres categorías. Ningún diccionario resuelve esto: hace falta el contexto. Etiquetar bien la categoría es el **primer paso** de casi todo lo demás — de ahí que aparezca tan pronto en el *pipeline* del §4.4.

7.2 Elegir el conjunto de etiquetas: un compromiso

El **conjunto de etiquetas** (*tagset*) que se elige cambia la dificultad del problema, y hay que equilibrar dos cosas que tiran en direcciones opuestas:

- **Más información:** etiquetas específicas — distinguir tiempo, persona, número, género.
- **Menos trabajo de desambiguación:** etiquetas generales — solo «verbo», «nombre», «adjetivo».

Cuantas más etiquetas, más informativo el resultado y **más difícil acertar**.

Conjunto de etiquetas	Número de etiquetas
Penn Treebank	45
Brown Corpus	87
LexEsp (PAROLE)	~250
Susanne	350

Las **categorías principales** son las de siempre: nombres (comunes y propios), pronombres, determinantes, adjetivos, verbos, adverbios, preposiciones y conjunciones. Se dividen en clases **abiertas** —admiten palabras nuevas: nombres, verbos, adjetivos— y **cerradas** —no: preposiciones, determinantes, conjunciones—.



Por qué esto es asunto del lingüista

Decidir el *tagset*, escribir la guía de anotación y resolver los casos dudosos **no es una tarea de programación**: es lingüística aplicada, y condiciona todo lo que el modelo podrá aprender después. Es el ejemplo más concreto del **RA3-b**, y por eso este bloque va justo antes del §9.

7.3 Cómo se etiqueta en la práctica

Enfoque	Cómo funciona	Dónde encaja
Basado en reglas	Reglas escritas a mano sobre el contexto	Dominios muy acotados; didáctico
Modelos de Markov (HMM, TnT)	Probabilidad de una secuencia de etiquetas dado el texto	El clásico; es lo que se practica con el corpus <code>cess_esp</code> de <code>nlTK</code>
Modelos neuronales	La categoría sale del modelo del lenguaje, con el contexto completo	<code>spaCy</code> y <i>transformers</i> : lo que se usa hoy

En la práctica de la unidad se hacen los tres: reglas y HMM con `nlTK`, y el enfoque neuronal con `spaCy`.

8. Las demás limitaciones (RA3-c)

8.1 Falta de comprensión real

Los modelos **generan respuestas plausibles sin comprender**: no verifican hechos, no razonan sobre el mundo y pueden **alucinar** — inventar datos con total seguridad. En cualquier tarea con consecuencias, la salida **se verifica**.

8.2 Sesgos

Los modelos aprenden de los textos que hay, y esos textos llevan **sesgos históricos y sociales**:

- Sesgo de **género** en los *embeddings*: «médico → hombre», «enfermera → mujer».
- Sesgos **raciales, étnicos y culturales** en los datos de entrenamiento.
- El AI Act advierte de que los sesgos pueden **perpetuarse y amplificarse** en bucles de retroalimentación.

**Inferir emociones está prohibido en dos contextos**

El AI Act **prohíbe** los sistemas que infieren el estado emocional **en el trabajo y en centros educativos**: la base científica es débil —la expresión de las emociones varía entre culturas— y su uso ahí es discriminatorio. Es un límite **legal**, no solo técnico, y afecta directamente a lo que se puede construir con análisis de sentimiento.

8.3 Lenguas con pocos recursos

Casi todos los modelos se entrenan con textos mayoritariamente en inglés. En lenguas con menos datos, las técnicas **estadísticas clásicas** pueden aún superar a las neuronales, que solo dan buenos resultados a partir de decenas de miles de *tokens*. No es un detalle menor: decide si un proyecto en una lengua minoritaria es viable.

8.4 Ironía y sarcasmo

«¡Qué buena idea, se me ha caído el móvil al agua!» es literalmente positivo y pragmáticamente negativo. Es ambigüedad **pragmática** pura, y los modelos fallan a menudo.

9. Cuándo es factible aplicar PLN (RA3-d)

9.1 Cinco criterios de decisión

Criterio	Pregunta guía
Tarea clara y acotada	¿Qué salida exacta quiero: clasificar, extraer, traducir, resumir?
Datos disponibles	¿Hay textos suficientes y representativos? ¿Están anotados?
Dominio con vocabulario conocido	¿El texto es de un ámbito concreto — médico, legal, atención al cliente?
Calidad esperada realista	¿Acepto un 5-10 % de error? ¿Cuánto cuesta cada error?
Regulación aplicable	¿Está el uso prohibido o restringido por el AI Act?

9.2 Factible y no factible

Factible hoy (bajo riesgo)	No factible o prohibido
Convertir datos no estructurados en estructurados	Inferir emociones en el trabajo o la educación
Clasificar documentos y detectar duplicados	Extracción no selectiva de rostros de internet
Mejorar el registro o el lenguaje de un documento	Manipulación subliminal del comportamiento
Traducción funcional de documentos	Comprensión profunda con inferencia abierta
Buscar y vincular datos en archivos	Respuesta fiable en dominios de alto riesgo sin verificación
Análisis de sentimiento en un dominio acotado	Detección de sarcasmo con alta precisión

**El AI Act como herramienta de diseño**

El **considerando 53** enumera tareas de PLN de **bajo riesgo** —procesamiento delimitado: transformar datos, clasificar, detectar duplicados, mejorar el lenguaje, traducción funcional— y el **artículo 5** prohíbe prácticas concretas. Leídos juntos, son una lista de comprobación excelente para decidir qué construir **antes** de empezar. Eso es el CE d.

10. Quién hace un proyecto de PLN (RA3-b, RA3-e, RA3-f)

Aquí están **tres de los siete criterios** de este RA, y son los que un material puramente técnico suele resolver en un párrafo. No se demuestran con código, pero deciden si un proyecto de PLN sale bien o sale caro.

10.1 Qué aporta el lingüista (RA3-b)

Aportación	En qué consiste
Anotación de datos	Etiquetar entidades, sentimiento o categorías en los textos de entrenamiento
Diseño del conjunto de etiquetas y de la guía	Definir el <i>tagset</i> del §7.2 y las reglas para aplicarlo de forma coherente
Recursos lingüísticos	Construir corpus y árboles sintácticos (<i>treebanks</i>), un trabajo que puede llevar años
Gramáticas y reglas	Las reglas morfológicas y sintácticas de la rama simbólica del PLN
Evaluación y análisis de errores	Explicar por qué falla un modelo: ambigüedad, registro, dominio



Las anotaciones deciden las predicciones

La calidad del etiquetado condiciona directamente lo que el modelo puede aprender: con etiquetas malas o **inconsistentes entre anotadores**, el modelo aprende esa inconsistencia. Y esto no se arregla con más datos ni con un modelo mejor.

Es la razón de fondo del CE b: **el lingüista no es un apoyo del proyecto, es parte de la tubería de datos**. Volviendo al §7.2: quien decide si el *tagset* tiene 45 etiquetas o 250 está decidiendo el techo de precisión del sistema.

10.2 El trabajo cooperativo (RA3-e)

El PLN es **interdisciplinar por definición**: lingüistas, informáticos y, según el proyecto, psicólogos o expertos en lógica. Dos casos reales que muestran cómo se hace bien:

- **Meta NLLB** — traducción entre 200 idiomas. Trabajaron con **hablantes nativos** para anotar y evaluar, y consiguieron **+44 % de BLEU medio** además de controlar la toxicidad de las traducciones. Sin hablantes nativos no había forma de evaluar la mayoría de esos idiomas.
- **Masakhane** — PLN para lenguas africanas. Más de **1.000 participantes de 30 países**, y una regla explícita: los lingüistas locales **no son «solo anotadores», son investigadores**. Rechazan la investigación *paracaidista*: entrar, tomar los datos y marcharse sin dejar capacidad local.

Lo que se gana cooperando	Lo que cuesta
Datos y evaluación de mucha más calidad	Vocabularios y métodos distintos: hay que traducirse
Cobertura de lenguas minoritarias	La anotación es lenta y cara
Mejor ciencia y sistemas más justos	Riesgo de usar a las comunidades como proveedores de datos



El riesgo tiene nombre

La cooperación puede degenerar en extracción: una empresa del norte global obtiene datos anotados baratos y publica; la comunidad que los produjo no queda con capacidad ni con crédito. Es el mismo problema que en la UD06 aparece como **el trabajo invisible detrás de la IA**. Por eso el CE e dice «**evaluar**» el trabajo cooperativo, no solo describirlo: hay formas mejores y peores de hacerlo.

10.3 La formación del investigador (RA3-f)

El perfil combina tres patas, y **ninguna sobra**:

Pata	Contenidos
Lingüística	Morfología, sintaxis, semántica, pragmática y discurso
Estadística y aprendizaje automático	Probabilidad, modelos, métricas y evaluación
Informática y métodos formales	Algoritmos, estructuras de datos, programación

El itinerario recomendado, en este orden:

1. **Jurafsky y Martin**, *Speech and Language Processing* — la referencia clásica, de acceso libre. Da la base teórica y el vocabulario.
2. **El manual de NLTK** — el mismo camino, pero con las manos: cada paso del *pipeline*, a mano.
3. **spaCy**, en inglés y en español — el salto a herramientas de producción.
4. **Modelos neuronales y transformers** — Hugging Face, y de ahí a plataformas de inferencia si hace falta desplegar.



Perfiles que no existían hace cinco años

Junto al lingüista computacional clásico han aparecido el **anotador o evaluador especializado** —con formación en derecho, medicina o ciencia, porque hay que entender el texto para etiquetarlo bien—, el perfil de **prompt engineer**, en evolución hacia LLMOps, y el **evaluador de sesgos y ética** de modelos. Los tres parten de la misma base: **lingüística más técnica**.



Cómo se evalúan estos tres criterios en esta unidad

No con un examen aparte: con la **parte escrita de los entregables EX2 y EX5**. En cada uno se pide justificar qué decisiones tomaría un lingüista en ese problema concreto, qué aporta cada perfil y qué formación haría falta. Está en la rúbrica, y cuenta.

11. Las herramientas en Python (RA3-c, RA3-g)



Dos entornos, y conviene saber cuál toca

Esta unidad usa las librerías más pesadas del módulo. La regla:

- **En el contenedor de la unidad** (`Dockerfile` y `docker-compose.yml` de la carpeta): `nlTK`, `spaCy`, `scikit-learn`, `gensim`, `textblob`. Todo lo que no entrena redes grandes.
- **En Colab**: todo lo que use `transformers`, entrenamiento de modelos o **audio**. Ahí hay GPU, y afinar un DistilBERT en CPU es cuestión de horas.

Cada notebook lo indica en su primera celda.

11.1 NLTK, para entender

NLTK es una plataforma de enseñanza e investigación con más de 50 corpus y recursos léxicos, como WordNet. Su virtud es que es **transparente**: cada paso del procesamiento se hace a mano, y por eso es la herramienta con la que se **aprende**.

```
1 pip install nltk
2 python -c "import nltk; nltk.download(['punkt_tab', 'stopwords', 'wordnet', 'cess_esp'])"
```

```
1 import nltk
2 from nltk.tokenize import word_tokenize
3 from nltk.corpus import stopwords
4 from nltk.stem.snowball import SnowballStemmer
5
6 texto = "La inteligencia artificial está transformando la atención al cliente."
7 tokens = word_tokenize(texto, language='spanish')
8 sin_vacias = [t for t in tokens if t.lower() not in stopwords.words('spanish')]
9 raiz = SnowballStemmer('spanish').stem
10
11 print("Tokens:", tokens)
12 print("Sin stopwords:", sin_vacias)
13 print("Stemming:", [raiz(t) for t in sin_vacias])
```

**El pipeline de NLTK, paso a paso**

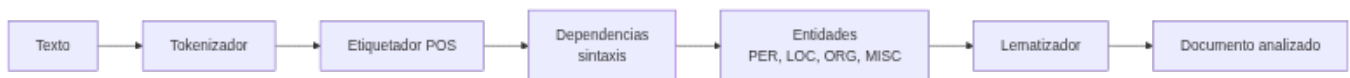
1. **Tokenizar:** `word_tokenize` separa palabras y signos.
2. **Limpiar:** se descartan artículos, preposiciones y palabras sin contenido.
3. **Normalizar:** el *stemming* recorta a una raíz aproximada («transformando» → «transform»); la **lematización** da el lema real («transformando» → «transformar»). El primero es rápido y tosco; la segunda necesita un diccionario.
4. **Enriquecer:** etiquetar POS (`pos_tag`), detectar entidades (`ne_chunk`), contar frecuencias (`FreqDist`) o buscar concordancias (`concordance`).

11.2 spaCy, para producir

```
1 pip install spacy
2 python -m spacy download es_core_news_sm
```

```
1 import spacy
2
3 nlp = spacy.load("es_core_news_sm")
4 doc = nlp("Maria trabaja en Madrid para una empresa de inteligencia artificial.")
5
6 for token in doc:
7     print(f"{token.text:<12} {token.pos_:<8} {token.lemma_:<12}")
8
9 print("\nEntidades:")
10 for ent in doc.ents:
11     print(f" {ent.text:<12} → {ent.label_}")
```

El pipeline de spaCy ejecuta toda la cadena **de una pasada**:



En una frase corriente, el analizador sintáctico puede encontrar **miles de análisis posibles**, y spaCy elige el más probable con su modelo estadístico. Es el \$6 en acción: la ambigüedad no se elimina, **se decide**.

**NLTK o spaCy: no compiten**

NLTK para **aprender** — cada paso a mano, todo visible. **spaCy** para **producir** — un objeto `nlp`, modelos preentrenados, mucho más rápida y con 25 idiomas. En los talleres se empieza con NLTK y se pasa a spaCy, en ese orden y por ese motivo.

**Por qué las entidades dan dinero**

La extracción de entidades es la base de leer facturas, contratos o historiales: el sistema encuentra fechas, importes, nombres y organizaciones **sin que nadie los marque a mano**. Es una de las aplicaciones más rentables del PLN, y de las menos vistas.

11.3 Análisis de sentimiento

Con **scikit-learn**, que funciona en español y sin GPU:

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.pipeline import make_pipeline
4 from nltk.corpus import stopwords
5
6 reseñas = ["me encantó la comida", "pésimo servicio", "todo perfecto", "no volveré"]
7 etiquetas = [1, 0, 1, 0]
8
9 pipe = make_pipeline(
10     TfidfVectorizer(stop_words=stopwords.words('spanish')),
11     LogisticRegression()
12 )
13 pipe.fit(reseñas, etiquetas)
14 print(pipe.predict(["la comida estaba buenisima"]))
```

**El detalle que rompe el ejemplo en español**

`TfidfVectorizer(stop_words=...)` **solo acepta la cadena 'english'**. Para español hay que pasar una **lista** — `stopwords.words('spanish')` — o no pasar nada y usar `max_df`. Si se escribe `stop_words='spanish'`, el error es poco claro.

11.4 Transformers y Hugging Face

Los modelos **Transformer** se usan con la librería `transformers` y sus *pipelines*. Funcionan en CPU, pero **entrenarlos o afinarlos, no**: eso va a Colab.

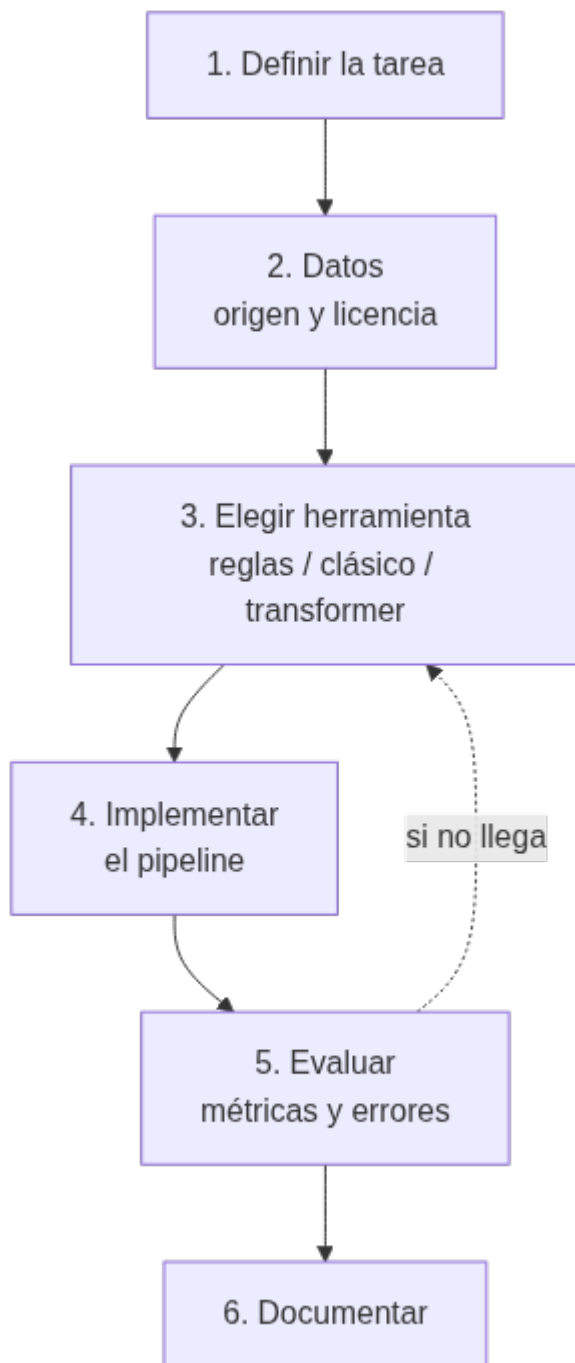
```
1 from transformers import pipeline
2
3 clasificador = pipeline("sentiment-analysis",
4                         model="pytorch/bert-base-sentiment-analysis")
5 print(clasificador("El servicio fue excelente, volveré seguro."))
```

**DistilBERT, y por qué se usa aquí**

DistilBERT tiene un **40 % menos de parámetros** y es un **60 % más rápido** que BERT, conservando el **97 %** de su capacidad. Por eso es el modelo de los notebooks de la unidad: cabe en una sesión de clase. El entregable **EX4** lo **afina** para reseñas de cine, que es *transfer learning* de manual.

12. Construir un sistema orientado a una tarea (RA3-g)**12.1 La metodología, en seis pasos**

1. **Definir la tarea**: qué entra, qué sale y para quién.
2. **Conseguir o anotar los datos**: origen, formato, tamaño y **licencia**.
3. **Elegir la herramienta**: reglas, modelo clásico o *transformer*, según datos y recursos.
4. **Implementar** el *pipeline*: preprocesado → representación → modelo → salida.
5. **Evaluar** sobre datos **no vistos**, y **analizar los errores** — no solo mirar la métrica.
6. **Documentar**: origen de los datos, licencia, decisiones y limitaciones.



El paso 5 es el que más se salta

Medir la exactitud es fácil; **mirar los errores** es lo que enseña. Casi siempre se agrupan: un registro que no estaba en el entrenamiento, un tipo de ambigüedad concreta, un vocabulario de dominio. Y eso apunta a qué hacer después — que es volver al paso 3, no al paso 4.

12.2 La progresión de los notebooks

Los notebooks de la unidad recorren esa metodología de menos a más, y conviene verlos como una escalera, no como piezas sueltas:

Notebook	Qué construye	Representación
N01 · introducción al PLN	Tokenizar, <i>stopwords</i> , BoW y tf-idf con <code>nlTK</code>	Cuenta de palabras
N02 · clasificación con PyTorch	Un clasificador de texto entrenado desde cero	De BoW a <i>word embeddings</i>
N03 · modelos de lenguaje	Afinar DistilBERT para sentimiento	<i>Embeddings</i> contextuales
N04 · spaCy	El <i>pipeline</i> completo: POS, dependencias, entidades	Modelo preentrenado
N05 · <code>nlTK</code> y Python	POS <i>tagging</i> con el corpus <code>cess_esp</code> y validación cruzada	Etiquetado estadístico

Y los cuatro entregables aplican cada nivel a un problema propio:

Entregable	Tarea	Qué demuestra
EX1	Representación de texto: tokenizar, <i>stopwords</i> , BoW y tf-idf	Los fundamentos, por dos caminos (NLTK y TextBlob)
EX2	Clasificar preguntas repitiendo el proceso de N02	Construir un clasificador propio
EX3	Etiquetado morfosintáctico del corpus <code>cess_esp</code> con NLTK	Trabajar con anotación real en español
EX4	Afinar DistilBERT para reseñas de cine	<i>Transfer learning</i> a una tarea nueva
EX5	Asistente virtual por voz	Un sistema de punta a punta, con audio



Ejemplo guiado: un clasificador de reseñas en seis pasos

1 · Tarea: clasificar una reseña de restaurante como positiva o negativa. **2 · Datos:** 60 reseñas anotadas a mano, 40 para entrenar y 20 para probar; licencia propia. **3 · Herramienta:** scikit-learn con tf-idf, porque con 60 ejemplos un *transformer* no aporta nada. **4 · Implementar:** `TfidfVectorizer` → `LogisticRegression`. **5 · Evaluar:** exactitud en las 20 de prueba y **leer los dos errores** — casi siempre son ironía o una negación. **6 · Documentar:** origen, licencia y el límite obvio (60 ejemplos de un solo dominio).

Fíjate en el paso 3: **la herramienta se elige por los datos que hay**, no por lo moderna que sea. Con 60 ejemplos, tf-idf gana.

13. Puntos clave de la unidad

- El **PLN** busca que las máquinas comprendan y se comuniquen con el lenguaje humano, combinando lingüística computacional, estadística y aprendizaje profundo.
- Sus tareas van de la **tokenización** al etiquetado POS, entidades, sentimiento, traducción, resumen y generación; el *pipeline* típico es **preprocesado** → **representación** → **modelo** → **salida**.
- El **potencial** es alto y medible: en respuesta a preguntas los sistemas **superan la precisión humana** (SQuAD 2.0), y el sentimiento en reseñas ronda el **95-97 %**.
- La **ambigüedad** es el problema de fondo, y tiene **seis formas**: sintáctica, léxica y morfológica, semántica, pragmática, fonológica y funcional. No se elimina: **se decide** con el contexto.
- El **etiquetado POS** es la herramienta central de desambiguación. Elegir el *tagset* es un compromiso: **más etiquetas = más información y más difícil acertar** (45 en Penn Treebank, 350 en Susanne).
- Las otras limitaciones: **forma ≠ significado** (Bender y Koller), alucinaciones, **sesgos**, lenguas con pocos recursos, ironía y sarcasmo.
- El **AI Act** sirve de herramienta de diseño: su considerando 53 lista PLN de bajo riesgo y su artículo 5 **prohíbe** prácticas concretas, como **inferir emociones en el trabajo o la educación**.
- El **lingüista** no es un apoyo: diseña el *tagset*, la guía de anotación y los recursos. **Las anotaciones deciden las predicciones**, y eso no se arregla con más datos.
- La **cooperación** entre perfiles se **evalúa**, no solo se describe: NLLB con hablantes nativos (+44 % BLEU) y Masakhane con 1.000 participantes son ejemplos de hacerlo bien; la investigación *paracaidista*, de hacerlo mal.
- La **formación** combina lingüística, estadística e informática, con un itinerario claro: Jurafsky → NLTK → spaCy → *transformers*.

- **NLTK para aprender, spaCy para producir;** scikit-learn cuando hay pocos datos y *transformers* cuando hay muchos y GPU. **La herramienta se elige por los datos que hay.**
- Construir un sistema son **seis pasos**, y el que más se salta es el quinto: **mirar los errores**, no solo la métrica.

14. Glosario

Término	Definición
PLN	Procesamiento del lenguaje natural
Token	Unidad mínima en que se divide un texto: palabra, signo, subpalabra
Tokenización	Dividir el texto en <i>tokens</i>
Stopwords	Palabras vacías (artículos, preposiciones) que se descartan
Stemming	Recortar la palabra a una raíz aproximada
Lematización	Reducir la palabra a su lema real, con diccionario
POS tagging	Asignar a cada palabra su categoría gramatical
Tagset	El conjunto de etiquetas gramaticales que se usa
Penn Treebank	<i>Tagset</i> de referencia en inglés: 45 etiquetas
PAROLE / LexEsp	<i>Tagset</i> de referencia en español: ~250 etiquetas
Categoría abierta / cerrada	Admite palabras nuevas (nombres, verbos) o no (preposiciones)
Ambigüedad sintáctica	La oración admite más de un análisis sintáctico
Ambigüedad léxica	Un término aislado admite varias interpretaciones
Ambigüedad morfológica	Una palabra puede tener más de una categoría en la frase
Ambigüedad semántica	Un elemento se puede interpretar de varios modos
Ambigüedad pragmática	El sentido depende del contexto y del hablante
Ambigüedad fonológica	Una cadena de sonidos resulta confusa
Ambigüedad funcional	Un término con doble función gramatical
Desambiguación	Elegir la interpretación correcta según el contexto
WSD	Desambiguación del sentido de las palabras
NER	Reconocimiento de entidades nombradas
Análisis de dependencias	Determinar las relaciones sintácticas entre palabras
Correferencia	Saber a qué se refiere un pronombre
Bolsa de palabras (BoW)	Representar un texto por la cuenta de sus palabras, sin orden
tf-idf	Pesar cada palabra por su frecuencia relativa en el documento y en el corpus
Word embedding	Representar palabras como vectores densos con significado
Embedding contextual	El vector de una palabra cambia según su contexto (BERT)
Transformer	Arquitectura basada en atención, base de los modelos actuales
BERT / DistilBERT	Modelo de lenguaje bidireccional, y su versión reducida
Transfer learning	Adaptar un modelo ya entrenado a una tarea nueva
Fine-tuning	Afinar los pesos de un modelo preentrenado con datos propios
LLM	Modelo grande de lenguaje
Modelo de uso general	En el AI Act, modelo con $\geq$ mil millones de parámetros y autosupervisión a gran escala

Término	Definición
Alucinación	Que el modelo genere información falsa con apariencia de certeza
NLTK	Biblioteca de PLN orientada a la enseñanza y la investigación
spaCy	Biblioteca de PLN orientada a producción
WordNet	Base de datos léxica de relaciones entre significados
cess_esp	Corpus del español anotado, incluido en NLTK
HMM / TnT	Modelos de Markov para etiquetado morfológico
Hugging Face	Repositorio de modelos y <i>datasets</i> de PLN
SQuAD	Conjunto de referencia para respuesta a preguntas
BLEU	Métrica de calidad de traducción automática
Corpus	Colección de textos usada para entrenar o evaluar
Treebank	Corpus anotado con árboles sintácticos
Guía de anotación	Documento que fija cómo etiquetar de forma coherente
Lengua con pocos recursos	Idioma con pocos datos disponibles para entrenar

15. FAQ

? ¿El PLN «entiende» de verdad el lenguaje? ▼

No. Procesa estadísticamente: aprende qué palabras y estructuras aparecen juntas. Bender y Koller lo argumentaron con precisión: un modelo entrenado **solo con la forma** del texto no tiene manera *a priori* de aprender el **significado**. Por eso puede escribir un párrafo impecable y equivocarse en el dato.

? Si la ambigüedad no se puede eliminar, ¿cómo funciona nada? ▼

Porque el sistema **no necesita acertar siempre**: necesita acertar lo suficiente para la tarea. Elige la interpretación más probable según el contexto, y en la mayoría de los casos es la buena. El problema aparece cuando el coste de equivocarse es alto — y ahí entra el \$9.

? ¿Cuántas etiquetas POS conviene usar? ▼

Las mínimas que resuelvan tu problema. Con 45 (Penn Treebank) se acierta más; con 250 (PAROLE) se sabe mucho más de cada palabra pero se falla más. Si solo necesitas distinguir nombres de verbos, usar 250 etiquetas es tirar precisión a la basura.

? ¿Para qué necesito un lingüista si tengo un modelo preentrenado? ▼

Para tres cosas que el modelo no hace: **decidir qué etiquetar y cómo** (el *tagset* y la guía), **anotar de forma coherente** los datos de tu dominio, y **explicar por qué falla** cuando falla. Un modelo preentrenado te ahorra entrenar, no te ahorra entender tu problema.

? ¿NLTK o spaCy? ▼

Las dos, en este orden. **NLTK** para aprender, porque cada paso es visible y se hace a mano. **spaCy** para trabajar, porque un solo objeto ejecuta todo el *pipeline* con modelos preentrenados y es mucho más rápida.

? ¿Puedo hacer análisis de sentimiento para detectar el ánimo de mis empleados? ▾

No. El AI Act **prohíbe** los sistemas que inferen el estado emocional en el **trabajo** y en **centros educativos**: la base científica es débil y el uso es discriminatorio. Es un límite legal, y es exactamente el tipo de decisión que pide el CE d.

? ¿Hace falta GPU para esta unidad? ▾

Para la mitad, no: `nlTK`, `spaCy`, `scikit-learn` y las representaciones de texto van en el contenedor. Para **afinar DistilBERT** y para el notebook de audio, sí — y por eso esos van en **Colab**, que la da gratis.

? ¿Por qué mi modelo va peor en español que en inglés? ▾

Porque casi todos se entrenan con textos mayoritariamente en inglés. En español hay recursos buenos, pero menos; en lenguas minoritarias, muy pocos. En esos casos, las técnicas estadísticas clásicas pueden **superar** a las neuronales, que necesitan decenas de miles de *tokens* para despegar.

? ¿Qué diferencia hay entre *stemming* y lematización? ▾

El *stemming* recorta a una raíz aproximada sin diccionario: rápido y tosco («transformando» → «transform», que no es una palabra). La lematización devuelve el **lema real** usando un diccionario: más lento y correcto («transformando» → «transformar»). Para buscar, suele bastar el primero; para analizar, hace falta el segundo.

16. Sesiones

Semana	Horas	Contenido	CE
16	3	Qué es el PLN, tareas y <i>pipeline</i> ; el potencial con cifras. <code>N01 (nlTK)</code> y <code>EX1</code>	RA3-a, RA3-c
17	3	La ambigüedad en sus seis formas ; desambiguación y POS <i>tagging</i> . <code>N04 (spaCy)</code> y <code>N05 (cess_esp)</code> ; Taller 1	RA3-a, RA3-c
18	3	Las demás limitaciones; cuándo es factible con la lupa del AI Act. <code>N02</code> y <code>EX2</code> ; Taller 2	RA3-c, RA3-d, RA3-g
19	3	El lingüista, la cooperación y la formación; sistemas orientados a tarea. <code>N03</code> , <code>EX4</code> y <code>EX5</code> ; evaluación	RA3-b, RA3-e, RA3-f, RA3-g

Sobre el reparto

Los tres CE «humanos» se tratan en la **semana 19**, cuando el alumnado ya ha peleado con los datos y con la anotación: así la discusión sobre el papel del lingüista deja de ser abstracta. Y el material de **ampliación** (el clasificador de géneros musicales) **no cuenta horas**.

17. Recursos

- [Diapositivas](#)
- [Ejercicios de la unidad](#)
- Talleres: [T01 · del texto al vector](#) · [T02 · un sistema de PLN de punta a punta](#)
- [Actividades guiadas](#) — 5 notebooks
- [Actividades entregables](#) — 5 notebooks evaluables, con sus rúbricas
- **Notebooks** — todos los de la unidad, con descarga y apertura en Colab, en el menú «Notebooks»



Referencias de la unidad ▾

- [Speech and Language Processing](#) — Jurafsky y Martin (acceso libre)
- [Introduction to Natural Language Processing](#) — Eisenstein
- [NLTK Book](#) · [spaCy](#) · [Usage](#) · [Hugging Face](#)
- [Bender y Koller \(2020\), Climbing towards NLU](#)
- [Meta NLLB](#) · [Masakhane](#)
- [Reglamento \(UE\) 2024/1689 · AI Act](#) — considerando 53 y art. 5
- [SQuAD](#) · [Penn Treebank POS tags](#)

18. Evaluación

Peso	Instrumento
40 % actividades	Media de los cinco entregables (EX1 - EX5); cada uno con su rúbrica sobre 10
60 % prueba escrita	Prueba del RA3 en Moodle: preguntas de test y de desarrollo sobre el contenido de la unidad

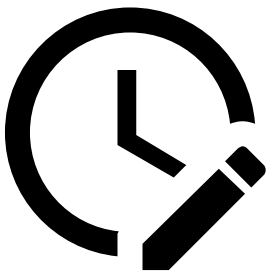
- **La normativa exige alcanzar todos los RA** del módulo para superarlo (art. 5.1 de la Orden 8/2025: la calificación del módulo está «en función de la consecución de los RA»; y las Instrucciones 26-27, que impiden calificar positivamente un módulo con RA no superados). El centro concreta ese mandato exigiendo **≥ 5 en cada RA**.
- **EX2 y EX5 llevan una parte escrita** que cubre los criterios b, e y f: el papel del lingüista, la cooperación entre perfiles y la formación necesaria. Está en su rúbrica y **cuenta**.

CE	Dónde se trabaja	Con qué se evalúa
RA3-a	\$4, \$7	Taller 1, EX1 , prueba del RA3
RA3-b	\$10.1	Parte escrita de EX2 y EX5 , prueba del RA3
RA3-c	\$5-8	EX1 , EX4 , Talleres 1 y 2, prueba del RA3
RA3-d	\$9	Taller 2, prueba del RA3
RA3-e	\$10.2	Parte escrita de EX2 y EX5 , prueba del RA3
RA3-f	\$10.3	Parte escrita de EX2 y EX5 , prueba del RA3
RA3-g	\$11-12	EX2 , EX4 , EX5 , Taller 2

19. Recuperación

Actividades del programa de recuperación individual por RA (art. 14.4 de la Orden 8/2025): construir un sistema de PLN orientado a una tarea distinta —otro dominio, otro tipo de salida— y las pruebas de autoevaluación de la unidad.

.....



24 de agosto de 2026

6.2 Diapositivas de la UD03

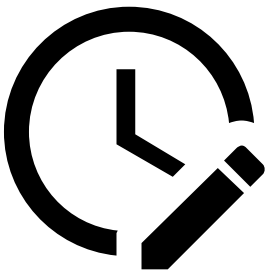


Diapositivas PDF



Cómo se manejan

Flechas o clic para avanzar · **F** para verlas a pantalla completa · **P** para las notas del ponente.



23 de agosto de 2026

6.3 UD03 · Ejercicios



Cómo se trabajan

Resuélvelos en tu cuaderno o en un documento Markdown. No se entregan por separado: son la preparación de la **prueba escrita del RA3** y de los talleres. Los entregables evaluables son los [cuatro notebooks](#).

A. Qué es el PLN (RA3-a)

1. Define **PLN** y di qué tres disciplinas combina, con la aportación de cada una.
2. Explica por qué un traductor automático «no entiende» aunque acierte, y por qué esa distinción no es filosófica sino operativa.
3. Relaciona cada nivel del lenguaje (fonología, morfología, sintaxis, semántica, pragmática) con una tarea de PLN.
4. Describe el *pipeline* típico en cuatro pasos y pon un ejemplo de cada paso con la frase «*El servicio fue excelente*».
5. Diferencia **tokenización**, **etiquetado POS** y **reconocimiento de entidades** con la frase «*María trabaja en Madrid desde 2019*».
6. Explica qué hace un asistente de voz desde que oye tu orden hasta que responde, nombrando las tareas de PLN implicadas.
7. Diferencia **stemming** y **lematización**, con un ejemplo en español, y di cuándo prefieres cada uno.

B. El potencial (RA3-c)

1. Los sistemas superan la precisión humana en **SQuAD 2.0**. ¿Significa que «entienden» mejor que una persona? Justifica.
2. El sentimiento en reseñas pasó de ~89 % a **95-97 %**. ¿Qué cambió técnicamente para conseguirlo?
3. Según el **AI Act**, ¿qué es un *modelo de IA de uso general*? Da los dos requisitos.
4. ¿Qué umbral de cómputo hace presumir **capacidades de gran impacto**, y qué implica?
5. Busca tres aplicaciones de PLN que uses cada semana sin darte cuenta y di qué tarea resuelve cada una.

C. La ambigüedad (RA3-c)

1. Enumera los **seis tipos de ambigüedad** y da un ejemplo propio de cada uno, distinto de los de la teoría.
2. Clasifica el tipo de ambigüedad de cada frase:

Frase	Tipo
Vino de la Rioja	
Compré unos zapatos de piel de señora	
La policía observó al sospechoso con unos prismáticos	
El pescado está listo para comer	
Antonio no nada nada	
Me quedé esperándote en el banco	

3. «*El Villarreal le ganó al Valencia en su campo*.» ¿Qué es exactamente lo ambiguo, y qué información resolvería la duda?
4. Explica por qué la ambigüedad **no se puede eliminar** y qué hace en su lugar un sistema de PLN.
5. ¿Qué es una **ventaja** de la ambigüedad para las personas? Da un caso donde nos resulte útil.
6. Resume el argumento de **Bender y Koller** sobre forma y significado, y qué predice sobre los errores de un modelo generativo.
7. Un modelo responde a una pregunta con un dato falso, redactado con total seguridad. ¿Es un fallo de ambigüedad, de comprensión o de datos? Justifica.

D. Desambiguación y POS (RA3-a, RA3-c)

1. Define **POS tagging** y explica por qué una palabra aislada suele ser ambigua respecto a su categoría.

- Etiqueta la categoría de **sobre** en las tres frases del ejemplo de la teoría, y explica qué información usas en cada caso.
- Explica el compromiso al elegir el **tagset**. ¿Qué se gana y qué se pierde al pasar de 45 a 250 etiquetas?
- Ordena por número de etiquetas: Penn Treebank, Brown, LexEsp (PAROLE), Susanne. ¿Cuál usarías para un buscador simple y cuál para un análisis morfológico del español?
- Diferencia categorías **abiertas** y **cerradas**, con dos ejemplos de cada una. ¿Por qué la distinción importa al etiquetar?
- Compara los tres enfoques de etiquetado (reglas, HMM/TnT, neuronal) en precisión, datos necesarios y transparencia.
- ¿Por qué decidir el *tagset* y escribir la guía de anotación **no es una tarea de programación**?

E. Las otras limitaciones (RA3-c)

- Explica el sesgo de género en los *embeddings* con un ejemplo, y por qué **no basta con quitar la variable** para eliminarlo.
- ¿Qué advierte el AI Act sobre los bucles de retroalimentación y los sesgos?
- El AI Act **prohíbe** inferir emociones en dos contextos. ¿Cuáles, y con qué dos argumentos?
- ¿Por qué en una lengua con pocos recursos una técnica estadística clásica puede **ganar** a una neuronal?
- «¡Qué buena idea, se me ha caído el móvil al agua!» Explica por qué falla un clasificador de sentimiento y de qué tipo de ambigüedad se trata.

F. Cuándo es factible (RA3-d)

- Enumera los **cinco criterios** de factibilidad y formula la pregunta guía de cada uno.
- Para cada caso, di si es factible hoy, no factible o **prohibido**, justificando:

Caso	Veredicto
Clasificar automáticamente los correos de soporte por tema	
Detectar el estado de ánimo de los alumnos por sus mensajes	
Extraer importes y fechas de 5.000 facturas	
Resumir informes médicos para el diagnóstico, sin revisión	
Traducir la documentación interna de una empresa	
Detectar ironía en reseñas con un 95 % de acierto	

- Tienes 200 correos anotados de un dominio muy concreto. ¿Elegirías tf-idf o afinar un *transformer*? Justifica con los criterios del §9.1.
- Usa el **considerando 53** y el **artículo 5** del AI Act para decidir si construirías un sistema que clasifica currículos por idoneidad.
- Un cliente quiere un chatbot que responda consultas legales «sin supervisión». ¿Qué le responderías, y con qué criterio?

G. El papel del lingüista (RA3-b)

- Enumera **cinco aportaciones** del lingüista a un proyecto de PLN y pon un ejemplo concreto de cada una.
- Explica la frase «**las anotaciones deciden las predicciones**» y por qué el problema **no se arregla con más datos**.
- Dos anotadores etiquetan el mismo corpus y coinciden en el 70 % de los casos. ¿Qué problema hay y qué harías antes de entrenar nada?
- ¿Por qué construir un *treebank* puede llevar años? ¿Qué se obtiene a cambio?
- Un modelo preentrenado falla en los textos de tu empresa. Describe qué haría un lingüista para averiguar por qué.
- Relaciona el §7.2 (elegir el *tagset*) con el papel del lingüista: ¿por qué esa decisión fija el techo de precisión del sistema?

H. El trabajo cooperativo (RA3-e)

- Explica qué hizo **Meta NLLB** con los hablantes nativos y qué consiguió, con la cifra.
- ¿Qué es la investigación **paracaidista** y por qué **Masakhane** la rechaza explícitamente?
- Completa la tabla con **dos ventajas** y **dos costes** del trabajo cooperativo entre lingüistas e informáticos.
- El CE dice «**evaluar**» el trabajo cooperativo, no describirlo. Propón **tres criterios** para juzgar si una colaboración se ha hecho bien o mal.

- Una empresa contrata anotadores en un país con salarios bajos, publica el modelo y no deja formación ni datos en el país. ¿Es cooperación? Argumenta las dos posturas.
- Además de lingüistas e informáticos, ¿qué otros perfiles pueden hacer falta? Pon un proyecto donde cada uno sea imprescindible.

I. La formación del investigador (RA3-f)

- Describe las **tres patas** de la formación en PLN y qué aporta cada una.
- Ordena el itinerario de lectura recomendado y justifica **por qué ese orden** y no otro.
- ¿Qué formación necesita un **anotador especializado** en textos médicos que no necesita uno generalista?
- Los perfiles de **prompt engineer** y **evaluador de sesgos** no existían hace unos años. ¿Qué base comparten con el lingüista computacional clásico?
- Un titulado en informática sin formación lingüística entra en un proyecto de PLN. ¿Qué le va a costar más, y qué leería primero?

J. Las herramientas (RA3-c, RA3-g)

- Escribe el código con `nlk` que tokenice una frase en español, quite las *stopwords* y aplique *stemming*.
- Escribe el código con `spaCy` que muestre, para cada token, su texto, categoría y lema, y luego las entidades.
- ¿Por qué **NLTK para aprender** y **spaCy para producir**? Da dos razones técnicas.
- Explica el error de escribir `TfidfVectorizer(stop_words='spanish')` y cómo se hace bien.
- ¿Qué es **DistilBERT** y por qué es el modelo elegido en esta unidad? Da las tres cifras.
- Di qué notebooks de la unidad van en el **contenedor** y cuáles en **Colab**, y por qué.
- ¿Qué diferencia hay entre usar un `pipeline` de `transformers` ya hecho y **afinar** el modelo?

K. Construir un sistema (RA3-g)

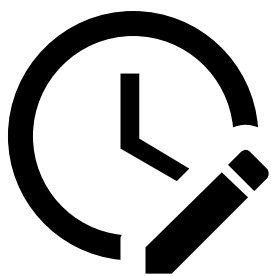
- Enumera los **seis pasos** de la metodología y di qué se decide en cada uno.
- ¿Por qué el paso de **analizar los errores** es el que más se salta, y qué se pierde al saltarlo?
- En el ejemplo guiado se elige `tf-idf` con 60 reseñas en vez de un *transformer*. Justifica esa decisión.
- Diseña en seis pasos un sistema que clasifique los correos de una FAQ por tema. Indica datos, herramienta y cómo lo evaluarías.
- Tu clasificador da un 92 % de exactitud, pero al leer los errores ves que **todos** son reseñas irónicas. ¿Qué harías: más datos, otro modelo o cambiar la tarea? Justifica.
- ¿Por qué hay que documentar el **origen y la licencia** de los datos? Relaciónalo con la UD06.
- Explica la progresión de los cinco notebooks guiados: qué representación del texto usa cada uno y por qué el orden es ese.
- Los cuatro entregables suben de nivel. Di qué demuestra cada uno y qué criterio cubre.

L. Integración

- Relaciona la ambigüedad del §6 con el papel del lingüista del §10.1: ¿por qué una explica la necesidad del otro?
- Un proyecto de PLN falla. Escribe tres hipótesis —una técnica, una de datos y una de **organización del equipo**— y cómo comprobarías cada una.
- Compara el PLN con los sistemas expertos de la UD05: ¿en qué se parecen las **reglas de anotación** a las **reglas de producción**? ¿En qué se diferencian?
- Enlaza esta unidad con la UD06: nombra **tres** decisiones de un proyecto de PLN que sean a la vez técnicas y éticas.

Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.



23 de agosto de 2026

6.4 Talleres

UD03 · Taller 1 — Del texto al vector: análisis de sentimiento



Entregable de la unidad

Cuenta en el 40 % de actividades del RA3, junto con los [notebooks entregables](#). Trabaja en parejas si lo indica el profesor.



Hazlo en el notebook

Tienes este taller como **notebook con las celdas a rellenar**: [UD03_T01_del_texto_al_vector.ipynb](#). Esta página es la referencia; lo que se entrega es el notebook completado.



Requisitos

Va en el **contenedor de la unidad** (`docker compose up -d` en la carpeta de la UD03), o en cualquier Python 3.12+ con:

```
1 pip install nltk spacy scikit-learn
2 python -m spacy download es_core_news_sm
```

Este taller **no necesita GPU ni Colab**.

Objetivo: construir un clasificador que determine si una reseña es positiva o negativa, usando un dataset anotado por el alumnado y `scikit-learn`.

FASE 1 — CREA TU DATASET (ORIGEN Y LICENCIA)

Crea un fichero `reseñas.csv` con al menos **30 reseñas cortas** en español (por parejas: 20 de entrenamiento, 10 de prueba) y su etiqueta (`1` = positiva, `0` = negativa). Anota en el informe el **origen** (opiniones propias o de un restaurante ficticio) y la **licencia** (p. ej. anotación propia bajo CC BY).

texto	etiqueta
La comida estaba riquísima y el servicio rápido	1
Pedimos y el plato llegó frío	0
...	...

FASE 2 — PREPROCESADO CON NLTK

```
1 import nltk
2 from nltk.tokenize import word_tokenize
3 from nltk.corpus import stopwords
4 nltk.download(['punkt_tab', 'stopwords'])
5
6 STOP = set(stopwords.words('spanish'))
7
8 def limpiar(texto):
9     tokens = word_tokenize(texto.lower(), language='spanish')
10    return ' '.join(t for t in tokens if t.isalpha() and t not in STOP)
```

Aplica `limpiar` a todas las reseñas y muestra 3 ejemplos antes/después.

FASE 3 — VECTORIZAR Y ENTRENAR

```

1 import pandas as pd
2 from sklearn.feature_extraction.text import TfidfVectorizer
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.pipeline import make_pipeline
5 from sklearn.model_selection import train_test_split
6
7 df = pd.read_csv('reseñas.csv')
8 X = df['texto'].apply(limpiar)
9 y = df['etiqueta']
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
11
12 pipe = make_pipeline(TfidfVectorizer(), LogisticRegression())
13 pipe.fit(X_train, y_train)
14 print("Exactitud:", pipe.score(X_test, y_test))

```

FASE 4 — MATRIZ DE CONFUSIÓN

```

1 from sklearn.metrics import confusion_matrix, classification_report
2 pred = pipe.predict(X_test)
3 print(confusion_matrix(y_test, pred))
4 print(classification_report(y_test, pred))

```

Interpeta la matriz: ¿cuántos positivos correctos, cuántos falsos positivos?

FASE 5 — PRUEBA CON RESEÑAS NUEVAS

```

1 for texto in ["El local está bien pero se notaba sucio", "!Espectacular, volveré mañana!"]:
2     print(pipe.predict([limpiar(texto)])[0], "-", texto)

```

FASE 6 — ANÁLISIS DE ERRORES

Busca en el test al menos **2 errores** del modelo e intenta explicarlos (jerga, ironía, ambigüedad, vocabulario nuevo). Documenta la conclusión.

ENTREGA DEL TALLER 1

Sube el **notebook ejecutado**. Se valora que **interpretes** los resultados, no solo que las celdas den un número.

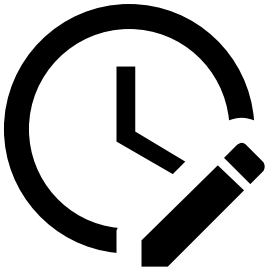
Fase	Evidencia mínima
1	El conjunto de reseñas, con su origen y licencia declarados
2	El texto preprocesado: tokens, sin <i>stopwords</i> , normalizado
3	El clasificador entrenado y su exactitud sobre datos no vistos
4	La matriz de confusión, interpretada : qué confunde con qué
5	Las predicciones sobre reseñas nuevas, tuyas
6	Dos errores leídos uno a uno y qué tipo de ambigüedad los explica



Soluciones

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD03 · Taller 2 · Ejercicios](#)



23 de agosto de 2026

UD03 · Taller 2 — Un sistema de PLN de punta a punta

**Entregable de la unidad**

Cuenta en el 40 % de actividades del RA3, junto con los [notebooks entregables](#). Trabaja en parejas si lo indica el profesor.

**Hazlo en el notebook**

Tienes este taller como **notebook con las celdas a rellenar**: [UD03_T02_sistema_pln.ipynb](#). Esta página es la referencia; lo que se entrega es el notebook completado.

**Requisitos**

Va en el **contenedor de la unidad** (`docker compose up -d` en la carpeta de la UD03), o en cualquier Python 3.12+ con:

```
1 pip install nltk spacy scikit-learn
2 python -m spacy download es_core_news_sm
```

Este taller **no necesita GPU ni Colab**.

Objetivo: construir un **extractor de entidades** (NER) para textos de un dominio concreto con `spacy`, siguiendo la metodología del CE g (tarea → datos → herramienta → implementar → evaluar → documentar).

FASE 1 — DEFINE LA TAREA

Elige un dominio (p. ej. **facturas, historias clínicas de ejemplo, noticias de tecnología**). Decide qué entidades quieres extraer (nombres de empresa, personas, lugares, fechas, importes, productos).

FASE 2 — DATOS

Crea al menos **10 textos de ejemplo** del dominio en un fichero `textos.txt` (pueden ser inventados). Indica origen y licencia en el informe.

FASE 3 — IMPLEMENTA CON SPACY

```
1 import spacy
2 from spacy.matcher import Matcher
3
4 nlp = spacy.load("es_core_news_sm")
5 matcher = Matcher(nlp.vocab)
6
7 # patrón propio: detectar "empresa de <algo>" o importes en euros
8 matcher.add("EMPRESA", [[{"LOWER": {"IN": ["empresa", "compañía"]}},
9                           {"POS": "ADP", "OP": "?"},
10                          {"IS_ALPHA": True, "OP": "+"}]]])
11 matcher.add("IMPORTE", [[{"LIKE_NUM": True}, {"LOWER": {"IN": ["€", "euros"]}}]])
12
13 def extraer(texto):
14     doc = nlp(texto)
15     entidades = [(e.text, e.label_) for e in doc.ents]
16     matches = matcher(doc)
17     return entidades, [(doc[s:e].text, "PATRON_EMPRESA/IMPORTE") for _, s, e in matches]
18
19 for linea in open('textos.txt', encoding='utf-8'):
20     print(extraer(linea.strip()))
```

FASE 4 — EVALÚA

Compara las entidades que devuelve spaCy con las que esperabas (anótalas a mano primero). Calcula cuántas acertó y dónde falló. ¿Qué patrones añadirías?

FASE 5 — DOCUMENTA

Escribe en el informe: la **tarea**, el **origen/licencia de los datos**, la **herramienta** (spaCy + Matcher), las **reglas/patrones** usados, la **evaluación** (aciertos/fallos) y las **limitaciones** (dominio, textos fuera de contexto, ambigüedad).

FASE 6 — MEJORA (RETO)

Añade 2 patrones más de tu dominio (p. ej. fechas en formato "dd/mm/aaaa" o códigos de producto) y repite la evaluación.
¿Mejoró la cobertura?

ENTREGA DEL TALLER 2

Sube el **notebook ejecutado**, siguiendo los seis pasos de la metodología del §12.1.

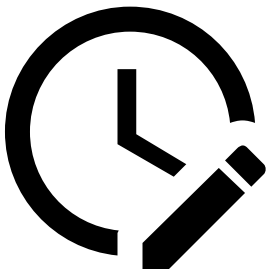
Fase	Evidencia mínima
1	La tarea definida: qué entra, qué sale y para quién
2	Los datos, con origen, tamaño y licencia
3	El sistema implementado con spaCy y funcionando
4	La evaluación sobre datos no vistos, con el análisis de errores
5	La documentación: decisiones tomadas y limitaciones asumidas
6	La mejora propuesta, aunque no esté implementada

Y la respuesta a: *¿por qué elegiste esa herramienta y no otra?* La respuesta buena habla de **los datos que tenías**, no de lo moderna que sea la técnica.

**Soluciones**

Las soluciones no se publican: se corrigen y comentan en clase.

[Volver a la UD03](#) · [Taller 1](#) · [Ejercicios](#)



23 de agosto de 2026

6.5 UD03 · Actividades guiadas

Cómo se trabajan

Estos cinco notebooks **no se entregan**: se hacen en clase, con el profesor, y son la preparación de los **cinco entregables**. Están ejecutados: puedes leerlos antes de tocar nada para ver qué hace cada celda.

Dos entornos

- **N01 y N04**: van en el **contenedor de la unidad** (`docker compose up -d` en esta carpeta) o en cualquier Python 3.12+ con `nltk` y `spaCy`.
- **N02, N03 y N05**: usan `transformers`, entrenan modelos o procesan audio. Van en **Colab**, que tiene GPU — en local, sin GPU, tardan horas.

N01 · Introducción al PLN

Los fundamentos con `nltk` y `TextBlob`: tokenizar, quitar *stopwords*, etiquetar categorías gramaticales, extraer frases nominales y construir representaciones bolsa de palabras, `tf-idf` y *word embeddings*.

Recurso	Enlace
Notebook	UD03_N01_introduccion_pln.ipynb

`gensim` no instala en Python 3.14

La celda de *Word2Vec* usa `gensim`, que **todavía no publica rueda para Python 3.14** (su última versión, 4.4.0, llega hasta 3.13). En el contenedor de la unidad no pasa nada —lleva Python 3.12—, pero si trabajas con un intérprete más nuevo, esa celda concreta fallará al instalar. El resto del notebook no depende de `gensim`.

N02 · Clasificación de texto con PyTorch

Construye un clasificador de texto **desde cero**: de la bolsa de palabras a los *word embeddings*, con un modelo de PyTorch entrenado y evaluado. Es el notebook que **EX2** repite sobre datos propios.

Recurso	Enlace
Notebook	UD03_N02_clasificacion_texto_torch.ipynb

N03 · Modelos de lenguaje: afinar DistilBERT

Transfer learning de manual: parte de un **DistilBERT** preentrenado y lo afina para análisis de sentimiento con una base de datos de tuits. **EX4** repite el proceso sobre reseñas de cine.

Recurso	Enlace
Notebook	UD03_N03_modelos_lenguaje_distilbert.ipynb

N04 · spaCy, primeros pasos

El *pipeline* de spaCy de una sola pasada: tokenización, POS, dependencias y entidades, sobre texto en español.

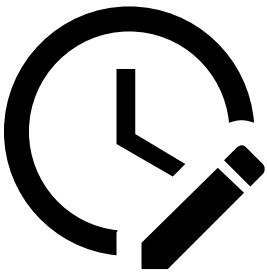
Recurso	Enlace
Notebook	UD03_N04_spacy_primeros_pasos.ipynb

N05 · Ampliación: clasificador de géneros musicales

Un sistema de PLN aplicado a **audio**: clasificar el género de una canción combinando `librosa` y `transformers`. Es el notebook más largo del módulo (240 celdas) y **no cuenta horas de la unidad**: queda como material de ampliación para quien quiera ir más allá de `EX5`.

Recurso	Enlace
Notebook	UD03_N05_ampliacion_clasificador_generos_musicales.ipynb

[Volver a la UD03](#) · [Entregables](#)



24 de agosto de 2026

6.6 UD03 · Actividades entregables



Los cinco cuentan para la nota del RA3

Los cinco notebooks forman el **40 % de actividades** del RA3. Cada uno se califica **sobre 10** con su [rúbrica](#), que tienes más abajo. EX2 y EX5 llevan además una **parte escrita** que cubre los criterios sobre el papel del lingüista, el trabajo cooperativo y la formación del investigador (RA3-b, RA3-e, RA3-f) —léela antes de empezar, porque cuenta en la rúbrica.



Dos entornos

EX1 y EX3 van en el **contenedor** de la unidad. EX2, EX4 y EX5 usan `transformers` o audio: van en **Colab**.



Identificadores de *dataset* que ya no funcionan

Los nombres clásicos de algunos conjuntos de datos de Hugging Face **ya no resuelven**: `datasets` exige el formato `namespace/name`. EX2 usa `SetFit/TREC-QC` en vez de `trec`, y EX4 usa `stanfordnlp/imdb` en vez de `imdb`.



EX5 : tres trampas verificadas

`pipeline("translation", ...)` ya no existe como tarea genérica (usa `AutoTokenizer` + `AutoModelForSeq2SeqLM`), cargar el audio con la ruta directa puede pedir `ffmpeg` (carga con `librosa` en su lugar) y el modelo de traducción necesita `sentencepiece`. Todo explicado en el propio notebook.

EX1 · Representación de texto, por dos caminos

Tokenizar, quitar *stopwords* y construir una bolsa de palabras y un vector tf-idf, resuelto **con NLTK y con TextBlob** — el mismo ejercicio, dos librerías.

Recurso	Enlace
Notebook	UD03_EX1_representacion_texto.ipynb

Se entrega: el notebook con los cuatro ejercicios resueltos por **uno** de los dos caminos —tú eliges— y una comparación breve con el otro.

EX2 · Clasificador de preguntas

Repite el proceso de N02 —de la bolsa de palabras a un clasificador entrenado— sobre un conjunto de **preguntas**, en vez de noticias.

Recurso	Enlace
Notebook	UD03_EX2_clasificador_preguntas.ipynb

Se entrega: el notebook con el clasificador entrenado y evaluado, **y una sección de conclusiones** que responda: ¿qué decisiones de anotación tomaría un lingüista con este conjunto de preguntas? ¿Qué perfiles harían falta si el proyecto creciera? ¿Qué formación necesitarías tú para mejorar este sistema?

EX3 · NLTK y Python: etiquetado con `cess_esp`

Procesa el corpus español anotado `cess_esp`: separa entrenamiento y prueba, reduce el conjunto de etiquetas morfosintácticas de 289 a un conjunto manejable, y valida con validación cruzada.

Recurso	Enlace
Notebook	UD03_EX3_nltk_python.ipynb

Se entrega: el notebook con los seis apartados resueltos.

EX4 · Análisis de sentimiento en reseñas de cine

Transfer learning: parte del DistilBERT de N03 —afinado para tuits— y adáptalo a reseñas de IMDb, un dominio distinto.

Recurso	Enlace
Notebook	UD03_EX4_sentimiento_imdb.ipynb

Se entrega: el notebook con el modelo afinado y evaluado sobre las reseñas.

EX5 · Asistente virtual por voz

Encadena tres modelos —voz a texto, traducción y texto a voz— en **una sola función** que reciba un audio y devuelva otro.

Recurso	Enlace
Notebook	UD03_EX5_asistente_virtual.ipynb
Audio de prueba	OpenTheDoor.wav

Se entrega: el notebook con la función conjunta funcionando de punta a punta, **y una sección final** que responda: ¿qué papel tendría un lingüista revisando las transcripciones y traducciones de este asistente? ¿Qué formación necesitaría el equipo si el asistente tuviera que funcionar en varios idiomas?

Rúbricas

Son las rúbricas reales del curso. Las cinco tareas se califican **sobre 10**; los puntos de cada rúbrica se **escalan** a esa nota.

EX1 · Representación de texto (8 puntos)

Cuatro criterios de 2 puntos, uno por ejercicio (correcta 2 · parcial 1 · incorrecta 0,5).

EX2 · Clasificador de preguntas (18 puntos)

Criterio	Puntos
Preparación del entorno	2
Cargar el <i>dataset</i>	2
Tokenización	2
Bolsa de palabras	2
<i>Word embeddings</i>	2
Crear el modelo neuronal	2
Entrenar	2
Evaluación	2
Conclusiones o comentarios	2

El último criterio es donde se puntúa la **parte escrita** sobre el lingüista y la cooperación.

EX3 · NLTK y `cess_esp` (10 puntos)

Cinco criterios de 2 puntos, apartados a) a e).

EX4 · Sentimiento en IMDb (10 puntos)

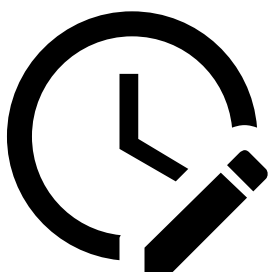
Criterio	Puntos
Cargar el <i>dataset</i>	2
Evaluación previa	2
Definir las etiquetas	2
Afinar el modelo (<i>fine-tuning</i>)	2
Inferencia	2

EX5 · Asistente virtual (8 puntos)

Criterio	Puntos
Modelo voz a texto	2
Modelo de traducción	2
Modelo texto a voz	2
Función conjunta con todos los pasos	2

El cuarto criterio es el que demuestra RA3-g: encadenar los tres modelos en un sistema, no usarlos por separado.

[Volver a la UD03](#) · [Actividades guiadas](#) · Talleres: [T01](#) · [T02](#)



24 de agosto de 2026

7. UD04

7.1 UD04 — Análisis de sistemas robotizados

Unidad 4 · 12 h · semanas 15-18

Cierra el bloque de aplicaciones de la IA. Se evalúa con **seis entregables prácticos** y la prueba escrita del RA4.

1. Introducción

Los robots ya no son una promesa de futuro: en 2024 se instalaron **más de 540.000 robots industriales** en el mundo y España fue el **tercer mercado europeo**, impulsado por la automoción. Entender cómo funciona un robot, cómo se modela su movimiento y cómo se diseña un sistema robotizado es competencia directa de un especialista en IA, porque un robot es un **agente encarnado**: el único sistema de IA de este curso que puede cambiar el estado del mundo físico.

Eso lo hace distinto de todo lo anterior. Un clasificador que se equivoca produce una etiqueta errónea; un robot que se equivoca rompe una pieza, o algo peor. Y su entorno es **parcialmente observable, estocástico y con más agentes dentro**: las cámaras no ven tras las esquinas, los engranajes patinan y las personas que comparten el espacio son impredecibles.

El recorrido de la unidad:

1. **Métodos y aplicaciones de la robótica**: qué es un robot, su hardware (sensores, actuadores, pinzas) y dónde se usa, con datos del sector.
2. **Qué clase de problema resuelve la robótica** y la jerarquía tarea → movimiento → control.
3. **Modelado y control cinemático** de manipuladores: grados de libertad, parámetros DH, cinemática directa e inversa.
4. **Los problemas típicos** (singularidades, redundancia) y sus **soluciones**: espacio de configuración, planificación de movimiento y seguimiento de trayectoria.
5. **Percepción robótica**: localización, mapeo y SLAM.
6. **Planificación con incertidumbre y aprendizaje** en robótica.
7. **Las técnicas de programación** de robots, del *teach pendant* al ROS 2 — y comparadas en la práctica sobre un mismo problema.
8. **Humanos y robots**: coordinación y aprender lo que la persona quiere.
9. **El diseño e implementación** de un sistema robotizado: selección, célula, seguridad y normativa.

Hilo conductor de la unidad

Un robot percibe, razona y actúa sobre el mundo físico. Primero vemos con qué lo hace (hardware); luego cómo se describe y controla su movimiento (cinemática); después cómo decide por dónde ir (planificación) y cómo sabe dónde está (percepción); y por último cómo se programa y cómo se integra en un sistema seguro.

De dónde sale el material de esta unidad

La parte de **percepción, planificación de movimiento, aprendizaje por refuerzo e interacción humano-robot** está construida a partir del capítulo 26 (*Robotics*) de *Artificial Intelligence: A Modern Approach*, 4.^a edición, de **Stuart Russell y Peter Norvig**, adaptada al nivel y a los criterios de evaluación de este módulo. Las prácticas con el simulador **AITK** provienen del curso **Models d'IA** de **Carles Gonzalez**, con licencia CC BY-NC-SA 4.0.

2. Resultado de aprendizaje y criterios de evaluación

RA4 — Analiza sistemas robotizados, evaluando opciones de diseño e implementación.

CE	Criterio de evaluación	Bloque
RA4-a		\$4-6

CE	Criterio de evaluación	Bloque
	Se han recopilado los problemas del modelado y control cinemático en robots manipuladores.	
RA4-b	Se han buscado soluciones a los problemas de los robots.	\$7-9
RA4-c	Se han valorado las características diferenciadoras de las técnicas de programación de robots y de sistemas robotizados.	\$10-11
RA4-d	Se han evaluado diferentes opciones en el diseño e implementación de sistemas robotizados.	\$12



Bloque de contenidos oficial (RD 279/2021, anexo I)

El currículo llama a este bloque, textualmente, «Análisis de sistemas robotizados», y le asigna estos contenidos:

- Métodos y aplicaciones de la robótica.
- Modelado y control de robots.
- Programación de robots y aplicaciones.
- Sistemas robotizados. Diseño e implementación.

Son **cuatro contenidos para cuatro criterios de evaluación** y, a diferencia del resto de los RA de este módulo, aquí **encajan casi uno a uno**: es la excepción. El único matiz es que «modelado y control» alimenta a la vez RA4-a (los problemas) y RA4-b (las soluciones).

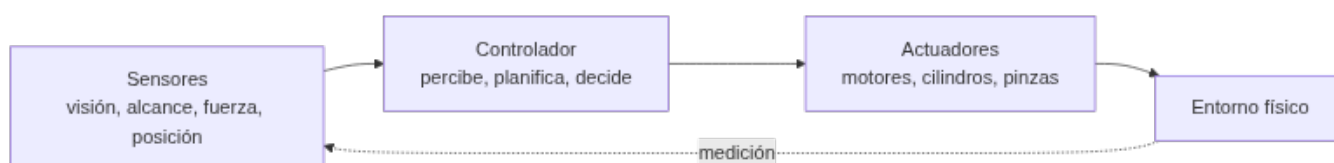
3. Objetivos de la unidad

Objetivo	Al terminar la unidad serás capaz de...
O1	Clasificar los robots por tipo y describir sus aplicaciones reales con datos del sector.
O2	Distintuir los sensores por lo que miden (entorno, ubicación, configuración interna) y elegir el adecuado para una tarea.
O3	Explicar grados de libertad, articulaciones y espacio articular frente a cartesiano.
O4	Resolver la cinemática directa de un manipulador con parámetros DH y entender por qué la inversa es más difícil.
O5	Identificar los problemas típicos (singularidades, redundancia, límites) y sus soluciones.
O6	Plantear un problema de planificación de movimiento en el espacio de configuración y elegir un método.
O7	Explicar cómo un robot se localiza y construye un mapa (SLAM) a partir de sensores con ruido.
O8	Comparar las técnicas de programación de robots resolviendo un mismo problema por reglas, lógica difusa, red neuronal y neuroevolución.
O9	Aplicar criterios de selección y diseño de un sistema robotizado (payload, alcance, sensores, seguridad).
O10	Valorar la normativa de seguridad (ISO 10218:2025) y la integración en la Industria 4.0.

4. Métodos y aplicaciones de la robótica (RA4-a)

4.1 ¿Qué es un robot?

Un **robot** es una máquina programable que **percibe su entorno**, **procesa información** y **actúa físicamente** sobre él. Tiene tres partes que se realimentan:



Los **efectores** son las piezas que ejercen fuerza sobre el entorno: ruedas, patas, articulaciones, pinzas. Cuando actúan, pueden cambiar tres cosas: el **estado del robot** (un coche gira las ruedas y avanza), el **estado del entorno** (un brazo empuja una taza) e incluso el **estado de las personas** alrededor (un exoesqueleto mueve una pierna; un robot se acerca al ascensor y alguien se aparta).

4.2 Tipos de robot, desde el hardware

La imagen popular del robot con cabeza y dos brazos —el **antropomórfico** de la ficción— es la menos frecuente en la realidad:

Tipo	Qué es	Ejemplo
Manipulador	Un brazo, no necesariamente unido a un cuerpo: se atornilla a una mesa o al suelo	Brazos de una línea de montaje; brazos montados en sillas de ruedas para asistencia
Robot móvil con ruedas	Se desplaza sobre ruedas, dentro o fuera	Aspiradora, AGV/AMR de almacén, coche autónomo, rover de Marte
Robot con patas	Atraviesa terreno accidentado donde la rueda no llega	Cuadrúpedos de inspección
Aéreo (UAV) y submarino (AUV)	Rotores o propulsión sumergida	Cuadricópteros, vehículos de exploración oceánica
Cobot	Manipulador que comparte espacio con personas, con potencia y fuerza limitadas	Montaje asistido, inspección
Otros	Prótesis, exoesqueletos, robots con alas, enjambres y entornos inteligentes , donde el robot es la habitación entera	Rehabilitación, agricultura de precisión

La diferencia entre un manipulador de una tonelada que ensambla coches y un brazo de asistencia no es solo el tamaño: el primero mueve mucha carga, el segundo mueve poca pero es **seguro entre personas**. Payload y seguridad son criterios distintos, y a menudo opuestos.

4.3 Sensores: qué se mide y con qué

Los sensores son la interfaz perceptiva del robot. Se clasifican de dos formas a la vez.

Por cómo obtienen la señal: los **pasivos** (cámaras) captan lo que el entorno emite; los **activos** (sonar, lidar) emiten energía y miden lo que vuelve. Los activos dan más información, pero consumen más y **se interfieren entre sí** cuando hay varios trabajando a la vez.

Por lo que miden:

Clase	Qué informa	Sensores típicos
Del entorno	Distancia y forma de lo que hay alrededor	Sonar, visión estéreo, luz estructurada (Kinect), cámara de tiempo de vuelo, lidar , radar, táctiles
De la ubicación	Dónde está el robot	GPS/GLONASS, balizas de interior, análisis de señal wifi, balizas de sonar bajo el agua
De la configuración interna (propioceptivos)	Cómo está el propio robot	Encoders de eje, odometría de rueda, giroscopios, sensores de fuerza y par

Las cifras ayudan a elegir:

Sensor	Alcance y precisión	Dónde encaja
Lidar de barrido	Precisión de ~1 cm a 100 m	El sensor de referencia en coches autónomos
Cámara de tiempo de vuelo	Imágenes de rango a hasta 60 fps	Interiores y distancias cortas; peor que el lidar a plena luz del día
Radar	Hasta kilómetros , y ve a través de la niebla	Vehículos aéreos y automoción

Sensor	Alcance y precisión	Dónde encaja
GPS	Unos metros; con GPS diferencial, precisión milimétrica en condiciones ideales	Exteriores. No funciona en interiores ni bajo el agua
Sensores de fuerza y par	Fuerzas en 3 traslaciones y 3 rotaciones, cientos de medidas por segundo	Manipular objetos frágiles o de forma desconocida



Por qué la odometría no basta

Contar vueltas de rueda parece una forma barata y exacta de saber cuánto has avanzado, y lo es — durante unos metros. Las ruedas **derrapan y patinan**, y el error se **acumula sin límite**: no hay nada que lo corrija. Por eso la odometría se combina siempre con sensores inerciales (giroscopios) y con alguna referencia externa. Es la razón de ser de la localización probabilística del §8.



El caso de la bombilla

Imagina un manipulador de **una tonelada** enroscando una bombilla. Es facilísimo aplicar demasiada fuerza y romperla. Los **sensores de fuerza** le dicen con qué fuerza agarra y los **de par**, con qué fuerza gira; midiendo cientos de veces por segundo, el robot detecta una fuerza inesperada y corrige **antes** de romper el cristal. La capacidad de manipular con cuidado no viene de tener un brazo más suave, sino de **medir rápido**.

4.4 Actuadores y pinzas

El mecanismo que produce el movimiento es el **actuador**:

Tipo	Cómo funciona	Dónde se usa
Eléctrico	Corriente que hace girar un motor	El más común; articulaciones de brazos robóticos
Hidráulico	Fluido a presión (aceite, agua)	Mucha fuerza: maquinaria pesada
Neumático	Aire comprimido	Movimientos rápidos y simples, pinzas

Los actuadores mueven **articulaciones** entre eslabones rígidos: de **revolución** (un eslabón gira respecto al otro) o **prismáticas** (uno se desliza sobre el otro). Las dos son de un solo eje; las esféricas, cilíndricas y planas son multieje.

Para agarrar, el robot usa **pinzas**, y aquí hay un compromiso claro:

- La **pinza de mordaza paralela** —dos dedos, un actuador— es «amada y odiada por su simplicidad»: hace poco, pero nunca falla y se programa en un minuto.
- Las de **tres dedos** ganan algo de flexibilidad sin complicarse.
- En el otro extremo, una **mano antropomórfica** como la Shadow Dexterous Hand tiene **20 actuadores**. Permite manipulación en la mano (coger el móvil y girarlo para orientarlo), pero **aprender a controlarla es mucho más difícil**.



Más grados de libertad no es mejor

Cada actuador que añades multiplica lo que el robot **puede** hacer y también lo que hay que **decidir** en cada instante. Veinte actuadores es un espacio de acción enorme. Casi toda la robótica industrial funciona con dos dedos y un actuador, porque la tarea no necesita más.

4.5 Aplicaciones, con datos (IFR World Robotics 2025)

Dato	Valor (2024)
Robots industriales instalados en el año	542.000 (cuarto año por encima de 500.000)
Stock operativo mundial	4,66 millones (+9 %)
Densidad robótica media	~132-151 robots por 10.000 empleados
País líder en densidad	Corea del Sur (1.012-1.220)

Dato	Valor (2024)
Líder de mercado	China (295.000 unidades, 54 % de las instalaciones)
España, 3.er mercado europeo	5.100 unidades , impulsado por la automoción
Robótica médica	+91 % (~16.700 unidades)
Robots de servicio profesional	~200.000 unidades (+9 %)



¿Por qué estos datos?

La **IFR (International Federation of Robotics)** publica cada año el informe *World Robotics* con las estadísticas oficiales del sector. Es la referencia para argumentar la importancia económica de la robótica con cifras y no con impresiones.

4.6 Cuatro casos reales

Fabricante o robot	Características	Uso real
KUKA KR AGILUS	6-11 kg de carga, 726-1.101 mm de alcance	Envasa 60 sándwiches por minuto; procesa 3.000 muestras de sangre al día
FANUC (familia)	Más de 100 modelos; cargas de hasta 2,3 t , alcance de 4,7 m	Paletizado (M-410, 700 kg), soldadura, manipulación pesada
Universal Robots e-Series	UR3e (3 kg / 500 mm) ... UR16e (16 kg / 900 mm); repetibilidad ±0,03-0,05 mm	Montaje asistido, inspección, carga y descarga de máquinas
Amazon Robotics	Más de 1 millón de robots de almacén	Logística: la flota reduce el tiempo de desplazamiento de los operarios



Pensar en «robot + tarea», no en «robot»

No se elige el robot más caro: se define primero **la tarea** —qué pieza, qué peso, qué ritmo, qué precisión, qué entorno— y después se busca el modelo que cumple payload, alcance, repetibilidad y seguridad. Es el enfoque del §12 y del Taller 2.

5. Qué clase de problema resuelve la robótica

Antes de elegir algoritmos conviene ver **qué tipo de problema** es. La robótica reúne casi todas las dificultades del curso a la vez: es **no determinista**, **parcialmente observable** y **multiagente**.

Y lo multiagente no es solo competitivo o solo cooperativo, sino los dos: en un pasillo estrecho por el que solo cabe uno, el robot y la persona **colaboran** —ninguno quiere chocar— y a la vez **compiten** un poco por pasar primero. Un robot demasiado educado, que siempre cede, se queda atascado para siempre en un sitio concurrido y nunca llega.

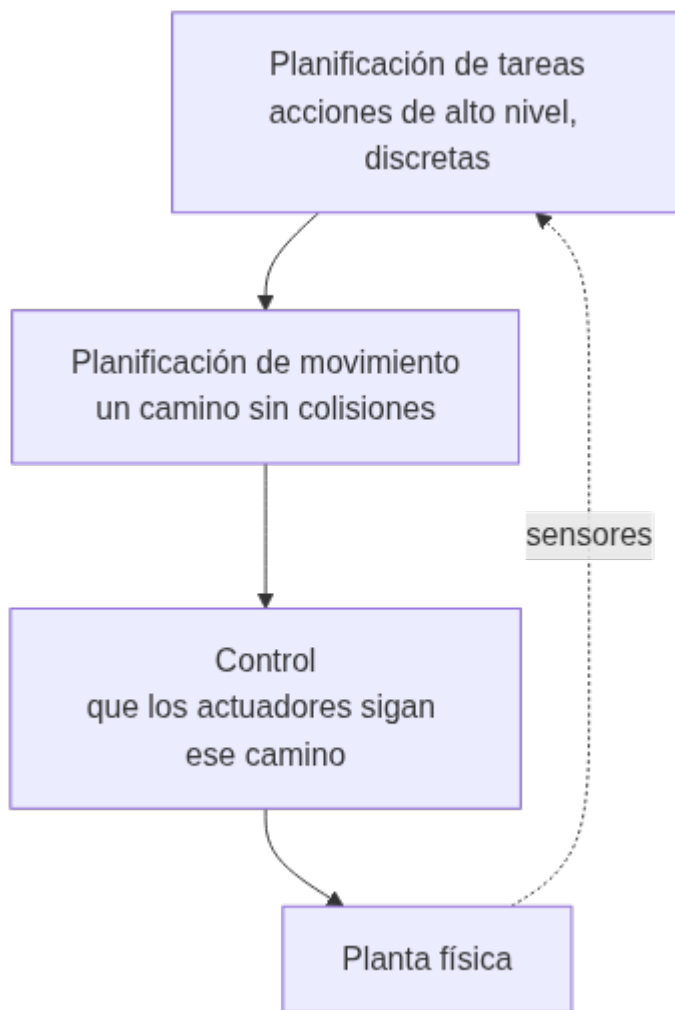


¿De quién es la recompensa?

En casi toda la robótica, el robot actúa **al servicio de una persona**: si lleva la comida a un paciente, la recompensa es del paciente, no suya. Y ahí está el problema: **la función de recompensa verdadera está en la cabeza del usuario**. El robot tiene que deducir lo que la persona quiere, o conformarse con la aproximación que le programó un ingeniero. Es el mismo problema de alineación que se trata en la UD06, aquí con un brazo de una tonelada.

5.1 La jerarquía de tres niveles

En crudo, las observaciones de un robot son señales de sensores (píxeles, impactos de láser) y sus acciones son corrientes eléctricas enviadas a los motores. Entre eso y un plan como «lleva la comida a la habitación 12» hay un abismo. La robótica lo salva **partiendo el problema**:



- **Planificación de tareas:** decide las submetas —ir a la puerta, abrirla, ir al ascensor, pulsar el botón—. Trabaja con estados y acciones **discretos**.
- **Planificación de movimiento:** encuentra un camino que lleva al robot de un punto a otro cumpliendo cada submeta (§7).
- **Control:** consigue que los actuadores ejecuten ese movimiento (§6.4).

⚠ Lo que se pierde al partir el problema

Dividir reduce la complejidad, pero **renuncia a que las partes se ayuden**. La acción puede mejorar la percepción (moverse para ver mejor) y decidir qué percepción hace falta. Un movimiento óptimo sobre el papel puede ser malo de seguir para el controlador. Y un plan de tareas puede ser **imposible de instanciar** a nivel de movimiento. Por eso la robótica actual avanza en cada nivel y, al mismo tiempo, en **volver a integrarlos**: planificar movimiento y control juntos, tareas y movimiento juntos, y cerrar el lazo con la percepción.

6. Modelado y control cinemático (RA4-a)

6.1 Conceptos básicos

Un manipulador se modela como una **cadena cinemática**: eslabones rígidos unidos por articulaciones.

Concepto	Definición
Grado de libertad (DoF)	Número de movimientos independientes. Hacen falta ≥ 6 para colocar el efector en cualquier posición y orientación en 3D
	Gira: su variable es un ángulo θ

Concepto	Definición
Articulación de revolución (R)	
Articulación prismática (P)	Se desliza: su variable es una distancia d
Eslabón	Pieza rígida entre dos articulaciones
Espacio articular	El vector q con la posición de cada articulación
Espacio cartesiano	La pose del efector: posición más orientación

Configuración	Articulaciones	Uso típico
Articulado / antropomórfico	≥ 3R	El más común en industria (6 ejes)
Cartesiano / pórtico	3P	Manipulación simple, gran alcance
SCARA	RRP	Montaje de componentes: rígido en Z, flexible en XY
Delta / paralelo	Paralelas	Empaquetado de alta velocidad

6.2 Cinemática directa (FK)

La **cinemática directa** calcula la pose del efector a partir de los valores articulares: $pose = f(q)$. Se resuelve multiplicando **matrices de transformación homogénea** de 4×4, que combinan rotación y traslación:

$${}^0T_n = {}^0T_1 \cdot {}^1T_2 \cdot \dots \cdot {}^{n-1}T_n$$

Cada transformación se describe con los **parámetros de Denavit-Hartenberg (DH)**: cuatro valores por articulación (θ , d , a , α). Se escribe la tabla DH del robot y la pose sale de encadenar las matrices.



Tabla DH del Puma 560 (fragmento)

El Puma 560, brazo industrial clásico, se describe con una tabla como esta (metros y grados):

Articulación	θ (variable)	d	a	α
1	q_1	0,6718	0	-90°
2	q_2	0	0,4318	0°
3	q_3	0,1500	0,0203	90°
4	q_4	0,4318	0	-90°

Con esos valores, `fkine([0, 0.2, 0.3, 0.4, 0.5, 0.6])` devuelve una matriz 4×4 con la pose del efector: una posición concreta del TCP, del orden de `[0,25; -0,13; 1,15]`. Se comprueba en el notebook de la unidad.

Cinemática directa de un brazo plano 3R, a mano

Para un brazo en el plano con eslabones l_1, l_2, l_3 :

$$\begin{aligned} x &= l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2) + l_3 \cos(\theta_1 + \theta_2 + \theta_3) \\ y &= l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2) + l_3 \sin(\theta_1 + \theta_2 + \theta_3) \\ \varphi &= \theta_1 + \theta_2 + \theta_3 \end{aligned}$$

Con $l_1 = l_2 = l_3 = 1$ y los tres ángulos a 0, el efector está en **(3, 0)** con orientación 0. Poniendo $\theta_1 = 90^\circ$, pasa a **(0, 3)**. El modelo geométrico predice exactamente dónde está la punta del brazo: eso es la cinemática directa, y **siempre tiene una única solución**.

6.3 Cinemática inversa (IK)

La **cinemática inversa** va al revés: dada una pose deseada, calcula los ángulos articulares que la consiguen, $q = f^{-1}(\text{pose})$. Es **mucho más difícil**, y ahí están casi todos los problemas que pide recopilar el criterio RA4-a:

Problema	En qué consiste	Cómo se aborda
Múltiples soluciones	Un 6R general puede tener hasta 16 soluciones ; con muñeca esférica, 8	Elegir por criterio: evitar obstáculos, minimizar el recorrido, codo arriba o abajo
Redundancia	Más DoF de los necesarios → infinitas soluciones	Optimizar en el espacio nulo del jacobiano
Sin solución	El objetivo está fuera del alcance	Detectarlo y avisar, no iterar sin fin
Singularidad	El jacobiano pierde rango → la velocidad articular tiende a infinito	Evitarla al planificar, o cruzarla bajando la velocidad cartesiana

Los métodos se dividen en **analíticos** —desacoplamiento cinemático en muñecas esféricas, teorema de Pieper— y **numéricos** —Newton-Raphson, pseudoinversa del jacobiano, CCD, FABRIK—.

6.4 Control

Estrategia	Qué controla	Cómo
Control de posición	El ángulo de cada articulación	PID por eje: la medida es el encoder, la consigna el ángulo objetivo
Control de velocidad	La velocidad del efector (<i>resolved-rate</i>)	$\dot{q} = J^+ v$, con la pseudoinversa de Moore-Penrose del jacobiano
Control de fuerza / impedancia	La interacción con el entorno	El robot se comporta como un sistema masa-muelle-amortiguador: ensamblaje, pulido

Los controladores de seguimiento van de menos a más:

- **P** — aplica una acción proporcional al error de posición. Simple, pero oscila.
- **PD** — añade un término proporcional a la derivada del error, que **amortigua** la oscilación.
- **PID** — añade la integral del error, que corrige los **errores sistemáticos persistentes**.
- **Par calculado** — usa la **dinámica inversa** del robot para calcular el par que hace falta, y deja al PID solo la corrección de lo que el modelo no acierta. Es lo que usan los robots industriales de verdad.

Los robots industriales trabajan con **servomotores con encoder** en lazo cerrado, no con motores paso a paso en lazo abierto: sin realimentación no hay forma de saber si el eje llegó.



De dónde salen las trayectorias

No se salta de un punto a otro: se **interpola**. `jt看aj` genera la trayectoria en el espacio articular con un polinomio de 5.º grado (aceleración y *jerk* continuos, movimiento suave); `ctr看aj` genera una **línea recta cartesiana**, que es lo que hace falta cuando el efector lleva una herramienta de corte. El controlador va siguiendo esa trayectoria punto a punto.

7. Los problemas y sus soluciones (RA4-b)

7.1 Singularidades cinemáticas

Una **singularidad** es una configuración en la que el jacobiano pierde rango: el robot **pierde capacidad de movimiento** en alguna dirección y los cálculos de velocidad se van a infinito.

Singularidad	Cuándo ocurre
De muñeca	Los ejes 4 y 6 se alinean ($\theta_5 = 0$): las dos articulaciones deberían girar a velocidad infinita
De hombro	El centro de la muñeca corta el eje de la base

Singularidad	Cuándo ocurre
De codo	El brazo queda completamente estirado o plegado
De límite	El efector llega al borde de su volumen de trabajo



Qué pasa de verdad si el robot entra en una singularidad

No es un problema abstracto de álgebra. Las velocidades articulares necesarias **tienden a infinito**, así que lo que se ve en la célula es **sobrecorriente en los servos, vibraciones o una parada de seguridad** en medio del ciclo. Por eso la planificación de trayectorias evita esas configuraciones, o las cruza reduciendo la velocidad cartesiana.

7.2 Soluciones

Problema	Solución
Singularidades	Planificar evitándolas; medir la manipulabilidad (índice de Yoshikawa) para saber cuánto margen queda
IK compleja	Desacoplamiento cinemático : una muñeca esférica separa el problema de posición del de orientación
Múltiples soluciones	Criterios de optimización: evitar obstáculos, minimizar recorrido
Límites articulares	Modelarlos y restringir el <i>solver</i>
Precisión deficiente	Calibrar el robot, que puede mejorar la precisión hasta $\times 10$, y medir la repetibilidad con la ISO 9283



Precisión no es repetibilidad

Un robot puede ser **repetible** y a la vez **impreciso**: vuelve siempre al mismo punto, pero ese punto no es el que se programó. Para trabajar con *teach pendant* basta la repetibilidad — el punto se enseñó ahí—, pero en **programación offline**, donde las coordenadas vienen de un CAD, la precisión es crítica y hay que calibrar (por ejemplo, un ABB IRB 1600 con láser *tracker*). La repetibilidad de los cobots UR e-Series es de $\pm 0,03-0,05$ mm.

7.3 El espacio de configuración

Para planificar el movimiento hay que cambiar de punto de vista. En vez de pensar en el robot moviéndose por una habitación, se piensa en un **punto moviéndose por el espacio de configuración**: el espacio abstracto de **todas las configuraciones posibles** del robot.

Para un brazo de 6 ejes, ese espacio tiene 6 dimensiones —una por articulación—; para un robot móvil en un plano, 3 (x, y, orientación). Los obstáculos del mundo real se convierten en **regiones prohibidas** de ese espacio, y lo que queda es el **espacio libre**. El problema deja de ser geométrico y se vuelve una búsqueda de camino.



El problema de la mudanza del piano

La planificación de movimiento se llama a veces así, por el esfuerzo de una empresa de mudanzas para llevar un piano grande y de forma irregular de una habitación a otra sin golpear nada. Formalmente hacen falta: un espacio de trabajo w , una región de obstáculos $o \subset w$, un robot con su espacio de configuración c , una configuración inicial q_s y una objetivo q_g . Y lo que se busca es un **camino continuo** por el espacio libre entre las dos.

El problema se complica de tres formas típicas: que el objetivo sea un **conjunto** de configuraciones válidas en vez de una sola; que haya que **minimizar un coste** (longitud del camino, energía); o que haya **restricciones** — si el robot lleva una taza de café, tiene que mantenerla vertical todo el trayecto.

7.4 Cómo se planifica el movimiento

Método	Idea	Fuerte en	Flojo en
Grafo de visibilidad	Nodos en los vértices de los obstáculos, aristas donde hay línea de visión clara; después A* o Dijkstra	Da el camino más corto en 2D con obstáculos poligonales	Escala mal con muchos obstáculos; el camino roza las esquinas
Diagrama de Voronoi	Divide el espacio en regiones por cercanía a cada obstáculo; el robot circula por los bordes entre regiones	Maximiza la distancia a los obstáculos: caminos seguros	Caminos más largos de lo necesario
Descomposición celular	Trocea el espacio libre en celdas y busca sobre el grafo de adyacencia	Sencillo y completo en su resolución	El número de celdas explota al subir de dimensión
Muestreo (RRT, PRM)	Va lanzando configuraciones al azar y conectando las válidas hasta unir inicio y meta	Es lo único práctico con muchas dimensiones (brazos de 6-7 ejes)	No garantiza el camino óptimo; el resultado es irregular y hay que suavizarlo

Los dos primeros ilustran bien un compromiso que reaparece siempre: el grafo de visibilidad busca el camino **más corto**, que pasa pegado a las esquinas; el diagrama de Voronoi busca el **más seguro**, que se aleja de todo y por eso es más largo. Cuál conviene depende de si el riesgo es chocar o es tardar.

Cualquiera de los cuatro devuelve un camino que suele necesitar un último paso de **suavizado** o de **replanificación** cuando el entorno cambia.

7.5 Seguimiento de trayectoria: del plan a la política

Aquí hay una distinción que conviene tener clara, porque explica cómo está montada la robótica de verdad.

Un **plan** dice qué camino recorrer. Una **política** dice qué acción tomar **desde cualquier estado** en el que el robot se encuentre. Las leyes de control del §6.4 son políticas, no planes.

Lo que se hace en la práctica es una **componenda en dos pasos**:

1. Se **planifica** en un espacio simplificado — solo el estado cinemático, suponiendo que se puede pasar de una configuración a la vecina sin preocuparse de la dinámica. Sale la **trayectoria de referencia**.
2. Se **convierte en política**: un controlador que intenta seguir ese plan y **vuelve a él** cuando se desvía.

⚠ Esa componenda introduce dos suboptimalidades

La primera, **planificar sin tener en cuenta la dinámica**: el camino más corto en el papel puede ser el que peor sigue el robot real. La segunda, **suponer que si te desvías lo mejor es volver al plan original**, cuando a veces lo óptimo es replanificar desde donde estás. El **control óptimo** ataca las dos a la vez: optimiza directamente sobre las acciones teniendo en cuenta la dinámica, con técnicas de optimización de trayectoria (*multiple shooting*, colocación directa) y, cuando el coste es cuadrático y la dinámica lineal, con el **LQR** —y su variante **iLQR**, que es lo que se usa cuando no se cumplen esas condiciones ideales, que es casi siempre.

8. Percepción robótica (RA4-b)

Percepción es convertir medidas de sensores en una representación interna del entorno. Y el problema de fondo es el del §4.3: **todos los sensores tienen ruido y ninguno lo ve todo**.

8.1 Localización y mapeo

Hay tres problemas, de menos a más difícil:

Problema	Lo que se sabe	Lo que se busca
Localización	El mapa	Dónde está el robot

Problema	Lo que se sabe	Lo que se busca
Mapeo	Dónde está el robot	El mapa
SLAM	Nada de las dos	Las dos a la vez

SLAM (*Simultaneous Localization And Mapping*) es el caso realista y parece un imposible —para saber dónde estás necesitas el mapa, y para hacer el mapa necesitas saber dónde estás—, pero se resuelve **probabilísticamente**: en vez de una respuesta, el robot mantiene una **distribución de probabilidad** sobre dónde puede estar (su *estado de creencia*) y la va afinando con cada medida.

Las herramientas son las mismas que aparecen en toda la IA con incertidumbre: **filtros de Kalman**, **modelos ocultos de Markov** y **redes bayesianas**, que modelan la transición de estado y el sensor.

Localización de Monte Carlo, en tres fotos

La localización con **filtro de partículas** (MCL) representa la creencia con una nube de «partículas», cada una una hipótesis de dónde está el robot:

1. Al arrancar, las partículas están **repartidas por todo el plano**: incertidumbre total.
2. Llega el primer conjunto de medidas y las partículas se **agrupan** en las zonas compatibles. Suelen quedar **varios grupos**: los pasillos de un edificio de oficinas se parecen entre sí, y el robot todavía no puede distinguirlos.
3. Con suficientes medidas, todas las partículas **colapsan en un solo sitio**.

Lo interesante es el paso 2: el robot representa explícitamente que hay **varias respuestas posibles** en vez de escoger una y equivocarse. Solo hacen falta dos modelos: el del movimiento y el del sensor.

8.2 Qué representación conviene

La tendencia en robótica es hacia representaciones con **semántica bien definida** — no solo «aquí hay algo», sino «esto es una puerta». Y las técnicas probabilísticas **ganan** a las alternativas en los problemas difíciles de percepción, como el propio SLAM.

Pero no siempre hace falta la artillería

Las técnicas estadísticas son a veces **demasiado engorrosas** para lo que se necesita, y una solución más simple es igual de efectiva en la práctica. Es el mismo criterio de la UD05 con el PID: empezar simple y añadir complejidad solo cuando se justifica. Los **EX2** y **EX3** de esta unidad navegan con reglas y con lógica difusa sobre los píxeles de una cámara, sin ningún filtro probabilístico, y funcionan.

8.3 Percepción que se adapta

Hay un tipo de aprendizaje que resuelve un problema muy concreto: que las medidas de los sensores cambian **de golpe** cuando cambia el entorno.

Piensa en entrar desde un exterior soleado a una habitación con fluorescentes. No solo hay menos luz: el fluorescente tiene **más componente verde** que el sol, así que todos los colores cambian. Y sin embargo no nos parece que a la gente se le haya puesto la cara verde. Nuestra percepción se **readapta** en segundos y el cerebro ignora la diferencia. Un robot que no haga eso creerá que ha cambiado de mundo.

9. Incertidumbre y aprendizaje (RA4-b)

9.1 Planificar cuando no se sabe el estado exacto

La incertidumbre viene de tres sitios: el entorno es **parcialmente observable**, los efectos de las acciones son **estocásticos** (o simplemente no están modelados) y los propios algoritmos de estimación son **aproximados** — un filtro de partículas no da el estado de creencia exacto ni con un modelo perfecto.

La mayoría de los robots en producción usan, aun así, **algoritmos deterministas**, y se apañan con dos atajos:

1. **Discretizar** el espacio de estados continuo (grafo de visibilidad, descomposición celular).
2. Ante la duda sobre el estado actual, **quedarse con el más probable** de la distribución.

Es más rápido y escala mejor. El precio es que el robot actúa como si estuviera seguro cuando no lo está.



Cuándo ese atajo se rompe

Quedarse con el estado más probable funciona mientras la distribución tenga **un solo pico claro**. Falla justo en los casos del §8.1, paso 2: cuando hay **varias hipótesis igual de buenas** —tres pasillos que se parecen—, elegir la más probable es tirar una moneda, y el robot actúa con plena confianza sobre una respuesta que tiene un 33 % de ser cierta.

9.2 Aprendizaje por refuerzo en robótica

El aprendizaje por refuerzo funciona muy bien en simulación y muy mal en un robot real, y la razón es sencilla: **el mundo real se niega a ir más rápido que el tiempo real**.

	En simulación	En un robot real
Velocidad	Millones de pruebas en unas horas	Las mismas pruebas tardarían años
Riesgo	Ninguno	El robot no puede arriesgarse a una prueba que lo dañe — y por tanto no puede aprender de ella

De ahí que el problema central sea la **transferencia de la simulación a la realidad** (*sim-to-real*), que es un área de investigación activa. Y de ahí también que los sistemas robóticos prácticos incorporen **conocimiento previo** sobre el robot, el entorno y la tarea: es la única forma de aprender rápido y de comportarse con seguridad mientras aprende.



Esto conecta con toda la unidad

EX4, EX5 y EX6 hacen exactamente este recorrido en pequeño y **en simulación**, que es donde se puede: generar datos conduciendo el robot, entrenar una red con ellos, y después dejar que la solución **evolucione** sin datos etiquetados.

10. Técnicas de programación de robots (RA4-c)

10.1 Las cinco formas de programar un robot industrial

Técnica	Cómo funciona	Ventajas	Desventajas	Cuándo
Teach pendant	Guías el brazo con la consola por los puntos de paso y los grabas	Curva de aprendizaje mínima, puesta a punto rápida	Para la producción mientras se programa; poca flexibilidad	Trayectorias básicas, paletizado, soldadura por puntos
Guiado manual	Mueves el efector con la mano y registras posiciones	Muy intuitivo, sin escribir código	Solo en cobots seguros	Cobots, montaje asistido
Programación textual	Código nativo del fabricante: RAPID, KRL, URScript	Determinista, se integra con PLC y seguridad	Sintaxis propietaria, no portable	Células industriales estables de alta cadencia
Programación offline (OLP)	Se programa en simulación sobre modelos CAD	Cero impacto en producción ; permite probar hipótesis	Coste de licencias; exige modelos precisos y robot calibrado	Geometrías complejas, células multimarca
ROS 2 y frameworks	Middleware de nodos, <i>topics</i> y servicios	Estándar abierto, acceso directo a algoritmos de IA	Curva de aprendizaje alta	Investigación, robots móviles, multi-robot



URScript se parece a Python

Los robots de **Universal Robots** se programan en **URScript**, un lenguaje muy parecido a Python que además puede enviarse al robot desde un cliente Python por FTP, SSH o RTDE. Es la puerta de entrada natural a la programación de cobots desde este módulo.